SSW 590 — DevOps

Group 1 DevOps IT Manual

Version 1.3+build.23.d1da656

Group 1 Submission

by

Group 1: Brahmi Dalvi and Sakshi Deshpande

Stevens Institute of Technology

November 11, 2026

© Stevens Institute of Technology 2026
ALL RIGHTS RESERVED

Template Attribution

This manual is built from the SSW 590 DevOps assignment template (version 1.3, distributed through Canvas), developed by **Darian Muresan, Ph.D.**, in conjunction with **OpenAI Codex** and **Anthropic Claude Code**.

SSW 590 — DevOps

Group 1 DevOps IT Manual

Version 1.3+build.23.d1da656
Group 1 Submission

Group 1: Brahmi Dalvi and Sakshi Deshpande
Stevens Institute of Technology
November 11, 2026

This is Group 1's living DevOps IT manual for SSW 590. It records what the group actually built, tested, and learned, one chapter per course module. The first chapter documents the startup's Gitea work hub: a student-pack domain, a version-pinned Gitea deployment, HTTPS with automated certificate renewal, a minimal firewall exposure, and an issue-to-reviewed-merge team workflow. Later chapters add an automated manual build with Gitea Actions and Jenkins continuous integration with a required merge gate on the product repository, and monitoring with Prometheus and Grafana: probes of both services, a dashboard, two alerts tested for real, and a controlled outage with a written review. Results are stated only where they were observed. The one thing not observed, the first automatic renewal of the production certificate (expected between December 13 and 15, 2026, after the due date), is labeled as such.

Document History

How to Use This Chapter

Add the newest approved change at the top. Retain every approved entry and link each entry to the changed chapter, table, figure, or controlled artifact. Every entry must say whether the Hosts and Passwords tables changed. Author initials: BD = Brahmi Dalvi, SD = Sakshi Deshpande, CC = Claude Code (AI assistant; see the AI Prompt Log).

Table 1: Document Update History

Date / Version	Author Initials	Change	Changed At
November 11, 2026 / 1.3	BD, SD, CC	New Chapter 6 (course Chapter 5): Prometheus, Grafana, Node Exporter, and Blackbox Exporter as four containers behind Caddy at `metrics.ssw590-g1.tech` after resizing the server to `t3.small`; HTTPS probes of the Gitea and Jenkins login pages; a six-panel dashboard; two alert rules (service availability and root-disk capacity) delivered to a Discord contact point; a real capacity test with a temporary threshold; a controlled three-minute Jenkins outage with timeline and recovery checks; a restart persistence test; and a worked SLI and SLO window (evidence 110–169 inline). Version increased from 1.2 to 1.3. **Hosts and Passwords tables: changed** (instance type; the second SSH address; the `metrics` host and DNS record with its four containers and private ports; Passwords hints for the Grafana administrator and user, the Discord webhook, the Discord server, and the extended team address list). Merged through the established review: infra pull request #2 (monitoring files, merge commit `c03fb1b`).	Chapter 6, Chapter 2, Section 6.8

Document Update History (continued)

Date / Version	Author Initials	Change	Changed At
November 2, 2026 / 1.2	BD, SD, CC (SD checked rows 4, 5, 6, 8 on November 3)	New Chapter 5 (course Chapter 4): Jenkins 2.580.2 LTS in a container behind Caddy at `ci.ssw590-g1.tech` with a public login page only, named accounts and matrix permissions, a Gitea webhook and integration user, a multibranch pipeline for `twintrack`, the required status check `twintrack/pipeline/pr-main` on `main`, a restart persistence test, and two controlled failure and repair runs on pull request #10 (evidence 70–106 inline; the failed first webhook is documented). Version increased from 1.1 to 1.2. **Hosts and Passwords tables: changed** (Jenkins host row and the `ci` DNS record; Passwords hints for the Jenkins accounts, the `jenkins-bot` user and token, the webhook with no secret, and the team address list). Pull request #10 was reviewed, approved, and merged by SD, and issue #9 closed (evidence 106).	Chapter 5, Chapter 2, Section 5.2.8
October 20, 2026 / 1.1	SD, BD, CC	New Chapter 4 (course Chapter 3): the manual's source moved to `group1-startup/manual` and is built on every push by Gitea Actions on a self-managed runner; the PDF carries a build identifier (run number and commit); evidence 50–66 inline. Version increased from 1.0 to 1.1. **Hosts and Passwords tables: changed** (runner host row; runner registration token hint; SD's Write access to the manual repository).	Chapter 4, Chapter 2, Section 4.2.8

Document Update History (continued)

Date / Version	Author Initials	Change	Changed At
October 15, 2026 / 1.0 (final)	BD, CC	Final fixes before submission: removed the last placeholder markers; set the repository link and the document status; corrected the Introduction link; marked the optional extra credit as not attempted; wrote the human-review lines in the AI Prompt Log; updated the front status table, the abstract, and notes inside captured evidence that had become out of date; made the repository public after a scan of its full history found no secrets. Earlier the same day: second-network firewall test, certificate-renewal dry run on Let's Encrypt staging, redacted AWS console evidence, cost estimate, row 6 setup run, and all evidence embedded inline. **Hosts and Passwords tables: updated** (Sakshi's SSH key recorded as having no passphrase; her Gitea MFA recorded as on, as confirmed by her; Brahmi's MFA and credential-storage hints added).	Chapter 3, Chapter 2, Chapter C
October 15, 2026 / 1.0	SD, CC	Submission version of the Gitea chapter (course Chapter 2): all eight rubric subsections carry real, dated evidence inside the PDF. Final quality pass: version increased from 0.1 to 1.0; secrets and address scan of the chapter sources and printed evidence; every screenshot inspected; failed tests from both members collected (Table 3.19). **Hosts and Passwords tables: updated in this version** (created from placeholders by BD: Gitea host, domain, earlier host, Gitea users, SSH keys, AWS, Name.com, and GitHub accounts; hints only).	Chapter 2, Chapter 3, Section 3.2.8

Document Update History (continued)

Date / Version	Author Initials	Change	Changed At
October 15, 2026 / 0.1 (in progress)	SD, CC	Row 6: wrote the startup team workflow subsection with real evidence (organization, public product repository, member roles, six-issue backlog, two issue-to-merge traces with peer approvals and automatic issue closure; evidence 25–30 inline) and recorded SD's expected HTTP 403 repository-creation failure in the failed-test log. Rows 3 and 7 evidence moved inline as listings and figures. Hosts and Passwords tables: **not changed** by this entry.	Section 3.2.6, Tables 3.14 and 3.15, Table 3.19
October 15, 2026 / 0.1 (in progress)	SD, CC	Row 7: rewrote the architecture and operation subsection for the live AWS deployment (figure labels, request and administration paths, interface table incl. Caddy admin port and IPv6, data locations, runbook, troubleshooting) and ran the read-only runbook commands on the server (evidence 24). Hosts and Passwords tables: **not changed** by this entry.	Section 3.2.7, Figure 3.17
October 15, 2026 / 0.1 (in progress)	BD, CC	Claimed the domain `ssw590-g1.tech` through the Student Pack ($0.00, automatic renewal off, stored card deleted); created the AWS security group, key pair, `t3.micro` instance, and Elastic IP with the AWS command line interface; created the DNS record; deployed Gitea 1.27.3 and Caddy 2.11.7 with Docker Compose and obtained a Let's Encrypt certificate; created the first administrator, the organization `group1-startup`, the team `developers`, and Sakshi's account and SSH key; wrote the domain, deployment, HTTPS, and firewall subsections from captured evidence, with the renewal test, the second-network test, and the open items marked as pending. **Hosts and Passwords tables: updated** (created from placeholders; the Gitea host, domain, earlier host, accounts, and keys are now listed, hints only).	Chapter 3, Chapter 2

Document Update History (continued)

Date / Version	Author Initials	Change	Changed At
October 15, 2026 / 0.1 (in progress)	SD, CC	Row 3: ran the persistence test on the EC2 server (container replaced, repository, issue, and commit unchanged) and replaced the pending results with the real log (evidence 20) and conclusion. Hosts and Passwords tables: **not changed** by this entry.	Section 3.2.3, Table 3.6
October 12, 2026 / 0.1 (SD draft, branch `sd/gitea`)	SD, CC	Drafted the four SD regions of the Gitea chapter: database and persistence (SQLite settings, data locations, scripted persistence-test procedure and a local rehearsal clearly marked as non-graded), startup team workflow (backlog and trace tables, pending G3), architecture and operation (new TikZ figure, request path, interface table, runbook, persistence-versus-backup), and manual quality controls (checklist and failed-test log). Added glossary terms (SQLite, named volume, pull request) and one citation. Hosts and Passwords tables: **not changed**. Version not yet increased; that happens at submission.	Sections 3.2.3, 3.2.6, 3.2.7, 3.2.8; Figure 3.17
October 12, 2026 / 0.1	BD, CC	Initial setup: created the separate IT manual from the course Canvas template (version 1.3, one chapter per file, template guides retained as appendices), the Gitea chapter outline with the eight rubric-aligned subsections and an extra-credit subsection, and the Hosts and Passwords tables. Hosts and Passwords tables created, with no real hosts or secrets recorded yet. No infrastructure exists yet; every evidence item is pending.	Chapter 1, Chapter 2, Chapter 3

Table of Contents

Template Attribution iii

Document History v

1 Introduction 1
- 1.1 Purpose . . . 1
- 1.2 Team and Repository . . . 1
- 1.3 Chapter Map . . . 1
- 1.4 Repository Layout . . . 2

2 Hosts and Passwords 3
- 2.1 Hosts . . . 3
- 2.2 Passwords (hints only) . . . 7

3 Startup Foundation with Gitea 12
- 3.1 Overview and Status . . . 12
- 3.2 Required Subsections . . . 13
  - 3.2.1 Student Pack domain and DNS (10 points) . . . 13
  - 3.2.2 Pinned Gitea deployment (10 points) . . . 18
  - 3.2.3 Database and persistence (10 points) . . . 26
  - 3.2.4 HTTPS and certificate renewal (15 points) . . . 30
  - 3.2.5 Firewall and external exposure (15 points) . . . 37
  - 3.2.6 Startup team workflow (20 points) . . . 42
  - 3.2.7 Architecture and operation (5 points) . . . 47
  - 3.2.8 Manual quality and controls (15 points) . . . 51
- 3.3 Extra Credit . . . 54
  - 3.3.1 Extra credit: verified backup and restore (up to 20 extra-credit points; not attempted) . . . 54

4 Automated Manual Build 55
- 4.1 Overview and Status . . . 55
- 4.2 Required Subsections . . . 55
  - 4.2.1 Source and trigger (15 points) . . . 55

4.2.2 Runner setup (15 points) . . . 56
4.2.3 Reproducible compilation (20 points) . . . 61
4.2.4 PDF version traceability (15 points) . . . 65
4.2.5 Downloadable artifact (10 points) . . . 65
4.2.6 Change, failure, and repair evidence (10 points) . . . 68
4.2.7 Runner and secret boundary (5 points) . . . 71
4.2.8 Manual quality and controls (10 points) . . . 72

5 Continuous Integration with Jenkins **75**
5.1 Overview and Status . . . 75
5.2 Required Subsections . . . 76
5.2.1 Jenkins deployment and persistence (10 points) . . . 76
5.2.2 HTTPS, access, and firewall (15 points) . . . 81
5.2.3 Gitea trigger and traceability (15 points) . . . 88
5.2.4 Versioned pipeline and tests (15 points) . . . 99
5.2.5 Required merge gate (15 points) . . . 102
5.2.6 Failure, repair, and review (10 points) . . . 107
5.2.7 Credentials and build boundary (10 points) . . . 114
5.2.8 Manual quality and controls (10 points) . . . 116

6 Monitoring and reliability (SRE) **120**
6.1 Deployment and capacity (15 points) . . . 120
6.2 Metrics and HTTPS probes (15 points) . . . 129
6.3 Access control and exposure (10 points) . . . 132
6.4 Dashboard (15 points) . . . 136
6.5 SLI, SLO, and error budget (10 points) . . . 143
6.6 Alerts (15 points) . . . 146
6.7 Controlled incident and review (10 points) . . . 157
6.8 Manual changes in this version (10 points) . . . 161

Bibliography **163**

A Document Authoring Guide **167**
A.1 How to Use This Appendix . . . 167
A.2 Template Provenance . . . 167
A.3 File Naming and Responsibilities . . . 167
A.4 Synonyms and Controlled Terms . . . 167
A.5 Building the PDF . . . 168
A.5.1 macOS and Linux Build . . . 168
A.5.2 Windows PowerShell Build . . . 168
A.6 Install LaTeX Tools on macOS . . . 168
A.6.1 Recommended Installation . . . 168
A.6.2 Verification and Compilation . . . 168
A.7 Install LaTeX Tools on Windows . . . 169
A.7.1 Recommended Installation . . . 169

A.7.2 Verification and Compilation . . . 169
A.8 Common Build Problems . . . 169
A.9 Authoring and Review Rules . . . 170

B AI Agent Usage Guide **171**
B.1 How to Use This Appendix . . . 171
B.2 Governing Principles . . . 171
B.3 Chapter File Convention . . . 171
B.4 AI Quick Start . . . 172
B.5 Task Routing . . . 172
B.6 DevOps-Specific Rules . . . 172
B.7 Example AI Requests . . . 173
B.7.1 Create an Assignment Brief . . . 173
B.7.2 Review a Student Workflow . . . 173
B.8 Human Decisions and Information Boundaries . . . 173
B.9 Completion Checklist . . . 173

C AI Prompt Log **175**
C.1 How to Use This Appendix . . . 175
C.2 Record: IT Manual Skeleton and Gitea Chapter Outline . . . 175
C.3 Record: SD Regions of the Gitea Chapter (Draft) . . . 176
C.4 Record: BD Deployment, Evidence, and Drafted Regions . . . 176
C.5 Record: SD Rows of the Automated Manual Build Chapter . . . 177
C.6 Record: Jenkins Integration and Chapter (Assignment 6) . . . 178
C.7 Record: SD Review of Rows 4, 5, 6, and 8 of the Jenkins Chapter . . . 178
C.8 Record: BD's Rows 4, 6, 7, and 8 of the Monitoring Chapter . . . 179

Glossary **181**

Acronyms **184**

Index **185**

List of Tables

1 Document Update History vi

2.1 Hosts 4
2.2 Passwords and keys (hints only; no secrets are recorded) 7

3.1 Rubric rows, owners, and status 12
3.2 Domain registration facts 13
3.3 Deployed components and versions (October 15, 2026) 20
3.4 Deployment checks, October 15, 2026 24
3.5 Where Gitea's persistent data lives 27
3.6 Persistence test on the cloud VM 28
3.7 Certificate options 31
3.8 Certificate served by `git.ssw590-g1.tech`, October 15, 2026 33
3.9 HTTPS checks, October 15, 2026 (rows 1 to 4 at 17:35 UTC from the administrator's computer; the renewal rows from the 19:03 to 19:08 UTC test) 37
3.10 Security group 37
3.11 Inbound rules, IPv4 (IPv6 has no rules) 38
3.12 External exposure checks 39
3.13 Startup organization and members 43
3.14 Startup backlog in `group1-startup/twintrack`, October 15, 2026 43
3.15 Issue-to-merge traces, October 15, 2026 44
3.16 Interfaces and exposure 49
3.17 Troubleshooting 50
3.18 Manual quality controls, checked October 15, 2026 51
3.19 Failed tests and corrective actions 52

4.1 Rubric rows, owners, and status 55
4.2 Network path of a push and of a runner job, with public and private ports . 56
4.3 The Gitea Actions runner 58
4.4 Build toolchain 61
4.5 One build traced from the run to the PDF 65
4.6 Successive changes, a controlled failure, and its repair, October 20, 2026 . . 69
4.7 Controlled failure: expected and actual results 69
4.8 Manual quality controls for this chapter, checked October 20, 2026 73

4.9 Failed tests and corrective actions in this chapter 74

5.1 Rubric rows, owners, and status 75
5.2 Jenkins deployment record 77
5.3 External tests, November 2, 2026, expected versus actual 86
5.4 Gitea to Jenkins integration 88
5.5 Pipeline stages 100
5.6 From push to status in Gitea for the two merge-gate states 103
5.7 Dated sequence of commits and Jenkins builds for pull request #10, October 29 to November 2, 2026 108
5.8 Secrets and where they live (hints only, no values) 114
5.9 Failed tests and mistakes of this assignment, and the corrections 118

6.1 Monitoring containers, versions, and limits 120
6.2 Metrics used in this chapter 131
6.3 Prometheus compared with the host 131
6.4 Grafana from a non-team network, November 11, 2026 133
6.5 Dashboard panels and why their queries are written this way 138
6.6 Host CPU and available memory around build #3 (UTC, 30 s samples) . . 141
6.8 SLI for the window 8:20–8:38 PM, November 11, 2026 144
6.9 Alert rules: owner, severity, query, threshold, timing, no-data policy 147
6.10 Incident timeline, November 11 to 12, 2026 157

B.1 AI Task-Routing Table 172

List of Figures

3.1 Name.com cart before payment, October 15, 2026: `ssw590-g1.tech` free for one year through the partner offer, Domain Safe Pro ($7.99) declined, total $0.00. This is the cart, not a receipt. . . . 14
3.2 Name.com order summary, October 15, 2026: domain and free Domain Safe, total $0.00. . . . 15
3.3 Domain page right after registration, October 15, 2026: "Renews Oct 15, 2027", automatic renewal ON, WHOIS privacy ON, transfer locked, "Renew for $89.99". . . . 15
3.4 Domain page, October 15, 2026: expires October 15, 2027, automatic renewal OFF, renewal price $89.99. . . . 16
3.5 Name.com DNS records for `ssw590-g1.tech`, October 15, 2026: one A record for `git`. . . . 17
3.6 Name.com billing page, October 15, 2026: "No saved payment profiles" after the card was deleted. . . . 19
3.7 Name.com order confirmation email, received Thursday October 15, 2026 at 00:36 (mailbox time): order 28974415, `ssw590-g1.tech`, one year, order total $0.00. The email gives the order date as October 14, 2026. . . . 19
3.8 The deployed Gitea at `git.ssw590-g1.tech`, signed in as the administrator, October 15, 2026. . . . 26
3.9 Repository `sakshi/persistence-check` after the container replacement, October 15, 2026: one commit, `4f15f70000`. . . . 30
3.10 Issue #1 still open after the container replacement, October 15, 2026. . . . 31
3.11 What the security group admits (green) and drops (red, dashed) for the Gitea VM [1, 2]. . . . 39
3.12 AWS console, instance `i-0c68ec1bc1e16a414`, Security tab, October 15, 2026 (redacted). . . . 42
3.13 Pull request #7 before approval, October 15, 2026: SD's review comment, and the merge blocked with "0 of 1 official approvals granted". . . . 45
3.14 Pull request #7 merged, October 15, 2026: "Closes #2", approved by `sakshi`, merged by `brahmi` as commit `0d0c3dac4d`. . . . 45
3.15 Pull request #8 merged, October 15, 2026: "Closes #1", approved by `brahmi`, merged by `sakshi` as commit `15f8a00343`. . . . 46

3.16 Closed issues in twintrack, October 15, 2026: #1 and #2 closed by their merges, #4 closed after the branch-protection test. 46
3.17 Group 1 Gitea deployment on one AWS EC2 instance, October 15, 2026. The security group admits HTTPS (443) and HTTP (80, redirect only) from anyone over IPv4, and SSH (22) only from the approved team address. Only Caddy publishes ports; Gitea's port 3000, its container sshd, Caddy's admin port 2019, and the SQLite file are private. There is no IPv6 path [1, 3, 4]. . 48

4.1 Actions tab of group1-startup/manual, October 19, 2026 (evening, US Eastern): run #1 for commit 1fb4e15 failed, run #2 for commit a5f3446 succeeded; both were started by a push to main. 57
4.2 Run #2 summary: triggered via push, status Success, duration 1 minute 11 seconds, workflow build.yml (on: push), one artifact it-manual-pdf. 57
4.3 Site Administration, Actions, Runners, October 19, 2026: group1-runner-1, version v0.6.1, type Global, label manual-builder, status Idle, last online "now". 59
4.4 Log of run #2, step *Set up job*: the job is assigned to group1-runner-1, runs in a temporary container from group1/texlive-node:2026.09, and the image is not pulled (forcePull=false). The first action fetched is actions/checkout from github.com. 59
4.5 Run #2 on the Gitea runner, October 20, 2026, seen signed out: every step succeeded and the run has one artifact, it-manual-pdf. 64
4.6 Title page of the PDF downloaded from run #2's artifact, October 20, 2026: the version reads 1.0+build.2.a5f3446, matching run #2 and commit a5f3446. 66
4.7 Title page of the PDF downloaded from the artifact of run #2 and opened in a viewer: Version 1.0+build.2.a5f3446. 67
4.8 Run #2 summary with the tooltip of it-manual-pdf: "Expires at Jan 17, 2027, 22:01" and 4.0 MiB. 68
4.9 Run #5, October 20, 2026, seen signed out: the compile step failed, and the verify and upload steps were skipped. 70
4.10 Settings, Collaborators of group1-startup/manual, October 19, 2026: sakshi with Write; teams Owners (Owner) and developers (unit permissions Code, Issues, Pull Requests, Releases, Wiki, Projects, Packages, Actions). 71

5.1 First-time setup finished, November 2, 2026: "Jenkins is ready! Your Jenkins setup is complete", Jenkins 2.580.2. The one-time unlock password was read on the server and is not shown. 82
5.2 The public login page https://ci.ssw590-g1.tech/login on November 2, 2026 (captured with headless Chrome from the administrator's computer): "Sign in to Jenkins", username, password, and Sign in. 83

5.3 External test from a phone on mobile data, November 2, 2026, 12:06 PM. Left: the Jenkins page opens without an account (the first-time "Unlock Jenkins" screen, because setup was not finished yet). Right: `https://ci.ssw590-g1.tech/` answers *Forbidden*. 84
5.4 Name.com Manage DNS, November 1, 2026: new A record `ci.ssw590-g1.tech` to the same address as `git.ssw590-g1.tech`, TTL 300. 85
5.5 Jenkins Users page, November 2, 2026: two named accounts, `brahmi` and `sakshi`. 88
5.6 Manage Jenkins, Security, November 2, 2026: sign-up off; matrix authorization with Brahmi = Administer, Sakshi = Read and Build, Anonymous and Authenticated Users = no permissions. 90
5.7 UML-style deployment diagram of the actual browser and webhook paths on one EC2 instance, adapted from the course manual's Figure 4.2. Solid green: browser and proxy paths. Dashed grey: private paths between Gitea and Jenkins. Builds run on the Jenkins controller executor [1, 5]. 91
5.8 `twintrack` Settings, Collaborators, November 2, 2026: `jenkins-bot` added with Write. 92
5.9 Jenkins Credentials, November 2, 2026: `gitea-jenkins-bot` (System, Global). The token is stored in Jenkins and is not displayed. 93
5.10 Multibranch scan of `group1-startup/twintrack`, November 2, 2026 (23:54 UTC): branches without a `Jenkinsfile` do not meet the criteria; pull request #10 does, and "Scheduled build for branch: PR-10". 94
5.11 `twintrack` Settings, Webhooks (repository level), November 2, 2026: the webhook to `http://jenkins:8080/gitea-webhook/post`, status dot green. 95
5.12 Recent Deliveries of the webhook: a Test Push Event to the private address, POST `application/json`, response 200. The signature headers are empty because no webhook secret is configured. 96
5.13 The webhook's events before the fix, November 2, 2026: *Pull Request* is ticked and *Pull Request Synchronized* is not; the three deliveries below all succeeded. 97
5.14 Branch rule for `main`, November 2, 2026: *Enable Status Check* ticked, pattern `twintrack/pipeline/pr-main`, shown as "Matched". Saved: Gitea's API reports `enable_status_check: true` with that context and one required approval. 103
5.15 **Failed check blocking the merge.** Pull request #10, November 2, 2026, head commit `86ee9fc`, required check `twintrack/pipeline/pr-main` failed. The merge button area is below the cropped part of the page. 104
5.16 Jenkins job `PR-10` after the failing push, November 2, 2026: red, last build #4 failed (12:30 AM UTC), last stable build #3. 105
5.17 **Passing check after repair.** Pull request #10, November 2, 2026, head commit `92c121d`, required check `twintrack/pipeline/pr-main` successful; the failed commits `eb97d7a` and `86ee9fc` are marked with red crosses; merge waiting for approval. 106
5.18 Jenkins job `PR-10` after the repair, November 2, 2026: build #5 green (12:32 AM UTC), build #4 failed, history #1 to #5. 106

5.19 Pull request #10 right after the first failing push (commit `eb97d7a`), November 2, 2026: the required check is marked *Required* and still pending, with "Some required checks are missing" and "0 of 1 official approvals"; the banner "As an administrator, you may still merge" is visible further down. 107
5.20 Issue #9 in `twintrack`, opened October 29, 2026 by `brahmi`. 108
5.21 Pull-request-to-merge activity for pull request #10, adapted from the course manual's Figure 4.1 [1] with the real commits, Jenkins builds, and times (UTC, November 3, 2026, unless noted; Listing 34). A new commit repeats the build; the check on the latest head commit, not the webhook delivery, gates the merge. 117

6.1 Deployment of the Group 1 server with monitoring, adapted from the course manual's Figure 5.1. Blue: the four new monitoring containers. Arrows start at the side that opens the connection. Gitea and Jenkins are probed only through their public HTTPS names; Prometheus, the exporters, and Grafana's port 3000 are reachable only on private Docker networks. Not drawn: the Gitea Actions runner and the Gitea–Jenkins webhook (Chapter 5). 123
6.2 Grafana home page after BD signed in as administrator over HTTPS with her own password, November 11, 2026 (BD's screenshot). 137
6.3 Dashboard, top half, November 11, 2026, last 3 hours EST: CPU about 2%, available memory about 750 MiB of 1.86 GiB, root filesystem free about 33%, probe duration of both URLs (one 4 s sample on the Jenkins line). 138
6.4 Dashboard scrolled down: Gitea and Jenkins both UP, probe duration with two labeled lines, and the scrape-health table. 140
6.5 Jenkins build #3 of `twintrack/main` (Nov 12, 2026, 1:41:47 AM server time): SUCCESS, took 2.4 s, and its console, `Running on Jenkins in /var/jenkins_home/workspace/twintrack_main`, the controller. 143
6.6 Contact points: `team-alerts` with one Discord integration (the webhook URL is not shown); `empty` has no integrations. 147
6.7 Grafana's test notification in the Discord channel, November 11, 2026, 19:30 EST. 149
6.8 Default notification policy: delivered to `team-alerts`, grouped by `grafana_folder` and `alertname`, group wait 30 s, group interval 1 m, repeat 4 h. 150
6.9 Rule `https-service-down`: query, threshold "below 1", labels `owner` and `severity`, notifications to `team-alerts`. 151
6.10 The same rule, lower part: pending 1 m, no data `NoData`, error `Error`, `runbook_url` and `summary`. 152
6.11 Rule `root-disk-below-15-percent`: query for the xfs root, graph at about 32.8%, threshold "below 15". 153
6.12 The same rule, lower part: pending 5 m, `NoData`, `Error`, `runbook_url`, `summary`. 153
6.13 Disk rule Firing with the temporary threshold 55, 20:08 EST. 154
6.14 The Discord Firing message for the disk rule, 20:08 EST. 155

6.15 The Discord Resolved message, 20:16 EST, after the threshold was restored to 15. 156
6.16 History of the rule for the Jenkins instance: Pending 20:25:10, Alerting 20:26:10, Normal 20:28:10 (EST). 159
6.17 Discord Firing message for `https://ci.ssw590-g1.tech/login`, 20:26 EST. 160
6.18 Discord Resolved message, 20:28 EST. 160
6.19 Jenkins `twintrack/main` build #2 after the outage, Nov 12, 2026, 1:29:12 AM server time (UTC): SUCCESS in 7.9 s. 161

Chapter 1

Introduction

1.1 Purpose

This is Group 1's operational IT manual for the SSW 590 startup. It is a living record: one chapter is added per delivered capability, and the compiled PDF is the only artifact submitted for each assignment. It is separate from the earlier assignment report in the `report/` folder of the repository.

1.2 Team and Repository

Item	Value
Group	Group 1
Members	Brahmi Dalvi (BD), Sakshi Deshpande (SD)
Course repository	https://github.com/brahmi9811/ssw-590-group1
Team Gitea	https://git.ssw590-g1.tech; organization group1-startup, product repository twintrack

1.3 Chapter Map

Where	Content
Chapter 2	Hosts and Passwords tables (hints only)
Chapter 3	Startup foundation with Gitea (course Chapter 2)
Chapter 4	Automated manual build with Gitea Actions (course Chapter 3)
Chapter 5	Continuous integration with Jenkins and a required merge gate (course Chapter 4)

Chapter 6	Monitoring and reliability: Prometheus, Grafana, probes, alerts, and a controlled outage (course Chapter 5)

1.4 Repository Layout

`assignment-4/manual/`	this LaTeX manual (source stays in the team; submit only the PDF)
`assignment-4/gitea/`	versioned, non-secret deployment files (compose, proxy config)
`assignment-4/*.md`	work split and per-person instructions
`report/`	earlier submitted assignment (do not modify)

Chapter 2

Hosts and Passwords

Keep both tables current; a missing or stale table costs 10 points. The Passwords table holds *hints and vault references only*: never a password, derivation key, recovery code, token, or private key. Last updated: November 11, 2026.

2.1 Hosts

Table 2.1: Hosts

Hostname	Role, environment, provider	Address, ports, owner, URL, backup, monitoring
`git.ssw590-g1.tech`	**Role:** Gitea 1.27.3 and Caddy 2.11.7 in Docker; the startup's work hub. **Environment:** the only one (no separate test environment). **Provider:** AWS EC2, `us-east-1c`, `t3.small` (resized from `t3.micro` on November 10, 2026; same instance, disk, and Elastic IP). **Instance:** `i-0c68ec1bc1e16a414`	**Address:** A record to Elastic IP `52.73.254.36`, TTL 300; no IPv6. **Exposed ports:** 443 and 80 from anyone; 22 from two single addresses only (the administrator's home address and, since November 11, 2026, the campus address used by both team members; the second rule is kept for the later assignments and is a shared campus address). **Owner:** Brahmi Dalvi. **HTTPS URL:** `https://git.ssw590-g1.tech` **Backup:** none yet (the optional backup and restore is unassigned). **Monitoring:** Docker health check, and since November 11, 2026 Prometheus, Node Exporter, and Blackbox Exporter probes with a Grafana dashboard and two alert rules (Chapter 6)
`gitea-runner` (added October 19, 2026)	**Role:** Gitea Actions runner `group1-runner-1` (`act_runner` 0.6.1) that builds this manual; a Docker container on the same host as Gitea, not a separate machine. **Environment:** the only one. **Provider:** AWS EC2, same instance as above	**Address:** none of its own; it connects out to `https://git.ssw590-g1.tech`. **Exposed ports:** none published by the runner (80 and 443 stay Caddy's, 22 admin only). **Owner:** Brahmi Dalvi. **Local job image:** `group1/texlive-node:2026.09`, about 5.8 GB on the host disk. **Backup:** none; the artifact expires after about 90 days. **Monitoring:** runner status page in Gitea (Site Administration, Actions, Runners)

Hostname	Role, environment, provider	Address, ports, owner, URL, backup, monitoring
`ci.ssw590-g1.tech` (added November 2, 2026)	**Role:** Jenkins controller (`jenkins/jenkins:2.580.2-lts-jdk21` plus `python3`) that builds and tests pull requests of `twintrack`; a Docker container on the same host as Gitea and Caddy. **Environment:** the only one. **Provider:** AWS EC2, same instance as above	**Address:** A record to the same Elastic IP, TTL 300; no IPv6. **Exposed ports:** none published by Jenkins; reached only through Caddy on 443; everyone may open only `/login`, the rest is limited to the team address. **Owner:** Brahmi Dalvi. **HTTPS URL:** `https://ci.ssw590-g1.tech/login` **Backup:** none; data in the named volume `jenkins_jenkins_home` on the same disk. **Monitoring:** an HTTPS probe of its login page every 30 s (Prometheus and Blackbox Exporter), the dashboard, and the alert rule `https-service-down` (Chapter 6); also Jenkins' own pages and the commit statuses in Gitea

Hostname	Role, environment, provider	Address, ports, owner, URL, backup, monitoring
`metrics.ssw590-g1.tech` (added November 10, 2026)	**Role:** Grafana 13.3.1 (dashboard and alert rules) reached through Caddy; with Prometheus v3.16.0, Node Exporter v1.12.1, and Blackbox Exporter v0.29.1 as four Docker containers on the same host as Gitea, Caddy, and Jenkins. **Environment:** the only one. **Provider:** AWS EC2, same instance as above	**Address:** A record to the same Elastic IP, TTL 300; no IPv6. **Exposed ports:** none published by the monitoring containers (Prometheus 9090, Node Exporter 9100, and Blackbox Exporter 9115 only on the private Docker network `monitoring`; Grafana 3000 only on the network shared with Caddy); reached through Caddy on 443; everyone may open only Grafana's login page and its static files, the rest is limited to the team addresses. **Owner:** Brahmi Dalvi. **HTTPS URL:** https://metrics.ssw590-g1.tech/login **Backup:** none; Prometheus and Grafana data in the named volumes `monitoring_prometheus_data` (kept 7 days or 1 GB) and `monitoring_grafana_data` on the same disk. **Monitoring:** Prometheus scrapes itself, Node Exporter, and Blackbox Exporter; its own health is on the dashboard; there is no external monitor, so a failure of the whole host stops monitoring too
`ssw590-g1.tech` (domain and DNS zone)	**Role:** domain registration and DNS records. **Provider:** Name.com, via the GitHub Student Pack offer	**DNS:** Name.com nameservers (`ns1bqx`, `ns2kry`, `ns3jmt`, `ns4ksy` at `name.com`); A records `git`, since November 1, 2026 `ci`, and since November 10, 2026 `metrics`. **Ports:** not a server. **Owner:** Brahmi Dalvi. **Expires:** October 15, 2027; automatic renewal OFF (renewal would cost $89.99); first year free

Hostname	Role, environment, provider	Address, ports, owner, URL, backup, monitoring
`ssw590-group1-web` (previous assignment)	**Role:** color-buttons website from the earlier assignment, unrelated to Gitea. **Provider:** AWS EC2, `us-east-1`, `t3.micro`. **Instance:** `i-00ee7edd4a5352f05`	**Address:** public DNS name recorded in the earlier report (it changes if the instance is stopped). **Exposed ports:** per the earlier report 80, 443, 22 (not re-checked here). **Owner:** Brahmi Dalvi. **Status:** not touched by this assignment; to be disabled after that assignment is graded

2.2 Passwords (hints only)

Table 2.2: Passwords and keys (hints only; no secrets are recorded)

Service / account	Owner	Vault reference or non-revealing hint	Privilege / MFA
Gitea user `brahmi`	Brahmi	Generated once by Gitea when the account was created and saved directly into Brahmi's password manager; never typed into chat, the repository, or this document. Entry name in the password manager: "Gitea admin"	Site administrator and organization owner. MFA: on (reported by Brahmi)
Gitea user `sakshi`	Sakshi	A temporary password was shared outside the chat; Sakshi changed it at first sign-in and keeps the new one herself (where is not recorded here)	Member of team `developers` (Write) and, since October 19, 2026, collaborator with Write on `group1-startup/manual`; not an administrator. MFA: on (confirmed by Sakshi to Brahmi on October 15, 2026)

Service / account	Owner	Vault reference or non-revealing hint	Privilege / MFA
Server login, key `ssw590-g1-gitea-key`	Brahmi	Private key file in the `.ssh` folder of Brahmi's computer, outside the repository, readable only by her Windows account. Created by AWS; no passphrase. Public half is on the server	Linux user `ec2-user`: `sudo` and the `docker` group (root-equivalent). Key login only. MFA: none
Server login, key `sakshi-ssw590-g1`	Sakshi	Private key stays on Sakshi's computer and was never shared; only the public half is on the server. No passphrase is set on the key (reported by Sakshi)	Same `ec2-user` account and privileges. Key login only. MFA: none
AWS IAM user `ssw-590`	Brahmi	Access key kept in the AWS command line configuration on Brahmi's computer, not in the repository or this document	Can create and manage EC2 resources in `us-east-1`; observed to be denied AWS Budgets and public SSM parameters. MFA: not applicable here: the key is used only through the command line, and commands ran without an MFA prompt
AWS account sign-in (console, billing)	Brahmi	Written down offline by Brahmi; the location is not recorded here	Account owner (root sign-in). MFA: on (reported by Brahmi)
Name.com account	Brahmi	Password kept by Brahmi (not in the repository or this document). No payment card is stored (deleted October 15, 2026)	Owner of the domain and its DNS records. MFA: on (reported by Brahmi)

Service / account	**Owner**	**Vault reference or non-revealing hint**	**Privilege / MFA**
GitHub account `brahmi9811`	Brahmi	Written down offline by Brahmi; the location is not recorded here	Owns the course repository and the Student Pack. MFA: on (reported by Brahmi)
Gitea runner registration token	Brahmi	Not stored anywhere. It is shown in Gitea's Site Administration (Actions, Runners), was used once on October 19, 2026 through the clipboard and an SSH pipe, and the server file that held it was emptied after registration. If lost or leaked, reset it on the same page	Lets whoever holds it register a runner for this Gitea. Rotation: on demand. MFA: not applicable
Jenkins accounts `brahmi` and `sakshi`	Brahmi, Sakshi	Brahmi's: saved in her password manager when the setup wizard created the account. Sakshi's: a temporary password was handed over privately (not in the repository or this document) and she changes it at first sign-in. The one-time unlock password was read on the server once and never copied	Brahmi: Overall Administer; Sakshi: Overall Read, Job Build and Read, View Read. Sign-up is off. MFA: none (Jenkins has no second factor configured)
Gitea user `jenkins-bot` and its token	Brahmi	The account password is in Brahmi's password manager. The access token (narrow scopes) was created once and is stored only in the Jenkins credential `gitea-jenkins-bot`; it is deleted after grading	Write on `twintrack` only; not an administrator. MFA: none
Jenkins webhook secret	none	The webhook `http://jenkins:8080/gitea-webhook/post` has no secret; it only travels over the private Docker network	not applicable

Service / account	**Owner**	**Vault reference or non-revealing hint**	**Privilege / MFA**
Team address list (`TEAM_IPS`)	Brahmi	Only in the server's untracked `.env` (mode 600; a backup copy `.env.bak-before-campus-ip` is on the server too); not an authenticator but kept out of the repository and this PDF. Since November 11, 2026 it holds two single addresses: the administrator's home address and the campus address	Decides who may open Jenkins and Grafana beyond their login pages. The campus address is shared by other people on that network. Rotation: when an address changes; remove the campus address at teardown
Grafana administrator `admin`	Brahmi	The first password was a random value generated on the server at the first start (file in a mode-700 folder, never printed, not in Git), so the default `admin`/`admin` was never valid. Brahmi replaced it with a password she typed at a prompt on the server (not in the shell history) and keeps in her password manager; never typed into chat, the repository, or this document	Grafana server administrator. MFA: none (not configured in Grafana)
Grafana user `sakshi`	Sakshi	A temporary password was handed over privately (not in the repository or this document); she changed it at her first sign-in on campus (reported by her on November 11, 2026) and keeps the new one herself	Role Editor in the main organization; not a Grafana administrator. Sign-up and anonymous access are off. MFA: none

Service / account	Owner	Vault reference or non-revealing hint	Privilege / MFA
Discord webhook URL of the contact point `team-alerts`	Brahmi	Created once in the settings of the team's private Discord channel and pasted only into Grafana, where it is stored in the volume `monitoring_grafana_data`; never in Git, chat, a screenshot, or this document. If it leaks, delete the webhook in Discord and create a new one	Lets whoever holds it post messages into the team's alert channel. Rotation: on demand. MFA: not applicable
Discord server "Group 1's server"	Brahmi	The account password is kept by Brahmi (not in the repository or this document); Sakshi was invited and has joined as a member (reported by her on November 11, 2026)	Owner: Brahmi. MFA: not recorded here

Changes in this update (November 11, 2026, version 1.3): Hosts and Passwords tables: updated. Hosts: the instance type is now `t3.small`; SSH (22) is allowed from two single addresses (home and campus); the new host `metrics.ssw590-g1.tech` with its four monitoring containers and private ports; the DNS record `metrics`; the monitoring notes of the Gitea and Jenkins rows. Passwords (hints only): the Grafana administrator, the Grafana user `sakshi`, the Discord webhook URL of the contact point, the Discord server, and the extended team address list. No secret is recorded in either table.

Changes in the earlier update (November 2, 2026, version 1.2): Hosts and Passwords tables: updated. Hosts: the Jenkins service `ci.ssw590-g1.tech` and the new DNS record `ci`. Passwords (hints only): the Jenkins accounts, the `jenkins-bot` user and token, the webhook (no secret), and the team address list. No secret is recorded in either table.

Changes in an earlier update (October 15, 2026): both tables were created from placeholders. They record the Gitea host, the domain, the earlier assignment's host, the two Gitea users, the two SSH keys, and the AWS, Name.com, and GitHub accounts. The hints and the MFA status were supplied by the account owners on October 15, 2026; MFA status is as reported and was not independently checked.

Chapter 3

Startup Foundation with Gitea

3.1 Overview and Status

This chapter answers the course manual's Chapter 2 [1]: the startup's Gitea work hub with a student domain, HTTPS, minimal public exposure, and persistent data. The eight numbered subsections below match the eight rubric rows (100 points) in the same order. Optional backup and restore evidence is a separate extra-credit subsection and is not part of the eight; it is optional, and the group did not attempt it.

Provider status

The assignment specifies a DigitalOcean Droplet. Group 1 asked the instructor whether AWS could be used instead. On October 12, 2026 (8:36 pm, Canvas message "Gitea Assignment — Question on Cloud Provider (Group 1)") the instructor replied that any hosting service is allowed, including AWS or the group's own servers, **as long as they are available online and can be used by the TA and the instructor to test the assignments.** Group 1 therefore uses AWS. The deployment must stay reachable and testable by the TA and instructor until grading is complete.

Table 3.1: Rubric rows, owners, and status

Subsection	Points	Owner	Status
Student Pack domain and DNS	10	BD	**PASS**
Pinned Gitea deployment	10	BD	**PASS**
Database and persistence	10	SD	**PASS**
HTTPS and certificate renewal	15	BD	**PASS**
Firewall and external exposure	15	BD	**PASS**
Startup team workflow	20	SD	**PASS**

Subsection	Points	Owner	Status
Architecture and operation	5	SD	**PASS**
Manual quality and controls	15	SD	**PASS**
Extra credit: backup and restore (optional)	up to 20	none	Not attempted

3.2 Required Subsections

3.2.1 Student Pack domain and DNS (10 points)

Owner: BD. Domain registration: **PASS** (order confirmed, $0.00). Stored card removed: **PASS**. DNS record, public-resolver lookup, and HTTPS link: **PASS**. A teammate's separate-network lookup was not done (see below).

Offer and eligibility. The domain comes from the GitHub Student Developer Pack, which lists domain offers from Namecheap, Name.com, and .TECH Domains [6]. The group's Student Pack is approved on Brahmi Dalvi's GitHub account. The Namecheap offer was started first but its GitHub authorization page requested the `repo` scope (read and write access to all public and private repositories), which the group judged too broad for a domain claim, so it was cancelled. The Name.com Education Pack authorization requested "public data only" and was accepted.

Domain and costs.

Table 3.2: Domain registration facts

Item	Value
Domain	`ssw590-g1.tech`, registered at Name.com, order #28974415. The confirmation email arrived Thursday, October 15, 2026 at 00:36 (mailbox local time) and gives the order date as October 14, 2026; the one-day difference is consistent with Name.com stamping the date in a different time zone, which we did not confirm
Planned Gitea hostname	`git.ssw590-g1.tech` (A record created, see below)
First-year cost	$0.00 (cart total $0.00; the regular price shown struck through in the cart was $89.99)
Renewal cost	$89.99 ("Renew for $89.99" on the domain page)
Expiration	October 15, 2027
Automatic renewal	OFF (turned off after registration)

Item	Value
Extras declined	Domain Safe Pro ($7.99/year); the free Domain Safe plan (domain lock, WHOIS privacy, free SSL certificate) is included

The order summary after checkout also shows a struck-through $7.99 next to the domain; the group did not investigate that figure and relies on the $0.00 total and the $89.99 renewal price shown on the domain page. Figure 3.1 shows the cart before payment: the domain is free and the paid Domain Safe Pro is declined, so the total is $0.00. Figure 3.2 shows the completed order, Figure 3.3 shows the domain page right after registration (automatic renewal still on, renewal price $89.99), and Figure 3.4 shows the same page after automatic renewal was turned off.

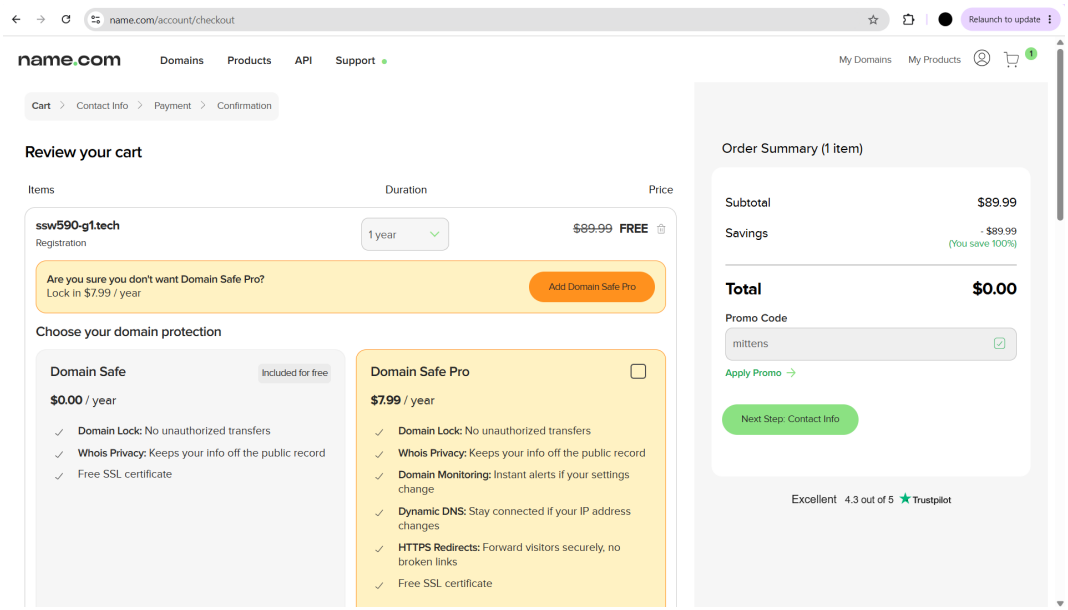


Figure 3.1: Name.com cart before payment, October 15, 2026: `ssw590-g1.tech` free for one year through the partner offer, Domain Safe Pro ($7.99) declined, total $0.00. This is the cart, not a receipt.

Renewal tracking. The group does not plan to renew: the domain is needed only through grading, and the TA and instructor must be able to reach the service until then. Expiry is October 15, 2027. Brahmi Dalvi owns this date. It is recorded in the Hosts table (Table 2.1), and issue #6 in the product repository, assigned to her, tracks adding a calendar reminder; that issue was still open on October 15, 2026, so no calendar event is claimed here.

DNS record. In Name.com's DNS manager the group created one record, `A git.ssw590-g1.tech 52.73.254.36` with TTL 300 (Figure 3.5). The address is the Elastic IP of the EC2 instance described in the deployment subsection. No AAAA record

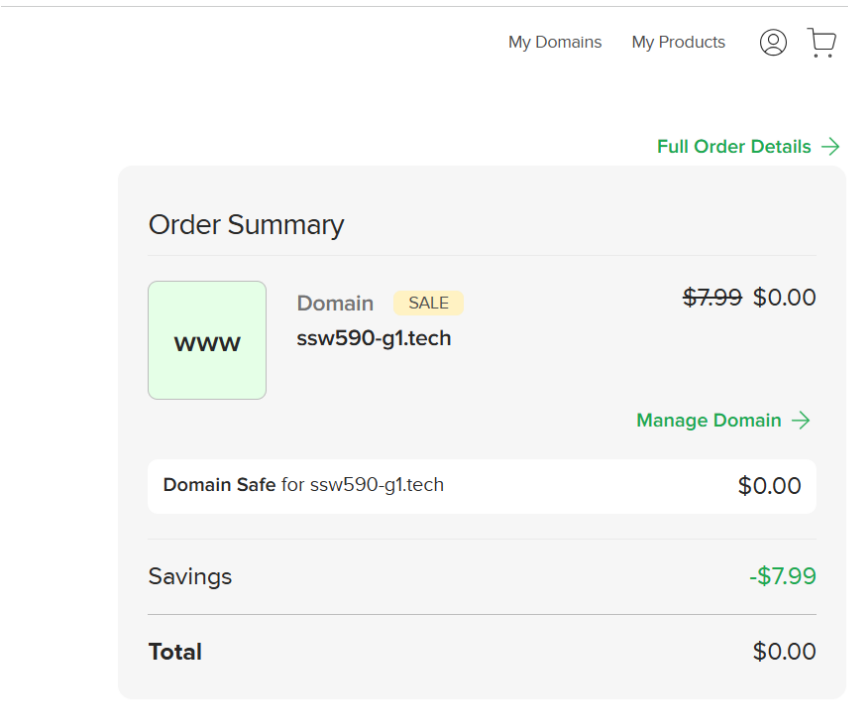


Figure 3.2: Name.com order summary, October 15, 2026: domain and free Domain Safe, total $0.00.

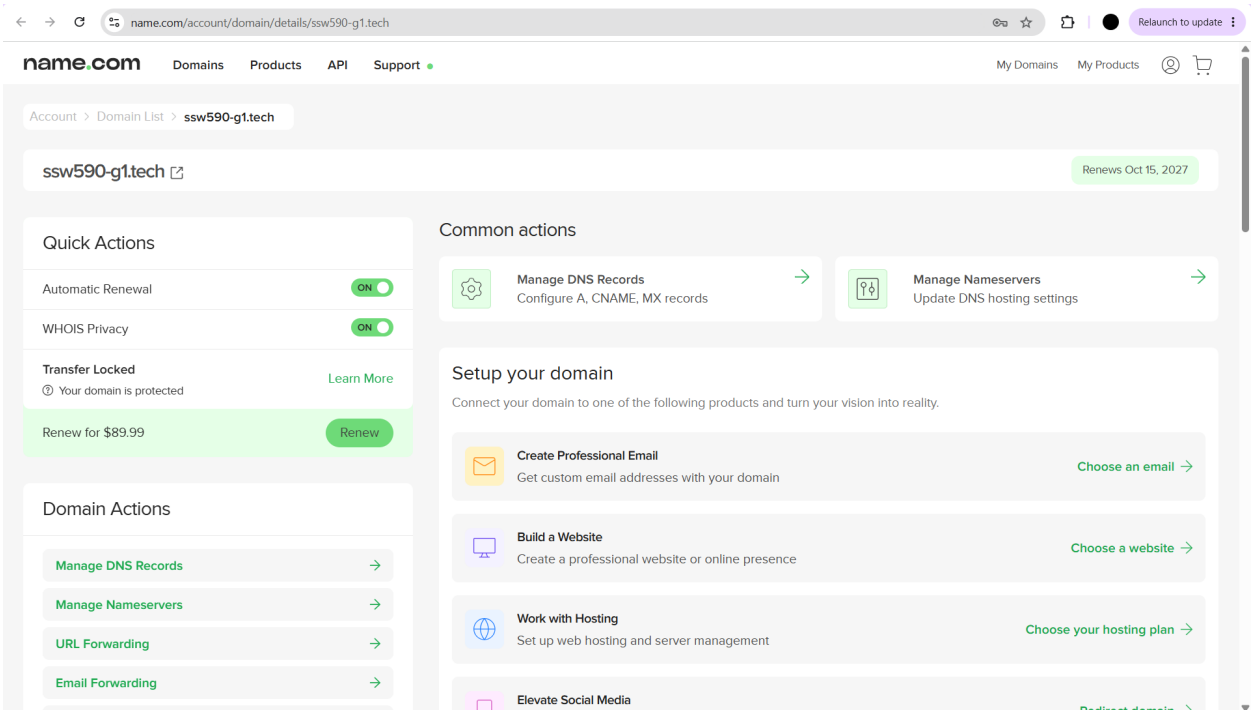


Figure 3.3: Domain page right after registration, October 15, 2026: “Renews Oct 15, 2027”, automatic renewal ON, WHOIS privacy ON, transfer locked, “Renew for $89.99”.

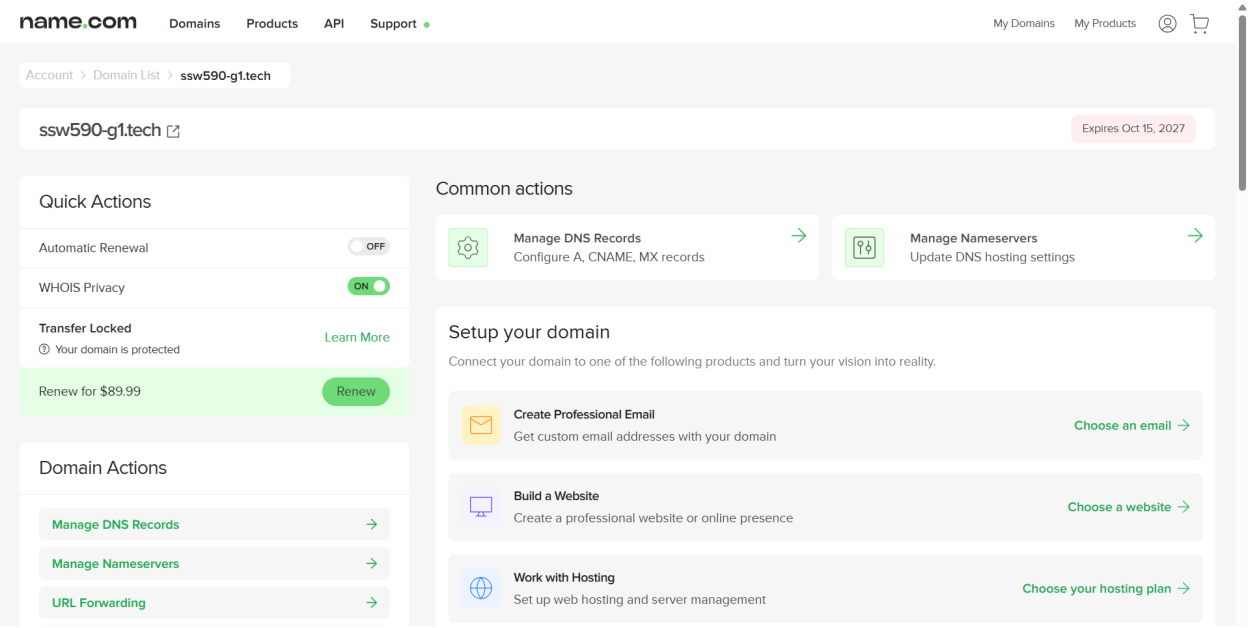


Figure 3.4: Domain page, October 15, 2026: expires October 15, 2027, automatic renewal OFF, renewal price $89.99.

was created, because the deployment is IPv4 only. The domain uses Name.com's own nameservers (`ns1bqx`, `ns2kry`, `ns3jmt`, `ns4ksy .name.com`).

Lookup result. At 17:30:34 UTC on October 15, 2026 the name resolved to the expected address through three public resolvers (expected: `52.73.254.36`; actual: the same).

Resolver	Result for `git.ssw590-g1.tech`	TTL
8.8.8.8 (Google)	A 52.73.254.36	300
1.1.1.1 (Cloudflare)	A 52.73.254.36	300
9.9.9.9 (Quad9)	A 52.73.254.36	300

These lookups were run from Brahmi Dalvi's computer through public resolvers; they were not run by a teammate on a separate network, so the independent-lookup wording of the assignment is met only in that sense. Listing 1 is the real output. A teammate has not repeated the lookup from a separate network; the three public resolvers above are the independent check. The Gitea site is live at the hostname: https://git.ssw590-g1.tech.

```
Real command output, Oct 15, 2026, run from Brahmi's machine (Windows PowerShell) by Claude Code on Brahmi's
↪  go-ahead.
Time of the DNS lookups: 2026-10-15 17:30:34 UTC. The admin's own public IP does not appear in this file.
These lookups were made from the admin's machine through three PUBLIC resolvers. They were not run by a
↪  teammate
on a separate network (the manual's wording for "independent lookup"); a teammate lookup can still be added.

=== Authoritative nameservers for ssw590-g1.tech (queried via 8.8.8.8) ===
ns4ksy.name.com
```

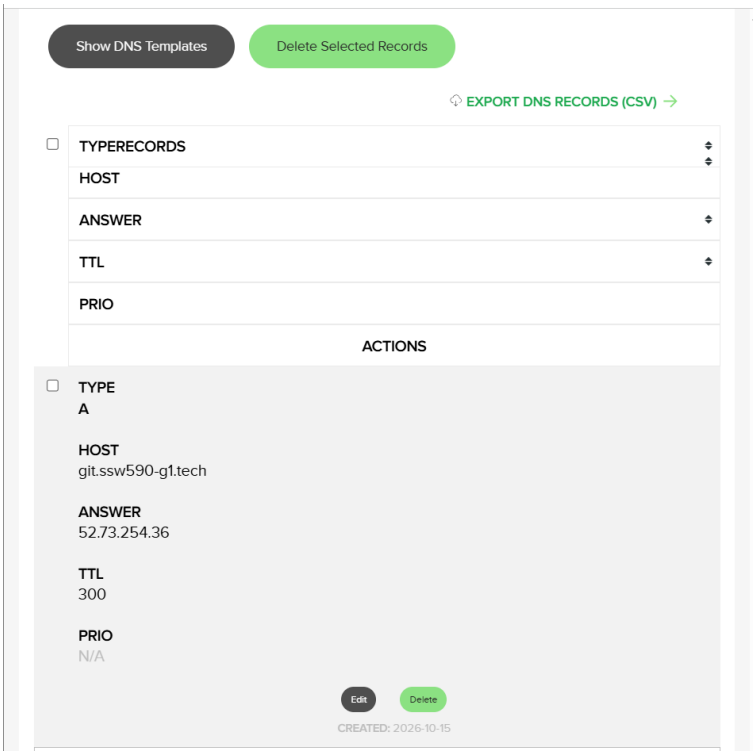


Figure 3.5: Name.com DNS records for `ssw590-g1.tech`, October 15, 2026: one A record for `git`.

```
ns2kry.name.com
ns1bqx.name.com
ns3jmt.name.com

=== A record lookups for git.ssw590-g1.tech ===
via 8.8.8.8 (Google):     git.ssw590-g1.tech  A  TTL 300  52.73.254.36
via 1.1.1.1 (Cloudflare): git.ssw590-g1.tech  A  TTL 300  52.73.254.36
via 9.9.9.9 (Quad9):      git.ssw590-g1.tech  A  TTL 300  52.73.254.36
AAAA via 8.8.8.8: no record returned (expected: IPv4 only by design)

=== SSH host-key verification before the first connection ===
Host key printed in the EC2 console log (ed25519):
↪  AAAAC3NzaC1lZDI1NTE5AAAAIP7bthP0bP6lOQ24+z2zzul/kB+xh4COxobDdf2gLDgY
Host key presented by 52.73.254.36 via ssh-keyscan: same key -> MATCH
(Connected with StrictHostKeyChecking=yes against a known_hosts file holding that verified key; key auth only,
↪  BatchMode.)

=== Read-only SSH check: ssh ec2-user@git.ssw590-g1.tech (connected by hostname) ===
ec2-user
ip-172-31-29-240.ec2.internal
PRETTY_NAME="Amazon Linux 2023.12.20261014"
6.18.54-121.166.amzn2023.x86_64
Docker version 25.0.14, build 0bab007
Docker Compose version v5.6.0
git version 2.50.1
active                     (systemctl is-active docker)
               total        used        free      shared  buff/cache   available
Mem:             913         259          77           0         575         492
NAME      TYPE SIZE USED PRIO
/swapfile file   2G 352K   -2
/dev/nvme0n1p1   20G  4.1G   16G  21% /
ec2-user adm wheel systemd-journal docker
exit code: 0

Notes:
- Proves: the A record is live on public resolvers, SSH by hostname works from the admin IP, the instance
↪  matches the planned setup.
- Does NOT prove: external exposure (ports 80/443 closed/open from other networks), HTTPS, or anything about
↪  Gitea (not deployed yet).
```

Listing 1: DNS lookups through three public resolvers, SSH host-key check, and a read-only SSH check of the server, October 15, 2026 (real output; the administrator's address is not part of it).

Payment safeguards. Name.com required a card even for the $0.00 order. To avoid an unintended renewal charge, the group turned automatic renewal off and deleted the stored payment profile; the billing page then reads "No saved payment profiles" (Figure 3.6). The confirmation email shows order #28974415 with an order total of $0.00 (Figure 3.7). The group has not yet compared a card statement; because the order total was $0.00 and the card was removed, no charge is expected.

3.2.2 Pinned Gitea deployment (10 points)

Owner: BD. Deployed and checked on October 15, 2026: **PASS**. Cost verification and the repository link are still open (see the end of this subsection).

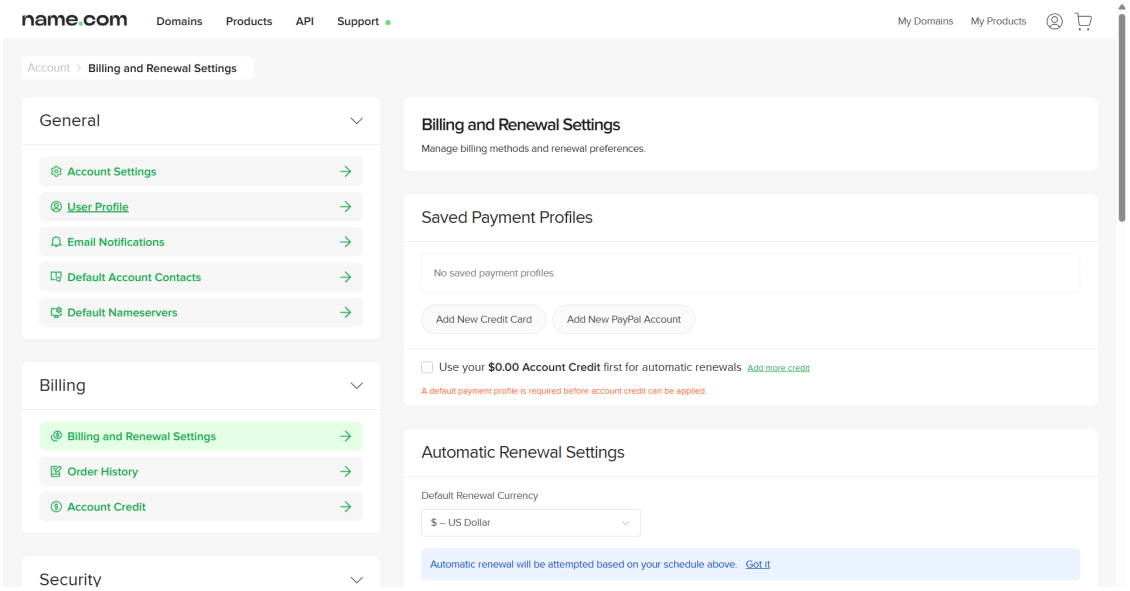


Figure 3.6: Name.com billing page, October 15, 2026: "No saved payment profiles" after the card was deleted.

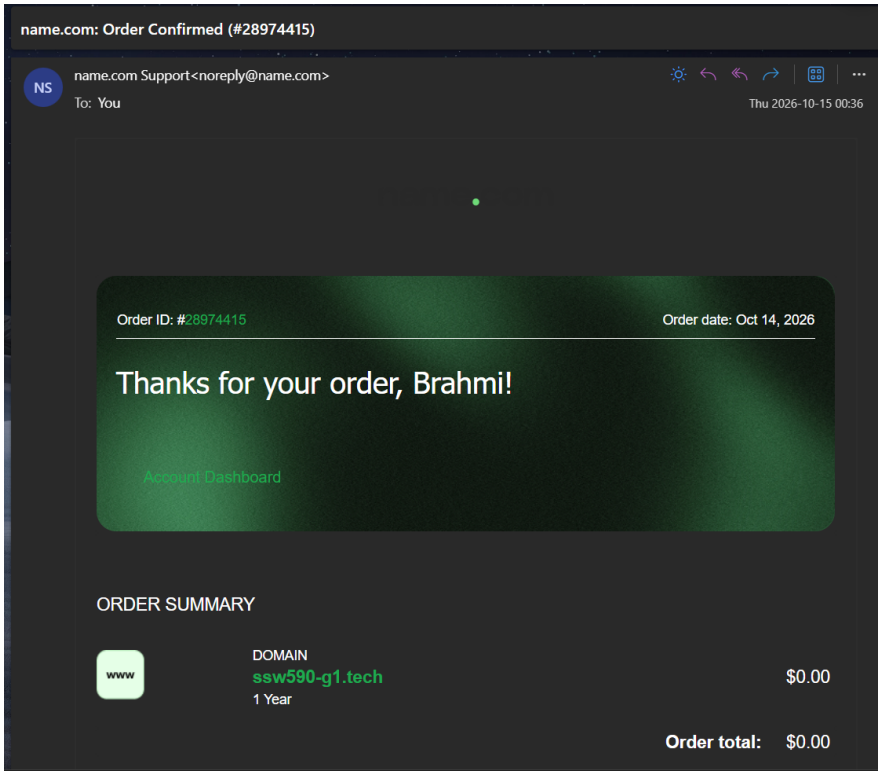


Figure 3.7: Name.com order confirmation email, received Thursday October 15, 2026 at 00:36 (mailbox time): order 28974415, `ssw590-g1.tech`, one year, order total $0.00. The email gives the order date as October 14, 2026.

Provider and versions. Gitea runs on one AWS EC2 instance instead of a DigitalOcean Droplet; the instructor approved this substitution (see the provider note at the start of this chapter). Table 3.3 records what was actually created. The instance was created with the AWS command line interface, not the web console, so every setting is repeatable.

Table 3.3: Deployed components and versions (October 15, 2026)

Item	Value
Cloud and region	AWS EC2, `us-east-1`, availability zone `us-east-1c`
Instance	`i-0c68ec1bc1e16a414`, type `t3.micro` (2 vCPU, 1 GiB RAM), Name tag `ssw590-g1-gitea`
Operating system	Amazon Linux 2023 (`al2023-ami-2023.12.20261014.0-kernel-6.18-x86_64`), image `ami-0ed59cbb87dfa8dba`, kernel `6.18.54-121.166.amzn2023.x86_64`
Disk and memory	20 GB gp3 root volume (deleted with the instance); 2 GB swap file added at first boot
Instance metadata	IMDSv2 only (`HttpTokens=required`)
Public address	Elastic IP `52.73.254.36`, named `git.ssw590-g1.tech` in DNS
Docker Engine	25.0.14 (Amazon Linux repository)
Docker Compose	v5.6.0 plugin, installed from the project's GitHub release and verified against a pinned SHA-256 (`40343e21...e76a`)
git	2.50.1
Gitea image	`gitea/gitea:1.27.3` from Docker Hub, image ID `79814cfc668d`, 191 MB [3]
Proxy image	`caddy:2.11.7` from Docker Hub, image ID `f77f856a30f0`, 66.3 MB
SSH key pair	`ssw590-g1-gitea-key` (ed25519); the private key stays on the administrator's computer, outside the repository

Both image tags were confirmed to exist on Docker Hub on October 12, 2026. `1.27.3` was the newest 1.27 release that day; a 28.1 release existed but was six days old and was not chosen. No image uses the `latest` tag.

```
AWS CLI output and EC2 console log excerpts, Oct 15, 2026 (UTC times from the console). Region us-east-1.
Commands were run by Claude Code on Brahmi's explicit go-ahead from Brahmi's machine (PowerShell, aws-cli
↪  2.37.9).
No credentials or private keys appear below. The admin's public IP is not part of this file.

=== Launch (aws ec2 run-instances, summarized) ===
image:         ami-0ed59cbb87dfa8dba  (Amazon Linux 2023, al2023-ami-2023.12.20261014.0-kernel-6.18-x86_64)
type:          t3.micro (2 vCPU, 1 GiB RAM)
disk:          20 GB gp3 root volume, DeleteOnTermination=true
key pair:      ssw590-g1-gitea-key (ed25519; private key kept on the admin's machine, outside the repo)
security grp:  sg-0415c75429368eb85 (ssw590-g1-gitea)
IMDS:          HttpTokens=required (IMDSv2 only)
user-data:     assignment-4/gitea/aws/ec2-user-data.sh (swap, Docker, git, pinned Compose)
tags:          Name=ssw590-g1-gitea, Project=ssw590-group1-assignment4
```

```
Instance: i-0c68ec1bc1e16a414
Elastic IP allocation: eipalloc-0dee5296b5382f367  (association eipassoc-0ea58688cb7153571)

=== Summary (describe-instances / describe-addresses) ===
i-0c68ec1bc1e16a414  running  t3.micro  ami-0ed59cbb87dfa8dba  us-east-1c  52.73.254.36  sg-0415c75429368eb85
↪  ssw590-g1-gitea-key  IMDS tokens: required
52.73.254.36  eipalloc-0dee5296b5382f367  i-0c68ec1bc1e16a414

=== Status checks (describe-instance-status) ===
running  instance-status: ok  system-status: ok

=== Boot script result (EC2 console output, snapshot 2026-10-15T17:22:54Z) ===
Oct 15 17:17:37 user-data: Complete!
Oct 15 17:17:40 user-data: Docker version 25.0.14, build 0bab007
Oct 15 17:17:40 user-data: Docker Compose version v5.6.0
Oct 15 17:17:40 user-data: git version 2.50.1
Oct 15 17:17:40 user-data: Mem:             913         254         121           0         537         498
Oct 15 17:17:40 user-data: Swap:           2047          16        2031
Oct 15 17:17:40 user-data: user-data finished OK
cloud-init v. 22.2.2 finished at Thu, 15 Oct 2026 17:17:40 +0000 (up 89.67 seconds)

Notes:
- Compose binary checksum: the script runs "sha256sum -c" with set -e; the script reached "finished OK", so
↪  the check passed.
  Expected SHA-256 40343e21ca777173e69cff5dbafeb37c6f81f3b0d57d9e597f036e95eb63e76a (GitHub asset digest and
  ↪  .sha256 file agreed).
- Gitea is NOT deployed yet; DNS is not created yet; no external exposure test has been run yet.
- Problems met: the AWS CLI on Windows crashed printing a non-ASCII character from the console log (charmap
↪  codec);
  an earlier poll looked like "still waiting" because that error was suppressed. Fixed by forcing UTF-8
  ↪  (PYTHONIOENCODING/PYTHONUTF8).
- The IAM user could not read the public SSM AMI parameter (AccessDenied); the AMI was found with
↪  describe-images instead.
```

Listing 2: The instance as created and the result of its first-boot script, October 15, 2026 (real AWS command line and console-log output).

How it is configured.

- **Restart behavior:** both containers use `restart: unless-stopped`, so they return after a reboot or a Docker restart.
- **Persistent data:** the named volume `gitea_gitea_data` holds Gitea's repositories and SQLite database at `/data`; `gitea_caddy_data` keeps Caddy's certificates and ACME account; `gitea_caddy_config` holds Caddy's saved configuration. The `Caddyfile` is mounted read-only from the project directory.
- **Proxy connection:** Caddy and Gitea share the private Compose network `gitea_web`; Caddy reaches Gitea at `gitea:3000` by service name. Gitea has no `ports:` entry, so it is not published on the host; only Caddy publishes 80 and 443.
- **Non-secret configuration (environment variables):** SQLite database at `/data/gitea/gitea.db`; `DOMAIN` and `ROOT_URL` set to the HTTPS hostname; Gitea's own SSH server disabled (Git over HTTPS only); web installer skipped (`INSTALL_LOCK`); self-registration disabled.
- **Secrets:** none appear in the Compose file, the `.env` file (hostname and image versions only), or this document. The first administrator password was generated once in the administrator's own terminal and stored in a password manager.

Compose and proxy configuration. The files below are exactly what runs on the server (their SHA-256 prefixes were compared after copying, Listing 3). They contain no secrets, so no redaction was needed. The first comment line of the Compose file still says it is a draft that has not been run; it was written before the first deployment and was left unchanged so that the file shown is exactly the one on the server. They are versioned in the team's course repository on GitHub, which was made public on October 15, 2026 so that graders can open it: https://github.com/brahmi9811/ssw-590-group1/tree/main/assignment-4/gitea. A second copy of the same files is published in the team's own Gitea at https://git.ssw590-g1.tech/group1-startup/infra. It was created on October 15, 2026 and is public; a signed-out check of Gitea's API listed the four files (`docker-compose.yml`, `Caddyfile`, `.env.example`, and `aws/ec2-user-data.sh`) plus a README, and their checksums match the files shown below.

```
# Gitea (SQLite) behind a Caddy TLS reverse proxy. Cloud-agnostic: works on a
# DigitalOcean Droplet or an AWS EC2 instance. DRAFT: not yet run on a server.
# Copy .env.example to .env (untracked) and set the values before `docker compose up -d`.
services:
  gitea:
    image: gitea/gitea:${GITEA_VERSION}          # pinned in .env
    container_name: gitea
    restart: unless-stopped
    environment:
      USER_UID: "1000"
      USER_GID: "1000"
      GITEA__database__DB_TYPE: sqlite3           # FINAL team decision (Oct 12, 2026):
      ↪  SQLite
      GITEA__database__PATH: /data/gitea/gitea.db # inside the persistent /data volume
      GITEA__server__DOMAIN: ${GITEA_DOMAIN}
      GITEA__server__ROOT_URL: https://${GITEA_DOMAIN}/
      GITEA__server__HTTP_PORT: "3000"
      GITEA__server__DISABLE_SSH: "true"          # HTTPS-only Git; no SSH listener
      GITEA__security__INSTALL_LOCK: "true"       # skip web installer; create admin via
      ↪  CLI
      GITEA__service__DISABLE_REGISTRATION: "true"
    volumes:
      - gitea_data:/data                          # repos, attachments, SQLite file
    networks:
      - web
    # No `ports:` on purpose: Gitea (3000) is reachable only from the proxy.
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost:3000/api/healthz"]
      interval: 30s
      timeout: 5s
      retries: 5

  caddy:
    image: caddy:${CADDY_VERSION}                 # pinned in .env
    container_name: caddy
    restart: unless-stopped
    depends_on:
      - gitea
```

```
    ports:
      - "80:80"     # HTTP-to-HTTPS redirect and ACME HTTP-01 validation only
      - "443:443"
    environment:
      GITEA_DOMAIN: ${GITEA_DOMAIN}
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data                          # certificates + ACME account: persist!
      - caddy_config:/config
    networks:
      - web

networks:
  web:

volumes:
  gitea_data:
  caddy_data:
  caddy_config:
```

```
# Caddy obtains and renews a Let's Encrypt certificate for this hostname and
# redirects HTTP to HTTPS automatically. Gitea is reached by Compose service name.
{$GITEA_DOMAIN} {
        reverse_proxy gitea:3000
}
```

Steps another team could repeat. These are the commands as they were run (abridged to the settings that matter). Placeholders in angle brackets were filled with the values in Table 3.3.

```
# 1. Firewall: security group with 443 and 80 open, 22 limited to the admin /32
aws ec2 create-security-group --group-name ssw590-g1-gitea \
  --description "SSW590 Group1 Gitea: 443/80 public, 22 admin only" --vpc-id <default
  ↪  VPC>
aws ec2 authorize-security-group-ingress --group-id <sg> --ip-permissions <443, 80, 22
↪  rules>
# 2. SSH key pair; the private key is saved to a file outside the repository
aws ec2 create-key-pair --key-name ssw590-g1-gitea-key --key-type ed25519 \
  --query KeyMaterial --output text
# 3. Instance with the versioned first-boot script (swap, Docker, git, pinned Compose)
aws ec2 run-instances --image-id ami-0ed59cbb87dfa8dba --instance-type t3.micro \
  --key-name ssw590-g1-gitea-key --security-group-ids <sg> \
  --block-device-mappings DeviceName=/dev/xvda,Ebs={VolumeSize=20,VolumeType=gp3} \
  --metadata-options HttpTokens=required \
  --user-data file://assignment-4/gitea/aws/ec2-user-data.sh
# 4. Fixed public address
aws ec2 allocate-address --domain vpc
aws ec2 associate-address --instance-id <instance> --allocation-id <allocation>
# 5. DNS (Name.com web console): A record  git  ->  <Elastic IP>, TTL 300
```

```
# 6. Copy the three non-secret files, create .env, and start the stack
scp docker-compose.yml Caddyfile .env.example ec2-user@git.ssw590-g1.tech:~/gitea/
sed 's/^GITEA_DOMAIN=.*/GITEA_DOMAIN=git.ssw590-g1.tech/' .env.example > .env
chmod 600 .env
docker compose config -q && docker compose pull && docker compose up -d
# 7. First administrator, run interactively by the administrator (password printed once)
docker compose exec -u git gitea gitea admin user create --admin \
  --username brahmi --email <email> --random-password
```

Expected versus actual.

Table 3.4: Deployment checks, October 15, 2026

Check	Expected	Actual	Evidence
Instance state	running, both status checks ok	running; instance and system checks ok	Listing 2
First-boot script	finishes without error, Compose checksum matches	"user-data finished OK" at 17:17:40 UTC	Listing 2
Installed tools	Docker, Compose, git, swap	Docker 25.0.14, Compose v5.6.0, git 2.50.1, 2 GB swap	Listing 2, Listing 1
Files on the server	identical to the repository copies	SHA-256 prefixes match for all three files	Listing 3
Containers	both up, Gitea healthy	`gitea` Up (healthy), `caddy` Up	Listing 3
Gitea settings	HTTPS `ROOT_URL`, SQLite	AppURL `https://git.ssw590-g1.tech/`; database ping `sqlite3`	Listing 3
Site and login	page loads, admin can sign in	HTTP 200 over HTTPS; signed in as the administrator	Listing 5, Figure 3.8

```
Real output from the server (ssh ec2-user@git.ssw590-g1.tech), Oct 15, 2026 (UTC). Run by Claude Code on
↪  Brahmi's go-ahead.
SANITIZED: log lines trimmed to the relevant ones; remote client IPs in the Caddy request/ACME lines are
↪  omitted. No secrets involved:
the server .env holds only non-secret values and no Gitea admin exists yet.

=== Files copied to ~/gitea on the server (scp), SHA-256 prefix identical to the repo copies ===
ed23b4136370d08a  docker-compose.yml
7705e125f84a2c82  Caddyfile
f42b0bd6bee72df6  .env.example
(source in the repo: assignment-4/gitea/)

=== ~/gitea/.env on the server (created from .env.example, chmod 600; no secrets) ===
GITEA_VERSION=1.27.3
CADDY_VERSION=2.11.7
```

```
GITEA_DOMAIN=git.ssw590-g1.tech

=== docker compose config -q  ->  "compose config OK" ===

=== docker compose pull (pinned images) ===
caddy:2.11.7        f77f856a30f0   66.3MB
gitea/gitea:1.27.3  79814cfc668d   191MB

=== docker compose up -d  (start: 2026-10-15 17:34:23 UTC) ===
Volume gitea_caddy_config Created, Network gitea_web Created
Container gitea Created/Started; Container caddy Created/Started
(volumes gitea_gitea_data, gitea_caddy_data, gitea_caddy_config; project directory name "gitea")

=== docker compose ps (about 40 s after start) ===
caddy  caddy:2.11.7          Up 40 seconds             0.0.0.0:80->80/tcp, [::]:80->80/tcp,
↪  0.0.0.0:443->443/tcp, [::]:443->443/tcp, 443/udp, 2019/tcp
gitea  gitea/gitea:1.27.3    Up 40 seconds (healthy)   22/tcp, 3000/tcp          (not published to the host)

=== Caddy log excerpts (JSON, trimmed) ===
acme_client: trying to solve challenge, identifier git.ssw590-g1.tech, challenge_type tls-alpn-01, ca
↪  https://acme-v02.api.letsencrypt.org/directory
tls: served key authentication certificate (tls-alpn-01) x5   (Let's Encrypt validation from several vantage
↪  points)
acme_client: authorization finalized, authz_status valid
acme_client: validations succeeded; finalizing order
acme_client: got renewal info (ACME Renewal Information), window_start 1797189600, window_end 1797345050
tls.obtain: certificate obtained successfully, issuer acme-v02.api.letsencrypt.org-directory
http.log.error: dial tcp 172.18.0.2:3000: connect: connection refused ... status 502
   -> one early 502: an internet scanner hit the site about a second before Gitea was listening. Transient;
   ↪  Gitea was up moments later.

=== Gitea log excerpts ===
cmd/web.go: Listen: http://0.0.0.0:3000
cmd/web.go: AppURL(ROOT_URL): https://git.ssw590-g1.tech/
HTTPRequest: GET / -> 200 OK (via the proxy)
HTTPRequest: GET /wp-json/ -> 404, GET /xmlrpc.php?rsd -> 404   (automated scanners probing the new site
↪  within seconds)
PING DATABASE sqlite3
HTTPRequest: GET /api/healthz -> 200 OK   (container healthcheck)

Notes:
- Certificate challenge used: TLS-ALPN-01 (port 443). Port 80 is still open for the HTTP-to-HTTPS redirect.
- Caddy's published ports show IPv6 bindings ([::]) on the host, but the security group has no IPv6 rules and
↪  no AAAA record exists, so IPv6 is not reachable.
- Port 2019/tcp in "ps" is Caddy's admin API inside the container; it is not published to the host.
- Gitea port 3000 and 22 are only exposed inside the Compose network.
- At the time of this capture (about 17:35 UTC) not yet done: admin account, persistence test, renewal test,
↪  external test from a second network. All were done later the same day (see the later listings).
```

Listing 3: Files copied to the server, image pull, `docker compose up -d`, container status, and the Caddy and Gitea logs, October 15, 2026 (real server output, trimmed, remote addresses removed).

Problems met and corrections.

- The AWS command line tool on Windows crashed while printing a non-ASCII character from the instance console log. An early check looked like "still waiting" because that error was hidden; the log was read again with UTF-8 forced and showed the script had finished.
- The IAM user was not allowed to read the public image-lookup parameter (access denied), so the Amazon Linux image was found with `describe-images` instead.

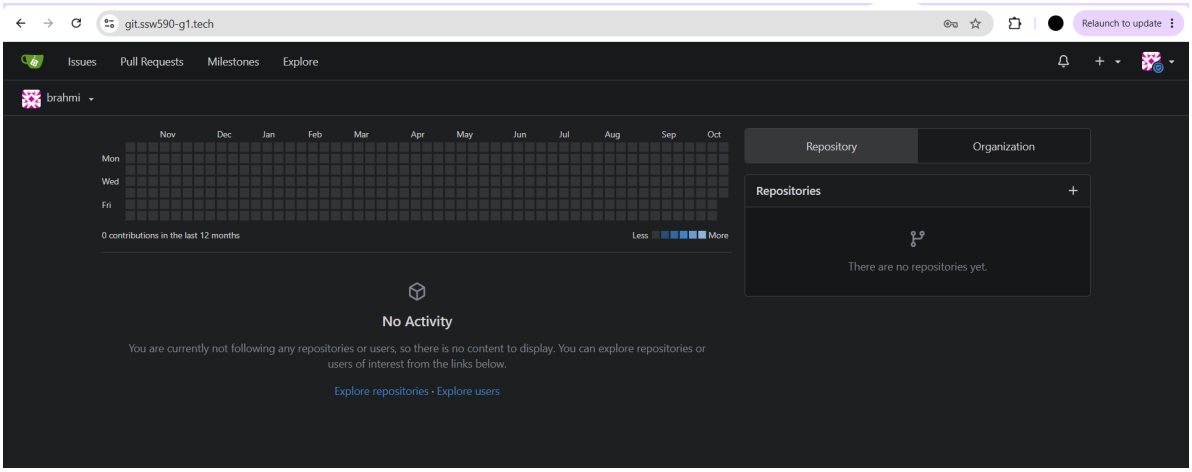


Figure 3.8: The deployed Gitea at `git.ssw590-g1.tech`, signed in as the administrator, October 15, 2026.

- Passing a multi-line script directly to `ssh` from Windows PowerShell 5.1 stripped quotes and failed with a shell syntax error; scripts are now copied to the server and run there.
- One 502 error appeared in the Caddy log about a second after start: an internet scanner reached the site before Gitea was listening. It did not recur.

Cost and cleanup. The running costs are the instance, the 20 GB volume, and the Elastic IP. The group has not confirmed the exact prices on AWS's own pricing pages. An estimate at on-demand list prices in `us-east-1`, for about 730 hours a month: the `t3.micro` instance at $0.0104 per hour is about $7.59; the public IPv4 address at $0.005 per hour is about $3.65 [7]; the 20 GB gp3 volume at $0.08 per GB-month is about $1.60 [8]. The total is **about $12.84 per month**. The instance rate comes from a third-party price calculator [9], and the gp3 rate from AWS blog and third-party guides; neither was confirmed on AWS's own pricing pages, so the figure is an estimate, not a bill. The group uses AWS credits and may qualify for free-tier hours (not checked), so the real charge may be lower. The account has a monthly cost budget of $15, created for the earlier assignment; its alert thresholds were not reviewed. The earlier assignment's instance, which keeps running until that assignment is graded, is a separate cost. The service must stay online until grading is finished because the instructor and TA test it. Teardown, after grading and only then: terminate the instance, release the Elastic IP, delete the security group and key pair, delete the DNS record, and let the domain expire (automatic renewal is already off, see the domain subsection).

3.2.3 Database and persistence (10 points)

Owner: SD. **Status:** persistence test on the cloud VM **PASS** (October 15, 2026, 18:29 UTC).

Decision: SQLite

Group 1 uses Gitea's built-in SQLite database [10]. This was a team decision recorded on October 12, 2026. Reasons:

- The startup has two members; Gitea's write load is a few pushes, issues, and reviews per day, well within SQLite's capability.
- No second container, database port, or database credential exists, so there is nothing extra to secure, patch, or expose.
- The whole service state (database file and repositories) lives in one volume, which keeps a future backup simple.

Limitation. SQLite serialises writes in a single file. A larger team with many concurrent pushes, or a deployment that runs several Gitea instances behind a load balancer, should move to PostgreSQL in its own container [1].

Settings

The database is configured with environment variables in the versioned Compose file (`gitea/docker-compose.yml`), which Gitea writes into `/data/gitea/conf/app.ini` at start-up [11]:

```yaml
services:
  gitea:
    environment:
      GITEA__database__DB_TYPE: sqlite3
      GITEA__database__PATH: /data/gitea/gitea.db
    volumes:
      - gitea_data:/data        # named volume, outside the container layer
    # no ports: Gitea is reached only by Caddy on the private network
volumes:
  gitea_data:
```

Table 3.5: Where Gitea's persistent data lives

Item	Location / value
Database type	`sqlite3` (file, no server)
Database file	`/data/gitea/gitea.db`
Git repositories	`/data/git/repositories/<owner>/<repo>.git`
Attachments, avatars, indexes	`/data/gitea/attachments`, `avatars`, `indexers`
Generated configuration	`/data/gitea/conf/app.ini` (contains generated secrets; never copied into this manual)
Docker volume	`gitea_gitea_data` (Compose project `gitea`) mounted at `/data`
Database network port	none: SQLite is a file, so there is no port to publish or firewall
Database credential	none: SQLite has no user or password

Because the volume is mounted from Docker's volume store, it is *outside* the container's disposable writable layer: removing or recreating the container does not touch it. Only an explicit `docker compose down -v` or `docker volume rm` deletes it, and both are forbidden in the team's runbook (Section 3.2.7).

Persistence test procedure

The test is scripted in `assignment-4/sd/persistence-test.sh` so it can be repeated exactly. Run on the Docker host from `assignment-4/gitea`, it:

1. creates repository `persistence-check` (with an initial README commit) and issue #1 through the Gitea API, unless they already exist;
2. records the container ID and creation time, the volume mount, the database file, and the repository, issue, and `main` commit IDs;
3. replaces *only* the Gitea container, keeping volumes:

```
docker compose up -d --force-recreate gitea
```

4. waits for the health check, then records the same items again.

The password is read from the terminal and never printed.

Evidence

Local rehearsal, October 12, 2026: not graded evidence

The procedure was rehearsed on SD's workstation (Docker Desktop, Gitea 1.27.3, Caddy 2.11.7, `https://localhost`). The container ID changed from `fdca1b83aa58` to `c1e3fb1ba256`; repository id 1, issue #1 (open) and commit `19b82109db90` were unchanged afterwards. This only shows the procedure works; the graded result is the run on the cloud VM below.

Table 3.6: Persistence test on the cloud VM

Check	Expected	Actual (VM)
Container ID before / after	different	`858a02ae8412` (created 17:34:23) → `834924d4bc0a` (created 18:29:21) **PASS**
Volume mounted at `/data`	`gitea_gitea_data` both times	`gitea_gitea_data` before and after **PASS**
Database file	same file, still present	`/data/gitea/gitea.db`, 2 187 264 bytes before and after **PASS**
Repository `persistence-check`	same id before and after	id 1, `sakshi/persistence-check`, created 18:29:20 both times **PASS**
Issue #1	present, same id and state	id 1, open, same title **PASS**

Check	Expected	Actual (VM)
`main` commit	same commit ID	`4f15f700003e` both times **PASS**

The test was run by SD on October 15, 2026 from her workstation over SSH, as `ec2-user` on the EC2 instance, from the Compose directory `~/gitea`:

```
ssh -t -i ~/.ssh/ssw590-g1-sakshi ec2-user@git.ssw590-g1.tech \
  'cd ~/gitea && GITEA_URL=https://git.ssw590-g1.tech GITEA_USER=sakshi \
   bash ~/persistence-test.sh | tee ~/persistence.log'
```

Listing 4 is the complete, unedited log. It contains no credential: the password was typed at a hidden prompt. The `host:` line is the instance's AWS-internal name, which is not reachable from the internet.

```
=== Gitea persistence test, started 2026-10-15T18:29:19Z ===
host: ip-172-31-29-240.ec2.internal  url: https://git.ssw590-g1.tech  user: sakshi
$ POST https://git.ssw590-g1.tech/api/v1/user/repos  (name=persistence-check, auto_init=true)
$ POST https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check/issues
--- BEFORE container replacement (2026-10-15T18:29:20Z) ---
$ docker inspect gitea --format {{.Id}} created={{.Created}}
858a02ae8412 created=2026-10-15T17:34:23.469902499Z
$ docker inspect gitea --format (mounts: type name -> destination)
volume gitea_gitea_data -> /data

$ docker compose exec gitea ls -l /data/gitea/gitea.db
-rw-r--r--    1 git      git        2187264 Oct 15 18:29 /data/gitea/gitea.db
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check
repo id=1 full_name=sakshi/persistence-check created=2026-10-15T18:29:20Z
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check/issues/1
issue #1 id=1 state=open title='Persistence check: this issue must survive container replacement'
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check/branches/main
branch main commit=4f15f700003e
--- replacing container (2026-10-15T18:29:21Z) ---
$ docker compose up -d --force-recreate gitea
 Container gitea Recreate
 Container gitea Recreated
 Container gitea Starting
 Container gitea Started
--- AFTER container replacement (2026-10-15T18:29:52Z) ---
$ docker inspect gitea --format {{.Id}} created={{.Created}}
834924d4bc0a created=2026-10-15T18:29:21.291713393Z
$ docker inspect gitea --format (mounts: type name -> destination)
volume gitea_gitea_data -> /data

$ docker compose exec gitea ls -l /data/gitea/gitea.db
-rw-r--r--    1 git      git        2187264 Oct 15 18:29 /data/gitea/gitea.db
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check
repo id=1 full_name=sakshi/persistence-check created=2026-10-15T18:29:20Z
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check/issues/1
issue #1 id=1 state=open title='Persistence check: this issue must survive container replacement'
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check/branches/main
branch main commit=4f15f700003e
=== finished 2026-10-15T18:29:52Z ===
```

Listing 4: Persistence test on the EC2 instance, October 15, 2026, 18:29:19–18:29:52 UTC (real, unedited output of `persistence-test.sh`).

Figure 3.9 and Figure 3.10 show the repository and the issue in the browser on October 15, 2026, a few minutes after the test finished (Gitea displays relative times such as "2 minutes ago"; the absolute times are in Listing 4). The commit shown, `4f15f70000`, is the `main` commit recorded before and after the replacement.

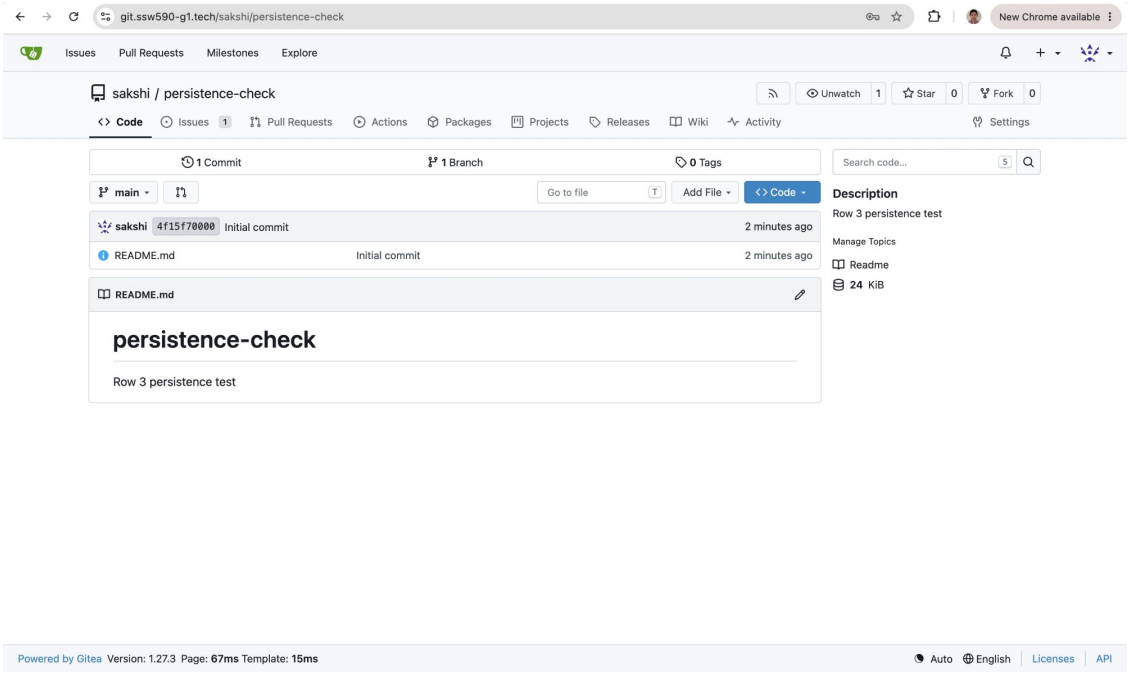


Figure 3.9: Repository `sakshi/persistence-check` after the container replacement, October 15, 2026: one commit, `4f15f70000`.

Conclusion

The Gitea container was replaced (new container ID and creation time) while the named volume was kept, and the repository, its `main` commit, and issue #1 were unchanged afterwards. They persisted because SQLite and the Git repositories live in the `gitea_gitea_data` volume, outside the container's disposable layer; the new container mounted the same volume. Gitea was unavailable for about 30 seconds during the replacement. This test shows that data survives replacement of the container. It is *not* a backup or restore test: no copy of the data exists outside the volume, so losing the volume or the VM disk would still lose the data (see Section 3.2.7).

3.2.4 HTTPS and certificate renewal (15 points)

Owner: BD. Certificate issued, redirect and TLS checked on October 15, 2026: **PASS**. Second-option research is partly done (open questions remain). The staging dry run passed as an issuance test; an actual renewal of the production certificate has not yet happened, so it is **PENDING**.

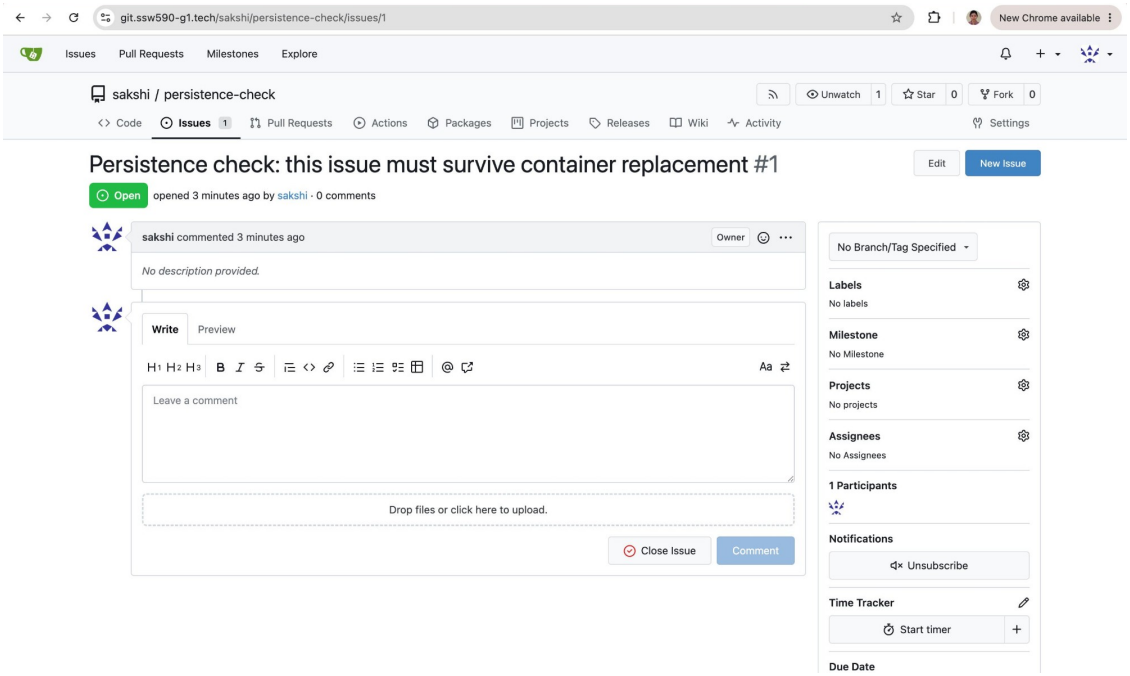


Figure 3.10: Issue #1 still open after the container replacement, October 15, 2026.

Certificate options compared. Two ways to get a certificate for `git.ssw590-g1.tech` were considered.

Table 3.7: Certificate options

Aspect	**A. Let's Encrypt through Caddy**	**B. Free certificate offered with the Student Pack or domain bundle**
Cost	Free [4]	Included free with Name.com's Domain Safe tier (visible in the cart); the Student Pack page also lists a free one-year Namecheap certificate [12, 6]
Issuance	Automatic: Caddy proves control of the name to Let's Encrypt with an ACME challenge (TLS-ALPN-01 was used) [13]	A domain-validated certificate from Name.com's free-certificate program. The support pages we could read do not say how it is requested, or whether it can be installed on a server hosted elsewhere [12]

Aspect	A. Let's Encrypt through Caddy	B. Free certificate offered with the Student Pack or domain bundle
Renewal	Automatic inside Caddy; certificates live in a persistent volume	Manual: in the Name.com account (My Products, then Renew; the checkout shows $0), and only within 90 days of expiry [14]. A missed window removes the renewal option
Fit for this project	Fits Docker Compose: no extra tool, no manual step, nothing to remember in a year	Not confirmed to work on our AWS-hosted server; adds a yearly manual step and an account dependency. For the Namecheap certificate, one student reports being asked to pay after claiming the free domain [15]

Option B's facts come from Name.com's support pages as summarized by a search engine, because the article bodies could not be retrieved in full; they have not been confirmed with the vendors. Whether the Domain Safe certificate can be installed on an external server, and whether its private key can be exported, is an open question that the group did not pursue, because option A already met the need.

Choice: option A. It is free, fully automatic, needs no extra software because Caddy is already the reverse proxy, and removes a manual renewal step. This rests on the documentation and on what the deployment actually did (a certificate was issued within seconds of the first start, see below). Option B was researched only through public support pages, so its limits are stated as open questions above, not as proven facts.

Where TLS terminates. TLS ends at Caddy, which is the only container that publishes ports 80 and 443. Caddy then forwards the request over plain HTTP on the private Compose network `gitea_web` to `gitea:3000`. That traffic never leaves the instance. Gitea's `ROOT_URL` is `https://git.ssw590-g1.tech/`, so the links and clone URLs it generates use HTTPS.

Certificate issued. On the first `docker compose up -d` (17:34 UTC) Caddy requested a certificate, completed the TLS-ALPN-01 challenge from several Let's Encrypt vantage points, and logged "certificate obtained successfully" within seconds of the stack starting (Listing 3). The certificate served at 17:35 UTC was read directly from the live connection (Listing 5).

Table 3.8: Certificate served by `git.ssw590-g1.tech`, October 15, 2026

Property	**Value**
Subject and name	`CN=git.ssw590-g1.tech`; subject alternative name `git.ssw590-g1.tech` only
Issuer	Let's Encrypt, intermediate `YE2`
Valid from / to	2026-10-15 16:35:56 UTC to 2027-01-13 16:35:55 UTC (90 days)
Key type and protocol	ECC key; TLS 1.3
Chain	`YE2`, `Root YE`, `ISRG Root X2`; Windows built the chain successfully (valid)
Client check	`curl` accepted the certificate without the insecure flag and received HTTP 200

```
Real output, Oct 15, 2026, 17:35 UTC, from Brahmi's Windows machine (outside the server) using curl.exe,
↪  Test-NetConnection, and .NET SslStream.
Run by Claude Code on Brahmi's go-ahead. The admin's own public IP is not in this file.
Limitation: this machine's IP is the ADMIN IP allowed to use SSH, so it cannot show that SSH is blocked for
↪  others.
A test from a second network (for example a phone hotspot) is still needed for the firewall row.

=== HTTP (port 80), curl -I http://git.ssw590-g1.tech/ ===
HTTP/1.1 308 Permanent Redirect
Location: https://git.ssw590-g1.tech/
Server: Caddy
Date: Thu, 15 Oct 2026 17:35:19 GMT

=== HTTPS (port 443), curl -I https://git.ssw590-g1.tech/ (no -k flag; certificate accepted) ===
HTTP/1.1 200 OK
Alt-Svc: h3=":443"; ma=2592000

=== Served certificate (read with .NET SslStream) ===
TLS protocol  : Tls13
Subject       : CN=git.ssw590-g1.tech
Issuer        : CN=YE2, O=Let's Encrypt, C=US
Not before    : 2026-10-15 16:35:56 UTC
Not after     : 2027-01-13 16:35:55 UTC      (90-day certificate)
SANs          : DNS Name=git.ssw590-g1.tech
Key algorithm : ECC
Chain valid   : True
  chain: CN=git.ssw590-g1.tech
  chain: CN=YE2, O=Let's Encrypt, C=US
  chain: CN=Root YE, O=ISRG, C=US
  chain: CN=ISRG Root X2, O=Internet Security Research Group, C=US

=== TCP probes from this machine (Test-NetConnection) ===
port 443  : OPEN
port 80   : OPEN
port 3000 : closed/filtered   (Gitea backend, expected not reachable)
port 2019 : closed/filtered   (Caddy admin API, expected not reachable)
port 5432 : closed/filtered   (PostgreSQL is not used; expected not reachable)
port 22   : OPEN from this machine only because its IP is the approved admin IP

Notes:
- Expected vs actual: 443 open, 80 open (redirect only), 3000/2019/5432 closed -> as expected.
- "closed/filtered" does not distinguish closed from firewall-dropped; both mean "not reachable".
```

```
- Renewal: not tested at the time of this capture (17:35 UTC). Caddy logged ACME renewal information (renewal
↪  window starts about 2026-12-14). A staging dry run was done later the same day (see the renewal-test
↪  listing); the real renewal of this certificate is expected between December 13 and 15, 2026 and has not
↪  been observed.
```

Listing 5: HTTP redirect, HTTPS response, the served certificate, and port probes from the administrator's computer, October 15, 2026, 17:35 UTC (real output).

HTTP to HTTPS redirect. Requesting `http://git.ssw590-g1.tech/` returned `HTTP/1.1 308 Permanent Redirect` with `Location: https://git.ssw590-g1.tech/` from Caddy (Listing 5, 17:35 UTC). Port 80 therefore serves only this redirect and certificate validation; it exposes no application.

Renewal. Caddy renews its certificates automatically before they expire; Let's Encrypt certificates last 90 days, so the first renewal is due about two months after issue [4]. Caddy's log already recorded the renewal window reported by the certificate authority (ACME Renewal Information), and the certificate and account are stored in the persistent volume `gitea_caddy_data`, so a container replacement does not lose them.

Renewal test (staging dry run). Caddy has no renew-now or dry-run command that we know of, so the test used the idea behind a certbot dry run: run the same ACME process against Let's Encrypt's *staging* server, which issues untrusted test certificates and does not touch the production certificate (Listing 6, October 15, 2026, 19:03 to 19:08 UTC). Three attempts were needed, and all are reported.

1. **Staging CA for the real hostname: inconclusive.** Adding a global staging CA and reloading Caddy produced no new order. Caddy already held a valid certificate for the name, kept serving it (same thumbprint), and logged only that automatic TLS was enabled. This proved nothing about renewal, and the production configuration was restored at once.
2. **Wrong directive: rejected.** The first version of the test block used `ca` inside the ACME issuer, which Caddy rejected ("unrecognized ACME issuer property: ca") and kept its previous working configuration, so the live site was unaffected. The correct directive is `dir`.
3. **Throwaway hostname with a staging-only issuer: succeeded.** A second site block was added for `52-73-254-36.sslip.io` (a public wildcard-DNS name that resolves to the server's address), configured with the staging directory, and the production block was left unchanged. Within about five seconds of the reload Caddy created an ACME account, solved the TLS-ALPN-01 challenge, finalized the order, downloaded the chain, and logged "certificate obtained successfully" with the staging issuer. The server then presented that certificate for the test name (issuer "(STAGING) Artificial Amaranth YE1", not trusted by Windows, as expected) while the production certificate stayed identical (same thumbprint).

The production Caddyfile was then restored (its SHA-256 matches the repository copy), the test certificate and backup file were removed, and normal checks confirmed the redirect, an HTTP 200 response, and the original production certificate (Listing 6).

```
Real output from the server (ssh ec2-user@git.ssw590-g1.tech) and from Brahmi's computer, Oct 15, 2026, 19:03
↪  to 19:08 UTC.
Run by Claude Code on Brahmi's go-ahead ("do the renewal test, she is not working right now").
SANITIZED: remote addresses in Caddy log lines replaced by <masked>; long JSON lines trimmed.

PURPOSE: show that Caddy, with this setup, can complete the full ACME certificate process. Caddy has no "renew
↪  now" or "dry run" command that we know of.
Method: run the same ACME process against Let's Encrypt's STAGING server (the idea behind "certbot renew
↪  --dry-run").
IMPORTANT LIMIT: this is a certificate ISSUANCE test for a different, throwaway hostname. It is NOT a renewal
↪  of the production certificate.

=== 0. Baseline (19:03:58 UTC): production certificate before the test ===
Issuer CN=YE2, O=Let's Encrypt | NotBefore 2026-10-15 16:35:56 | NotAfter 2027-01-13 16:35:55
Thumbprint 62AC575CAC34B6E3D61F581D5DCFFB321EB7C4AF | chain trusted: True

=== 1. ATTEMPT 1: global staging CA for the real hostname (19:04:08 UTC) ===  RESULT: INCONCLUSIVE
Caddyfile: added a global block { acme_ca https://acme-staging-v02.api.letsencrypt.org/directory }, then
↪  "caddy reload".
After 30 s and again at 19:04:54: the only certificate-related log line was "enabling automatic TLS
↪  certificate management";
no staging certificate was requested, storage held only the production certificate directory,
and the served certificate was still the production one (same thumbprint 62AC575C...C4AF).
Reading: Caddy already held a valid certificate for the name, so a reload with a different CA did not start a
↪  new order.
Nothing about renewal was demonstrated. Production config restored at 19:05:28 UTC (Caddyfile sha256 prefix
↪  7705e125f84a2c82 = repo copy).

=== 2. ATTEMPT 2a: invalid syntax (19:05:52 UTC) ===
Test block used "ca <url>" inside "issuer acme { }".  caddy reload -> error:
  adapting config using caddyfile: parsing caddyfile tokens for 'tls': unrecognized ACME issuer property: ca
Caddy refused the reload and kept the previous working configuration, so the live site was not affected.
↪  Correct directive is "dir <url>".

=== 3. ATTEMPT 2b: throwaway hostname with a staging-only issuer (19:06:44 UTC) ===  RESULT: SUCCESS
Hostname: 52-73-254-36.sslip.io (public wildcard DNS service; resolves to the server's Elastic IP, checked via
↪  8.8.8.8).
Caddyfile = production block UNCHANGED + this block:
  52-73-254-36.sslip.io {
      tls { issuer acme { dir https://acme-staging-v02.api.letsencrypt.org/directory } }
      respond "renewal test" 200
  }
caddy validate -> "Valid configuration"; caddy reload -> no error.
Caddy log (ACME lines, trimmed):
  using ACME account  https://acme-staging-v02.api.letsencrypt.org/acme/acct/353835223
  trying to solve challenge  identifier 52-73-254-36.sslip.io  challenge_type tls-alpn-01  ca
  ↪  .../acme-staging-v02.api.letsencrypt.org/directory
  served key authentication certificate (tls-alpn-01) x5   (remote <masked>)
  authorization finalized  authz_status valid
  validations succeeded; finalizing order
  ↪  https://acme-staging-v02.api.letsencrypt.org/acme/order/353835223/51532536033
  got renewal info (ACME Renewal Information) x3   names 52-73-254-36.sslip.io  window_start 1795720004
  ↪  window_end 1795830526
  successfully downloaded available certificate chains  count 3
  tls.obtain: certificate obtained successfully  issuer acme-staging-v02.api.letsencrypt.org-directory
Time from reload (ts 1792091205) to "certificate obtained successfully" (ts 1792091210): about 5 seconds.
Storage after: .../acme-v02.api.letsencrypt.org-directory/git.ssw590-g1.tech  (production)  and
               .../acme-staging-v02.api.letsencrypt.org-directory/52-73-254-36.sslip.io  (test)

=== 4. Certificates served at 19:07:40 UTC (read with .NET SslStream, validation bypass ON for this inspection
↪  only) ===
```

```
SNI 52-73-254-36.sslip.io : Issuer CN=(STAGING) Artificial Amaranth YE1, O=Let's Encrypt | NotBefore
↪  2026-10-15 18:08:16 | NotAfter 2026-12-18 18:08:15
                           Thumbprint 44202E6BCB0E5097A10AAD60D95B695E1DD9153F | trusted by Windows: False
                           ↪  (expected: staging CA is not trusted)
SNI git.ssw590-g1.tech    : Issuer CN=YE2, O=Let's Encrypt | Thumbprint
↪  62AC575CAC34B6E3D61F581D5DCFFB321EB7C4AF (IDENTICAL to baseline) | trusted: True
Production site during the test: https://git.ssw590-g1.tech/ -> HTTP/1.1 200 OK

=== 5. Restore and cleanup (19:07:56 UTC) ===
cat Caddyfile.prod > Caddyfile ; sha256 prefix 7705e125f84a2c82 (equals the repo copy)
caddy validate -> "Valid configuration" ; caddy reload ; throwaway staging certificate directory removed from
↪  storage;
backup file Caddyfile.prod removed. ~/gitea contains only .env, .env.example, Caddyfile, docker-compose.yml.
containers: caddy Up 2 hours, gitea Up 38 minutes (healthy).

=== 6. Final public checks (19:08:02 UTC, normal certificate validation, no bypass) ===
http://git.ssw590-g1.tech/  -> HTTP/1.1 308 Permanent Redirect
https://git.ssw590-g1.tech/ -> HTTP/1.1 200 OK
production certificate: issuer CN=YE2, O=Let's Encrypt; thumbprint 62AC575CAC34B6E3D61F581D5DCFFB321EB7C4AF
↪  (unchanged)

Notes:
- What it shows: Caddy as deployed (same image, same ports, TLS-ALPN-01 challenge reachable from the internet,
↪  writable certificate storage) completes a full ACME order, a fresh certificate is stored and served, and
↪  the process takes seconds.
- What it does NOT show: that the production certificate has been renewed. That certificate was issued
↪  2026-10-15 17:34 UTC; Caddy logged a renewal window starting at Unix time 1797189600 (2026-12-13 19:20:00
↪  UTC) and ending 1797345050 (2026-12-15 14:30:50 UTC), so the first automatic renewal is expected between
↪  December 13 and 15, 2026. That is a prediction from Caddy's log, not a test result.
- The test used a third-party service (sslip.io) for DNS and Let's Encrypt's staging server; no production
↪  certificate was requested and no rate limit on the real hostname was used.
- Live-site impact: no downtime, no change to the production certificate; Caddy reloaded its configuration
↪  four times (reload at 19:04:08 and 19:05:28 and 19:06:44 and 19:07:56).
```

Listing 6: The staging dry run, with all three attempts, the restore, and the final checks, October 15, 2026, 19:03 to 19:08 UTC (real output; remote addresses masked).

What this shows and what it does not. It shows that Caddy as deployed (same image, same ports, a reachable challenge port, writable certificate storage) completes a full certificate order in seconds and stores the result. It is an *issuance* test for a different hostname; it is **not** an observed renewal of the production certificate. That certificate was issued at 17:34 UTC on October 15, 2026, and Caddy's log gives it a renewal window from December 13, 2026 (19:20 UTC) to December 15, 2026. The first automatic renewal is therefore expected in that window; this is a prediction from the log, not a result. That window falls after this document is due, so the real renewal could not be observed before submission.

Proxy configuration. The proxy configuration is the `Caddyfile` shown in the deployment subsection: one site block for `git.ssw590-g1.tech` with a `reverse_proxy` to `gitea:3000`. No certificate settings are needed because Caddy's automatic HTTPS is the default. The file contains no secrets.

Expected versus actual.

Table 3.9: HTTPS checks, October 15, 2026 (rows 1 to 4 at 17:35 UTC from the administrator's computer; the renewal rows from the 19:03 to 19:08 UTC test)

Check	Expected	Actual	Evidence
HTTP on port 80	redirect to HTTPS	308 to `https://git.ssw590-g1.tech/`	Listing 5
HTTPS on port 443	valid certificate, page served	HTTP 200, certificate accepted	Listing 5
Certificate identity	issued for the hostname by a public CA	Let's Encrypt, subject and SAN match the hostname	Listing 5
Validity	not expired, about 90 days	expires 2027-01-13	Listing 5
Renewal process (staging dry run)	Caddy completes a full ACME order and serves the new certificate	staging order for a throwaway name succeeded in about 5 seconds; production certificate unchanged	Listing 6; **PASS** (issuance only)
Renewal of the production certificate	renews automatically before expiry	window predicted Dec 13 to 15, 2026; not yet observed	**PENDING**

3.2.5 Firewall and external exposure (15 points)

Owner: BD. Security group created and first outside checks run on October 15, 2026: **PASS**. Second-network (hotspot) test: **PASS**. Console screenshot of the rules: **PASS**.

The firewall. The public boundary is an AWS security group, a stateful firewall attached to the instance's network interface. Docker-published ports can bypass a host firewall such as `ufw` [2], so the group did not rely on any host firewall; none was configured on the instance.

Table 3.10: Security group

Property	Value
Name and ID	`ssw590-g1-gitea`, `sg-0415c75429368eb85`
Network	default VPC (`vpc-08a308ecc88120f9f`) in `us-east-1`
Attached to	instance `i-0c68ec1bc1e16a414`; this is the only security group passed at launch
Created	October 15, 2026, with the AWS command line interface (Listing 7)
Default network ACL	left unchanged and not examined

Rules.

Table 3.11: Inbound rules, IPv4 (IPv6 has no rules)

Port	Source	Purpose	Notes
443/TCP	`0.0.0.0/0`	HTTPS to Caddy	the one public application entry point
80/TCP	`0.0.0.0/0`	redirect and certificate validation	only a 308 redirect is served
22/TCP	the administrator's public IPv4 address, one /32 (redacted)	administrator SSH	key login only; the two administrators share one home network address

No other inbound rule exists, so everything else is dropped: Gitea's port 3000, Caddy's administration port 2019, and port 5432 (no database service exists because SQLite is used). **IPv6:** the security group has no IPv6 rules and no AAAA DNS record was created, so the service is IPv4 only. Caddy's published ports show IPv6 bindings (`[::]`) on the host, but they are not reachable because nothing admits IPv6 traffic. **UDP:** Caddy advertises HTTP/3 on UDP 443, but the group admits only TCP, so clients fall back to HTTPS over TCP. **Outbound:** the AWS default (all traffic allowed) was kept, because the instance must reach Docker Hub, GitHub releases, package mirrors, DNS, time servers, and Let's Encrypt.

```
AWS CLI output, run from Brahmi's machine (PowerShell, aws-cli 2.37.9), region us-east-1, Oct 15, 2026.
SANITIZED: the admin's public IPv4 address was replaced by <ADMIN-IP>. Nothing else was edited.
The commands were run by Claude Code on Brahmi's go-ahead; no credentials appear in this output.

=== Step 1: default VPC (read-only) ===
VPC: vpc-08a308ecc88120f9f
=== Step 2: create security group ===
  aws ec2 create-security-group --group-name ssw590-g1-gitea \
     --description "SSW590 Group1 Gitea: 443/80 public, 22 admin only" --vpc-id <default VPC> \
     --tag-specifications Name=ssw590-g1-gitea, Project=ssw590-group1-assignment4
SG: sg-0415c75429368eb85
=== Step 3: ingress rules (authorize-security-group-ingress, one call per rule) ===
sgr-07ffad0f8d05c8f97        443
sgr-09bfd2a236c9aa42a        80
sgr-071a77a76f43108d2        22
=== Step 4: verify (describe-security-groups) ===
80        80        tcp        0.0.0.0/0        HTTP redirect and ACME validation only
22        22        tcp        <ADMIN-IP>/32        Admin SSH from Brahmi only
443        443        tcp        0.0.0.0/0        HTTPS to Caddy
--- egress ---
-1        0.0.0.0/0

Notes:
- Inbound IPv4 only; no IPv6 rules and no AAAA DNS record are planned.
- Egress is the AWS default (all outbound allowed).
- This security group is not attached to any instance yet (no instance exists).
- This output proves the rules exist on the group. It is NOT an external exposure test.
```

Listing 7: Creation of the security group and its rules with the AWS command line interface, October 15, 2026 (real output; the administrator's address is replaced by a placeholder).

Host and Compose port mappings. `docker compose ps` on the server (Listing 3) shows:

- `caddy`: publishes `0.0.0.0:80`, `0.0.0.0:443`, their `[::]` equivalents, and `443/udp`; port `2019/tcp` (Caddy's administration interface) is only exposed inside the container and is not published.
- `gitea`: `22/tcp` and `3000/tcp` are exposed only to the Compose network; nothing is published on the host.

Network diagram. Figure 3.11 shows what the security group admits and drops. The request path through the proxy is in Figure 3.17.

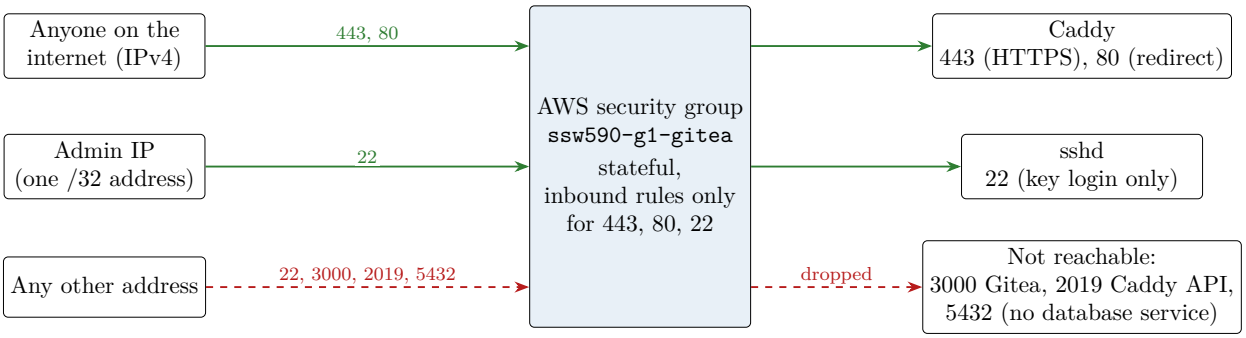


Figure 3.11: What the security group admits (green) and drops (red, dashed) for the Gitea VM [1, 2].

External tests. Two sets of outside checks were run from the administrator's computer, which is not the server (Listing 5 and Listing 8). The first was at 17:35 UTC on October 15, 2026 on the administrator's home network, which uses the approved administrator address, so it could not show that SSH is blocked for anyone else. The second was at 18:57:54 UTC the same day with the computer connected to a phone hotspot: a different network and a different public IPv4 address (the program confirmed the address differed from the approved one before probing; the addresses are not recorded here).

Table 3.12: External exposure checks

Port	Expected	Admin network, 17:35	Hotspot network, 18:57	Result
443	open, HTTPS	open, HTTP 200	open, HTTP 200	**PASS**

Port	Expected	Admin network, 17:35	Hotspot network, 18:57	Result
80	open, redirect only	open, 308 redirect	open, 308 redirect	**PASS**
3000 (Gitea)	not reachable	closed or filtered	closed or filtered	**PASS**
2019 (Caddy API)	not reachable	closed or filtered	closed or filtered	**PASS**
5432 (database)	not reachable	closed or filtered	closed or filtered	**PASS**
22 (SSH)	open from the admin address only	open	blocked: connection timed out	**PASS**

"Closed or filtered" means the connection attempt failed; the test cannot tell a refused connection from a dropped one, and both mean the port is not usable from that network. For SSH, the hotspot attempt used the *valid* private key and still failed after about 20 seconds with "Connection timed out". That is a network-level drop, not an authentication failure, which confirms that the security group, not the key, keeps other addresses out. **Limits:** this is one mobile network at one moment, and it does not test IPv6 (no AAAA record exists).

```
Real output, Oct 15, 2026, 18:57:54 UTC, from Brahmi's Windows laptop connected to her PHONE HOTSPOT (a
↪  different network and public IPv4 address from the approved admin address).
Run by Claude Code on Brahmi's go-ahead. Checked before probing: the laptop's public IPv4 differs from the
↪  approved admin address (neither address is printed here).
Target: git.ssw590-g1.tech (Elastic IP of the instance). Tools: Test-NetConnection, curl.exe, OpenSSH client.

=== TCP probes (Test-NetConnection) ===
port 443  : OPEN
port 80   : OPEN
port 22   : closed/filtered (no connection)
port 3000 : closed/filtered (no connection)
port 2019 : closed/filtered (no connection)
port 5432 : closed/filtered (no connection)

=== HTTP and HTTPS ===
curl -I http://git.ssw590-g1.tech/   ->  HTTP/1.1 308 Permanent Redirect   Location:
↪  https://git.ssw590-g1.tech/
curl -I https://git.ssw590-g1.tech/  ->  HTTP/1.1 200 OK

=== SSH with the VALID private key from this unapproved address ===
ssh -i <valid key> -o StrictHostKeyChecking=yes -o BatchMode=yes -o ConnectTimeout=10
↪  ec2-user@git.ssw590-g1.tech "echo connected"
ssh: connect to host git.ssw590-g1.tech port 22: Connection timed out      (exit 255, about 20 s)

Notes:
- Expected vs actual: 443 open, 80 open (redirect only), 22 blocked, 3000/2019/5432 not reachable -> all as
↪  expected.
- SSH failed at the network level (timeout), not at authentication, even though the key is valid: the security
↪  group drops the packets from addresses other than the approved admin /32.
- "closed/filtered" cannot tell a refused connection from a dropped one; both mean the port is not usable from
↪  this network.
```

```
- This closes the second-network row in the firewall section. It is one test, from one mobile network, at one
↪  moment; it does not test IPv6 (no AAAA record exists).
```

Listing 8: Probes from the phone-hotspot network, October 15, 2026, 18:57 UTC (real output; neither network address is printed).

The server's own SSH log gives a second view. It was read at 18:34 UTC, about an hour after the instance started. It lists only successful key logins from the two administrators' keys and **zero** failed or invalid attempts (Listing 9), which is consistent with port 22 being reachable only from the approved address. One hour of log is a weak measure, not proof.

```
Real output from the server (journalctl for sshd), Oct 15, 2026, read at 18:34:18 UTC. Run by Claude Code on
↪  Brahmi's go-ahead.
SANITIZED: source addresses replaced by <masked>; remote port numbers replaced by <n>. Only public key
↪  fingerprints are shown.

--- successful key logins, newest 12 ---
2026-10-15T17:55:28+0000 ... Accepted publickey for ec2-user from <masked> port <n> ssh2: ED25519
↪  SHA256:Wb244wRaDU7Dd+q7qlh7Lr2g2838XDRUGSS+4zGYX34   (Brahmi's key)
2026-10-15T17:55:29+0000 ... same key
2026-10-15T18:15:23+0000 ... same key
2026-10-15T18:15:24+0000 ... same key (two lines)
2026-10-15T18:25:52+0000 ... Accepted publickey for ec2-user from <masked> port <n> ssh2: ED25519
↪  SHA256:1oYCcSuCBcdksbkTTJ91uPmkJ8MQmvWDUJuENqtQfPo   (Sakshi's key: FIRST LOGIN)
2026-10-15T18:28:39+0000 ... Sakshi's key
2026-10-15T18:28:40+0000 ... Sakshi's key
2026-10-15T18:29:10+0000 ... Sakshi's key
2026-10-15T18:30:33+0000 ... Sakshi's key
2026-10-15T18:34:17+0000 ... Brahmi's key (this check)
2026-10-15T18:34:18+0000 ... Brahmi's key (this check)

--- count of successful logins by key fingerprint (whole log since boot) ---
      5 SHA256:1oYCcSuCBcdksbkTTJ91uPmkJ8MQmvWDUJuENqtQfPo     (Sakshi)
     18 SHA256:Wb244wRaDU7Dd+q7qlh7Lr2g2838XDRUGSS+4zGYX34     (Brahmi, including the AI assistant's checks
     ↪  run with her key)

--- failed or invalid SSH attempts in the same log (count) ---
0

Notes:
- Proves: Sakshi's key authenticates to the server (first success 18:25:52 UTC); the security group admitted
↪  her connection.
- The zero count means no failed, invalid-user, or pre-authentication-close lines appeared since boot (about 1
↪  hour). It is consistent with port 22 being reachable only from the approved address, but it is one hour of
↪  logs, not a long-term measure.
- Does NOT prove Sakshi's Gitea sign-in (she reported it works; not independently checked).
- Both administrators use the same Linux account (ec2-user); the key fingerprint in the log is what tells them
↪  apart.
```

Listing 9: SSH journal on the server, read at 18:34 UTC on October 15, 2026 (source addresses masked; only public-key fingerprints shown).

Figure 3.12 shows the same rules in the AWS console, on the instance's Security tab, with the account number and the administrator's address blacked out. It confirms that the instance is running with the Elastic IP, has no IPv6 address, requires IMDSv2, and has the security group `ssw590-g1-gitea` attached (launched October 15, 2026, 13:16 Eastern time). The rule list is scrolled to its end; the three rules are 80 and 443 from anyone and 22 from the redacted administrator address, with the descriptions given when they were created. A command-line listing of the same rules is in Listing 7.

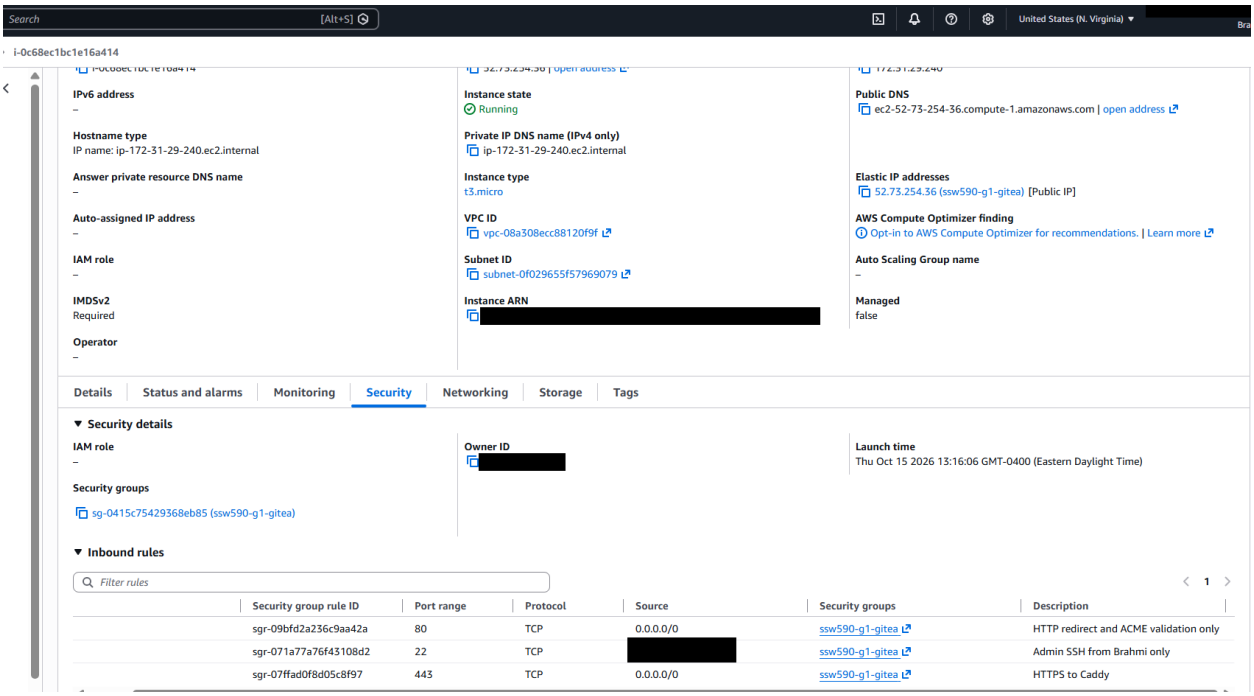


Figure 3.12: AWS console, instance `i-0c68ec1bc1e16a414`, Security tab, October 15, 2026 (redacted).

Corrections. None have been needed so far. The only exposure found beyond the plan is informational: internet scanners reached the HTTPS port within seconds of the site coming online (Listing 3) and probed for unrelated software, which returned 404 responses. Nobody could create an account, because registration is disabled.

Why HTTPS does not replace a firewall. HTTPS protects the content of a connection between a browser and Caddy; it says nothing about which ports exist or who may connect to them. Without the security group, ports such as 22, 3000, or 2019 would accept connections from the whole internet, and encryption on port 443 would not help. The firewall decides which services can be reached at all; TLS only protects the ones that are reachable. The two address different risks, so both are needed [1].

3.2.6 Startup team workflow (20 points)

Owner: SD (BD is the second member and reviewer). **Status:** organization, repository, backlog, and two reviewed, merged, issue-closing pull requests on October 15, 2026 **PASS**.

Organization, repository, and members

Table 3.13: Startup organization and members

Item	Value
Organization	group1-startup
Product repository	group1-startup/twintrack (public, so the TA and instructor can inspect it signed out); default branch main
README	README.md on main, added by pull request #8
Member: Brahmi Dalvi (brahmi)	organization owner and Gitea site administrator (infrastructure); also in team developers
Member: Sakshi Deshpande (sakshi)	team developers: write access to code, issues, and pull requests; cannot create repositories in the organization or change settings (product)
Branch rule on main	no direct pushes, one approving review required; a direct commit by the administrator was refused with HTTP 403 (issue #4)

The least-privilege split was confirmed by a real failure: SD's first attempt to create the repository returned HTTP 403 "User is not allowed to create repositories in this organization", so BD, as owner, created it (Table 3.19).

Backlog

Six actionable issues mix product and operations work (Table 3.14). They were created through the Gitea API by the setup script assignment-4/gitea/row6-setup.py, run by each member with a personal token typed at a hidden prompt and deleted afterwards (Listing 10).

Table 3.14: Startup backlog in group1-startup/twintrack, October 15, 2026

#	Issue	Type	Assignee	State
1	Write the product README	product	SD	closed by PR #8
2	Add CONTRIBUTING.md for the issue, branch, PR, review flow	operations	BD	closed by PR #7
3	Define the TwinTrack task data model	product	SD	open
4	Protect main: require PR + 1 approval	operations	BD	closed after the refusal test
5	Document the Gitea restart and troubleshooting runbook	operations	SD	open
6	Add the domain expiry date (October 15, 2027) to the shared calendar and name its owner	operations	BD	open

Issue-to-merge traces

Each trace has an issue, a branch named after it, a pull request whose description contains `Closes #N`, a comment and an approving review by the *other* member, a merge, and automatic closure of the issue. Gitea numbers pull requests in the same sequence as issues, so the first pull request is #7. All times are UTC from the Gitea API (Listing 11).

Table 3.15: Issue-to-merge traces, October 15, 2026

Step	Trace 1 (author SD)	Trace 2 (author BD)
Issue	#1 Write the product README	#2 Add CONTRIBUTING.md
Branch	`issue-1-readme`	`issue-2-contributing`
Pull request	#8 “Add product README”, body “Closes #1”, opened 19:40:12	#7 “Add CONTRIBUTING.md”, body “Closes #2”, opened 19:31:18
Reviewer comment	BD, 19:47:09	SD, 19:39:04 (asks that authors reply to or resolve each comment before approval)
Approval	BD, APPROVED, 19:47:21	SD, APPROVED, 19:46:27
Merge	by SD, 19:47:45, commit `15f8a00343`	by BD, 19:49:04, commit `0d0c3dac4d`
Issue closed	#1 closed 19:47:45 by the merge	#2 closed 19:49:04 by the merge

Review quality, stated honestly. Both reviews were real approvals by the other member, enforced by the branch rule (“0 of 1 official approvals” blocked the merge until then; Figure 3.13). They were light: BD’s comment on #8 was a placeholder (“comment from brahmi”), and SD’s request on #7 was approved and merged without a reply or a change to `CONTRIBUTING.md`. SD’s suggestion remains open as a follow-up for the next contribution.

```
Real output of assignment-4/gitea/row6-setup.py run by Sakshi in her own terminal (macOS), Oct 15, 2026 (about
↪  19:39 UTC).
The access token was typed at a hidden prompt and is not in this output. Followed by an independent check made
↪  with NO token (anonymous Gitea API).

=== Script output (Sakshi's stages) ===
Gitea access token (hidden, not saved):
Signed in as: sakshi  (site admin=False)
[repo] exists already: https://git.ssw590-g1.tech/group1-startup/twintrack  (private=False)
[issues] exists: #1 Write the product README
[issues] exists: #2 Add CONTRIBUTING.md for the issue, branch, PR, review flow
[issues] exists: #3 Define the TwinTrack task data model
[issues] exists: #4 Protect `main`: require PR + 1 approval
[issues] exists: #5 Document the Gitea restart and troubleshooting runbook
[issues] exists: #6 Add the domain expiry date (Oct 15, 2027) to the shared calendar and name its owner
[readme-pr] branch issue-1-readme created with the product README (from sd/backlog-draft.md)
[pr] opened: https://git.ssw590-g1.tech/group1-startup/twintrack/pulls/8
    NEXT: Brahmi reviews it (at least one comment), Sakshi addresses it, Brahmi approves, then merge.

Your stages are done. Brahmi runs her part next (protection and the CONTRIBUTING.md PR).

Now DELETE the token in Gitea > Settings > Applications.
```

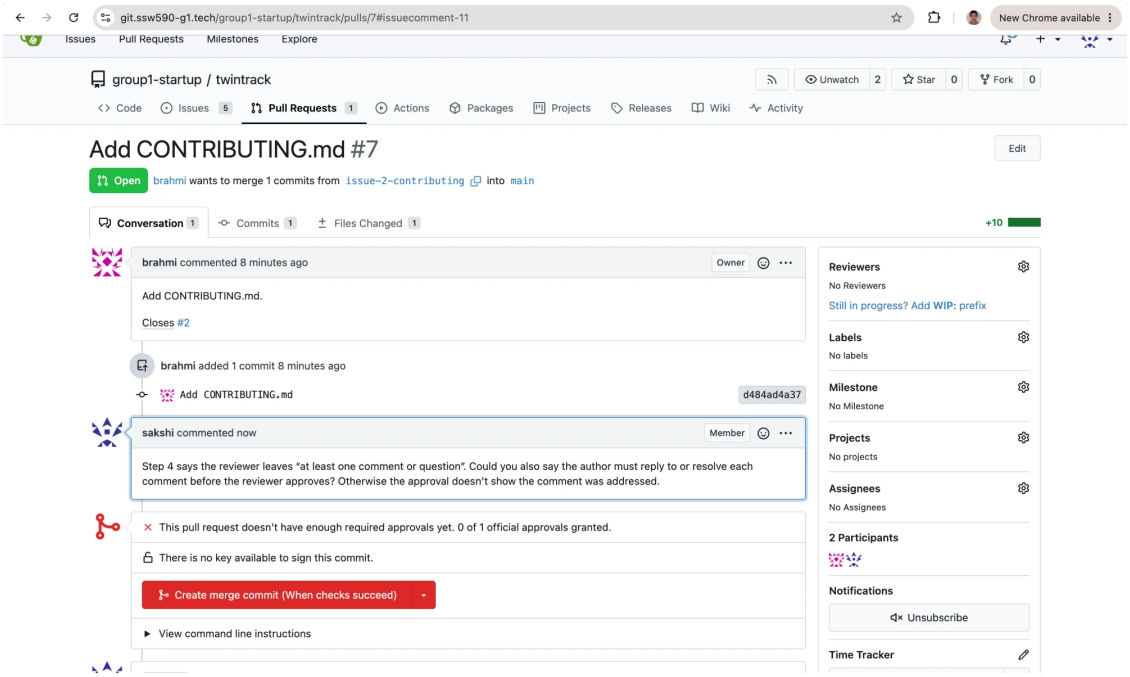


Figure 3.13: Pull request #7 before approval, October 15, 2026: SD's review comment, and the merge blocked with "0 of 1 official approvals granted".

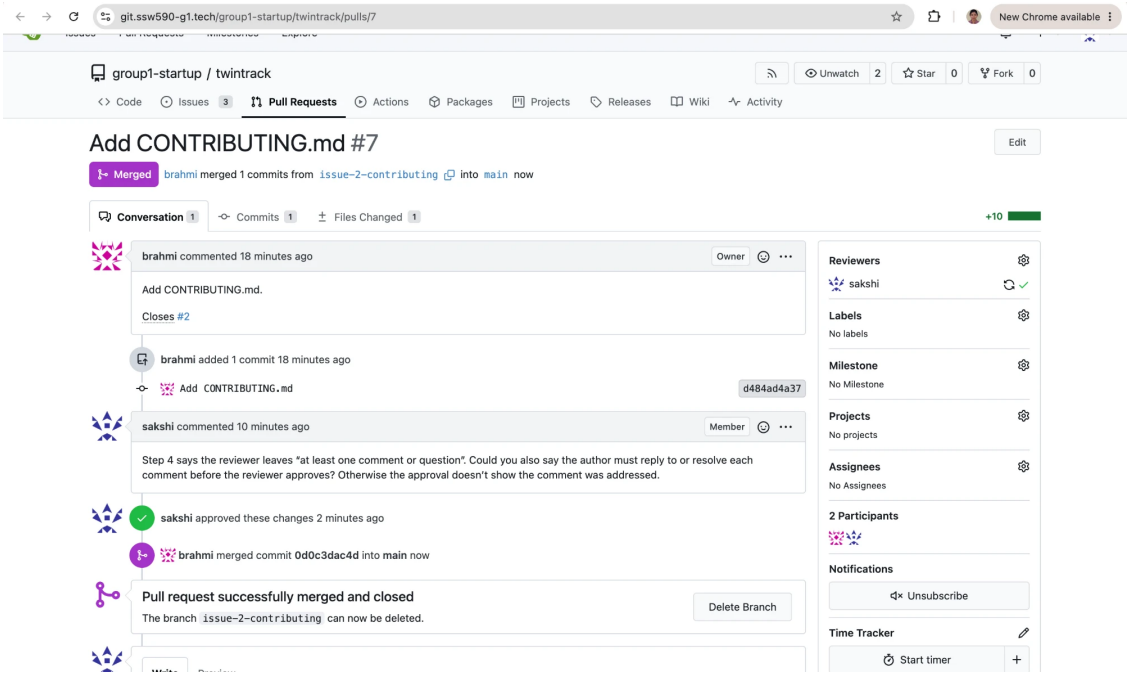


Figure 3.14: Pull request #7 merged, October 15, 2026: "Closes #2", approved by `sakshi`, merged by `brahmi` as commit `0d0c3dac4d`.

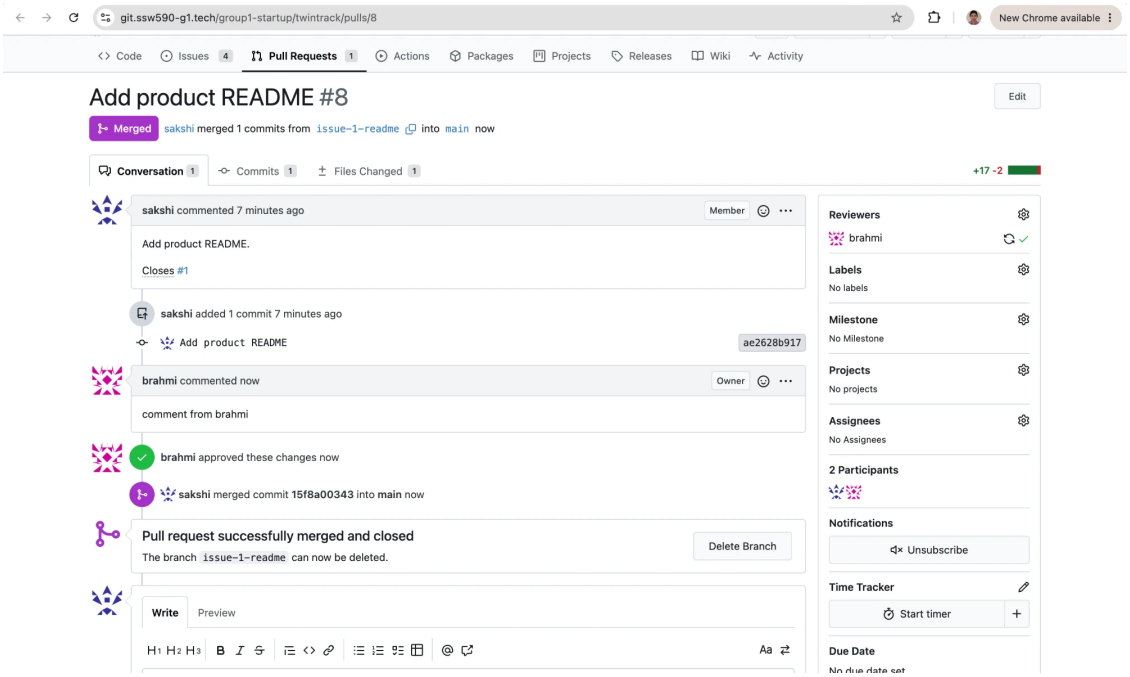


Figure 3.15: Pull request #8 merged, October 15, 2026: "Closes #1", approved by `brahmi`, merged by `sakshi` as commit `15f8a00343`.

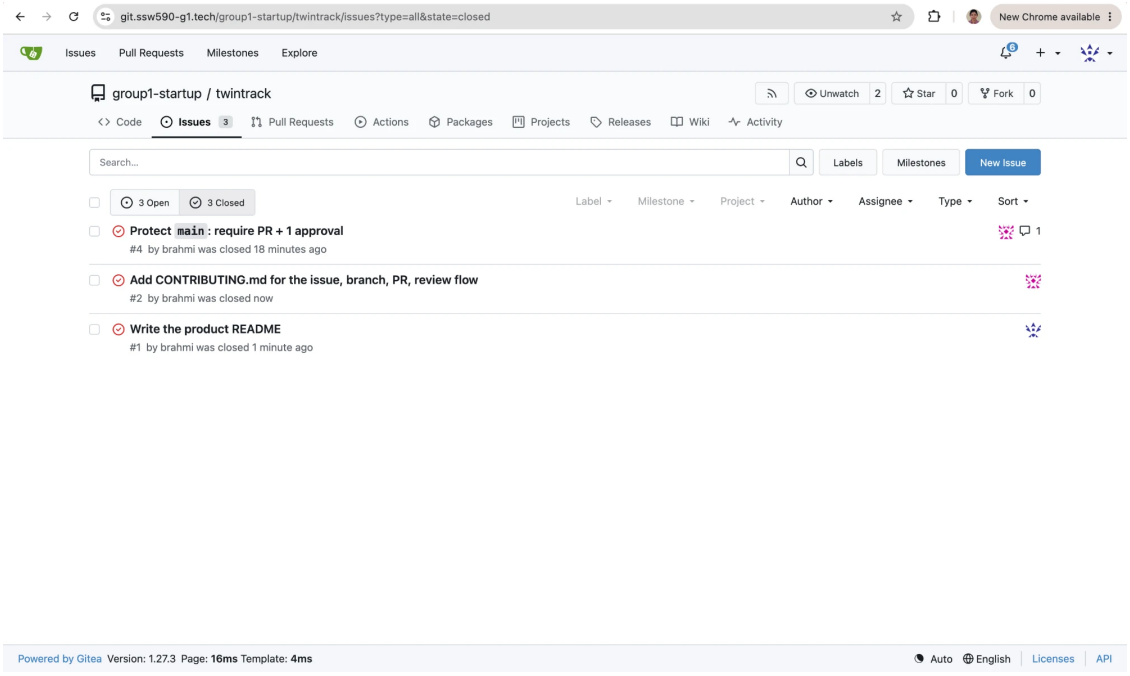


Figure 3.16: Closed issues in `twintrack`, October 15, 2026: #1 and #2 closed by their merges, #4 closed after the branch-protection test.

```
Repository: https://git.ssw590-g1.tech/group1-startup/twintrack

=== Independent check, anonymous API, 2026-10-15 19:40:30 UTC ===
#8 'Add product README' state=open head=issue-1-readme base=main by=sakshi body='Add product README.\n\nCloses
↪  #1'

Notes:
- Sakshi's first run (before Brahmi's) stopped with HTTP 403 "User is not allowed to create repositories in
↪  this organization";
  nothing was created. Expected: the team `developers` cannot create repositories. Brahmi then ran her stages
  ↪  first.
```

Listing 10: SD's run of the setup script, which opened pull request #8, October 15, 2026 (real output; no token in it).

```
Anonymous Gitea API listing (no token), group1-startup/twintrack, read 2026-10-15 19:49:50 UTC
Captured by Claude Code from Sakshi's machine. Public data only; nothing below is secret. Curly quotes and
↪  apostrophes in comment text are written as straight ones.

repo: private=False default_branch=main
branch main: protected=True required_approvals=1

=== Issues ===
#1  closed assignee=sakshi  closed_at=2026-10-15T19:47:45Z  Write the product README
#2  closed assignee=brahmi  closed_at=2026-10-15T19:49:04Z  Add CONTRIBUTING.md for the issue, branch, PR,
↪  review flow
#3  open   assignee=sakshi  closed_at=None  Define the TwinTrack task data model
#4  closed assignee=brahmi  closed_at=2026-10-15T19:31:18Z  Protect `main`: require PR + 1 approval
#5  open   assignee=sakshi  closed_at=None  Document the Gitea restart and troubleshooting runbook
#6  open   assignee=brahmi  closed_at=None  Add the domain expiry date (Oct 15, 2027) to the shared calendar
↪  and name its owner

=== Pull request #7 ===
title='Add CONTRIBUTING.md' author=brahmi head=issue-2-contributing base=main
body='Add CONTRIBUTING.md.\n\nCloses #2'
created_at=2026-10-15T19:31:18Z state=closed merged=True merged_at=2026-10-15T19:49:04Z merged_by=brahmi
↪  merge_commit=0d0c3dac4d
comment  2026-10-15T19:39:04Z by sakshi  'Step 4 says the reviewer leaves "at least one comment or question".
↪  Could you also say the author must reply to or resolve each comment before the reviewer approves?
↪  Otherwise the approval doesn't show the comment was addressed.'
review   2026-10-15T19:46:27Z by sakshi  state=APPROVED body=''

=== Pull request #8 ===
title='Add product README' author=sakshi head=issue-1-readme base=main
body='Add product README.\n\nCloses #1'
created_at=2026-10-15T19:40:12Z state=closed merged=True merged_at=2026-10-15T19:47:45Z merged_by=sakshi
↪  merge_commit=15f8a00343
comment  2026-10-15T19:47:09Z by brahmi  'comment from brahmi'
review   2026-10-15T19:47:21Z by brahmi  state=APPROVED body=''
```

Listing 11: The whole workflow read from the public Gitea API without a token, October 15, 2026, 19:49:50 UTC: issues, branch rule, both pull requests with comments, approvals, merges, and merge commits.

3.2.7 Architecture and operation (5 points)

Owner: SD. **Status:** figure and text describe the live deployment of October 15, 2026 **PASS**; runbook commands run on the server (Listing 12) **PASS**.

Architecture

Figure 3.17 shows the deployment, adapted from the course reference design [1] for the SQLite path only. Black arrows cross the public internet; blue arrows stay inside the instance. The facts come from the deployment evidence (Sections 3.2.2 to 3.2.5): one AWS EC2 instance `ssw590-g1-gitea` (`t3.micro`, Amazon Linux 2023, region `us-east-1`) with an Elastic IP, security group `ssw590-g1-gitea`, and Docker Compose project `gitea` in `~/gitea`.

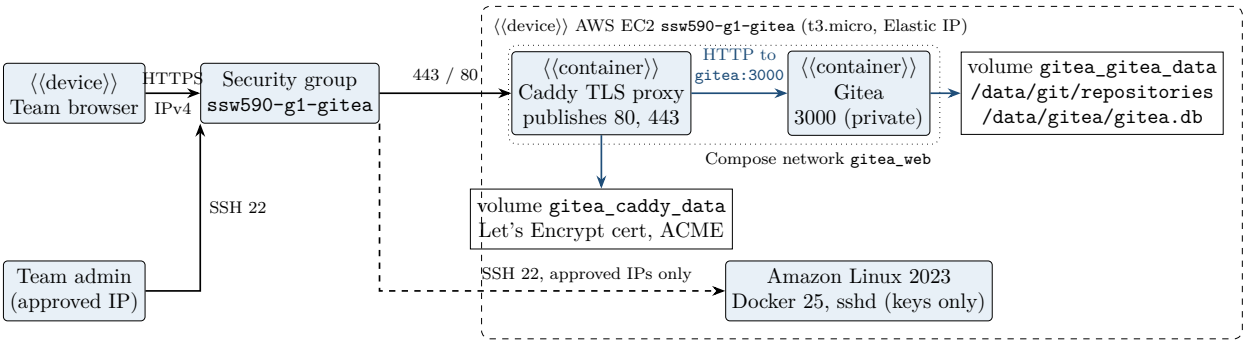


Figure 3.17: Group 1 Gitea deployment on one AWS EC2 instance, October 15, 2026. The security group admits HTTPS (443) and HTTP (80, redirect only) from anyone over IPv4, and SSH (22) only from the approved team address. Only Caddy publishes ports; Gitea's port 3000, its container `sshd`, Caddy's admin port 2019, and the SQLite file are private. There is no IPv6 path [1, 3, 4].

Request path

1. The browser resolves git.ssw590-g1.tech through public DNS (an `A` record at Name.com, TTL 300; no `AAAA` record) to the instance's Elastic IP and connects on 443.
2. The security group admits 443 and 80 from anywhere and 22 only from the approved address; everything else is dropped.
3. Caddy terminates TLS with a Let's Encrypt certificate for git.ssw590-g1.tech, obtained with the TLS-ALPN-01 challenge on port 443 and stored in the `gitea_caddy_data` volume. A request on port 80 receives `308 Permanent Redirect` to HTTPS.
4. Caddy forwards the request as plain HTTP over the private Compose network `gitea_web` to `gitea:3000`, found by service name.
5. Gitea reads and writes the SQLite file and the bare Git repositories in the `gitea_gitea_data` volume and answers through Caddy.

Git clone and push use the same HTTPS path; Gitea's own SSH clone is disabled (`DISABLE_SSH=true`). The image's `sshd` still listens on port 22 *inside* the container, but that port is not published.

Administration path

Administrators connect with SSH keys only, as `ec2-user`, from the approved address. Each member has a separate key, and the server's `sshd` log tells them apart by key fingerprint.

Before the first connection, each member compared the server's host key with the one published by the administrator (`SHA256:sJSqc/LAVTJ...buvgosM`). Docker is run by `ec2-user`, which is in the `docker` group.

Public and private interfaces

Table 3.16: Interfaces and exposure

Interface	Exposure	Notes
Caddy 443/TCP	public	HTTPS for the web UI, API, and Git
Caddy 80/TCP	public	`308` redirect to HTTPS only
Host SSH 22/TCP	restricted	approved address only (Section 3.2.5)
Gitea 3000/TCP	private	Compose network only, no host port mapping
Container `sshd` 22	private	not published
Caddy admin 2019/TCP	private	inside the Caddy container, not published
SQLite	none	a file, not a network service
IPv6	none	no security-group rule and no `AAAA` record

Where data lives

All Gitea state is in the named volume `gitea_gitea_data` (Table 3.5); TLS certificates and the ACME account are in `gitea_caddy_data`, and Caddy's runtime configuration in `gitea_caddy_config`. The volumes are on the instance's 20 GB `gp3` root disk. Containers hold no state and can be replaced at any time; row 3 did exactly that.

Operating runbook

All commands run on the instance from `~/gitea`.

```
docker compose ps                       # both services Up; gitea (healthy)
docker compose logs --tail=100 gitea    # application errors
docker compose logs --tail=100 caddy    # TLS issuance/renewal, proxy errors
docker compose restart gitea            # restart without recreating
docker compose up -d --force-recreate gitea   # replace the container, keep volumes
docker compose up -d                    # re-apply the Compose file after a change
curl -sS -o /dev/null -w "%{http_code}\n" https://git.ssw590-g1.tech/api/healthz
```

The read-only commands were run on October 15, 2026 at 18:50 UTC (Listing 12). Container replacement was exercised by the row 3 test (Table 3.6): Gitea was back and healthy about 30 seconds later. A plain `docker compose restart` was not run separately, because it does strictly less than that replacement.

```
Real output from the server, run by Claude Code from Sakshi's machine (ssh ec2-user@git.ssw590-g1.tech,
↪  Sakshi's key), Oct 15, 2026.
Purpose: the read-only commands of the row 7 runbook, run for real. Restart command not run here (see the
↪  manual).
SANITIZED: Caddy log lines reduced to level, logger, message and upstream, so client IP addresses are not
↪  shown. The only addresses left are 0.0.0.0 (listen on all), 127.0.0.1 (loopback) and 172.18.0.x (the
↪  private Docker network). The single-character ellipsis (U+2026) that docker prints in the COMMAND column
↪  is written as "...".

$ date -u
2026-10-15T18:50:22Z
$ docker compose ps
NAME      IMAGE                COMMAND                  SERVICE   CREATED             STATUS
↪  PORTS
caddy     caddy:2.11.7         "caddy run --config ..."   caddy     About an hour ago   Up About an hour
↪  0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 443/udp, 2019/tcp
gitea     gitea/gitea:1.27.3   "/usr/bin/entrypoint..."   gitea     21 minutes ago      Up 21 minutes
↪  (healthy)   22/tcp, 3000/tcp
$ docker compose logs --tail=3 gitea
gitea  | 2026/10/15 18:49:54 HTTPRequest [I] router: completed GET /api/healthz for 127.0.0.1:55688, 200 OK in
↪  0.4ms @ healthcheck/check.go:76(healthcheck.Check)
gitea  | 2026/10/15 18:50:10 .../src/net/http/server.go:2286:HandlerFunc.ServeHTTP() [I] PING DATABASE sqlite3
gitea  | 2026/10/15 18:50:10 HTTPRequest [I] router: completed GET /api/healthz for 172.18.0.3:32964, 200 OK
↪  in 0.5ms @ healthcheck/check.go:76(healthcheck.Check)
$ docker compose logs --tail=3 caddy   (fields: level, logger, msg, upstream)
warn http.handlers.reverse_proxy aborting with incomplete response upstream=gitea:3000
warn http.handlers.reverse_proxy aborting with incomplete response upstream=gitea:3000
warn http.handlers.reverse_proxy aborting with incomplete response upstream=gitea:3000
$ curl -sS -o /dev/null -w "%{http_code}" https://git.ssw590-g1.tech/api/healthz
healthz HTTP 200
$ docker volume ls --filter name=gitea_
DRIVER    VOLUME NAME
local     gitea_caddy_config
local     gitea_caddy_data
local     gitea_gitea_data
$ docker network ls --filter name=gitea_
NETWORK ID     NAME        DRIVER    SCOPE
ea58c8a1e342   gitea_web   bridge    local
```

Listing 12: Read-only runbook commands on the EC2 instance, October 15, 2026, 18:50 UTC (real output; client addresses removed from the Caddy lines).

The Caddy warnings "aborting with incomplete response" appear when a response stream to a client is cut off (for example, a browser closing the connection); users did not see errors, and the health check returned 200.

Table 3.17: Troubleshooting

Symptom	**Check**
Browser cannot connect	DNS still resolves to the Elastic IP; security group allows 443; `docker compose ps` shows Caddy Up
502 Bad Gateway	Gitea is down or still starting: `docker compose logs gitea`; one transient 502 was seen at first start (Section 3.2.2)
Certificate error	`docker compose logs caddy`; port 443 reachable for TLS-ALPN-01; `gitea_caddy_data` still mounted
SSH refused or times out	the member's public address has changed; ask the administrator to update the security group

Symptom	Check
Data appears missing	`docker volume ls` still lists `gitea_gitea_data`; Compose is run from `~/gitea` so the project name is still `gitea`

Never run `docker compose down -v` or `docker volume rm` on the instance: they delete the volumes and all data.

Persistence is not backup

The persistence test (Section 3.2.3) proves data survives a *container* replacement because it lives on a volume. A backup is a separate copy of that data, stored elsewhere, that has been restored successfully. Without one, deleting the volume, terminating the instance (its root disk has `DeleteOnTermination=true`), or corrupting the database still loses everything. A verified backup and restore is the optional extra credit (Section 3.3.1).

3.2.8 Manual quality and controls (15 points)

Owner: SD. **Status:** final review of the whole manual on October 15, 2026, before submission **PASS**.

Table 3.18: Manual quality controls, checked October 15, 2026

Control	Status	Where / how checked
Manual version increased	**PASS**	0.1 → 1.0 (title page and page footers), dated October 15, 2026
History: initial setup entry	**PASS**	Version 0.1, October 12, 2026, Table 1
History: this update, with links	**PASS**	Version 1.0 entry and the dated entries for each row, each linking its section, table, or figure
Hosts and Passwords tables current, hints only, named in history	**PASS**	Chapter 2; every row holds a vault reference or hint, never a secret; the version 1.0 history entry names both tables as updated
Headings, captions, labels	**PASS**	One subsection per rubric row in rubric order; every figure, table, and listing has a caption and label; the final build has no undefined or multiply defined references
Evidence inside the PDF	**PASS**	Command output as listings and screenshots as figures, referenced by number rather than by file name
Citations for external facts	**PASS**	Course manual, Gitea, Caddy, Let's Encrypt, Docker, and GitHub Education sources in the Bibliography

Control	Status	Where / how checked
Glossary and index entries for new terms	**PASS**	Glossary, Acronyms, Index
Secrets and addresses redacted	**PASS**	Scripted scan of all chapter sources and of every command-output file printed in the PDF for private keys, access tokens, AWS keys and account numbers, passwords, e-mail addresses, and IPv4 addresses. Only expected addresses remain: the server's public Elastic IP (published in DNS anyway), public DNS resolvers, loopback, and private Docker and AWS addresses. Every screenshot was inspected by eye; the AWS console screenshot is redacted, and browser tab bars with unrelated personal tabs were cropped
Hyperlinks to the Gitea service, organization, repositories, issues, and pull requests	**PASS**	Repositories are public, so the links work for the TA and instructor without an account
No `\Todo`, undefined references, or build errors	**PASS**	Final build of October 15, 2026: no undefined or multiply defined references, no errors, and no `\Todo` marker in the sources or in the text of the PDF (both searched). Five markers that remained outside SD's regions (domain calendar owner, extra-credit placeholder, Introduction link, and two AI-log review lines) were removed by BD

Failed tests and corrective actions

All failed or inconclusive attempts made for this chapter are listed here, with the section where each is described in full.

Table 3.19: Failed tests and corrective actions

Date	What failed	Correction
October 12, 2026	SD's first local rehearsal of `persistence-test.sh` stopped with a Python syntax error in the script's output formatting.	Fixed the quoting; the rerun completed. Rehearsal only; no server involved.

Date	What failed	Correction
October 15, 2026	The AWS command line tool on Windows crashed printing a non-ASCII character from the instance console log, so an early check looked like "still waiting" (Section 3.2.2).	Read the log again with UTF-8 forced.
October 15, 2026	The IAM user was denied the public image-lookup parameter (Section 3.2.2).	Found the Amazon Linux image with `describe-images`.
October 15, 2026	A multi-line script passed to `ssh` from Windows PowerShell 5.1 lost its quotes and failed (Section 3.2.2).	Scripts are copied to the server and run there.
October 15, 2026	One transient 502 at first start: a scanner reached Caddy before Gitea was listening (Section 3.2.2).	None needed; it did not recur.
October 15, 2026	Renewal test, attempt 1 (staging CA for the real hostname): inconclusive; attempt 2: Caddy rejected the directive `ca` (Section 3.2.4).	Attempt 3 used a throwaway hostname with a staging-only issuer and succeeded; the production Caddyfile was restored.
October 15, 2026	SD's first run of `row6-setup.py` could not create the product repository: HTTP 403 "User is not allowed to create repositories in this organization" (Section 3.2.6).	Expected under least privilege. BD, as owner, ran her stages first; SD's rerun opened pull request #8.
October 15, 2026	A manual build failed: a single-character ellipsis in captured `docker compose ps` output could not be line-broken by `minted`.	The character is written as three dots in that evidence file, and the file's header says so.

Accepted limitations

- No backup exists; the persistence test is not a backup (Section 3.2.7). The extra credit was not attempted.
- The peer reviews in row 6 were light (Section 3.2.6).
- MFA statuses in the Passwords table are as reported by each account owner and were not independently checked.

3.3 Extra Credit

3.3.1 Extra credit: verified backup and restore (up to 20 extra-credit points; not attempted)

Status: not attempted. The group did not carry out the optional backup and restore, and no backup, restore, or result is claimed here. The persistence test in the database subsection (a repository and an issue surviving the replacement of the Gitea container) is a different test and is not a backup.

Chapter 4

Automated Manual Build

4.1 Overview and Status

This chapter answers the course manual's Chapter 3 [1]: every push of this manual's LaTeX source to the team's Gitea repository builds the PDF on a self-managed Gitea runner and publishes it as a downloadable artifact whose title page and footers carry the commit and run that produced it. The eight numbered subsections match the eight rubric rows in order.

Table 4.1: Rubric rows, owners, and status

Subsection	Points	Owner	Status
Source and trigger	15	BD	**PASS**
Runner setup	15	BD	**PASS**
Reproducible compilation	20	SD	**PASS**
PDF version traceability	15	SD	**PASS**
Downloadable artifact	10	BD	**PASS**
Change, failure, and repair evidence	10	SD	**PASS**
Runner and secret boundary	5	BD	**PASS**
Manual quality and controls	10	SD	**PASS**

4.2 Required Subsections

4.2.1 Source and trigger (15 points)

Owner: BD. **Status:** two push-triggered runs observed on October 19–20, 2026 **PASS**.

What is where. The source of this manual lives in the Gitea repository `group1-startup/manual` (public, created by BD on October 19, 2026). The main file is `devopsITManual.tex`; the compile command is the `latexmk` command in the workflow `.gitea/workflows/build.yml`.

The workflow runs on a `push` to the branch `main` (and on a manual start). The repository's two members, BD (organization owner) and SD (collaborator with write access), push over HTTPS; the course's Chapter 2 repository `twintrack` is the product, and this repository holds the manual (Table 4.2).

Table 4.2: Network path of a push and of a runner job, with public and private ports

Step	Port	Detail
`git push` from a laptop	**443/TCP, public**	HTTPS remote `https://git.ssw590-g1.tech/group1-startup/manual.git`; the AWS security group admits 443 from anyone.
TLS termination	443 (Caddy)	Caddy terminates TLS with the Let's Encrypt certificate and forwards the request to Gitea.
Caddy to Gitea	**3000/TCP, private**	Only on the private Docker network `gitea_web`; not published to the host and not reachable from the internet (Chapter 3).
Gitea Git over SSH	none	Disabled (`DISABLE_SSH`); HTTPS-only Git.
Runner to Gitea	443 (outbound)	The runner calls `https://git.ssw590-g1.tech` like a browser does. **No inbound port exists for the runner.**

Why the runner needs no public submission port. The runner works by pulling: it connects out to Gitea over HTTPS, asks for waiting jobs, and reports the log and result back over the same connection. Gitea never opens a connection to the runner, and a push goes to Gitea, not to the runner. The runner container therefore publishes no port at all (the `Ports` column of `docker ps` is empty for it, and the only published ports on the host remain Caddy's 80 and 443).

A run started by a push. SD's first push, commit `1fb4e15`, started run #1, and her second push, commit `a5f3446`, started run #2; both appear in the repository's Actions list (Figure 4.1). Run #2's summary shows "Triggered via push" on `main` and `build.yml (on: push)` (Figure 4.2). Run #1 compiled the manual but failed at the upload step, which is explained with the other failed attempts in Section 4.2.3.

4.2.2 Runner setup (15 points)

Owner: BD. **Status:** runner online and two jobs run on it, October 19–20, 2026 **PASS**.

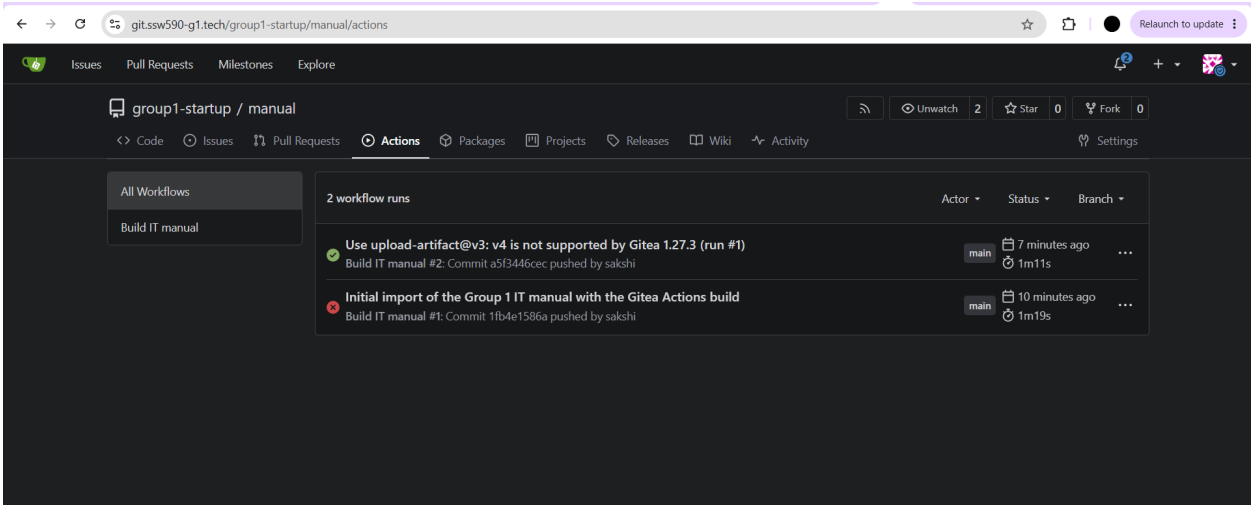


Figure 4.1: Actions tab of `group1-startup/manual`, October 19, 2026 (evening, US Eastern): run #1 for commit `1fb4e15` failed, run #2 for commit `a5f3446` succeeded; both were started by a push to `main`.

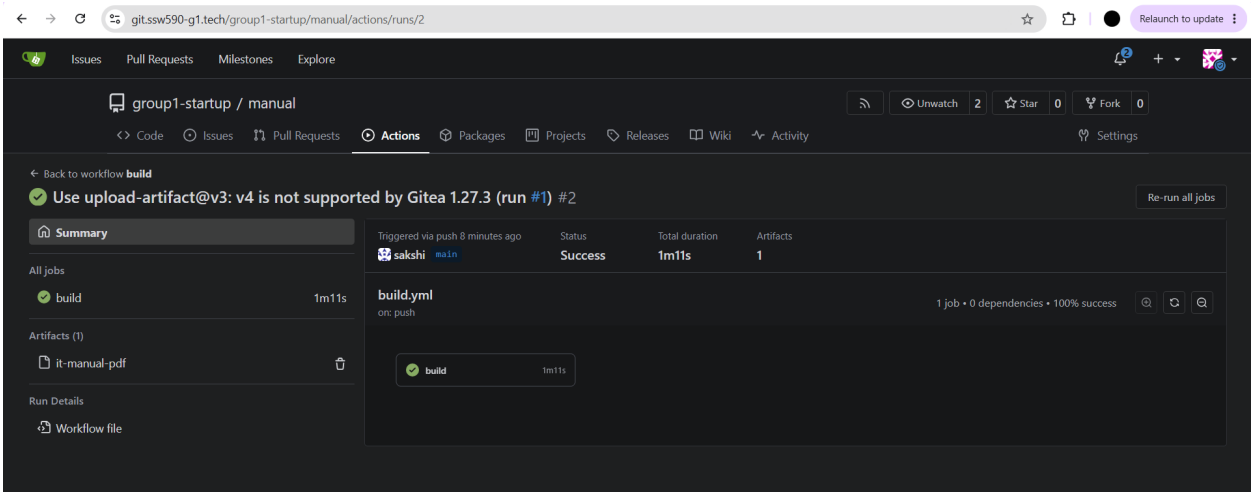


Figure 4.2: Run #2 summary: triggered via push, status Success, duration 1 minute 11 seconds, workflow `build.yml` (`on: push`), one artifact `it-manual-pdf`.

Table 4.3: The Gitea Actions runner

Property	Value
Name and version	`group1-runner-1`, `act_runner` v0.6.1 (Docker image `gitea/act_runner:0.6.1`, pinned) [16]
Location	A Docker container named `gitea-runner` on the same EC2 `t3.micro` host as Gitea (single-host layout of the course's Figure 3.2)
Scope	Global: registered with the instance-level registration token from Site Administration, so it can serve any repository of this Gitea (Figure 4.3); only `group1-startup/manual` uses it
Label	`manual-builder:docker://group1/texlive-node:2026.09` [17]; the workflow selects it with `runs-on: manual-builder`
Execution mode	`docker://`: every job runs in a temporary job container created from the label's image, not directly in the runner
Job image	Built on the host from `ci/Dockerfile` in the manual repository (TeX Live 2026 pinned by digest plus Node.js), 5.82 GB; it is local only, so the runner is configured with `force_pull: false` and never downloads it
Network path	Runner and job container → `https://git.ssw590-g1.tech` (public name, port 443). The host reaches its own public address; this was checked before installing (HTTP 200 on `/api/healthz`)
Capacity	One job at a time (`capacity: 1`) because the host has 2 vCPUs, about 1 GB of RAM and 2 GB of swap; 20 GB disk with about 9.4 GB free after the image
Restart behavior	`restart: unless-stopped`; the registration is kept in the Docker volume `runner_runner_data`
Published ports	None

Decision: same host. Three options were compared on October 19, 2026: the runner on the Gitea host (chosen), a larger instance, or a laptop. The Gitea host costs nothing extra and stays online for the instructor and TA; the risks are the small memory and the shared disk. Observed: run #2 took 1 minute 11 seconds, Gitea stayed healthy, and swap use was about 110 MB afterwards.

Registered and online, with a job assigned. The runner shows as *Idle* with its label in the administration page (Figure 4.3). After SD's push, the first lines of run #2's *Set up job* log show the runner receiving the job and starting the pinned image without pulling it (Figure 4.4): `group1-runner-1(version:v0.6.1) received task 2 of job build, be triggered by event: push`, `forcePull=false`, `Image exists? true`, then a new container on its own network. The container is removed after the job.

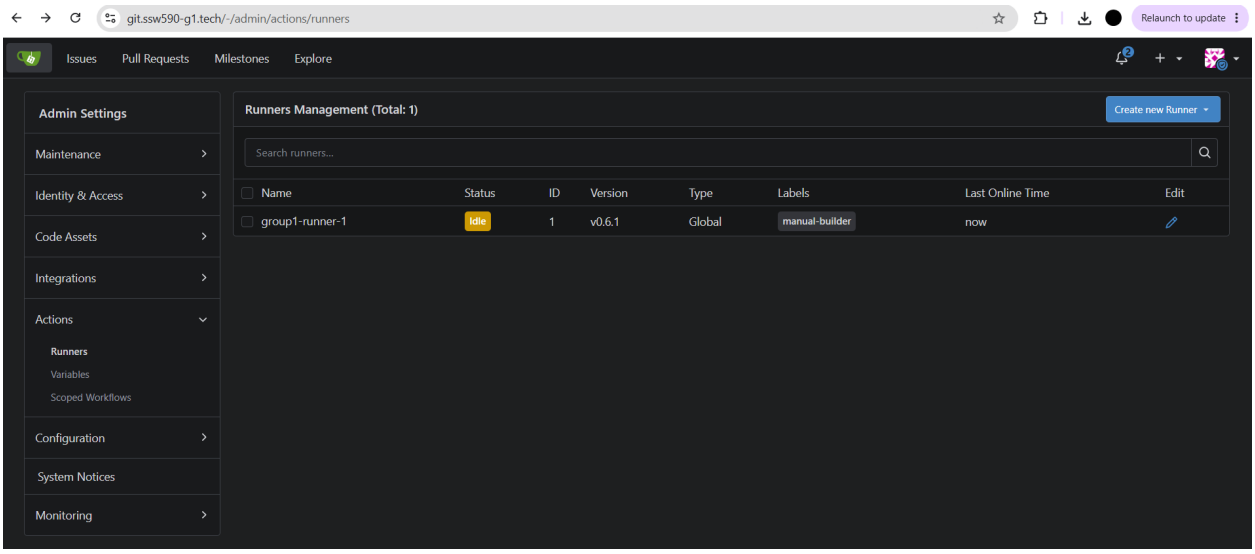


Figure 4.3: Site Administration, Actions, Runners, October 19, 2026: `group1-runner-1`, version v0.6.1, type Global, label `manual-builder`, status Idle, last online "now".

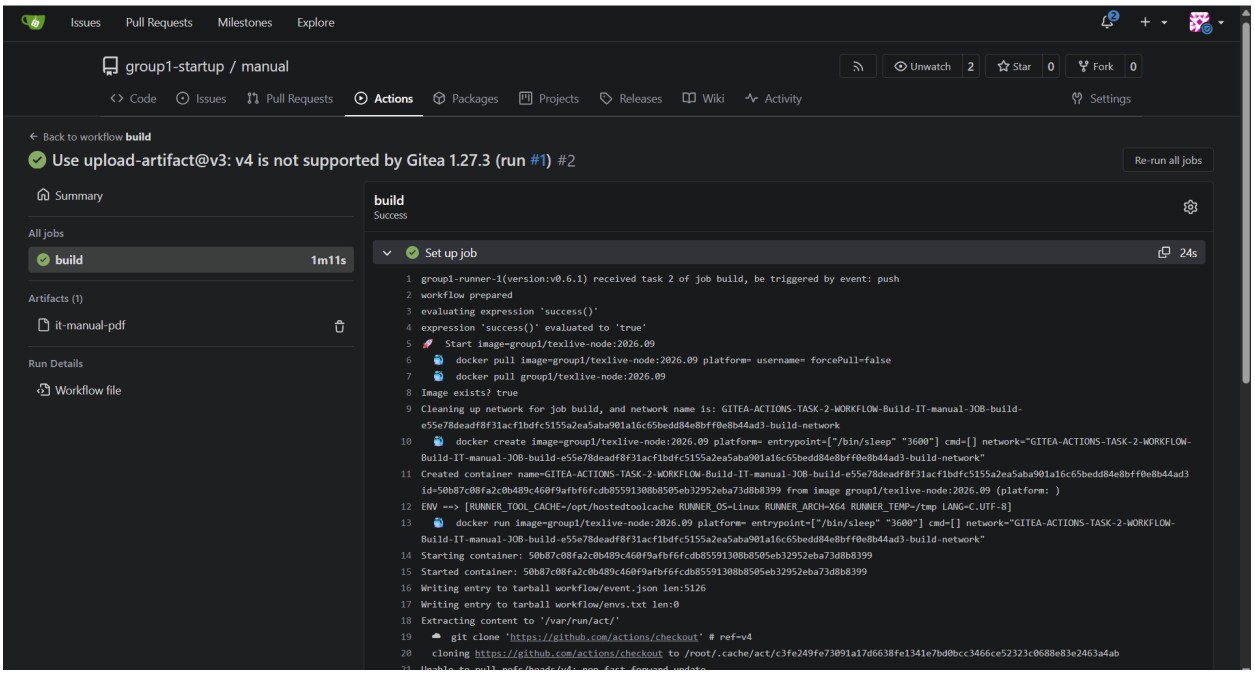


Figure 4.4: Log of run #2, step *Set up job*: the job is assigned to `group1-runner-1`, runs in a temporary container from `group1/texlive-node:2026.09`, and the image is not pulled (`forcePull=false`). The first action fetched is `actions/checkout` from github.com.

Isolation and ports. The runner container publishes no port; the only host ports open to the internet are still 80 and 443 on Caddy. Jobs run in their own containers on a fresh network per job; the runner is not run in privileged mode, and no host volumes are allowed for jobs (the configuration sets no `valid_volumes`). Whether a workflow can still reach the Docker socket was *not* tested; the risk is discussed in Section 4.2.7.

```yaml
# Gitea Actions runner (act_runner) for git.ssw590-g1.tech, same EC2 host as Gitea.
# Non-secret. The registration token lives only in runner.env (chmod 600, emptied after registration).
# The runner publishes NO ports; it only makes outbound HTTPS calls to Gitea.
# It mounts the Docker socket to start job containers: root-equivalent on this host, so only
# BD and SD may edit workflows (see the runner and secret boundary section of the manual).
services:
  runner:
    image: gitea/act_runner:0.6.1
    container_name: gitea-runner
    restart: unless-stopped
    env_file: runner.env
    environment:
      CONFIG_FILE: /config.yaml
      GITEA_INSTANCE_URL: https://git.ssw590-g1.tech
      GITEA_RUNNER_NAME: group1-runner-1
      GITEA_RUNNER_LABELS: "manual-builder:docker://group1/texlive-node:2026.09"
    volumes:
      - runner_data:/data
      - ./config.yaml:/config.yaml:ro
      - /var/run/docker.sock:/var/run/docker.sock

volumes:
  runner_data:
```

Listing 13: Runner Compose file (non-secret; also in the repository's `config/runner/` folder). The registration token is read from a root-only file that is emptied after registration.

```yaml
# act_runner configuration (non-secret).
log:
  level: info

runner:
  file: /data/.runner
  capacity: 1          # one job at a time: the host has about 1 GB RAM
  timeout: 1h

cache:
  enabled: false       # artifacts go through Gitea, not the runner cache

container:
  network: ""          # default: a fresh network per job
  privileged: false
  force_pull: false    # the job image is local-only (built on this host); never re-download it
  force_rebuild: false
```

Listing 14: Runner configuration (non-secret): one job at a time, no cache server, no privileged jobs, never re-pull the local image.

Failed attempts and corrections. Registration took four attempts on October 19, 2026; the first three failed. Attempt 1 was rejected: Gitea answered "runner registration token not

found" and the runner retried in a loop; the container was stopped and the token file on the server was emptied. The script was changed to report how many characters it received, and attempt 2 showed the cause: only *one* character of the 40-character token arrived, because pasting into a hidden terminal prompt kept only the first character. The script was changed again to read the token one key at a time, but attempt 3 still read one character; the script stopped on the administrator's machine before sending anything. The third change reads the token from the clipboard (which holds the full 40 characters, as checked with a length-only command) and refuses to send anything shorter than 20 characters. Attempt 4 printed `Runner registered successfully`. Afterwards the token file was emptied again (0 bytes, mode 600) and the runner restarted without it. The token itself is never written in this manual.

4.2.3 Reproducible compilation (20 points)

Owner: SD. **Status:** successful build on the Gitea runner, run #2, October 20, 2026, 03:00 UTC **PASS**.

Toolchain

Table 4.4: Build toolchain

Item	Value
Job image	`group1/texlive-node:2026.09`, built from `ci/Dockerfile`
Base image	`texlive/texlive`, pinned by digest `sha256:7334b00bf8e7...3083ff` (image created September 20, 2026)
TeX	TeX Live 2026, pdfTeX 3.141592653-2.6-1.40.29, `latexmk` 4.88
Other tools in the image	Python 3 with Pygments (`minted`), `bibtex`, `makeglossaries`, `makeindex`, `git`
Added for Actions	Node.js 24 (Debian package), needed by `actions/checkout` and `actions/upload-artifact`
Main file	`devopsITManual.tex` at the repository root
Build command	`latexmk -pdf -shell-escape -interaction=nonstopmode -halt-on-error -usepretex=... devopsITManual.tex`
Expected output	`devopsITManual.pdf`, copied to `out/IT-manual+build.<run>.<sha>.pdf`

TeX is installed only in the job image, which is stored in the server's Docker engine; it is not in the Gitea container and not in the server's own operating system. The runner starts a fresh container from that image for each job and removes it afterwards.

Workflow

```yaml
# Builds the Group 1 DevOps IT manual on every push to main and publishes the PDF as an
↪  artifact.
# Runs on the team's self-managed Gitea runner; the label below must map to the job image
↪  built
# from ci/Dockerfile (TeX Live pinned by digest + Node.js for the JavaScript actions).
name: Build IT manual

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  build:
    runs-on: manual-builder
    timeout-minutes: 30
    defaults:
      run:
        shell: bash
    steps:
      - name: Check out the triggering commit
        uses: actions/checkout@v4

      - name: Record the toolchain
        run: |
          set -euo pipefail
          pdflatex --version | head -n 1
          latexmk --version | tail -n 1
          pygmentize -V
          node --version

      - name: Compute the build identifier
        id: buildid
        run: |
          set -euo pipefail
          short="$(printf '%s' "${{ gitea.sha }}" | cut -c1-7)"
          echo "short=${short}" >> "$GITHUB_OUTPUT"
          echo "suffix=+build.${{ gitea.run_number }}.${short}" >> "$GITHUB_OUTPUT"
          echo "Build identifier: +build.${{ gitea.run_number }}.${short} (attempt ${{
          ↪  gitea.run_attempt }})"

      - name: Compile the manual (fails on any LaTeX error)
        run: |
          set -euo pipefail
          latexmk -pdf -shell-escape -interaction=nonstopmode -halt-on-error \
            -usepretex='\def\BuildSuffix{${{ steps.buildid.outputs.suffix
            ↪  }}}\def\BuildRun{${{ gitea.run_number }}}\def\BuildCommit{${{
            ↪  steps.buildid.outputs.short }}}' \
            devopsITManual.tex

      - name: Verify the PDF exists and carries the build identifier
```

```
      run: |
        set -euo pipefail
        test -s devopsITManual.pdf || { echo "::error::devopsITManual.pdf is missing or
        ↪  empty"; exit 1; }
        if grep -E "undefined references|multiply defined" devopsITManual.log; then
          echo "::error::unresolved or duplicate references in the final pass"; exit 1
        fi
        grep -F "Build identifier: Version" devopsITManual.log
        grep -qF "${{ steps.buildid.outputs.suffix }}" devopsITManual.log \
          || { echo "::error::build identifier not found in the LaTeX log"; exit 1; }
        mkdir -p out
        cp devopsITManual.pdf "out/IT-manual${{ steps.buildid.outputs.suffix }}.pdf"
        ls -l out

    # upload-artifact@v4 is rejected by Gitea 1.27.3 ("GHESNotSupportedError", run #1);
    ↪  v3 is supported.
    - name: Upload the PDF artifact
      uses: actions/upload-artifact@v3
      with:
        name: it-manual-pdf
        path: out/*.pdf
        if-no-files-found: error
```

Failure detection

The build fails, and no artifact is uploaded, when:

- `pdflatex` reports any error: `-halt-on-error` stops it and `latexmk` exits non-zero;
- `devopsITManual.pdf` is missing or empty;
- the final pass still reports undefined or multiply defined references;
- the build identifier is not in the LaTeX log (row 4);
- no PDF matches the upload path (`if-no-files-found: error`).

Before pushing, the same commands were run in the same image on SD's workstation (`ci/local-build.sh`): a clean build took about 18 seconds, and a copy with an undefined command inserted exited with code 1, reported "Undefined control sequence" with the line number, and produced no PDF.

Checkout and artifact path

The runner polls Gitea over HTTPS for work. In the job container, `actions/checkout` fetches the triggering commit from `https://git.ssw590-g1.tech`, and `actions/upload-artifact` sends the PDF back to the same run over HTTPS. Gitea and the job share no filesystem.

Runs on the server

The first push (commit `1fb4e15`, run #1) compiled the manual but failed at the upload step: Gitea 1.27.3 rejects `actions/upload-artifact@v4` with "GHESNotSupportedError". Version 3 of the action is supported, so commit `a5f3446` changed only that line, and run #2

succeeded (Figure 4.5, Listing 15). The compile step took 36 seconds on the `t3.micro` server, and the whole job 1 minute 11 seconds, including 24 seconds to start the job container. The run pages and the full job logs are public, so they can be opened without an account.

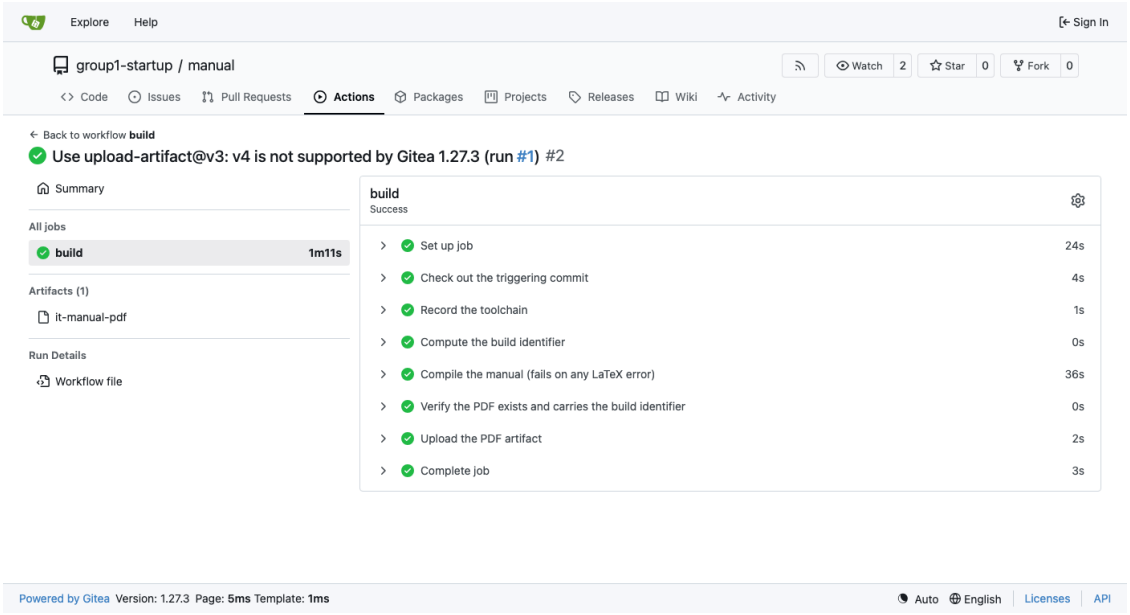


Figure 4.5: Run #2 on the Gitea runner, October 20, 2026, seen signed out: every step succeeded and the run has one artifact, `it-manual-pdf`.

```
Gitea Actions run #2, read anonymously 2026-10-20 03:09:31 UTC
commit a5f3446  event push  runner group1-runner-1  label manual-builder
started 2026-10-20T02:59:52Z  finished 2026-10-20T03:01:03Z  conclusion success
steps:
  Check out the triggering commit                        success  03:00:16 -> 03:00:20
  Record the toolchain                                   success  03:00:20 -> 03:00:21
  Compute the build identifier                           success  03:00:21 -> 03:00:21
  Compile the manual (fails on any LaTeX error)          success  03:00:21 -> 03:00:57
  Verify the PDF exists and carries the build identifier success  03:00:58 -> 03:00:58
  Upload the PDF artifact                                success  03:00:58 -> 03:01:00

Key lines from the job log (timestamps UTC):
  03:00:20 pdfTeX 3.141592653-2.6-1.40.29 (TeX Live 2026)
  03:00:20 Latexmk, John Collins, 9 March 2026. Version 4.88
  03:00:21 Pygments version 2.20.0, (c) 2006-present by Georg Brandl, Matthaus Chajdas and contributors.
  03:00:21 v24.21.0
  03:00:21 Build identifier: +build.2.a5f3446 (attempt 1)
  03:00:21 Latexmk: This is Latexmk, John Collins, 9 March 2026. Version 4.88.
  03:00:23 Build identifier: Version 1.0+build.2.a5f3446
  03:00:29 Output written on devopsITManual.pdf (65 pages, 4095419 bytes).
  03:00:39 Output written on devopsITManual.pdf (86 pages, 4226429 bytes).
  03:00:48 Output written on devopsITManual.pdf (87 pages, 4228163 bytes).
  03:00:57 Output written on devopsITManual.pdf (87 pages, 4228180 bytes).
  03:00:58 -rw-r--r--. 1 root root 4228180 Oct 20 03:00 IT-manual+build.2.a5f3446.pdf
  03:00:59 With the provided path, there will be 1 file uploaded
  03:00:59 Artifact name is valid!
  03:01:00 Total size of all the files uploaded is 4076563 bytes
  03:01:00 Artifact has been finalized. All files have been successfully uploaded!
  03:01:00 The size of all the files that were uploaded is 4076563 bytes. This takes into account any gzip
  ↪  compression use
```

```
03:01:00 Artifact it-manual-pdf has been successfully uploaded!
03:01:03 Job succeeded
```

Listing 15: Run #2: step results from the Gitea API and key lines of the job log (toolchain versions, build identifier, compile passes, artifact upload), October 20, 2026.

4.2.4 PDF version traceability (15 points)

Owner: SD. **Status:** identifier verified on the PDF downloaded from run #2 **PASS**.

The human-maintained version stays `\DocumentVersion` in `devopsITManual.tex`, changed only by a person with a history entry. The workflow appends a build suffix `+build.<run_number>.<first 7 characters of the commit>`, taken from Gitea's `gitea.run_number` and `gitea.sha` [18]. It reaches LaTeX through `latexmk -usepretex`, which defines `\BuildSuffix`, `\BuildRun`, and `\BuildCommit` before the main file is read. The main file provides `+local` as the default, so a local build is clearly unofficial. The suffix appears on the title page and in every page footer, and is written to the LaTeX log, where the workflow checks it.

Nothing is written back to the repository: the identifier exists only inside the generated PDF and the run's log, so a build cannot trigger another push. This PDF itself shows **Version 1.3+build.23.d1da656**.

Evidence

Table 4.5: One build traced from the run to the PDF

Item	Value (run #2)
Human-maintained version	`1.0` (`\DocumentVersion`)
Gitea run	#2, attempt 1 (Figure 4.5)
Triggering commit	`a5f3446`
Identifier in the job log	`Build identifier: Version 1.0+build.2.a5f3446` (Listing 15)
Artifact file name	`IT-manual+build.2.a5f3446.pdf`
Identifier printed in the PDF	`Version 1.0+build.2.a5f3446` on the title page (Figure 4.6) and in every page footer

A re-run of the same run keeps its run number; the job log also records the attempt number ("attempt 1" in Listing 15), which tells re-runs apart.

4.2.5 Downloadable artifact (10 points)

Owner: BD. **Status:** artifact of run #2 downloaded without signing in and opened, October 19–20, 2026 **PASS**.

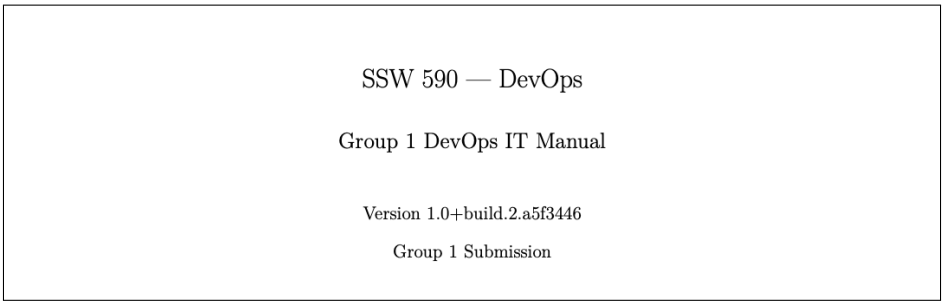


Figure 4.6: Title page of the PDF downloaded from run #2's artifact, October 20, 2026: the version reads `1.0+build.2.a5f3446`, matching run #2 and commit `a5f3446`.

Where it is. The workflow's last step uploads `out/*.pdf` as the artifact `it-manual-pdf` (the step fails if no file matches, so a missing PDF cannot pass silently). On the successful run #2 the artifact is listed under *Artifacts* in the left column of the run summary (Figure 4.2). A team member opens the `group1-startup/manual` repository, chooses *Actions*, opens the run, and clicks the artifact name; the browser downloads a zip file that holds the PDF. A direct link, which also works without signing in because the repository is public, is https://git.ssw590-g1.tech/group1-startup/manual/actions/runs/2/artifacts/it-manual-pdf.

Downloaded and opened. The download of run #2 is a 4.0 MiB zip with one file, `IT-manual+build.2.a5f3446.pdf`. Opened in a PDF viewer, its title page reads `Version 1.0+build.2.a5f3446`, the same run number and commit as the run (Figure 4.7). An anonymous download (no login, no cookie) from the administrator's machine on October 20, 2026 at 03:16 UTC returned HTTP 200, 4,087,975 bytes, content type `application/zip`, and the same file name (Listing 16).

```
Real output. Run by Claude Code on Brahmi's go-ahead from Brahmi's Windows machine, with no cookie and no
↪  login.
Purpose: show that the artifact of run #2 can be downloaded without signing in (the repository is public).
Time: 2026-10-20 03:16 UTC

$ curl -sL -o art.zip -w "HTTP %{http_code} bytes %{size_download} type %{content_type}"
↪  https://git.ssw590-g1.tech/group1-startup/manual/actions/runs/2/artifacts/it-manual-pdf
HTTP 200 bytes 4087975 type application/zip

$ unzip -l art.zip
Archive:  /tmp/anon2/art.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
  4228180  1980-00-00 00:00   IT-manual+build.2.a5f3446.pdf
---------                     -------
  4228180                     1 file
```

Listing 16: Anonymous download of the run #2 artifact, October 20, 2026, 03:16 UTC (real output): HTTP 200, a zip that contains the versioned PDF.

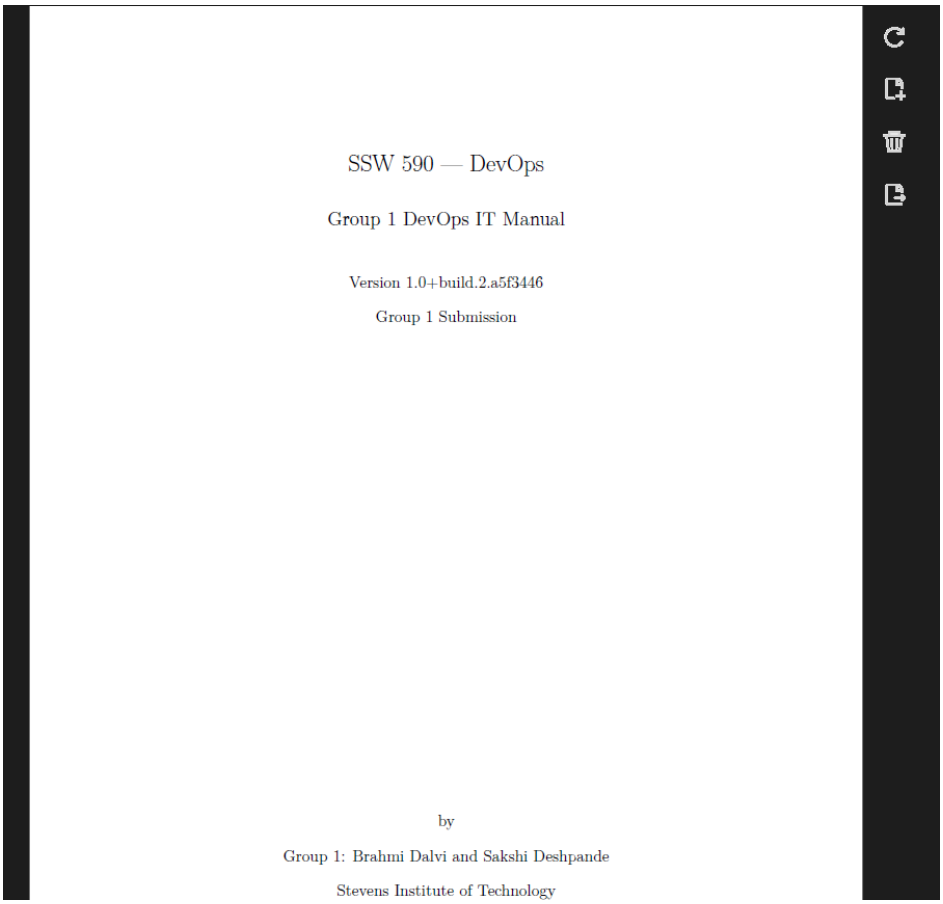


Figure 4.7: Title page of the PDF downloaded from the artifact of run #2 and opened in a viewer: `Version 1.0+build.2.a5f3446`.

Retention. No retention setting was configured by the team. The observed default is shown in the artifact's tooltip: the artifact of run #2 *expires at January 17, 2027, 22:01* (Figure 4.8), about 90 days after the run, and is 4.0 MiB. This is the observed value on this Gitea 1.27.3 installation, not a documented guarantee.

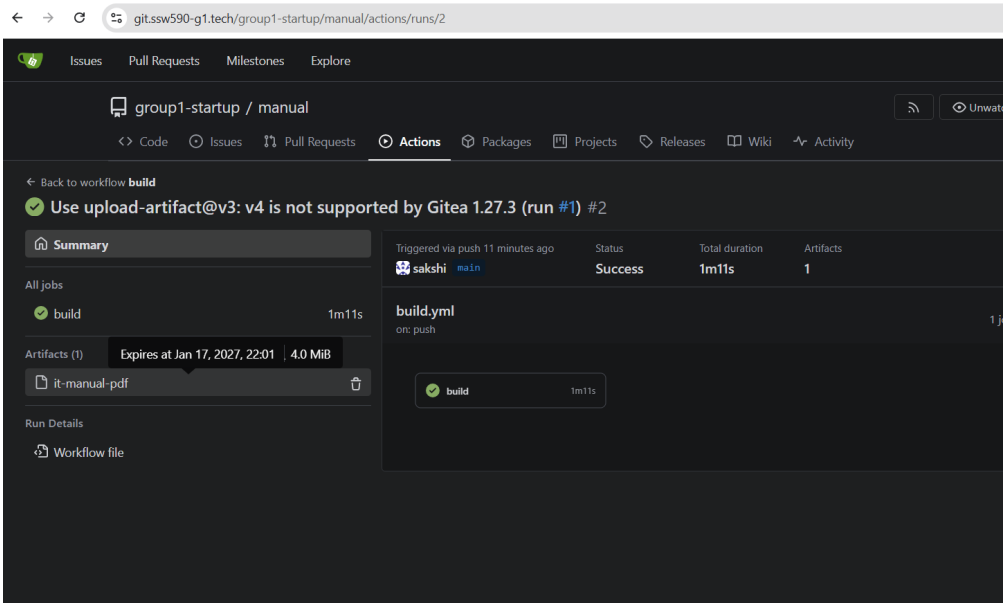


Figure 4.8: Run #2 summary with the tooltip of `it-manual-pdf`: "Expires at Jan 17, 2027, 22:01" and 4.0 MiB.

An artifact is not a backup. Artifacts expire and live on the same disk as Gitea; deleting the run or the repository removes them, and the server has no backup (the optional backup test of Chapter 2 was not attempted). The permanent copies are the LaTeX source in Git (which can rebuild any version from its commit) and the PDF that the team submits to the course. The artifact is a delivery path, not storage.

4.2.6 Change, failure, and repair evidence (10 points)

Owner: SD. **Status:** two successive changes, one controlled failure, and its repair, all built on the Gitea runner on October 20, 2026 **PASS**.

Each row is a separate push to `main`. The PDF identifier was read from the title page of the artifact downloaded from that run (Listing 18); times are UTC.

Table 4.6: Successive changes, a controlled failure, and its repair, October 20, 2026

Step	Change	Commit	Run / result	PDF identifier
Change 1	Row 3 text and evidence added	c59a784	#3 success, 03:10	1.0+build.3.c59a784
Change 2	Row 4 text and evidence added	34ae070	#4 success, 03:11	1.0+build.4.34ae070
Failure	One line \ControlledFailureForRowSix (an undefined command) added on purpose	dcdf778	#5 **failure**, 03:13	none: no PDF, no artifact
Repair	That line removed	a9636e4	#6 success, 03:13	1.0+build.6.a9636e4

Table 4.7: Controlled failure: expected and actual results

Check	Expected	Actual (run #5)
Compile step	fails at the bad line	failed: "! Undefined control sequence" at l.199 \ControlledFailureForRowSix; latexmk exit code 12
Later steps	not run	"Verify" and "Upload" skipped
Artifact	none	none (run page shows no artifact; download returns nothing)
Run status	failure	failure, in 14 seconds (Figure 4.9)
Gitea service	unaffected	unaffected: the failing commit changed only the LaTeX source, and the repair run #6 started 40 seconds later and succeeded

```
Gitea Actions run #5, read anonymously 2026-10-20 03:13:48 UTC
commit dcdf778  event push  runner group1-runner-1  label manual-builder
started 2026-10-20T03:13:02Z  finished 2026-10-20T03:13:16Z  conclusion failure
steps:
  Check out the triggering commit                          success  03:13:04 -> 03:13:06
  Record the toolchain                                     success  03:13:06 -> 03:13:07
  Compute the build identifier                             success  03:13:07 -> 03:13:07
  Compile the manual (fails on any LaTeX error)            failure  03:13:07 -> 03:13:14
  Verify the PDF exists and carries the build identifier   skipped  03:13:15 -> 03:13:15
  Upload the PDF artifact                                  skipped  03:13:15 -> 03:13:15
```

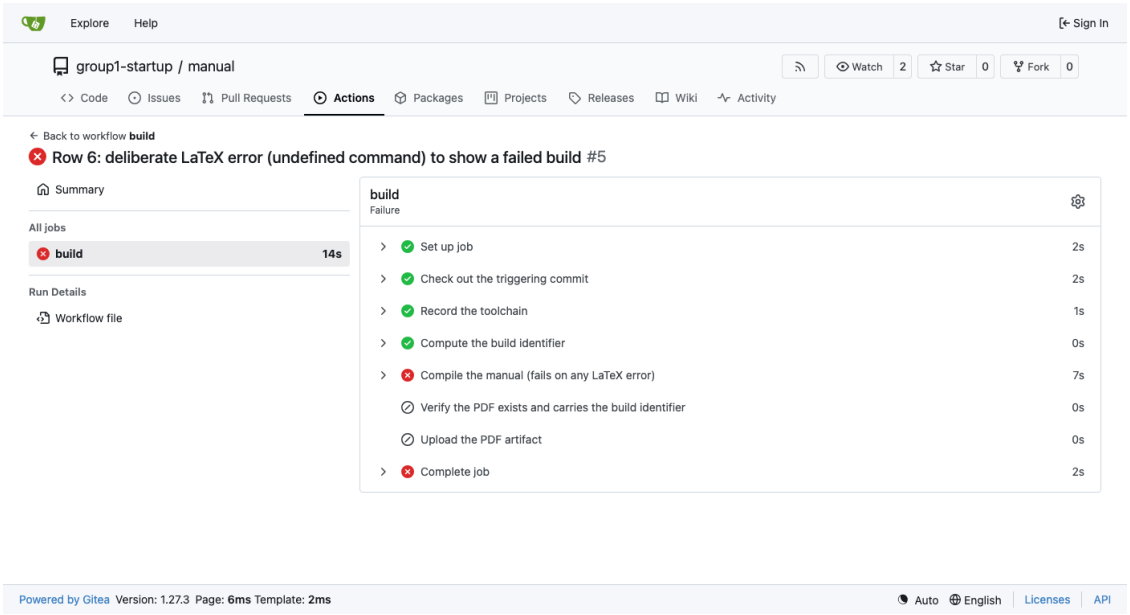


Figure 4.9: Run #5, October 20, 2026, seen signed out: the compile step failed, and the verify and upload steps were skipped.

```
Key lines from the job log (timestamps UTC):
  03:13:06 pdfTeX 3.141592653-2.6-1.40.29 (TeX Live 2026)
  03:13:07 Latexmk, John Collins, 9 March 2026. Version 4.88
  03:13:07 Pygments version 2.20.0, (c) 2006-present by Georg Brandl, Matthaus Chajdas and contributors.
  03:13:07 v24.21.0
  03:13:07 Build identifier: +build.5.dcdf778 (attempt 1)
  03:13:07 Latexmk: This is Latexmk, John Collins, 9 March 2026. Version 4.88.
  03:13:09 Build identifier: Version 1.0+build.5.dcdf778
  03:13:14 ! Undefined control sequence.
  03:13:14 l.199 \ControlledFailureForRowSix
  03:13:14 !  ==> Fatal error occurred, no output PDF file produced!
  03:13:14 Failure - Main Compile the manual (fails on any LaTeX error)
  03:13:15 exitcode '12': failure
  03:13:16 Job failed
```

Listing 17: Run #5: step results and the LaTeX error in the job log, October 20, 2026.

```
Runs of group1-startup/manual, read anonymously (no token) 2026-10-20 03:15:08 UTC
Artifact checked by downloading it-manual-pdf and reading the PDF title page.

run  commit   result   started (UTC)         artifact file                    title-page version
#1   1fb4e15  failure  2026-10-20 02:56:10   (none)                           -
#2   a5f3446  success  2026-10-20 02:59:52   IT-manual+build.2.a5f3446.pdf    Version 1.0+build.2.a5f3446
#3   c59a784  success  2026-10-20 03:10:16   IT-manual+build.3.c59a784.pdf    Version 1.0+build.3.c59a784
#4   34ae070  success  2026-10-20 03:11:45   IT-manual+build.4.34ae070.pdf    Version 1.0+build.4.34ae070
#5   dcdf778  failure  2026-10-20 03:13:02   (none)                           -
#6   a9636e4  success  2026-10-20 03:13:56   IT-manual+build.6.a9636e4.pdf    Version 1.0+build.6.a9636e4

Commit messages:
  1fb4e15  Initial import of the Group 1 IT manual with the Gitea Actions build
  a5f3446  Use upload-artifact@v3: v4 is not supported by Gitea 1.27.3 (run #1)
  c59a784  Row 3: record the first server builds (runs #1 and #2) with evidence 61-62
  34ae070  Row 4: trace run #2 to its PDF identifier (evidence 63)
```

```
dcdf778  Row 6: deliberate LaTeX error (undefined command) to show a failed build
a9636e4  Row 6: repair the deliberate LaTeX error from dcdf778
```

Listing 18: Runs #1–#6 with commit, result, artifact file name, and the version printed on the downloaded PDF's title page, read without an account, October 20, 2026.

4.2.7 Runner and secret boundary (5 points)

Owner: BD. **Status:** described from the observed configuration; the Docker-socket limits below were not tested **PASS**.

Who can edit the workflow. A workflow is code that the runner executes, so whoever can push to `group1-startup/manual` controls what runs. Two people can: BD (organization owner) and SD (collaborator with *Write*, Figure 4.10). The team `developers` also has the Actions unit, and registration on this Gitea is closed (`DISABLE_REGISTRATION`), so no outsider can create an account, fork the repository, or open a pull request. **Limitation:** `main` of this repository has no branch protection (unlike `twintrack`), so either member can push a workflow change that runs immediately without review; this is accepted for a two-person team and was not changed.

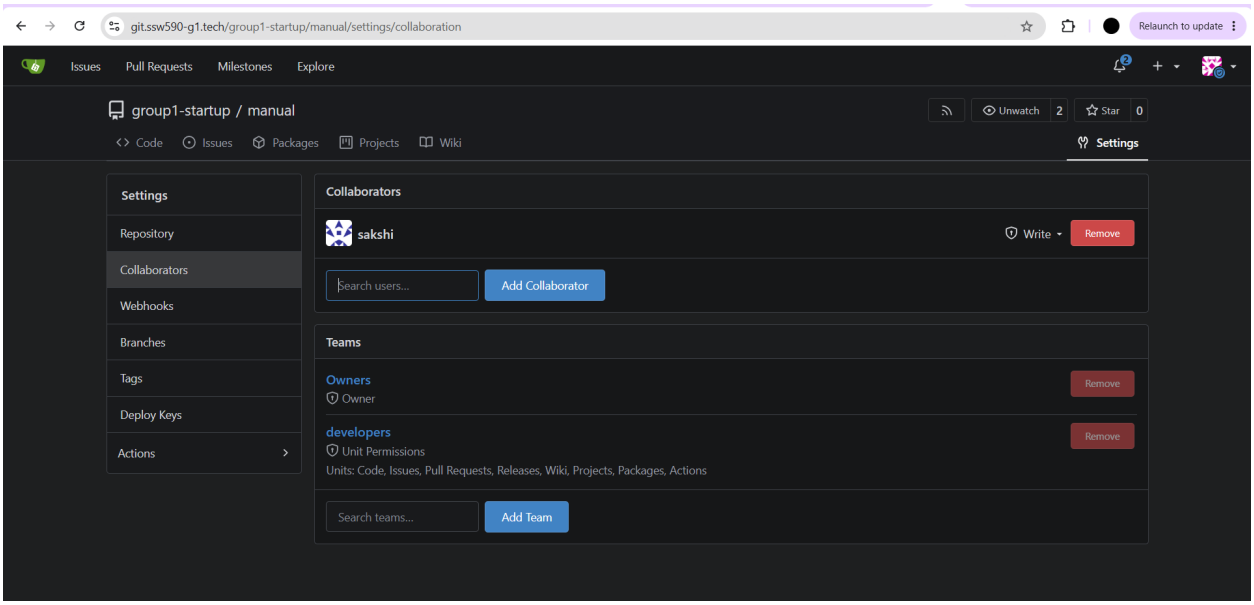


Figure 4.10: Settings, Collaborators of `group1-startup/manual`, October 19, 2026: `sakshi` with Write; teams `Owners` (Owner) and `developers` (unit permissions Code, Issues, Pull Requests, Releases, Wiki, Projects, Packages, Actions).

Where the secrets are.

- *Runner registration token* (hint only): shown in Gitea's Site Administration, Actions, Runners page; BD copies it into the clipboard and a script on BD's machine sends it over SSH to a root-only file on the server (mode 600). After a successful registration the file was emptied (0 bytes) and the runner was restarted without it; the registration itself is kept in the Docker volume. The token is not in the repository, the workflow, the logs, or this PDF. If it leaks, reset it on the same page.
- *Repository secrets:* none. The workflow uses no secret and no access token of its own.
- *Gitea access token* used once to create the repository: deleted right after use.

Docker privilege risk. The runner container mounts the host's Docker socket (Listing 13) so that it can create job containers. Anyone who controls the runner process can start any container on the host, which also holds Gitea's data volume; that is root-equivalent on this host, the same host that serves the team's Gitea [16]. The jobs themselves are not given the socket: the configuration sets `privileged: false` and no `valid_volumes`, which by design keeps a workflow from mounting host paths. *This was not tested*, so the manual does not claim it as verified. Compensating controls: only two trusted people can edit workflows, there are no outside accounts, no repository secrets exist to steal, and the runner publishes no port. A stronger design would use a separate runner host, which was not chosen because of cost.

Why untrusted workflow changes must not get privileged access or secrets. A pull request from a stranger could rewrite the workflow to run any command in the job, read every secret the job can see, and, with Docker access, attack the host and the other containers. Therefore this runner is registered only for this repository's trusted authors, holds no secrets, and must never be given privileged mode, host mounts, or secrets while outsiders can propose workflow changes.

Third-party code in the build. The workflow's two steps `actions/checkout` and `actions/upload-artifact` are fetched from github.com at the time of each run (the *Set up job* log, Figure 4.4, shows the clone), selected by a version tag that its owner can move. This is a supply-chain dependency that was accepted for simplicity; pinning each action to a commit hash is a possible improvement.

Non-secret configuration. The runner's Compose file and configuration are printed in Section 4.2.2 (Listings 13 and 14) and are kept in the repository under `config/runner/`. Hosts and Passwords tables: **updated** for the runner (Chapter 2).

4.2.8 Manual quality and controls (10 points)

Owner: SD. **Status:** final review of this chapter on October 20, 2026 **PASS**.

Table 4.8: Manual quality controls for this chapter, checked October 20, 2026

Control	Status	Where / how checked
Manual version increased	**PASS**	1.0 → 1.1, dated October 20, 2026; the build suffix is added automatically, the version itself only by a person
Dated history entry linking this chapter	**PASS**	Version 1.1 entry in Table 1
Hosts and Passwords tables	**PASS**	**Changed** (by BD): new Hosts row for the runner `gitea-runner` (same EC2 host, no published ports); new Passwords row for the runner registration token (hint only, not stored); SD's Gitea row now records her Write access to `group1-startup/manual`. Named in the history entry
Citations	**PASS**	Gitea Actions variables, runner labels, runner Docker installation, configuration (artifact retention), Let's Encrypt and Caddy sources in the Bibliography
Glossary and index terms	**PASS**	Gitea Actions, runner, artifact, job container in the Glossary; index entries for these and for the build identifier and controlled failure
Evidence inside the PDF, dated and sanitized	**PASS**	Evidence 50–58 (BD) and 60–66 (SD) as figures and listings. Scripted scan of every chapter source, printed configuration and command-output file for private keys, tokens, AWS keys and account numbers, passwords, e-mail addresses, and IPv4 addresses found only expected public or private addresses; every screenshot was inspected by eye
Links	**PASS**	Repository, workflow, runs, commits, and artifact links are public and open without an account
Failed-build troubleshooting procedure	**PASS**	Below; checked against the real failure of run #5
Build	**PASS**	This PDF was built by the workflow itself; its identifier is on the title page and in every footer

Failed-build troubleshooting

1. Open the failed run in Gitea (*Actions* tab) and expand the first red step.
2. Compile step failed: search the log for the first line starting with `!`; the following `l.<n>` gives the file line. Fix it and run `ci/local-build.sh` before pushing again.
3. "PDF is missing" or "build identifier not found": the compile ended early or

`-usepretex` was changed; compare with the workflow in Section 4.2.3.

4. Job never starts ("waiting for a runner"): the runner is offline or its label is not `manual-builder`; ask the runner owner (Section 4.2.2).
5. Checkout or upload fails: the runner cannot reach `https://git.ssw590-g1.tech`; check DNS and the Caddy container. An upload error naming "GHES" means an artifact action newer than Gitea supports (run #1).

Check against a real failure. For run #5, steps 1 and 2 led directly to the cause: the red step was "Compile the manual", and the log showed `! Undefined control sequence.` followed by `l.199 \ControlledFailureForRowSix` (Listing 17). Removing that line (commit `a9636e4`) gave the green run #6. Step 5 matches run #1.

Failed tests and corrective actions

Table 4.9: Failed tests and corrective actions in this chapter

When / who	What failed	Correction
October 19, 2026, BD	Runner registration, attempts 1–3: the token arrived truncated to one character (Section 4.2.2)	Read the token from the clipboard with a length check; attempt 4 registered; token file emptied afterwards
October 20, 2026, SD	Run #1: `upload-artifact@v4` rejected by Gitea 1.27.3 ("GHESNotSupportedError"; Section 4.2.3)	Switched to `@v3` (commit `a5f3446`); run #2 green
October 20, 2026, SD	The first screenshot tool run captured an empty page (the run page draws itself with JavaScript after loading)	Waited for rendering before the capture; no evidence used the empty image
October 20, 2026, SD	Evidence 63 (title page) rendered as a black box in the PDF of run #7: transparent background	Re-rendered on white (commit `1eaddcc`); run #8 checked
October 20, 2026, SD	Run #5: *controlled* failure, intended (Section 4.2.6)	Repaired by `a9636e4`; run #6 green

Chapter 5

Continuous Integration with Jenkins

5.1 Overview and Status

This chapter answers the course manual's Chapter 4 [1]: a Gitea pull request on the startup's product repository `twintrack` starts a Jenkins pipeline, Jenkins reports a named status on the proposed commit, and Gitea refuses the merge while that required check fails. Jenkins runs as a controlled service on the same AWS EC2 instance as Gitea. The eight numbered subsections match the eight rubric rows in order. Evidence is dated November 2, 2026 (US Eastern) unless stated otherwise; the Jenkins log times are in UTC and fall on November 3.

Table 5.1: Rubric rows, owners, and status

Subsection	Points	Owner	Status
Jenkins deployment and persistence	10	BD	**PASS**
HTTPS, access, and firewall	15	BD	**PASS**
Gitea trigger and traceability	15	BD	**PASS**
Versioned pipeline and tests	15	SD (draft BD, checked SD)	**PASS**
Required merge gate	15	SD (draft BD, checked SD)	**PASS**

Subsection	Points	Owner	Status
Failure, repair, and review	10	SD (draft BD, checked SD)	**PASS**
Credentials and build boundary	10	BD	**PASS**
Manual quality and controls	10	SD (draft BD, checked SD)	**PASS**

Key facts. Jenkins: https://ci.ssw590-g1.tech/login. Product repository: group1-startup/twintrack; pull request #10 closes issue #9. Required status check on `main`: **`twintrack/pipeline/pr-main`**. Deployment files: group1-startup/infra, folder jenkins.

5.2 Required Subsections

5.2.1 Jenkins deployment and persistence (10 points)

Owner: BD. **Status:** deployed, restarted, and verified on November 2, 2026 **PASS**.

What runs. Jenkins is an official LTS image, `jenkins/jenkins:2.580.2-lts-jdk21` from Docker Hub [19]. The tag is pinned to an exact version (it was the newest 2.580 LTS tag, published October 28, 2026, about 244 MB for amd64) and was not `lts` or `latest`. A thin local `Dockerfile` adds `python3`, because the course's example `Jenkinsfile` runs Python tests on the controller executor, and installs the plugins at build time so the plugin versions are part of the image (Listing 21). The file set is in the `infra` repository, folder jenkins; the sources are also printed below. No password, token, or key is in any of them.

```
# Jenkins controller for the Group 1 startup (Assignment 6).
# Official Jenkins LTS image, pinned to an exact version, plus python3 because the course's example
# Jenkinsfile runs `python3 -m unittest` on the controller executor (decision G6: builds on the controller).
# Plugins are installed at build time so the versions are recorded in the image, not downloaded at run time.
FROM jenkins/jenkins:2.580.2-lts-jdk21

USER root
RUN apt-get update \
 && apt-get install -y --no-install-recommends python3 \
 && rm -rf /var/lib/apt/lists/*
USER jenkins

COPY plugins.txt /usr/share/jenkins/ref/plugins.txt
```

```
RUN jenkins-plugin-cli --plugin-file /usr/share/jenkins/ref/plugins.txt
```

Listing 19: `Dockerfile` of the Jenkins image: official LTS image, `python3`, and four plugins (their dependencies are resolved at build time).

```
# Jenkins for the Group 1 startup, on the same host as Gitea and Caddy (decision G1: same server, capped
↪  memory).
# Non-secret. Start from ~/jenkins on the server: docker compose up -d --build
# - NO `ports:` on purpose: neither 8080 nor the agent port 50000 is published. Caddy reaches Jenkins by
↪  service name
#   on the shared Docker network, and Gitea's webhook reaches it on that private network.
# - The memory limits protect Gitea: if Jenkins outgrows them, only the Jenkins container is killed.
services:
  jenkins:
    build: .
    image: group1/jenkins-python:2.580.2
    container_name: jenkins
    restart: unless-stopped
    environment:
      # Heap capped for a ~1 GB host; the inbound agent port is disabled (no agents are used).
      JAVA_OPTS: "-Xms128m -Xmx384m -Djenkins.model.Jenkins.slaveAgentPort=-1"
      JENKINS_OPTS: "--httpPort=8080"
    mem_limit: 640m
    memswap_limit: 1g
    volumes:
      - jenkins_home:/var/jenkins_home      # jobs, settings, credentials: persists across container
      ↪  replacement
    networks:
      - web

networks:
  web:
    external: true
    name: gitea_web                          # the network Gitea and Caddy already share

volumes:
  jenkins_home:
```

Listing 20: Compose file: no `ports:` on purpose, a 640 MiB memory limit, a 384 MB Java heap, a named volume, and the Docker network that Gitea and Caddy already share.

Table 5.2: Jenkins deployment record

Property	Value
Image and versions	`group1/jenkins-python:2.580.2`, built on the host from `jenkins/jenkins:2.580.2-lts-jdk21`; Jenkins 2.580.2, OpenJDK 21.0.13, Python 3.13.5; 547 MB (Listing 21)
Key plugins (versions)	`gitea` 286.v0740a_413c1eb_, `git` 5.11.0, `workflow-aggregator` (Pipeline) 608.v67378e9d3db_1, `matrix-auth` 3.3; the complete list of 60 installed plugins with versions is in Listing 21
Host capacity	AWS EC2 `t3.micro`: 913 MB RAM, 2 GB swap, 20 GB disk (about 9.3 GB free). The same host runs Gitea, Caddy, and the Assignment 5 runner

Property	Value
Limits	Container limit 640 MiB (swap up to 1 GB); Java heap `-Xms128m -Xmx384m`; if Jenkins outgrows its limit only the Jenkins container is killed, not Gitea
Persistent volume	Named volume `jenkins_jenkins_home` mounted at `/var/jenkins_home`: settings, accounts, credentials, jobs, and build history
Restart policy	`restart: unless-stopped`
Network	Joins the existing `gitea_web` network; **no published ports**, so neither 8080 nor the agent port 50000 exists on the host
Deployment command	`docker compose up -d --build` in `~/jenkins` on the server

Capacity decision and first start. The server is small, so the decision to place Jenkins beside Gitea (instead of resizing or adding an instance) was made on October 29, 2026 with a rule: resize only if Jenkins is killed for memory, swap stays above about 1 GB, or Gitea slows. The image was built with a watchdog that would abort at 1 GB of swap or an unhealthy Gitea; it did not trigger (lowest free memory 131 MB, swap peak 147 MB). Jenkins started and logged "fully up and running" about 25 seconds after the start, peaking at 408 MiB of its 640 MiB limit, and once settled the machine had about 98 MB of RAM free with swap absorbing the rest (Listing 22). It fits, but with little headroom; later checks during builds showed 91–213 MB free and swap use between 187 and 531 MB, with Gitea healthy throughout.

Restart test. On November 2, 2026 the container was restarted with `docker compose restart jenkins`. The two named accounts, the global configuration (which holds the matrix security settings and the Jenkins URL), the credentials store, the multibranch job, and the build history (builds 1 to 5) have the *same checksums* before and after, and the service answered again about 12.6 seconds later (log line "fully up and running" at 00:44:03 UTC after the restart at 00:43:50 UTC); anonymous `/login` returned HTTP 200 and the API stayed refused (Listing 23). A first reading of "9 seconds" was wrong (its log filter matched the previous start) and is corrected in that file. **This is a persistence test, not a backup test:** the data stays on the same server disk, and the optional backup and restore of Chapter 3 was not attempted.

```
Real output from the server (ssh ec2-user@git.ssw590-g1.tech), Oct 29, 2026 evening US Eastern (00:12-00:14
↪  UTC Oct 30). Run by Claude Code on Brahmi's go-ahead.
Nothing was started: this is the image BUILD and a read-only inspection with throwaway containers. Jenkins
↪  itself is not running yet.

=== Build (docker build -t group1/jenkins-python:2.580.2 ~/jenkins; Dockerfile and plugins.txt are in the
↪  repository, assignment-6/jenkins/) ===
Base image: jenkins/jenkins:2.580.2-lts-jdk21 (pinned). Added: python3 (apt). Plugins installed at build time
↪  from plugins.txt: gitea, workflow-aggregator, git, matrix-auth.
Result: BUILD_EXIT=0; image group1/jenkins-python:2.580.2, 547MB.
```

```
=== Memory watchdog during the build (every 10 s; abort rule: swap above 1000 MB or Gitea not healthy) ===
00:12:32 start: availMB=445 swapMB=114
00:12:32 swapMB=114 availMB=444 gitea=healthy
00:12:53 swapMB=127 availMB=326 gitea=healthy
00:13:03 swapMB=147 availMB=131 gitea=healthy
BUILD_EXIT=0
(Lowest free memory seen: 131 MB; highest swap: 147 MB; Gitea stayed healthy; the abort rule never fired.)

=== Inside the image ===
python3: Python 3.13.5
Jenkins: 2.580.2
Java: openjdk version "21.0.13" 2026-10-20 LTS

=== Plugins installed in the image (60) ===
apache-httpcomponents-client-4-api 4.5.14-269.vfa_2321039a_83
asm-api 9.11.1-226.vff833a_e4c58e
authentication-tokens 1.144.v5ff4a_5ec5c33
bouncycastle-api 2.30.1.86.2-310.v4655f6742752
branch-api 2.1303.v9f3b_95dc329d
caffeine-api 3.3.1-214.v18eb_56b_34920
cloudbees-folder 6.1107.v31f8955765c8
commons-lang3-api 3.21.0-122.v6a_ea_4fd34049
commons-text-api 1.15.0-218.va_61573470393
credentials-binding 728.v902a_273b_8947
credentials 1514.vb_01eb_014e7d4
display-url-api 2.217.va_6b_de84cc74b_
durable-task 686.v80ff80875b_82
git-client 6.6.1
git 5.11.0
gitea 286.v0740a_413c1eb_
gson-api 2.15.0-212.v085f60a_8093b_
handy-uri-templates-2-api 2.1.8-38.vcea_5d521d5f3
instance-identity 203.v15e81a_1b_7a_38
ionicons-api 94.vcc3065403257
jackson-annotations2-api 2.22-19.v10a_a_582ea_26e
jackson2-api 2.22.4-449.v4ee9f4ec4123
jakarta-activation-api 2.1.4-1
jakarta-mail-api 2.1.5-1
jakarta-xml-bind-api 4.0.9-19.v2b_a_5b_44d9a_1c
javax-activation-api 1.2.0-8
jaxb 2.3.9-143.v5979df3304e6
joda-time-api 2.15.1-211.ve499f52b_da_33
json-api 20260814-226.v20f9685d642c
mailer 534.v1b_36f5864073
matrix-auth 3.3
mina-sshd-api-common 2.19.0-192.v2b_a_7b_2c1dc71
mina-sshd-api-core 2.19.0-192.v2b_a_7b_2c1dc71
pipeline-build-step 601.v6d4c6d1a_9dc7
pipeline-groovy-lib 822.vc95f70dea_db_7
pipeline-input-step 560.v56198a_642157
pipeline-milestone-step 152.v6e22b_8cfc66c
pipeline-model-api 2.2293.v6e7193cec599
pipeline-model-definition 2.2293.v6e7193cec599
pipeline-model-extensions 2.2293.v6e7193cec599
pipeline-stage-step 345.va_96187909426
pipeline-stage-tags-metadata 2.2293.v6e7193cec599
plain-credentials 199.v9f8e1f741799
scm-api 728.vc30dcf7a_0df5
script-security 1436.v1082791135e4
snakeyaml-api 2.7-152.vc672728cf536
ssh-credentials 372.va_250881b_08cd
structs 362.va_b_695ef4fdf9
variant 70.va_d9f17f859e0
woodstox-core-api 7.3.0-16.v017f30ed004c
workflow-aggregator 608.v67378e9d3db_1
workflow-api 1429.v7a_1b_efb_a_f7c5
workflow-basic-steps 1129.v60e3c7e6654d
```

```
workflow-cps 4395.v738456a_5da_6b_
workflow-durable-task-step 1479.v56e587f413a_7
workflow-job 1622.vc41c189cb_e5f
workflow-multibranch 842.v3a_b_59b_57b_e6e
workflow-scm-step 487.v41313a_96b_65d
workflow-step-api 724.v538c2362b_dfb_
workflow-support 1028.vc22910142650

=== Server after the build ===
Mem:  913 total, 369 used, 110 free, 433 buff/cache, 398 available (MB)
Swap: 2047 total, 135 used (MB)
gitea-runner Up 9 days; gitea Up 2 weeks (healthy); caddy Up 2 weeks
```

Listing 21: Image build on the server (exit code 0, 547 MB), memory watchdog, and the 60 plugins with exact versions, October 29, 2026 (real output).

```
Real output from the server (ssh ec2-user@git.ssw590-g1.tech) and from Brahmi's Windows machine, Oct 29, 2026
↪  evening US Eastern (00:22-00:26 UTC Oct 30). Run by Claude Code on Brahmi's go-ahead.
Jenkins was started with `docker compose up -d` in ~/jenkins (compose file:
↪  assignment-6/jenkins/docker-compose.yml). The startup log was NOT printed: it contains a one-time admin
↪  password.

=== First start, watchdog every 10 s (abort rule: swap above 1000 MB or Gitea not healthy) ===
00:22:25 before: availMB=438 swapMB=133
00:22:38 swapMB=141 availMB=247 gitea=healthy jenkins=running mem=[270MiB / 640MiB] fullyUp=no
00:22:50 swapMB=152 availMB=88 gitea=healthy jenkins=running mem=[407.8MiB / 640MiB] fullyUp=yes
READY
(About 25 seconds from start to "Jenkins is fully up and running".)

=== Container state ===
jenkins Up 31 seconds; gitea-runner Up 10 days; gitea Up 2 weeks (healthy); caddy Up 2 weeks
restarts=0 oomkilled=false memlimit=671088640 (640 MiB)
Published ports of the jenkins container: none (empty)

=== Settled state, about 2 minutes after start ===
Mem:  913 total, 671 used, 61 free, 180 buff/cache, 98 available (MB)
Swap: 2047 total, 176 used, 1871 free (MB)
jenkins 390.4MiB / 640MiB cpu=0.15%
gitea-runner 10.99MiB / 913.1MiB cpu=0.00%
gitea 81.02MiB / 913.1MiB cpu=0.35%
caddy 11.19MiB / 913.1MiB cpu=0.00%
load average: 0.91 0.28 0.11

=== Private-network reachability (from the caddy container to the jenkins service name) ===
wget http://jenkins:8080/login  ->  HTTP/1.1 200 OK

=== Raw port 8080 from outside the server (Test-NetConnection from Brahmi's computer, public name
↪  git.ssw590-g1.tech) ===
port 8080 TcpTestSucceeded: False

Notes:
- Only about 98 MB of RAM was free once Jenkins had settled; swap absorbed the rest. It fits, but with little
↪  headroom. The decision "same server, capped memory" stays under review: resize only if Jenkins is killed,
↪  swap stays high, or Gitea slows during a build.
- A (failed) quoting error in one of the checking commands printed an unrelated shell message; it did not
↪  affect any service.
- The inbound agent port 50000 is not published (no ports at all) and the Java property disabling it was set
↪  in JAVA_OPTS; it was not separately probed from outside yet.
```

Listing 22: First start: time to ready, memory, no published ports, private reachability from Caddy, and raw port 8080 closed from outside, October 29, 2026 (real output).

```
Real output, Nov 2, 2026 about 7:44 PM US Eastern (00:43-00:45 UTC Nov 3). Run by Claude Code on Brahmi's
↪  go-ahead over SSH on the server. Nothing was deleted: the restart used `docker compose restart jenkins` in
↪  ~/jenkins (the named volume was not touched).
Purpose (course manual, Chapter 4, row 1): show that Jenkins' settings, accounts, credentials, job and build
↪  history survive a container restart. This is a persistence test, NOT a backup test: the data stays on the
↪  same server disk.
SANITIZED: file checksums and names are shown, not contents; the credentials file content is never shown.
↪  Jenkins user directory names carry a hash suffix that is cut off in the summary below (the full names are
↪  in the original output).

=== BEFORE the restart (2026-11-03 00:43:45 UTC) ===
container started at : 2026-10-30T00:22:26 UTC   restart count 0   (it had been up for 3 days)
volume               : jenkins_jenkins_home -> /var/jenkins_home
users in the volume  : brahmi, sakshi
job twintrack config : sha256 prefix 4136a7eb7f1d0263
global config.xml    : sha256 prefix df12215633771922   (matrix security lives here)
credentials.xml      : sha256 prefix 2568f5ee03b9865e   (content not shown)
PR-10 builds on disk : 1 2 3 4 5, nextBuildNumber = 6
Jenkins URL setting  : https://ci.ssw590-g1.tech/
anonymous /login: HTTP 200      anonymous /api/json: HTTP 403
memory               : available 134 MB, swap used 531 MB

=== docker compose restart jenkins ===
Container jenkins Restarting / Started

=== AFTER the restart (taken while Jenkins was still starting; /login answered 503 at that moment) ===
container started at : 2026-11-03T00:43:50 UTC   restart count 0 (a `restart` command, not a crash)
volume               : jenkins_jenkins_home -> /var/jenkins_home      (same named volume)
users in the volume  : brahmi, sakshi                                  (same)
job twintrack config : sha256 prefix 4136a7eb7f1d0263                  (identical)
global config.xml    : sha256 prefix df12215633771922                  (identical)
credentials.xml      : sha256 prefix 2568f5ee03b9865e                  (identical)
PR-10 builds on disk : 1 2 3 4 5, nextBuildNumber = 6                  (identical)
Jenkins URL setting  : https://ci.ssw590-g1.tech/                      (identical)

=== Readiness, measured properly with a second command ===
container StartedAt (this start): 2026-11-03T00:43:50.68 UTC
"Jenkins is fully up and running" logged at 2026-11-03 00:44:03.28 UTC  ->  about 12.6 seconds after the
↪  restart
anonymous /login : HTTP 200      anonymous /api/json (must stay refused): HTTP 403      anonymous / : HTTP 403
memory: available 213 MB, swap used 187 MB; gitea Up 19 hours (healthy), caddy Up 20 hours, gitea-runner Up 2
↪  weeks

=== Correction ===
The first version of the test script printed "Jenkins fully up again: yes after 9 s". That reading was WRONG:
↪  its log filter used a bad time window and matched the "fully up" line of the PREVIOUS start (Oct 29/30).
↪  The 503 in the same output shows Jenkins was still starting. The figure above (about 12.6 s) comes from
↪  the second command, which only searches the log of the current start.

=== Result ===
Settings, the two named accounts, the credentials store, the multibranch job and its build history are
↪  byte-for-byte identical after the restart, and the service answers again in about 13 seconds. This
↪  verifies persistence of /var/jenkins_home across a container restart; it is not a backup or restore test.
```

Listing 23: Restart test: state before and after (identical checksums), readiness time, and the correction of a wrong first reading, November 2, 2026 (real output).

5.2.2 HTTPS, access, and firewall (15 points)

Owner: BD. **Status:** live and tested from outside, IPv4 only **PASS**.

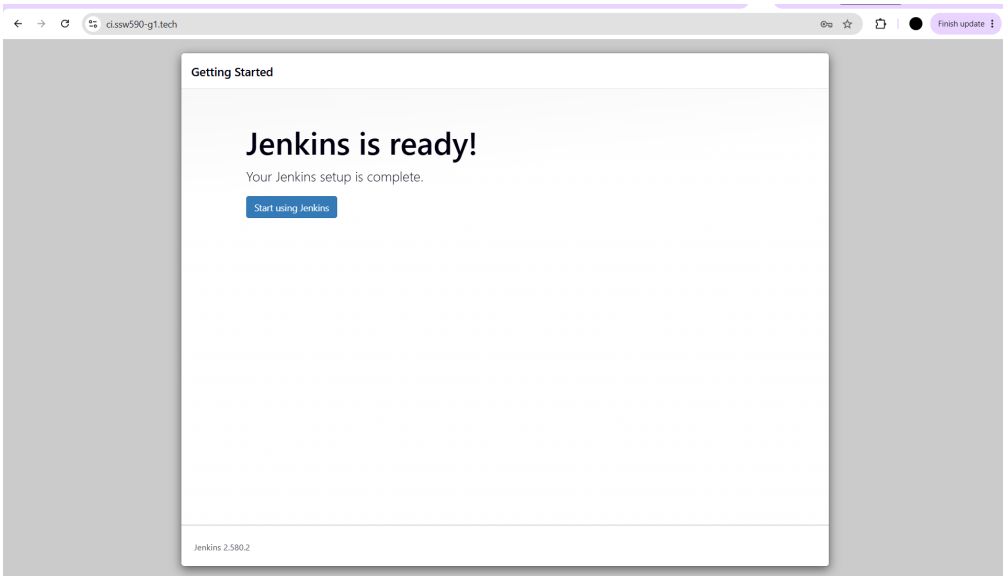


Figure 5.1: First-time setup finished, November 2, 2026: "Jenkins is ready! Your Jenkins setup is complete", Jenkins 2.580.2. The one-time unlock password was read on the server and is not shown.

Login page and the instructor's access path. The complete, working URL of the Jenkins login page is `https://ci.ssw590-g1.tech/login`. **No account is needed to open it.** Caddy lets *anyone* open exactly the login page and the few static files it needs; every other path (`/`, the dashboard, the API) answers a plain *Forbidden* unless the request comes from the team's approved address. So the instructor opens the URL in any browser and sees the sign-in page (Figure 5.2); to inspect the rest of Jenkins a team member signs in from the approved address. *This check shows that the web service is available; it does not show that a pipeline works*, which is shown in the following subsections. The phone test in Figure 5.3 (left and right images) was made on mobile data, outside the team address: the Jenkins page opened without an account (it was still the first-time unlock screen then) and `/` answered *Forbidden*.

DNS and certificate. An A record `ci.ssw590-g1.tech` points to the same Elastic IP as the Gitea host (TTL 300, created November 1, 2026; Figure 5.4); no AAAA record exists. Caddy obtained a Let's Encrypt certificate for the new name automatically on its first request (TLS-ALPN-01 on port 443). It is valid from **November 2, 2026 04:04 UTC to January 31, 2027 04:04 UTC**, issuer `CN=YE2, O=Let's Encrypt`, with a valid chain (Listing 24). HTTP is redirected to HTTPS with a 308. **Renewal** uses the same automatic mechanism and the same Caddy instance and storage volume as the Gitea certificate (Chapter 3); the staging dry-run described there covers it. The first real renewal of this certificate (due before January 31, 2027) has not happened yet and is not claimed.

Caddy, the Jenkins URL, and proxy settings. Caddy terminates TLS and proxies to the container by service name (Listing 24 shows the change and the new site block); it

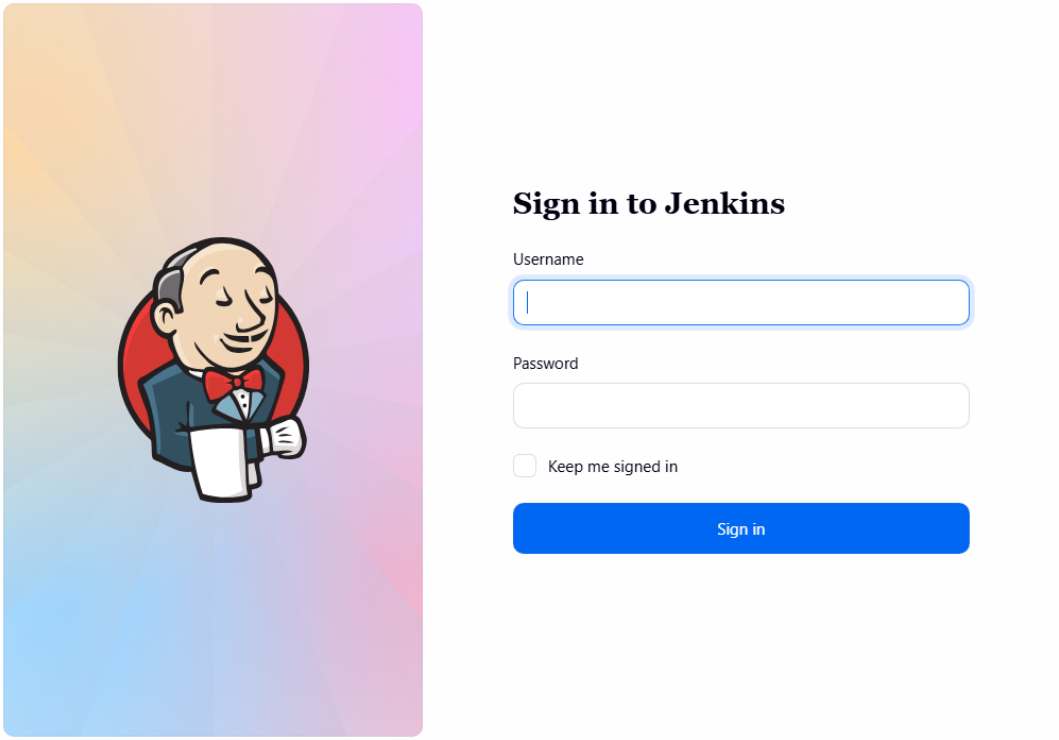


Figure 5.2: The public login page `https://ci.ssw590-g1.tech/login` on November 2, 2026 (captured with headless Chrome from the administrator's computer): "Sign in to Jenkins", username, password, and Sign in.

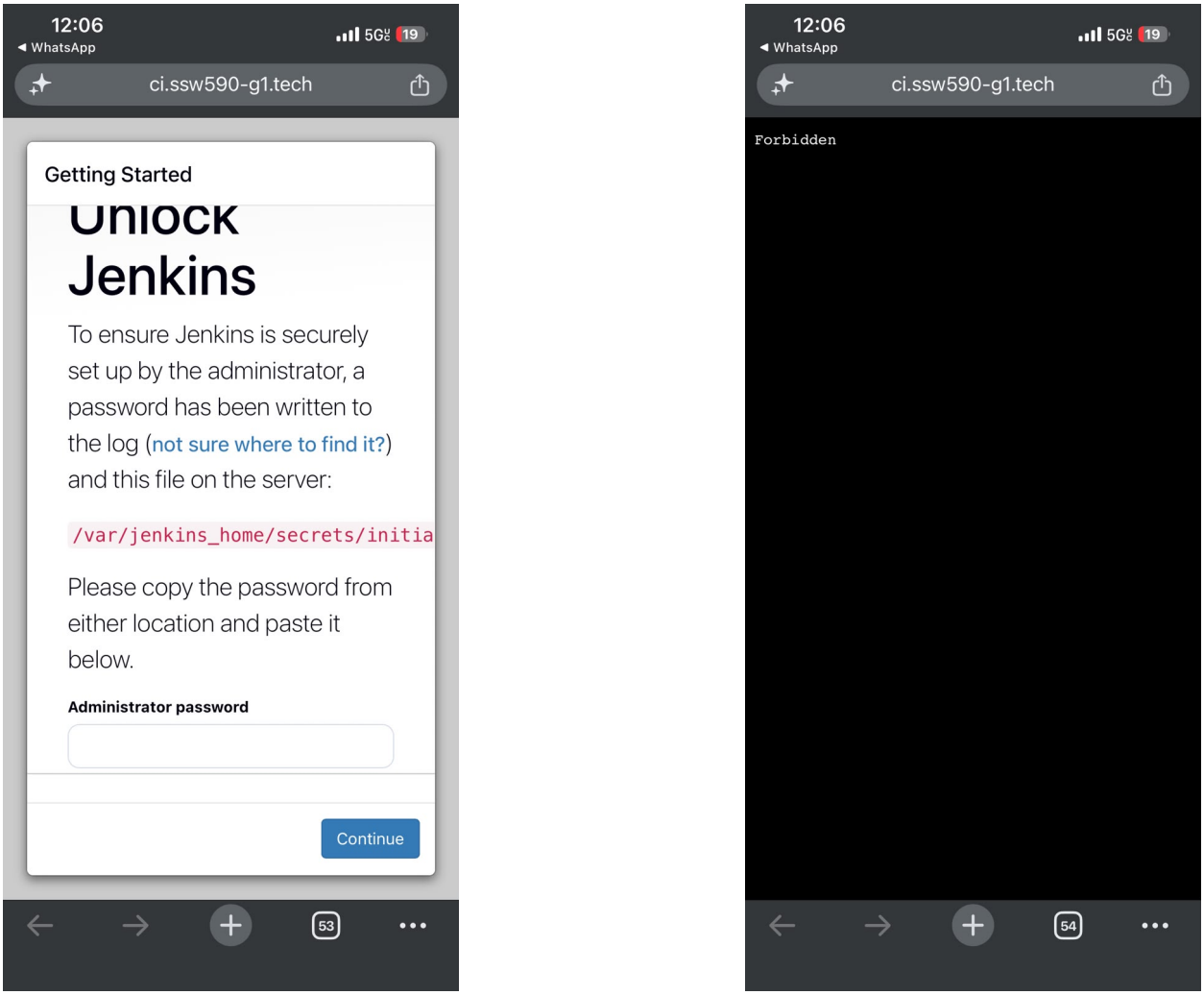


Figure 5.3: External test from a phone on mobile data, November 2, 2026, 12:06 PM. Left: the Jenkins page opens without an account (the first-time "Unlock Jenkins" screen, because setup was not finished yet). Right: `https://ci.ssw590-g1.tech/` answers *Forbidden*.

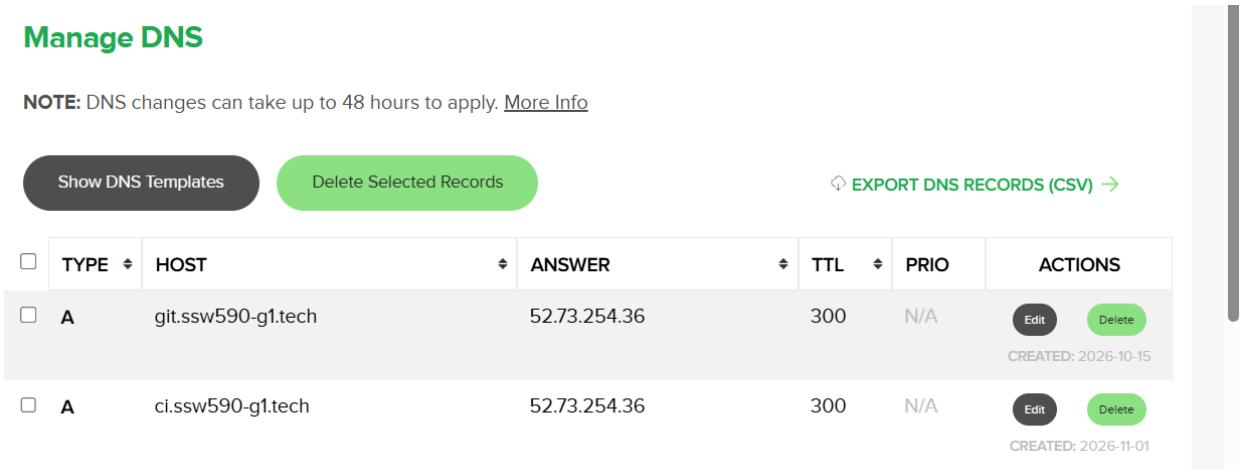


Figure 5.4: Name.com Manage DNS, November 1, 2026: new A record `ci.ssw590-g1.tech` to the same address as `git.ssw590-g1.tech`, TTL 300.

was applied with a backup of the old Caddyfile and compose file and an automatic rollback, and only the Caddy container was recreated (Gitea stayed healthy). The team's address list lives only in the server's untracked `.env` (mode 600), never in the repository. The Jenkins URL is `https://ci.ssw590-g1.tech/` (read from the saved configuration in Listing 23). Caddy's reverse proxy adds the forwarding headers itself; a screenshot of Jenkins' "reverse proxy setup" check was not captured, so that check is not claimed.

```
Real output, Nov 2, 2026 about 00:00-00:06 US Eastern (04:00-05:06 UTC). Run by Claude Code on Brahmi's
↪  go-ahead ("yes, apply Caddy") from Brahmi's Windows machine over SSH.
SANITIZED: the team's allowed address (TEAM_IPS) is stored only in the server's untracked .env and is never
↪  printed. Backups made first: docker-compose.yml.bak-20261102, Caddyfile.bak-20261029, .env.bak-20261102.

=== 1. Compose change: pass TEAM_IPS (a variable reference) to the caddy service ===
diff docker-compose.yml.bak-20261102 docker-compose.yml:
41a42
>       TEAM_IPS: ${TEAM_IPS}
compose config: OK (checked quietly, without printing interpolated values)

=== 2. Caddyfile replaced by the candidate validated on Oct 29 (cp, so the bind mount keeps working) ===
Caddyfile checksum before: 7705e125f84a2c82   after: 7a3679d4ff1ba6bb
New site block: ci.ssw590-g1.tech { @team remote_ip {$TEAM_IPS}; @loginpage path /login /login/* /static/*
↪  /adjuncts/* /favicon.ico; handle @team {reverse_proxy jenkins:8080}; handle @loginpage {reverse_proxy
↪  jenkins:8080}; handle {respond "Forbidden" 403} }

=== 3. Only the caddy container was recreated (gitea was not touched) ===
Container gitea Running
Container caddy Recreate / Recreated / Starting / Started
caddy state: running
After: caddy Up 20 seconds; jenkins Up 3 days; gitea-runner Up 13 days; gitea Up 2 weeks (healthy)
gitea via public name: HTTP 200

=== 4. Caddy log: automatic certificate for the new name (excerpt; JSON trimmed) ===
http: enabling automatic TLS certificate management, domains: ["git.ssw590-g1.tech","ci.ssw590-g1.tech"]
tls.obtain: obtaining certificate, identifier ci.ssw590-g1.tech
http.acme_client: trying to solve challenge, challenge_type tls-alpn-01, ca
↪  https://acme-v02.api.letsencrypt.org/directory
tls: served key authentication certificate (tls-alpn-01) for ci.ssw590-g1.tech, 5 times (Let's Encrypt
↪  validation from several vantage points)
```

```
=== 5. Certificate served for ci.ssw590-g1.tech (read with .NET SslStream) ===
TLS: Tls13
Subject: CN=ci.ssw590-g1.tech
Issuer: CN=YE2, O=Let's Encrypt, C=US
Not before: 2026-11-02 04:04 UTC
Not after : 2027-01-31 04:04 UTC
Chain valid: True

=== 6. HTTP and HTTPS behaviour, from the administrator's computer (on the team address) ===
http://ci.ssw590-g1.tech/ -> 308 https://ci.ssw590-g1.tech/        (redirect to HTTPS)
https://ci.ssw590-g1.tech/login -> 200
https://ci.ssw590-g1.tech/ -> 403

The 403 at / is Jenkins' own response, not Caddy's block: the headers show "Server: Jetty(12.1.14)",
↪  "X-Jenkins: 2.580.2", "Via: 1.1 Caddy", and the body says "Authentication required" (Jenkins' setup wizard
↪  has not been completed, so anonymous requests are refused). This shows the team address is passed through
↪  to Jenkins. The path for outsiders (plain "Forbidden" from Caddy at /, with /login open) still has to be
↪  tested from a different network.

Notes:
- First attempt at reading the Caddy "obtained" log line failed because of a quoting error in the checking
↪  command; the certificate check in section 5 is the proof of issuance.
- Rollback was prepared (restore the three backups and recreate caddy) and not needed.

UPDATE (later the same evening): the test from a phone on mobile data was made after this file was written.
↪  The Jenkins page opened without an account and the root path answered a plain "Forbidden" (evidence 76 and
↪  77); the 403 seen above from the team address was therefore Jenkins' own response, as explained in the
↪  note.
```

Listing 24: Caddy change with backups and rollback prepared, certificate for the new name, and the first HTTP/HTTPS checks, November 2, 2026 (real output; the allowed address is never printed).

Named accounts and roles. Registration is closed (*Allow users to sign up* is off). There are two named accounts in Jenkins' own user database, `brahmi` and `sakshi` (Figure 5.5). Authorization uses *Matrix-based security*: Brahmi has *Overall Administer*; Sakshi has Overall Read, Job Build and Read, and View Read; *Anonymous* and *Authenticated Users* have no permissions (Figure 5.6). Anonymous requests to the API are refused (HTTP 403).

Firewall and ports. No AWS rule was added for Jenkins. The security group still admits only 443 and 80 from anyone and SSH (22) from the administrator's address, has no IPv6 rule, and has no rule for 8080, 50000, or 3000; the Jenkins container publishes nothing, and Caddy's 80 and 443 are the only published ports. No host firewall such as `ufw` is used, because Docker-published ports bypass it [2]; the security group is the boundary and exposure was tested from outside.

Table 5.3: External tests, November 2, 2026, expected versus actual

Target	Expected	Actual	Source
HTTPS 443, `/login`	opens for anyone	opens (HTTP 200; phone on mobile data)	Figure 5.3, Listing 24

Target	Expected	Actual	Source
HTTPS 443, `/`	plain Forbidden for non-team addresses	Forbidden on the phone	Figure 5.3, right image
HTTP 80	redirect to HTTPS	308 to `https://ci.ssw590-g1.tech/`	Listing 24
Raw port 8080	unreachable	closed/filtered	Listing 25, Listing 22
Agent port 50000	unreachable	closed/filtered	Listing 25
Gitea backend 3000	unreachable	closed/filtered	Listing 25
IPv6	not applicable	no AAAA record, no IPv6 rule	Listing 25

The 8080/50000/3000 probes ran from the administrator's computer; because the security group has no rule for those ports for any address, a probe from another network cannot behave differently. The outsider's view of 443 was tested from the phone. IPv6 is not enabled, so the equivalent IPv6 checks are not applicable; the deployment is IPv4 only.

```
Real output, Nov 2, 2026 about 7:45 PM US Eastern, from Brahmi's Windows machine (AWS command line, read-only
↪  describe call; Test-NetConnection; nslookup). Run by Claude Code on Brahmi's go-ahead. Nothing was
↪  changed.
SANITIZED: the administrator's address in the SSH rule is replaced by <admin /32>. No account number is shown.
Limitation: the probes ran from the administrator's own computer, which is the allowed SSH address. That does
↪  not weaken the result for 8080, 50000 and 3000, because the security group has no rule for them for any
↪  address; the earlier test from a phone on mobile data (evidence 76 and 77) covers the outsider's view of
↪  443.

=== A. AWS security group attached to the server (the same group as Assignment 4; no rule was added for
↪  Jenkins) ===
group: ssw590-g1-gitea   id: sg-0415c75429368eb85
inbound  TCP 80-80   from 0.0.0.0/0     (HTTP redirect and ACME validation only)
inbound  TCP 22-22   from <admin /32>   (Admin SSH from Brahmi only)
inbound  TCP 443-443 from 0.0.0.0/0     (HTTPS to Caddy)
IPv6 inbound rules: 0   (none expected)
No rule exists for 8080 (Jenkins), 50000 (Jenkins agents) or 3000 (Gitea): none of them is published. The
↪  jenkins container also publishes no port on the host (evidence 73).

=== B. Probes from the administrator's computer to the public name (Test-NetConnection, 2026-11-02 19:45
↪  local) ===
ci.ssw590-g1.tech port 443    -> OPEN
ci.ssw590-g1.tech port 80     -> OPEN
ci.ssw590-g1.tech port 8080   -> closed/filtered
ci.ssw590-g1.tech port 50000  -> closed/filtered
ci.ssw590-g1.tech port 3000   -> closed/filtered
("closed/filtered" cannot tell closed from firewall-dropped; both mean not reachable.)

=== C. DNS: A and AAAA records of the Jenkins hostname (public resolver 8.8.8.8) ===
A    : 52.73.254.36
AAAA : none (no AAAA record), so the deployment is IPv4 only and no IPv6 check applies
```

Listing 25: Security group rules (administrator address redacted), external port probes, and DNS records, November 2, 2026 (real output).

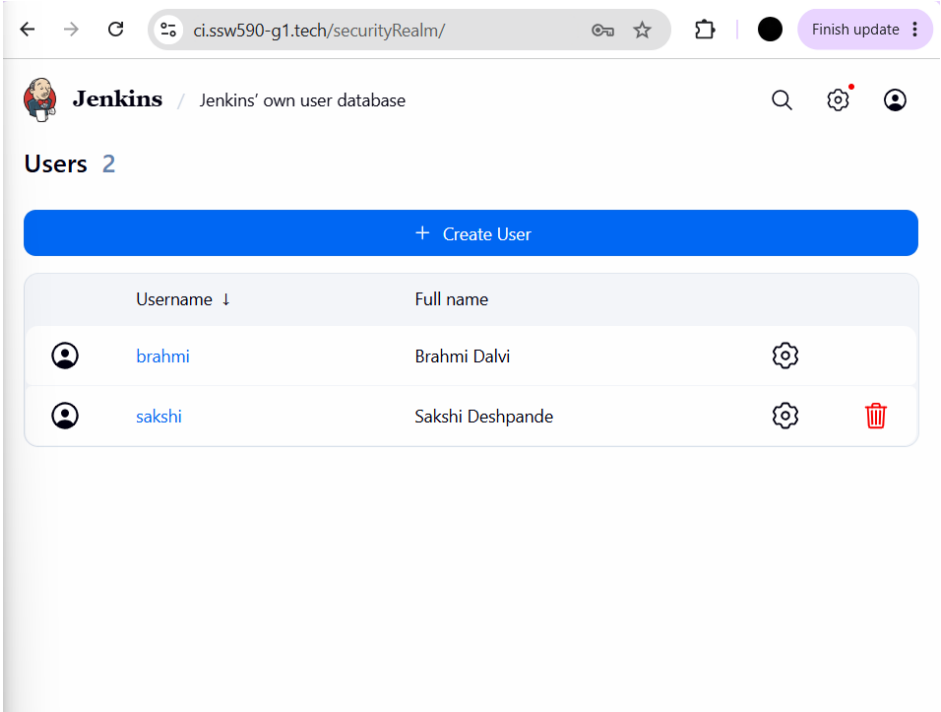


Figure 5.5: Jenkins Users page, November 2, 2026: two named accounts, `brahmi` and `sakshi`.

5.2.3 Gitea trigger and traceability (15 points)

Owner: BD. **Status:** a webhook-started build observed after one corrected attempt **PASS**.

Table 5.4: Gitea to Jenkins integration

Item	Value
Repository	`group1-startup/twintrack` (public)
Integration identity	A dedicated Gitea user `jenkins-bot` with *Write* on `twintrack` only (Figure 5.8); its token is stored only in the Jenkins credential `gitea-jenkins-bot` (Figure 5.9). The scopes were chosen narrow (user and organization read, repository write); the exact scope list was not captured as evidence
Jenkins job	Multibranch Pipeline `twintrack` (Gitea plugin), which found the `Jenkinsfile` in pull request #10 and scheduled build `PR-10` (Figure 5.10)
Webhook	Repository level: `http://jenkins:8080/gitea-webhook/post` on the private Docker network, POST with JSON; events *Push*, *Pull Request*, and *Pull Request Synchronized* (the last was added after a failed attempt, see below); no webhook secret (Figure 5.11, Figure 5.12)

Item	Value
Gitea setting	`[webhook] ALLOWED_HOST_LIST = jenkins`: Gitea blocks webhooks to private addresses by default [11, 20]; only the host name `jenkins` is allowed (Listing 28)
Where code runs	Checkout and tests run in a workspace inside the Jenkins container (controller executor). Jenkins fetches the code from `https://git.ssw590-g1.tech` (the public name, through Caddy) and posts statuses back to the same API
Status reported	Context **`twintrack/pipeline/pr-main`**, posted as `jenkins-bot`: pending, then success or failure (Listing 32)

Delivery result and the linked Jenkins run. A *Test Push Event* reached `http://jenkins:8080/gitea-webhook/post` and returned HTTP 200 (Figure 5.12). The first build that the webhook itself started is Jenkins build #3 (cause `BranchEventCause`, console line "Pull request #10 event") for commit `d5e5c64`, which passed in about 5 seconds; Gitea holds the matching status for the same commit (Listing 26). Builds #4 and #5 (commits `86ee9fc` and `92c121d`) were also started by the webhook (Listing 35, Listing 36). The commit ID is therefore the same in Gitea, in the Jenkins log, and in the status.

```
Real output, Nov 2, 2026 (UTC times below are Nov 3). Run by Claude Code on Brahmi's go-ahead: read-only look
↪  inside the Jenkins container (build.xml and console log, annotation blobs removed) and anonymous reads of
↪  Gitea's API.
Context: after evidence 92/93 (webhook missing the "Pull Request Synchronized" event), Brahmi ticked that
↪  event in the twintrack webhook and then pushed a no-change commit (d5e5c64) to the pull request branch.
↪  This is the first build that the webhook started by itself.

=== Jenkins build #3 of job twintrack/PR-10 ===
start: epoch 1793665478961 (2026-11-03 00:24:38 UTC)   duration: 5265 ms (about 5 s)   result: SUCCESS
Cause recorded in build.xml: jenkins.branch.BranchEventCause (branch-api 2.1303.v9f3b_95dc329d), i.e. started
↪  by an SCM event, not by a user.
First line of the console log: "Pull request #10 event"
(Build #1 began with "Branch indexing"; build #2 was started manually, see evidence 89.)

=== Console log, key lines ===
Current revision of pull request #10 is d5e5c6476ed81f6d1fe6f21b9b8816bfcc968b0d
Obtained Jenkinsfile from 0d0c3dac4dbf24eeb90965c1bd2850a5e1d91b94+d5e5c6476ed81f6d1fe6f21b9b8816bfcc968b0d
Checking out Revision d5e5c6476ed81f6d1fe6f21b9b8816bfcc968b0d (issue-9-pipeline)
Commit message: "Trigger Jenkins through the webhook: pull request synchronized event (issue #9)"
[Gitea] Notifying pull request build status: PENDING Build started...
+ python3 -m py_compile pricing.py
+ python3 -m unittest discover -s tests -v
Ran 2 tests in 0.000s
OK
[Gitea] Notifying pull request build status: SUCCESS This commit looks good
Finished: SUCCESS

=== Gitea statuses for the new head commit d5e5c64 (anonymous API), by jenkins-bot ===
2026-11-03T00:24:44 | 'twintrack/pipeline/pr-main' | success | This commit looks good
2026-11-03T00:24:43 | 'twintrack/pipeline/pr-main' | pending | Build started...
2026-11-03T00:24:30 | 'twintrack/pipeline/pr-main' | pending | Build queued...

=== Pull request #10 as Gitea sees it ===
```

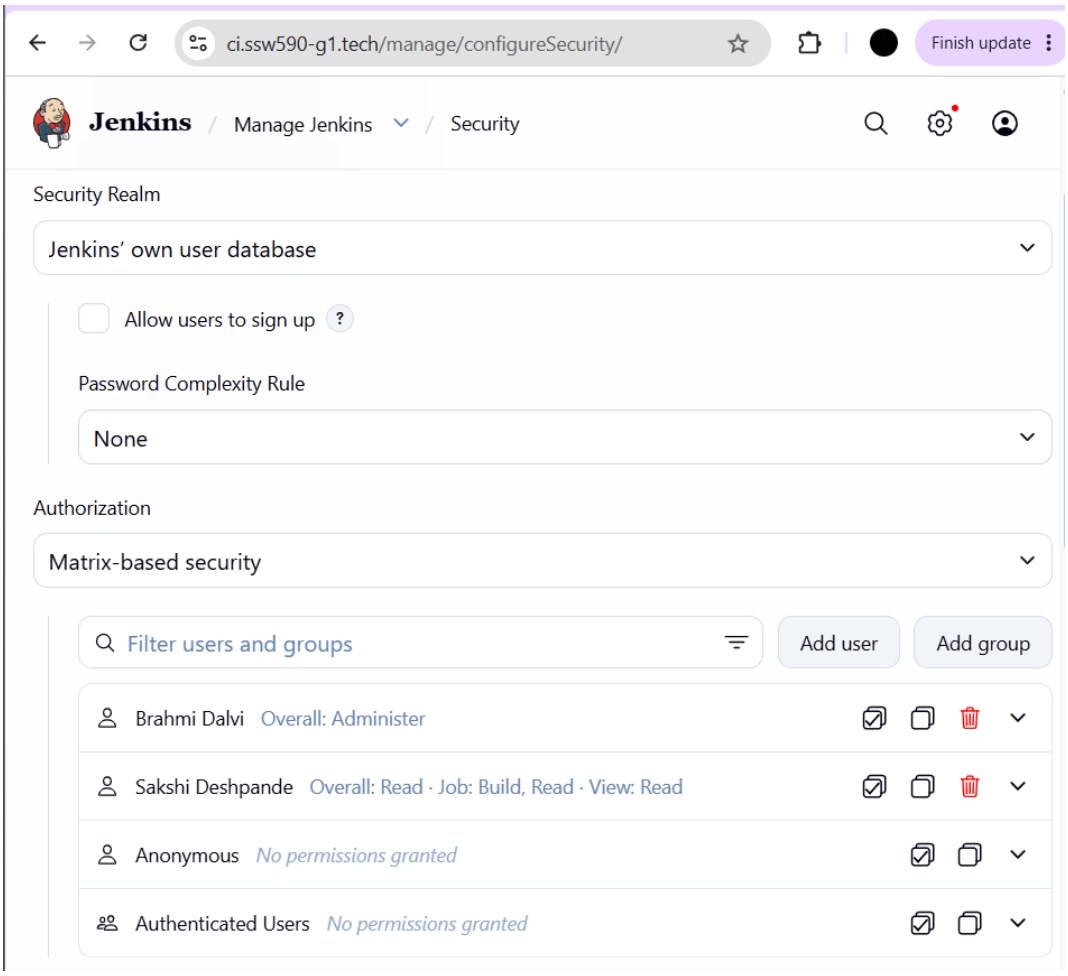


Figure 5.6: Manage Jenkins, Security, November 2, 2026: sign-up off; matrix authorization with Brahmi = Administer, Sakshi = Read and Build, Anonymous and Authenticated Users = no permissions.

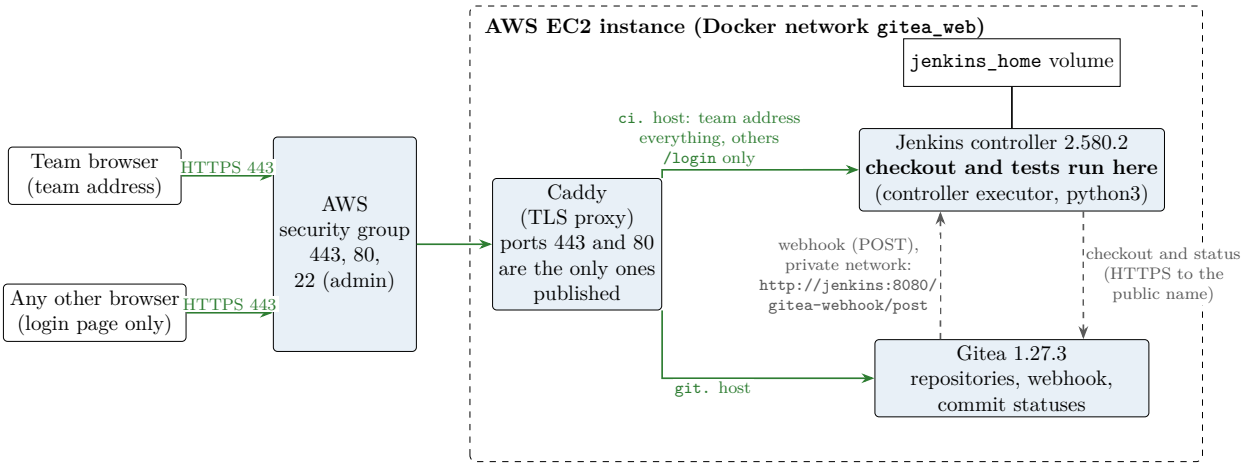


Figure 5.7: UML-style deployment diagram of the actual browser and webhook paths on one EC2 instance, adapted from the course manual's Figure 4.2. Solid green: browser and proxy paths. Dashed grey: private paths between Gitea and Jenkins. Builds run on the Jenkins controller executor [1, 5].

```
head: d5e5c64 | state: open | merged: False | mergeable: True
reviews so far: sakshi REQUEST_REVIEW (no approval yet)

=== Server memory at the time ===
Mem: 913 total, 674 used, 62 free, 176 buff/cache, 102 available (MB); Swap 2047 total, 386 used
```

Listing 26: Build #3, the first build started by the webhook: cause, console, and Gitea statuses for commit `d5e5c64`, November 2, 2026 (real output).

A failed delivery, diagnosed and corrected. The first webhook attempt did not start builds for new commits, and the cause was found in two steps. (1) BD first created the webhook in her *personal* account settings (the page said it fires for "all repositories that you own"), but `twintrack` belongs to the organization, so it never fired; it was recreated on the repository. (2) Even then, a real push to the pull request branch reached Jenkins (its *Multibranch Pipeline Events* page lists three push events processed in under a second each, Listing 27) but no build was scheduled, because the webhook did not have the *Pull Request Synchronized* event ticked (Figure 5.13): a plain push event only re-checks branches, and the branch behind a pull request is skipped, so only the pull-request event tells Jenkins to rebuild the new head commit. Ticking that event fixed it, proven by build #3. In the meantime build #2 had been started by hand and the repair commit `7d7848f` was never built. **Operator steps for a failed delivery:** open the webhook's *Recent Deliveries* in Gitea (did it send, to which URL, which response?); open *Multibranch Pipeline Events* in Jenkins (was the event received, and what did it do?); check the events ticked in the webhook; check the allowed-host setting and that the URL uses the container name `jenkins`; then use *Scan Repository Now* as a manual fallback.

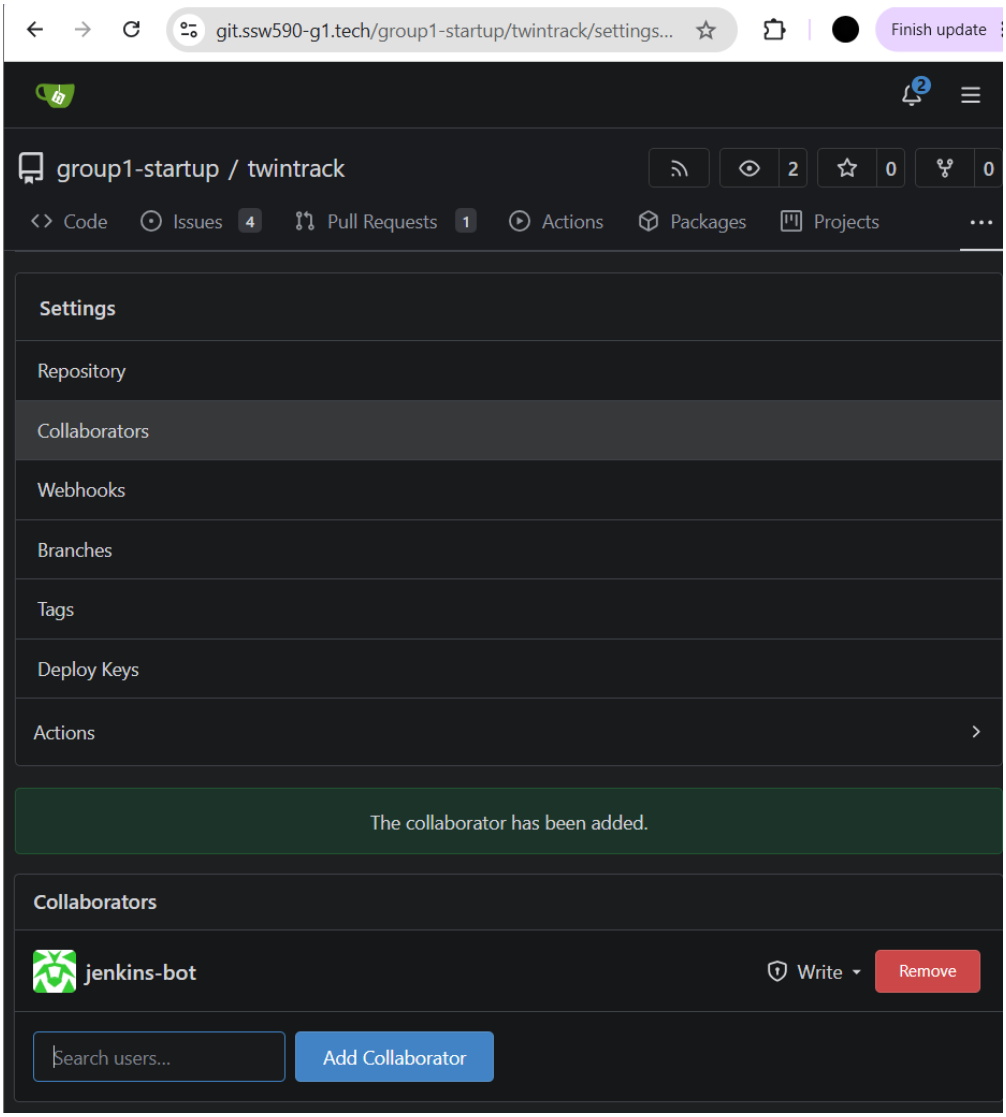


Figure 5.8: `twintrack` Settings, Collaborators, November 2, 2026: `jenkins-bot` added with Write.

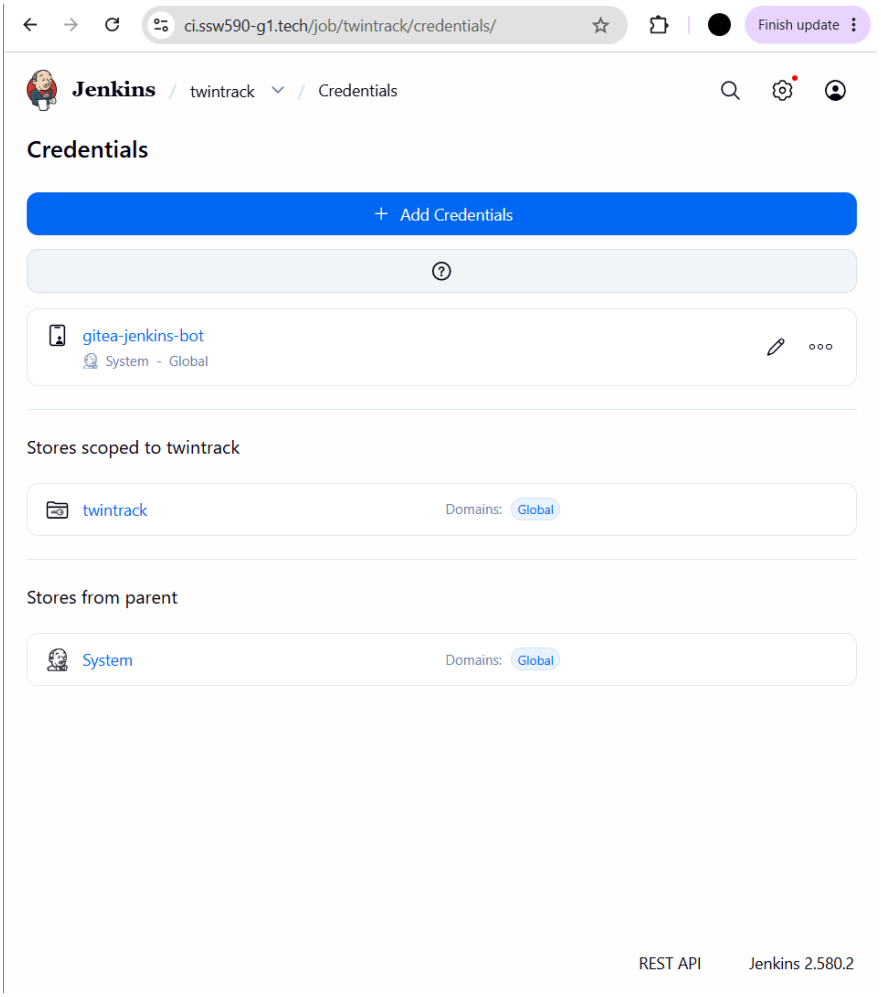


Figure 5.9: Jenkins Credentials, November 2, 2026: `gitea-jenkins-bot` (System, Global). The token is stored in Jenkins and is not displayed.

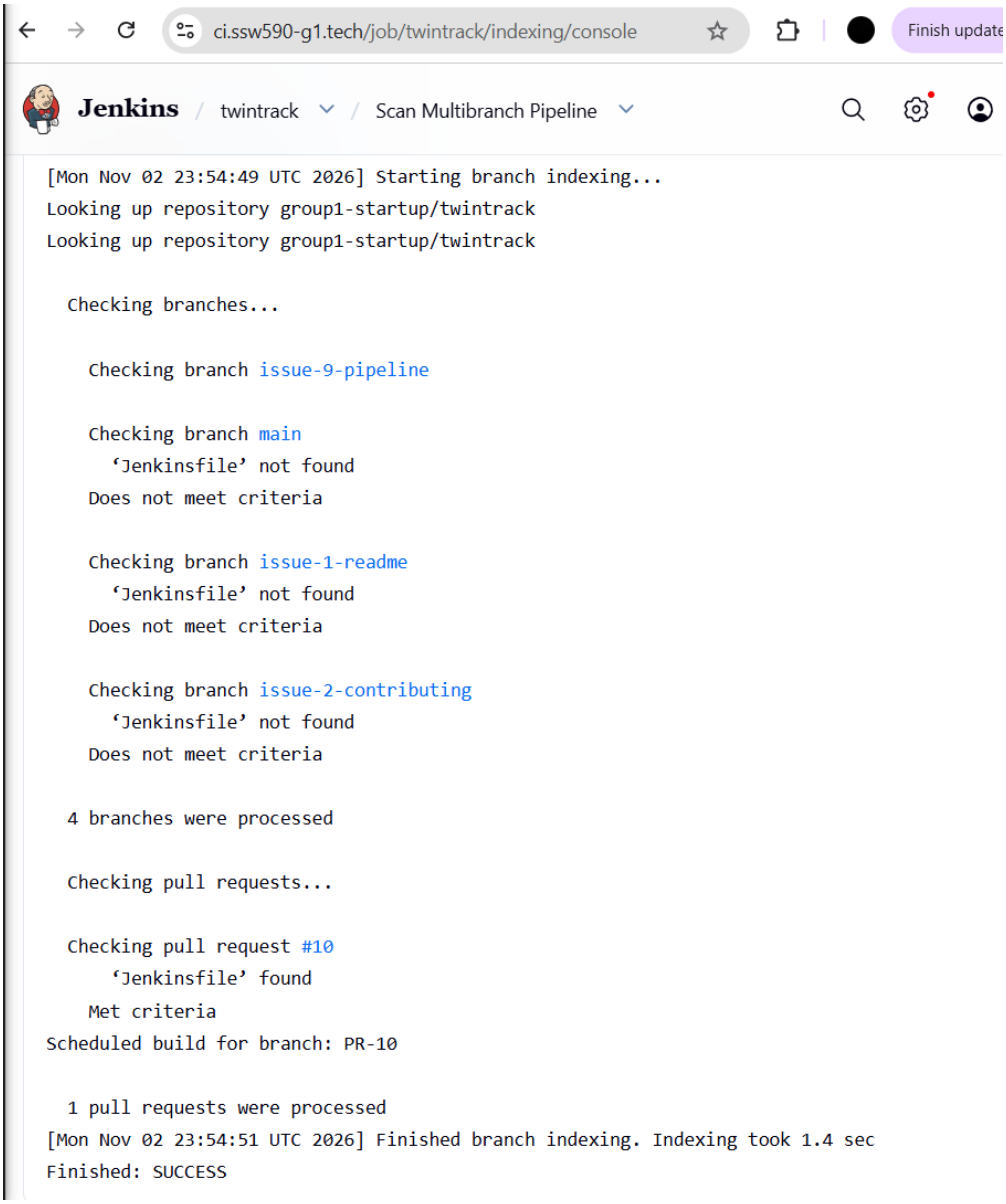


Figure 5.10: Multibranch scan of `group1-startup/twintrack`, November 2, 2026 (23:54 UTC): branches without a `Jenkinsfile` do not meet the criteria; pull request #10 does, and "Scheduled build for branch: PR-10".

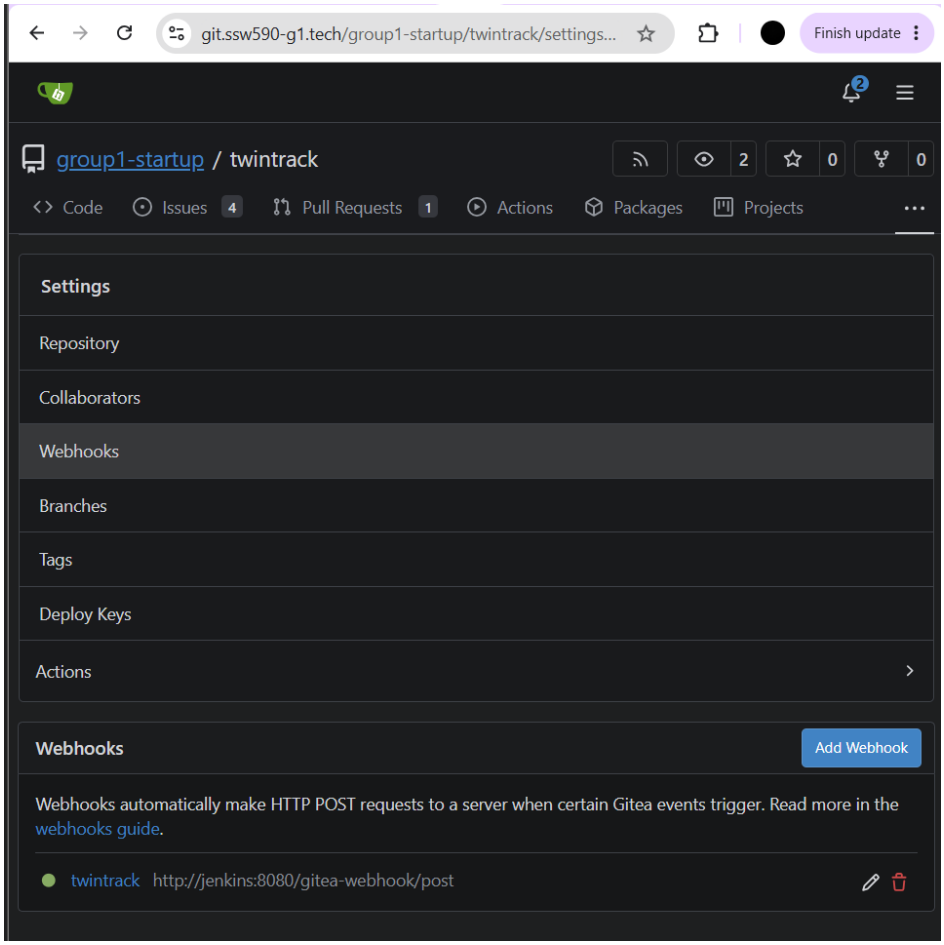


Figure 5.11: `twintrack` Settings, Webhooks (repository level), November 2, 2026: the webhook to `http://jenkins:8080/gitea-webhook/post`, status dot green.

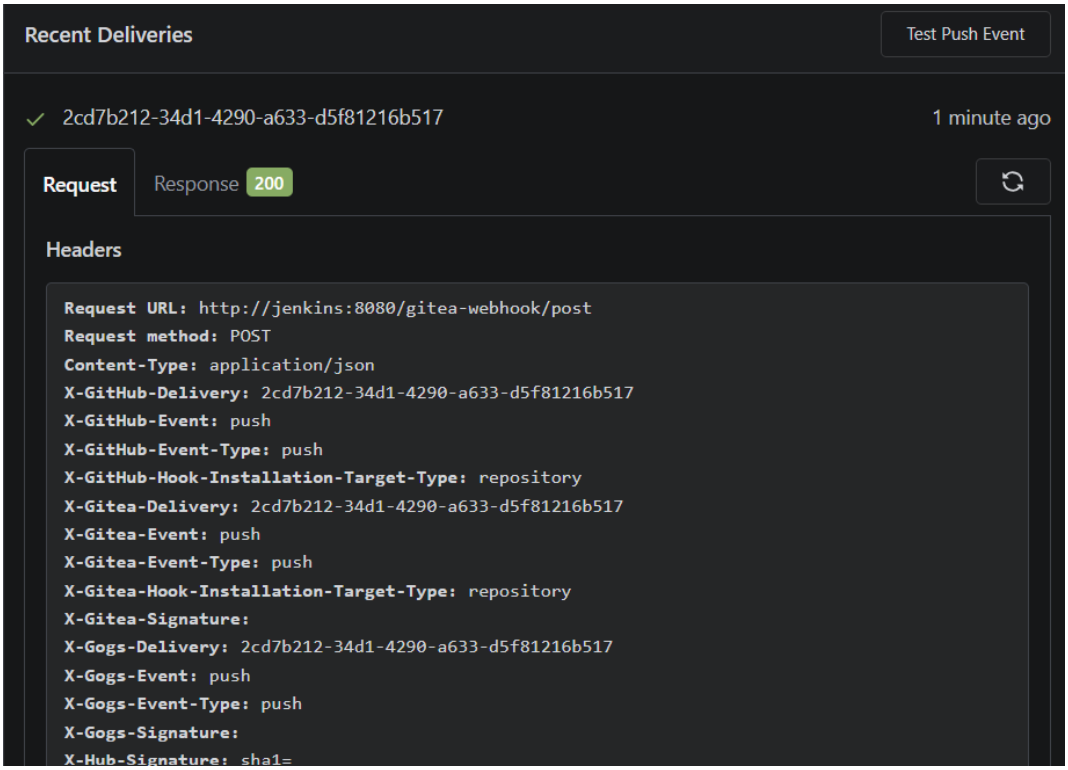


Figure 5.12: Recent Deliveries of the webhook: a Test Push Event to the private address, POST `application/json`, response 200. The signature headers are empty because no webhook secret is configured.

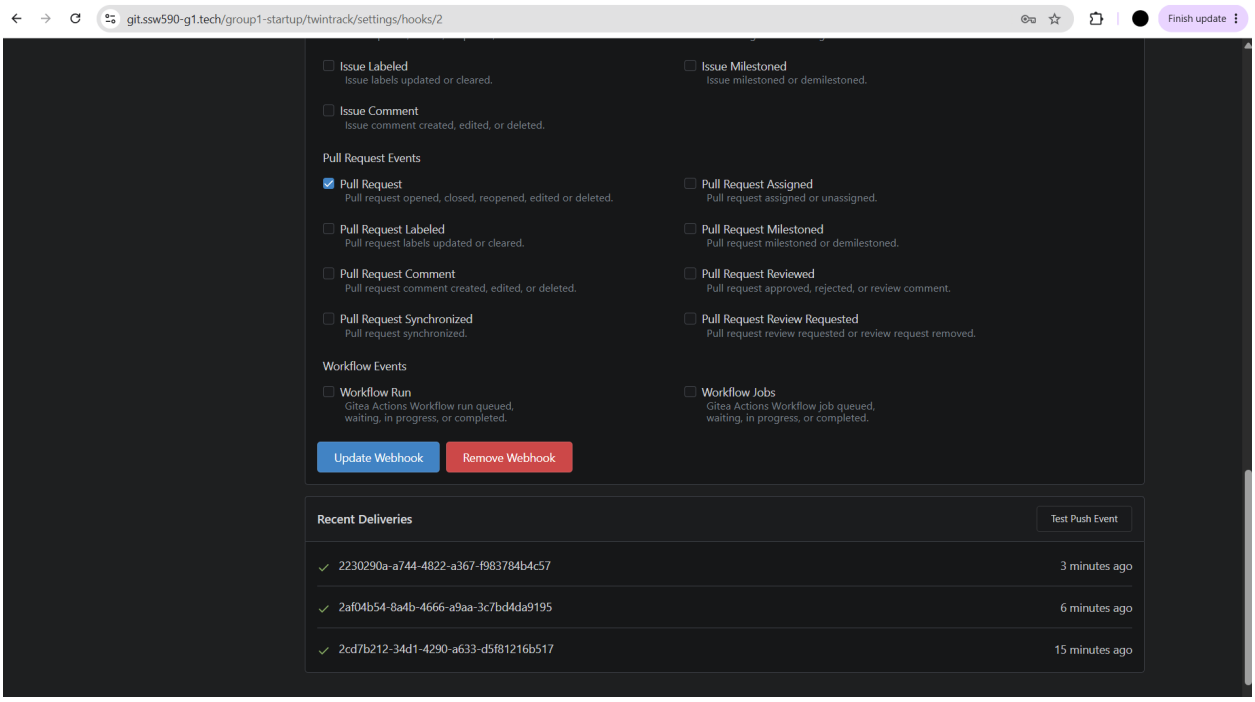


Figure 5.13: The webhook's events before the fix, November 2, 2026: *Pull Request* is ticked and *Pull Request Synchronized* is not; the three deliveries below all succeeded.

```
Text copied by Brahmi from Jenkins, job twintrack, "Multibranch Pipeline Events" page, Nov 2, 2026 (the times
↪  are UTC, already Nov 3). Pasted into the session and saved unchanged; the sender 172.18.0.2 is the private
↪  Docker address of Gitea on the shared network.
What it shows: the Gitea webhook DOES reach Jenkins (push events were received and processed in under a second
↪  each), but these push events only re-check branches and the pull request; none of them scheduled a build.
↪  Pull Request "Synchronized" events were not enabled in the webhook at this time (screenshot 92), and that
↪  is the event that makes Jenkins rebuild a pull request after a new commit.

[Tue Nov 03 00:05:06 UTC 2026] Received Push event to branch main in repository group1-startup/twintrack
↪  CREATED event from 172.18.0.2 => http://jenkins:8080/gitea-webhook/post with timestamp Tue Nov 03 00:05:06
↪  UTC 2026
Looking up repository group1-startup/twintrack

  Checking branches...

    Checking branch issue-9-pipeline

    Checking branch main
      'Jenkinsfile' not found
    Does not meet criteria

    Checking branch issue-1-readme

    Checking branch issue-2-contributing

  4 branches were processed

  Checking pull requests...

  Checking pull request #10

  1 pull requests were processed
```

```
[Tue Nov 03 00:05:07 UTC 2026] Push event to branch main in repository group1-startup/twintrack CREATED event
↪  from 172.18.0.2 => http://jenkins:8080/gitea-webhook/post with timestamp Tue Nov 03 00:05:06 UTC 2026
↪  processed in 0.69 sec
[Tue Nov 03 00:14:35 UTC 2026] Received Push event to branch issue-9-pipeline in repository
↪  group1-startup/twintrack UPDATED event from 172.18.0.2 => http://jenkins:8080/gitea-webhook/post with
↪  timestamp Tue Nov 03 00:14:35 UTC 2026
Looking up repository group1-startup/twintrack

  Checking branches...

    Checking branch issue-9-pipeline

    Checking branch main

    Checking branch issue-1-readme

    Checking branch issue-2-contributing

  4 branches were processed

  Checking pull requests...

  Checking pull request #10

  1 pull requests were processed
[Tue Nov 03 00:14:35 UTC 2026] Push event to branch issue-9-pipeline in repository group1-startup/twintrack
↪  UPDATED event from 172.18.0.2 => http://jenkins:8080/gitea-webhook/post with timestamp Tue Nov 03 00:14:35
↪  UTC 2026 processed in 0.25 sec
[Tue Nov 03 00:17:24 UTC 2026] Received Push event to branch issue-9-pipeline in repository
↪  group1-startup/twintrack UPDATED event from 172.18.0.2 => http://jenkins:8080/gitea-webhook/post with
↪  timestamp Tue Nov 03 00:17:24 UTC 2026
Looking up repository group1-startup/twintrack

  Checking branches...

    Checking branch issue-9-pipeline

    Checking branch main

    Checking branch issue-1-readme

    Checking branch issue-2-contributing

  4 branches were processed

  Checking pull requests...

  Checking pull request #10

  1 pull requests were processed
[Tue Nov 03 00:17:25 UTC 2026] Push event to branch issue-9-pipeline in repository group1-startup/twintrack
↪  UPDATED event from 172.18.0.2 => http://jenkins:8080/gitea-webhook/post with timestamp Tue Nov 03 00:17:24
↪  UTC 2026 processed in 0.2 sec
```

Listing 27: Jenkins *Multibranch Pipeline Events* (copied by Brahmi): push events received and processed, but no build scheduled (real output).

```
Real output from the server, Nov 2, 2026 about 12:30 PM US Eastern. Run by Claude Code on Brahmi's go-ahead
↪  ("yes, recreate Gitea").
Purpose: Gitea blocks webhooks to private addresses by default ([webhook] ALLOWED_HOST_LIST). Allowing the
↪  single host name "jenkins" lets Gitea call the Jenkins container on the private Docker network without
↪  opening Jenkins to the internet.
Backup made first: docker-compose.yml.bak-20261102b (rollback was automatic on any failure and was not
↪  needed).
```

```
=== 1. Compose change (gitea service environment) ===
references now: 1
compose config: OK (checked quietly)
19a20
>       GITEA__webhook__ALLOWED_HOST_LIST: jenkins

=== 2. Only gitea was recreated; its data volume was not touched ===
Container gitea Recreate / Recreated / Starting / Started
gitea health: healthy after 37 s   (so Gitea was unavailable for up to about 37 seconds)

=== 3. The setting inside Gitea's configuration file ===
[webhook]
ALLOWED_HOST_LIST = jenkins

=== 4. Everything else ===
gitea Up 35 seconds (healthy); caddy Up 20 minutes; jenkins Up 3 days; gitea-runner Up 13 days
gitea via public name: HTTP 200
jenkins login via public name: HTTP 200

=== 5. Data still present ===
twintrack open pull request: "number":10
```

Listing 28: Allowing the webhook host `jenkins` in Gitea: compose change, only Gitea recreated (healthy after 37 seconds), setting confirmed, data intact, November 2, 2026 (real output).

5.2.4 Versioned pipeline and tests (15 points)

Owner: SD. **Status:** drafted by BD from the real runs; every statement checked by SD on November 3, 2026 against Gitea's records, the Jenkins evidence, and a clean local run **PASS**.

The application is the course's optional minimal Python example (manual Chapter 4, step 4), committed to `twintrack` through the issue-linked pull request #10 (issue #9): `pricing.py`, `tests/test_pricing.py`, and a `Jenkinsfile` at the repository root. The files are linked at the commit that Jenkins tested last (`Jenkinsfile`, `pricing.py`).

```
pipeline {
    agent any
    options { skipDefaultCheckout() }
    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Syntax check') {
            steps { sh 'python3 -m py_compile pricing.py' }
        }
        stage('Test') {
            steps { sh 'python3 -m unittest discover -s tests -v' }
        }
    }
}
```

Listing 29: The declarative `Jenkinsfile`: Checkout, Syntax check, Test.

```
def total_price(unit_price, quantity):
```

```
    if quantity < 1:
        raise ValueError("Quantity must be positive")
    return unit_price * quantity
```

```
import unittest
from pricing import total_price


class PricingTests(unittest.TestCase):
    def test_total(self):
        self.assertEqual(total_price(10, 3), 30)

    def test_invalid_quantity(self):
        with self.assertRaises(ValueError):
            total_price(10, 0)
```

Listing 30: `pricing.py` and `tests/test_pricing.py`.

Table 5.5: Pipeline stages

Stage	What it does and expected result
Checkout	`checkout scm` fetches the pull request head, merged with `main`, from Gitea; the log names the exact revision
Syntax check (build/lint)	`python3 -m py_compile pricing.py`: fails on any syntax error; silent success otherwise
Test	`python3 -m unittest discover -s tests -v`: two tests; the stage fails if any test fails, with the assertion in the log

Dependencies and clean run. The only dependency is Python 3 on the executor (3.13.5, installed in the image by `apt`); there are no packages from the internet. A clean run is reproduced on any computer with Python 3: clone the repository, then run `python3 -m py_compile pricing.py` and `python3 -m unittest discover -s tests -v` in it. This was done before the first push (Listing 31) and again by SD on November 3, 2026, in a fresh clone of the merged `main` (commit `a0094a8`, Listing 33): syntax OK, two tests `ok`, exit code 0, the same result as Jenkins.

Run view and stage results. The Jenkins Pipeline Stage View plugin is not installed, so there is no graphical stage view; the distinct stage results are in the console log (Listing 32): after the checkout, the *Syntax check* stage runs `py_compile`, and the *Test* stage prints the two tests as `ok`, followed by `Finished: SUCCESS`. The first passing run tested commit `da16c9b` (build #1); the log also shows what exactly is tested: Jenkins fetches the pull request head and `main`, then reports "Merging remotes/origin/main commit `0d0c3da` into PR head commit `da16c9b`", so a pull request build tests the result of the merge, not the branch alone. The Gitea statuses and the build records are in the same listing. The failing stage is visible in the failing builds (Listing 35).

```
Real output. Run by Claude Code on Brahmi's go-ahead on Brahmi's Windows machine (Python 3.14.6; Jenkins will
↪  use python3 on the server).
Time: 2026-10-29 23:48 UTC. Files are the course manual's Python example (V1.38, Chapter 4, step 4),
↪  unchanged.

$ python -m py_compile pricing.py
(exit 0: syntax OK)

$ python -m unittest discover -s tests -v
test_invalid_quantity (test_pricing.PricingTests.test_invalid_quantity) ... ok
test_total (test_pricing.PricingTests.test_total) ... ok

----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
(exit 0)
```

Listing 31: Local run of the course example before the first pull request, October 29, 2026 (real output).

```
Real output, Nov 2, 2026. Run by Claude Code on Brahmi's go-ahead: (1) read-only look inside the Jenkins
↪  container on the server (docker exec jenkins ... tail of the build log), (2) anonymous read of Gitea's
↪  commit-status API for the PR head commit.
SANITIZED: Jenkins console annotation blobs (lines beginning "ha:////", an internal encoding for the web view)
↪  were removed; nothing else was changed. No credential appears in the log (Jenkins masks them; the log only
↪  says "using GIT_ASKPASS to set credentials").

=== Build record (Jenkins build.xml of job twintrack, branch PR-10, build #1) ===
start: 2026-11-02 23:54:58 UTC   duration: 12108 ms (about 12 s)   result: SUCCESS

=== Console log, last 45 lines (annotation blobs removed) ===
using GIT_ASKPASS to set credentials
 > git fetch --no-tags --force --progress -- https://git.ssw590-g1.tech/group1-startup/twintrack.git
 ↪  +refs/pull/10/head:refs/remotes/origin/PR-10 +refs/heads/main:refs/remotes/origin/main # timeout=10
Merging remotes/origin/main commit 0d0c3dac4dbf24eeb90965c1bd2850a5e1d91b94 into PR head commit
↪  da16c9b006a6cfedbc1b81ffc9b1c6fd23e0c307
 > git checkout -f da16c9b006a6cfedbc1b81ffc9b1c6fd23e0c307 # timeout=10
 > git merge 0d0c3dac4dbf24eeb90965c1bd2850a5e1d91b94 # timeout=10
Merge succeeded, producing da16c9b006a6cfedbc1b81ffc9b1c6fd23e0c307
Checking out Revision da16c9b006a6cfedbc1b81ffc9b1c6fd23e0c307 (issue-9-pipeline)
Commit message: "Add pricing module, unit tests and Jenkinsfile for CI (issue #9)"
First time build. Skipping changelog.
[Gitea] Notifying pull request build status: PENDING Build started...
[Gitea] Notified
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Syntax check)
[Pipeline] sh
+ python3 -m py_compile pricing.py
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] sh
+ python3 -m unittest discover -s tests -v
test_invalid_quantity (test_pricing.PricingTests.test_invalid_quantity) ... ok
test_total (test_pricing.PricingTests.test_total) ... ok

----------------------------------------------------------------------
Ran 2 tests in 0.000s
```

```
OK
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
[Gitea] Notifying pull request build status: SUCCESS This commit looks good
[Gitea] Notified
Finished: SUCCESS

=== Commit statuses that Gitea holds for commit da16c9b (anonymous API:
↪  /api/v1/repos/group1-startup/twintrack/commits/da16c9b/statuses) ===
2026-11-02T23:55:11 | context = 'twintrack/pipeline/pr-main' | state = success | by jenkins-bot | This commit
↪  looks good
2026-11-02T23:55:09 | context = 'twintrack/pipeline/pr-main' | state = pending | by jenkins-bot | Build
↪  started...
2026-11-02T23:54:51 | context = 'twintrack/pipeline/pr-main' | state = pending | by jenkins-bot | Build
↪  queued...

The exact status-check context to require on the protected branch is:  twintrack/pipeline/pr-main
Server memory at the time: 105 MB available, swap 352 MB; jenkins 391 MiB of its 640 MiB cap.
```

Listing 32: Build #1 of `PR-10` (commit `da16c9b`): console, stage commands, and the Gitea statuses (real output; console annotation blobs removed).

```
Clean run of the twintrack pipeline commands on SD's workstation (macOS), 2026-11-03 01:11:49 UTC, run by
↪  Claude Code for SD.
Fresh clone of the public repository, no other setup; same commands as the Jenkinsfile stages.

$ git clone https://git.ssw590-g1.tech/group1-startup/twintrack.git && cd twintrack
$ git log -1 --format="%h %s"
a0094a8 Merge pull request 'Add pricing module, unit tests and Jenkinsfile for CI' (#10) from issue-9-pipeline
↪  into main
$ python3 --version
Python 3.13.7
$ python3 -m py_compile pricing.py && echo "syntax OK"
syntax OK
$ python3 -m unittest discover -s tests -v
test_invalid_quantity (test_pricing.PricingTests.test_invalid_quantity) ... ok
test_total (test_pricing.PricingTests.test_total) ... ok

----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
exit code: 0
```

Listing 33: Clean-run check by SD: fresh clone of `twintrack` at the merged commit `a0094a8`, the two pipeline commands, November 3, 2026 (real output).

5.2.5 Required merge gate (15 points)

Owner: SD. **Status:** drafted by BD from the real runs; checked by SD on November 3, 2026 against Gitea's branch, status, and timeline records (Listing 34) **PASS**.

The rule. The default branch `main` of `twintrack` is protected in Gitea: pushes go through pull requests, *one approving review* is required, and the **status check pattern**

`twintrack/pipeline/pr-main` is required [21]. That is the exact context Jenkins reports; Gitea itself listed it as "Matched" among the status checks found in the last week before the rule was saved (Figure 5.14). A delivered webhook or a green page inside Jenkins is not the gate: the gate is Gitea's own record of this status on the pull request's latest head commit.

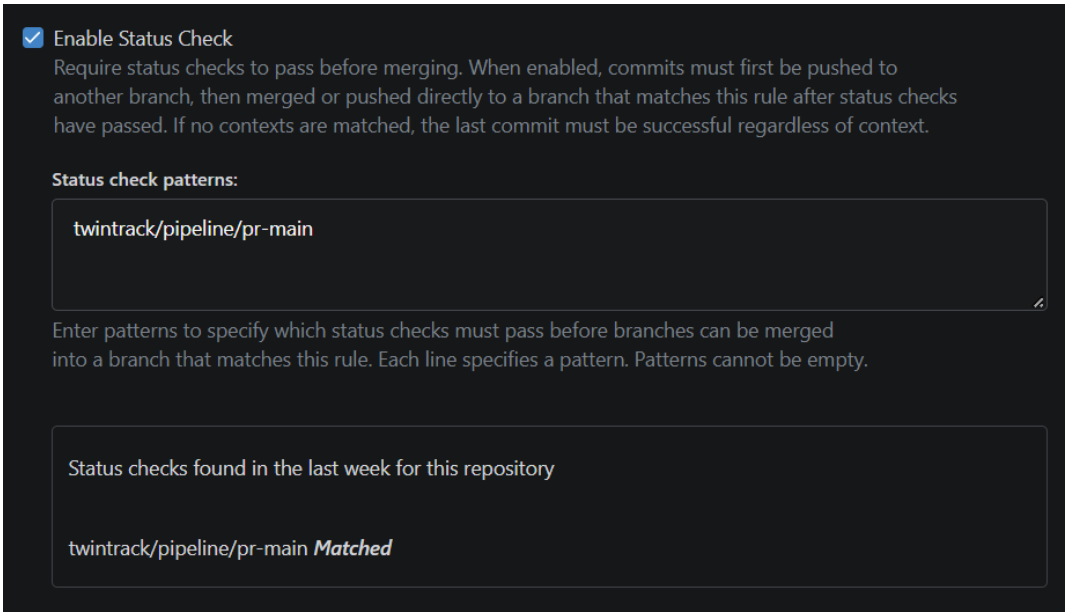


Figure 5.14: Branch rule for `main`, November 2, 2026: *Enable Status Check* ticked, pattern `twintrack/pipeline/pr-main`, shown as "Matched". Saved: Gitea's API reports `enable_status_check: true` with that context and one required approval.

How the status reaches Gitea. Jenkins' Gitea plugin posts it through Gitea's commit-status API with the token of `jenkins-bot`: *pending* when the build is queued and started, then *success* or *failure* when it ends; the console shows the lines "[Gitea] Notifying pull request build status" for each step [5, 22]. Gitea shows the result next to the commit and uses the required context to allow or block the merge. Gitea's own records show the round trip for the two merge-gate states (Listing 34, UTC, November 3, 2026):

Table 5.6: From push to status in Gitea for the two merge-gate states

Head commit	Pushed	First *pending*	Final status	Jenkins run
`86ee9fc`	00:30:22	00:30:24	**failure** 00:30:38	`PR-10` #4
`92c121d`	00:32:51	00:32:53	**success** 00:33:18	`PR-10` #5

The Jenkins run links need a Jenkins sign-in (Section 5.2.2); the same run numbers are visible without one as the target of each status in Gitea, and the pull request is public: pull

request #10.

Failed check blocking the merge. After the controlled failure (commit `86ee9fc`, November 2, 2026), the pull request page reads *Some required checks failed*: the required check `twintrack/pipeline/pr-main` shows "There was a failure building this commit", and below it "Some required checks were not successful" and "0 of 1 official approvals granted" (Figure 5.15). The failing commits carry red crosses in the commit list, and Jenkins shows the job red (Figure 5.16).

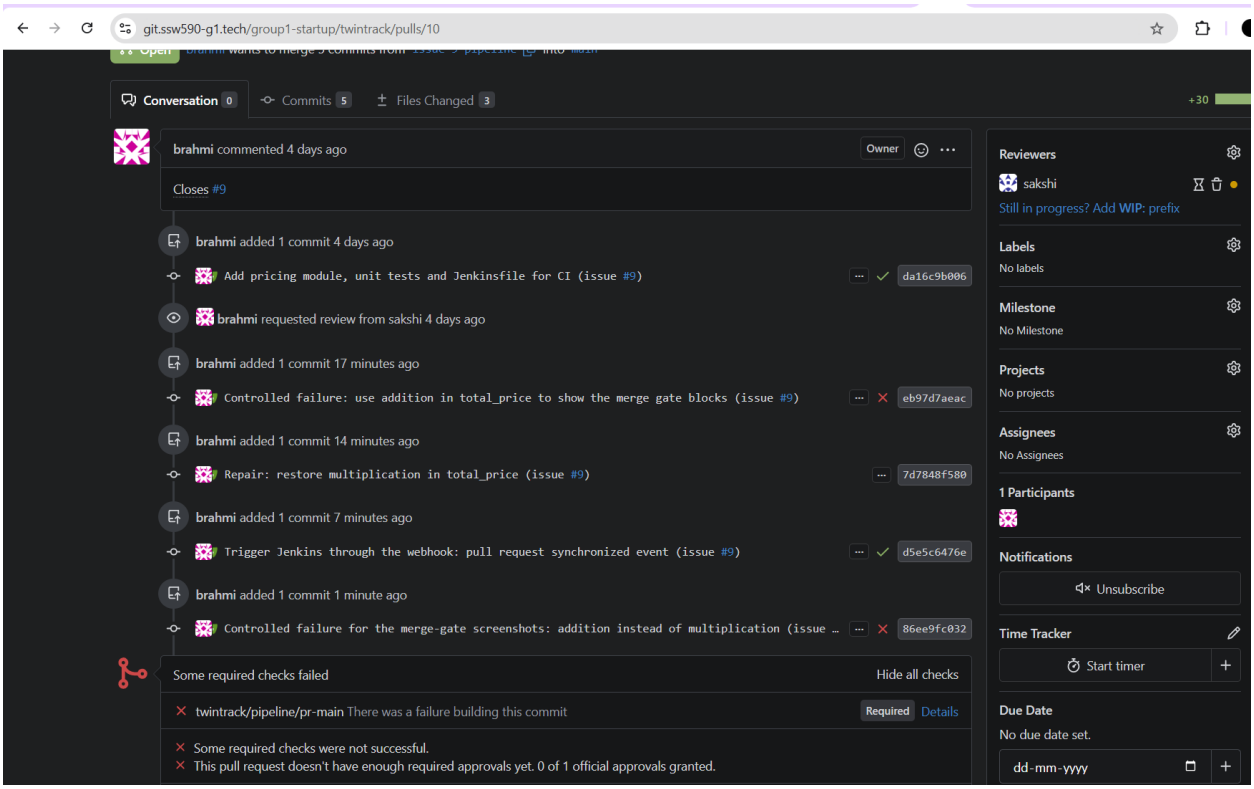


Figure 5.15: **Failed check blocking the merge.** Pull request #10, November 2, 2026, head commit `86ee9fc`, required check `twintrack/pipeline/pr-main` failed. The merge button area is below the cropped part of the page.

Passing check after the repair. After the repair (commit `92c121d`) the page reads *All checks were successful*: the required check shows "This commit looks good" and the merge now waits only for the approving review (Figure 5.17); Jenkins shows build #5 green (Figure 5.18).

A limit of this gate. The repository owner is also a Gitea administrator, and an earlier screenshot of the pull request page showed "As an administrator, you may still merge this pull request" (Figure 5.19). An owner could therefore override the rule; the team agreed not to, and Gitea's option "Administrators must follow branch protection rules", which removes

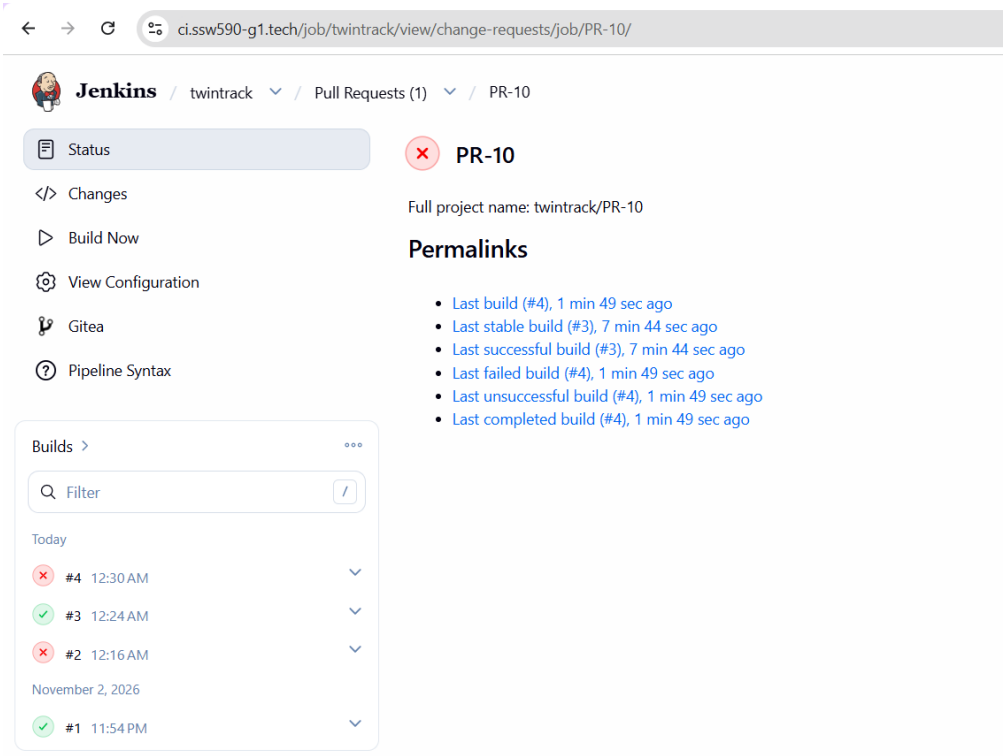


Figure 5.16: Jenkins job `PR-10` after the failing push, November 2, 2026: red, last build #4 failed (12:30 AM UTC), last stable build #3.

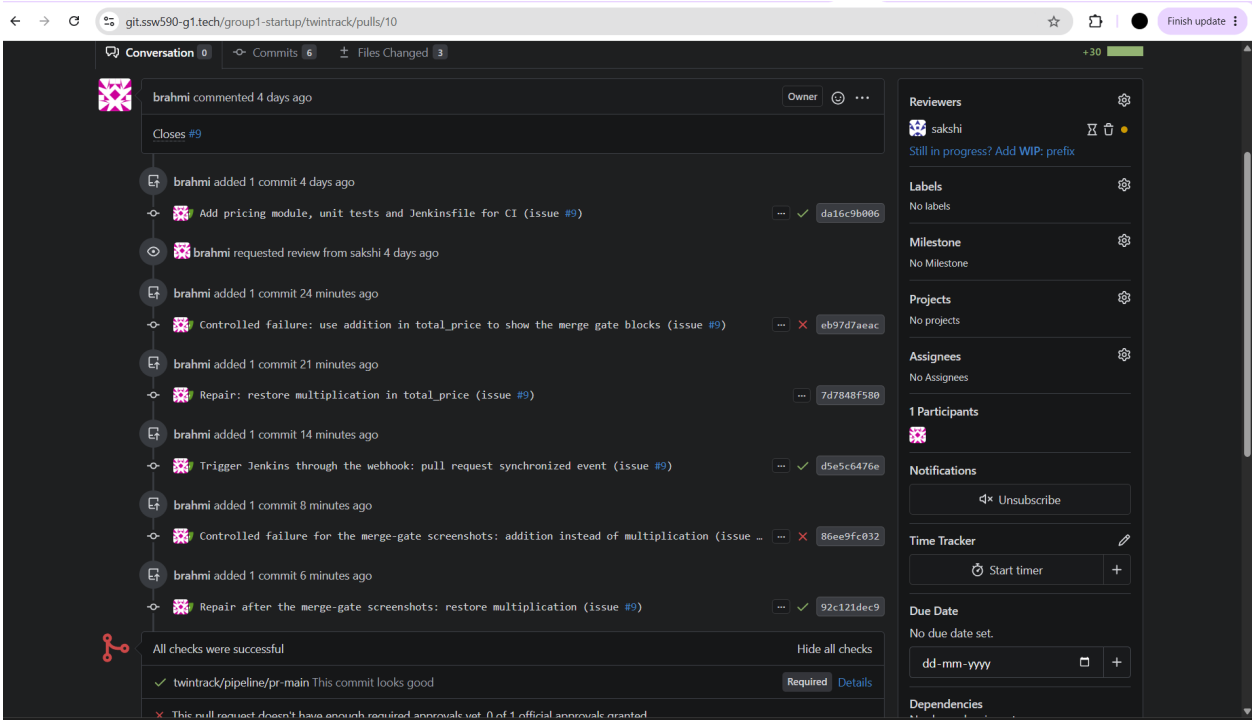


Figure 5.17: **Passing check after repair.** Pull request #10, November 2, 2026, head commit `92c121d`, required check `twintrack/pipeline/pr-main` successful; the failed commits `eb97d7a` and `86ee9fc` are marked with red crosses; merge waiting for approval.

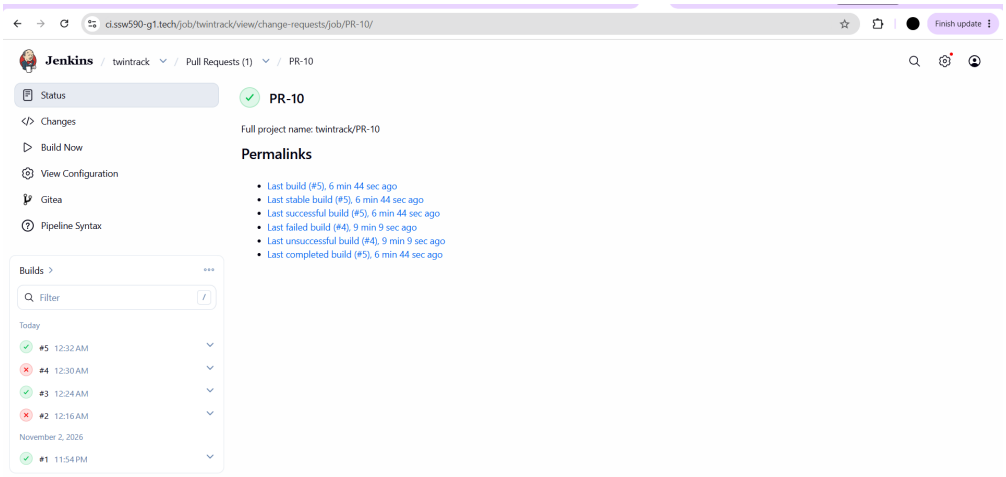


Figure 5.18: Jenkins job `PR-10` after the repair, November 2, 2026: build #5 green (12:32 AM UTC), build #4 failed, history #1 to #5.

that override [21], was not enabled. No merge into `main` used the override: pull request #10 was merged only after the check passed and SD approved it (Section 5.2.6).

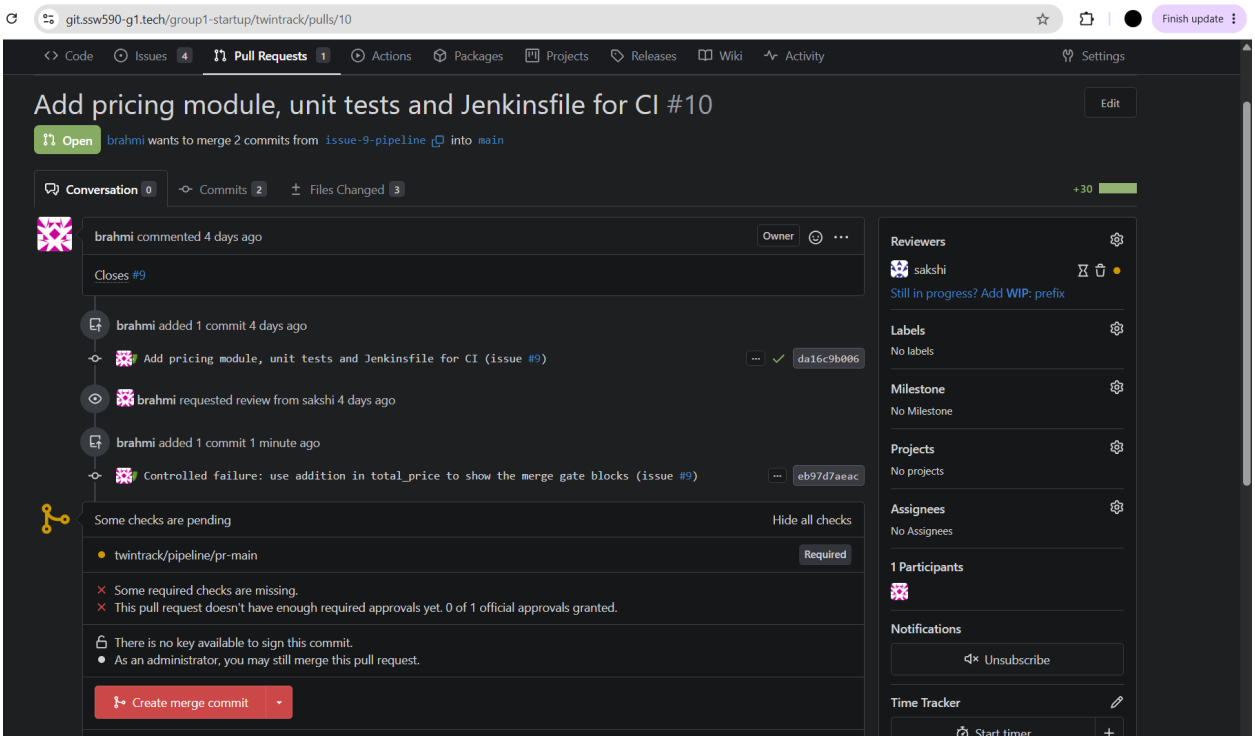


Figure 5.19: Pull request #10 right after the first failing push (commit `eb97d7a`), November 2, 2026: the required check is marked *Required* and still pending, with "Some required checks are missing" and "0 of 1 official approvals"; the banner "As an administrator, you may still merge" is visible further down.

5.2.6 Failure, repair, and review (10 points)

Owner: SD. **Status:** two failures and repairs with matching IDs, peer review by SD, merge, and issue closure all observed; checked by SD on November 3, 2026 against Gitea's pull-request timeline **PASS**.

The work item is issue #9 ("Add pricing module, unit tests and Jenkinsfile for CI", Figure 5.20) and the pull request #10 with `Closes #9` in its description, from branch `issue-9-pipeline` into `main`. The controlled failure was made twice, the second time to capture the merge-gate screenshots with automatic builds.

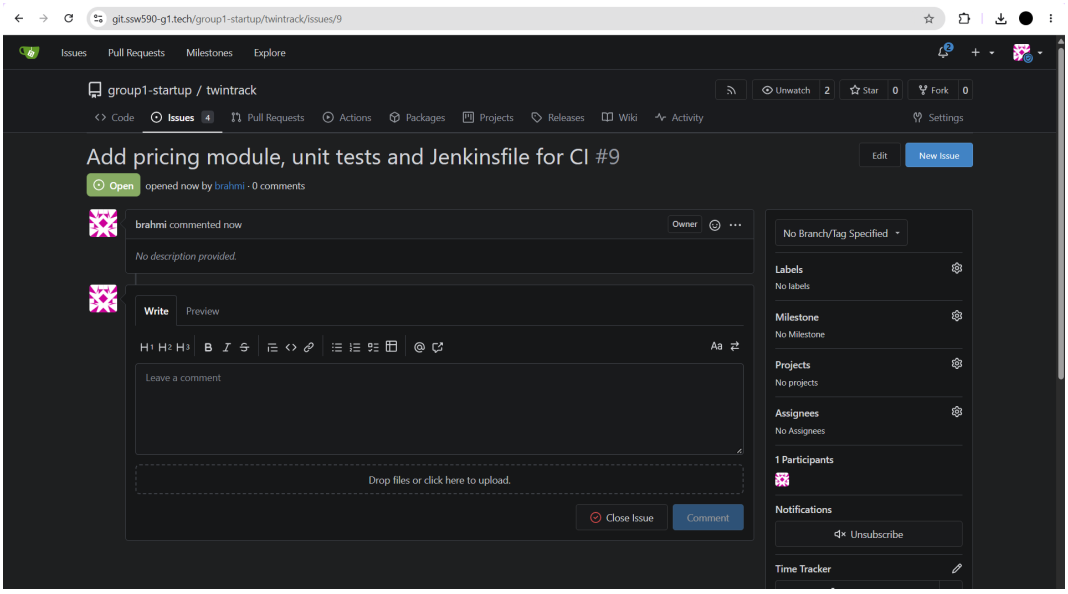


Figure 5.20: Issue #9 in `twintrack`, opened October 29, 2026 by `brahmi`.

Table 5.7: Dated sequence of commits and Jenkins builds for pull request #10, October 29 to November 2, 2026

Build	Commit	Change	Result	Started by, and evidence
#1	`da16c9b`	bootstrap: Python example, tests, `Jenkinsfile`	success	repository scan (Listing 32)
#2	`eb97d7a`	controlled failure: addition instead of multiplication	**failure** (`13 != 30`)	started *by hand* by BD (Listing 37, Figure 5.19)
none	`7d7848f`	repair	*never built*	the webhook lacked the Synchronized event (Listing 27)
#3	`d5e5c64`	no file change; proves the corrected webhook	success	**webhook** (Listing 26)
#4	`86ee9fc`	second controlled failure	**failure** (`13 != 30`)	**webhook** (Listing 35, Figure 5.15)
#5	`92c121d`	repair: multiplication restored	success	**webhook** (Listing 36, Figure 5.17)
review	`92c121d`	SD approves pull request #10 with a review comment (00:51:00 UTC)	approved	SD in Gitea (Listing 38)

Build	Commit	Change	Result	Started by, and evidence
merge	`a0094a8`	SD merges #10 into `main` (00:51:29 UTC); issue #9 closes	merged	SD in Gitea (Listing 38)
`main` #1	`a0094a8`	Jenkins builds the merged `main`	success	**webhook** (Listing 38)

Expected versus actual. The test `test_total` expects $10 \times 3 = 30$; with the broken line it computes 13, so the *Test* stage must fail and the required check must turn red. That is what builds #2 and #4 show (Listing 37, Listing 35); builds #1, #3 and #5 pass. The final content of the pull request equals the original bootstrap (the diff between `da16c9b` and `92c121d` is empty). The first demonstration was imperfect and is kept as a failed attempt, not hidden: build #2 was started manually and the repair commit was never built, because of the webhook event described in 5.2.3; the second round was fully automatic. Nothing was done to the deployed Gitea or Jenkins services.

Push times and matching IDs. Gitea's timeline of the pull request records every push with its commit (Listing 34): `da16c9b` on October 30, 00:07 UTC (October 29 US Eastern), then on November 3 (UTC) `eb97d7a` 00:14:34, `7d7848f` 00:17:23, `d5e5c64` 00:24:27, `86ee9fc` 00:30:22, and `92c121d` 00:32:51. Each commit's statuses in Gitea name the Jenkins run that produced them (`PR-10/2` for `eb97d7a`, ..., `PR-10/5` for `92c121d`), which is how the IDs in Table 5.7 are matched across the two systems. Two details a reader may notice: the second failure and its repair were *committed* one second apart (19:25:40 and 19:25:41 US Eastern) because both were prepared together, but they were *pushed* separately, two and a half minutes apart, so the failing commit was the pull request's head, and was tested and shown as blocking, before the repair existed on the server. For `eb97d7a`, the first status appeared about two minutes after the push, when BD started build #2 by hand; for the webhook-started builds it appeared within three seconds of the push.

```
Pull request #10 of group1-startup/twintrack, read anonymously (no token) 2026-11-03 01:11:36 UTC
Source: Gitea API /issues/10/timeline and /commits/<sha>/statuses. Times UTC.

=== Timeline (pushes, review request, review, merge) ===
2026-10-30T00:07:32  push    by brahmi commits da16c9b
2026-10-30T00:08:17  review requested by brahmi from sakshi
2026-11-03T00:14:34  push    by brahmi commits eb97d7a
2026-11-03T00:17:23  push    by brahmi commits 7d7848f
2026-11-03T00:24:27  push    by brahmi commits d5e5c64
2026-11-03T00:30:22  push    by brahmi commits 86ee9fc
2026-11-03T00:32:51  push    by brahmi commits 92c121d
2026-11-03T00:51:00  review  by sakshi (state from /reviews below)
2026-11-03T00:51:29  merge   by sakshi
review   2026-11-03T00:51:00  sakshi  APPROVED
merged   2026-11-03T00:51:29  by sakshi  merge commit a0094a8
issue #9 closed at 2026-11-03T00:51:29

=== Commit statuses (context, state, time, Jenkins run) ===
```

```
da16c9b  committed 2026-10-29T19:56:02-04:00
    2026-11-02T23:54:51  twintrack/pipeline/pr-main   pending  by jenkins-bot run PR-10
    2026-11-02T23:55:09  twintrack/pipeline/pr-main   pending  by jenkins-bot run PR-10/1
    2026-11-02T23:55:11  twintrack/pipeline/pr-main   success  by jenkins-bot run PR-10/1
eb97d7a  committed 2026-11-02T19:02:22-05:00
    2026-11-03T00:16:47  twintrack/pipeline/pr-main   pending  by jenkins-bot run PR-10
    2026-11-03T00:16:51  twintrack/pipeline/pr-main   pending  by jenkins-bot run PR-10/2
    2026-11-03T00:16:52  twintrack/pipeline/pr-main   failure  by jenkins-bot run PR-10/2
7d7848f  committed 2026-11-02T19:09:42-05:00
    (no status: never built)
d5e5c64  committed 2026-11-02T19:22:59-05:00
    2026-11-03T00:24:30  twintrack/pipeline/pr-main   pending  by jenkins-bot run PR-10
    2026-11-03T00:24:43  twintrack/pipeline/pr-main   pending  by jenkins-bot run PR-10/3
    2026-11-03T00:24:44  twintrack/pipeline/pr-main   success  by jenkins-bot run PR-10/3
86ee9fc  committed 2026-11-02T19:25:40-05:00
    2026-11-03T00:30:24  twintrack/pipeline/pr-main   pending  by jenkins-bot run PR-10
    2026-11-03T00:30:36  twintrack/pipeline/pr-main   pending  by jenkins-bot run PR-10/4
    2026-11-03T00:30:38  twintrack/pipeline/pr-main   failure  by jenkins-bot run PR-10/4
92c121d  committed 2026-11-02T19:25:41-05:00
    2026-11-03T00:32:53  twintrack/pipeline/pr-main   pending  by jenkins-bot run PR-10
    2026-11-03T00:33:00  twintrack/pipeline/pr-main   pending  by jenkins-bot run PR-10/5
    2026-11-03T00:33:18  twintrack/pipeline/pr-main   success  by jenkins-bot run PR-10/5
a0094a8  committed 2026-11-03T00:51:28+00:00
    2026-11-03T00:51:32  twintrack/pipeline/head      pending  by jenkins-bot run main
    2026-11-03T00:51:46  twintrack/pipeline/head      pending  by jenkins-bot run main/1
    2026-11-03T00:51:49  twintrack/pipeline/head      success  by jenkins-bot run main/1

main: protected=True required_approvals=1 enable_status_check=True contexts=['twintrack/pipeline/pr-main']
```

Listing 34: Pull request #10 from Gitea's API without a token, November 3, 2026: pushes with commit IDs, the review request, SD's approval, the merge, issue #9's closure, every commit status with its Jenkins run, and the branch rule (real output).

```
Real output, Nov 2, 2026 evening US Eastern (UTC times below are Nov 3). Run by Claude Code on Brahmi's
↪  go-ahead: the commit was pushed by Claude Code with Git's saved sign-in on Brahmi's computer (no token was
↪  handled); Jenkins was read-only inspected with docker exec; Gitea's status API was read anonymously.
Purpose: the second, fully automatic controlled failure, done to capture the "failed required check blocks the
↪  merge" state on the open pull request #10 (the first failure, build #2, had been started manually).

=== The change pushed to the branch of pull request #10 ===
commit 86ee9fc "Controlled failure for the merge-gate screenshots: addition instead of multiplication (issue
↪  #9)"
pricing.py:   return unit_price * quantity   ->   return unit_price + quantity

=== Jenkins build #4 of job twintrack/PR-10 ===
start: epoch 1793665833958 (2026-11-03 00:30:33 UTC)   duration: 3984 ms (about 4 s)   result: FAILURE
Cause recorded in build.xml: jenkins.branch.BranchEventCause (started by an SCM event, not by a user)
First line of the console log: "Pull request #10 event"
Checking out Revision 86ee9fc032fedfb78cf7462a048f66f09ef253bf (issue-9-pipeline)
Commit message: "Controlled failure for the merge-gate screenshots: addition instead of multiplication (issue
↪  #9)"
[Gitea] Notifying pull request build status: PENDING Build started...
FAIL: test_total (test_pricing.PricingTests.test_total)
AssertionError: 13 != 30
Ran 2 tests in 0.004s
FAILED (failures=1)
ERROR: script returned exit code 1
[Gitea] Notifying pull request build status: FAILURE There was a failure building this commit
Finished: FAILURE

=== Commit statuses Gitea holds for 86ee9fc (anonymous API), by jenkins-bot, context
↪  twintrack/pipeline/pr-main ===
00:30:24 pending (Build queued...)
```

```
00:30:36 pending (Build started...)
00:30:38 failure (There was a failure building this commit)

=== All Jenkins builds of the job so far ===
#1 SUCCESS (scan, da16c9b)   #2 FAILURE (manual, eb97d7a)   #3 SUCCESS (webhook, d5e5c64)   #4 FAILURE
↪  (webhook, 86ee9fc)
Next: the repair commit 92c121d (restores the multiplication) is pushed after the screenshots of this red
↪  state.
```

Listing 35: Build #4, the second controlled failure (commit `86ee9fc`): cause, failing test, and Gitea statuses (real output).

```
Real output, Nov 2, 2026 evening US Eastern (UTC times below are Nov 3). Run by Claude Code on Brahmi's
↪  go-ahead: the repair commit was pushed by Claude Code with Git's saved sign-in on Brahmi's computer (no
↪  token handled); Jenkins was read-only inspected with docker exec; Gitea's API was read anonymously.
Purpose: the repair after the second controlled failure (evidence 96-98), built automatically by the webhook.

=== The repair pushed to the branch of pull request #10 ===
commit 92c121d "Repair after the merge-gate screenshots: restore multiplication (issue #9)"
pricing.py:   return unit_price + quantity   ->   return unit_price * quantity
Final content of the pull request equals the original (diff against d5e5c64 is empty).

=== Jenkins build #5 of job twintrack/PR-10 ===
start: epoch 1793665978962 (2026-11-03 00:32:58 UTC)   duration: 18951 ms (about 19 s)   result: SUCCESS
Cause recorded in build.xml: jenkins.branch.BranchEventCause (started by an SCM event, not by a user)
First line of the console log: "Pull request #10 event"
Checking out Revision 92c121dec9da4c51e5080b198e82be7c405df799 (issue-9-pipeline)
Commit message: "Repair after the merge-gate screenshots: restore multiplication (issue #9)"
[Gitea] Notifying pull request build status: PENDING Build started...
+ python3 -m py_compile pricing.py
+ python3 -m unittest discover -s tests -v
Ran 2 tests in 0.000s
OK
[Gitea] Notifying pull request build status: SUCCESS This commit looks good
Finished: SUCCESS

=== Gitea statuses for 92c121d (anonymous API), by jenkins-bot, context twintrack/pipeline/pr-main ===
00:32:53 pending   00:33:00 pending   00:33:18 success
Combined status of 92c121d: success

=== Pull request #10 as Gitea sees it ===
head: 92c121d | open: True | merged: False | mergeable: True (approval still outstanding)
Commits on the PR, oldest first: da16c9b (bootstrap), eb97d7a (failure 1), 7d7848f (repair 1, never built),
↪  d5e5c64 (no-change commit that proved the webhook), 86ee9fc (failure 2), 92c121d (repair 2)

=== All Jenkins builds of the job ===
#1 SUCCESS (scan, da16c9b)   #2 FAILURE (started manually, eb97d7a)   #3 SUCCESS (webhook, d5e5c64)   #4
↪  FAILURE (webhook, 86ee9fc)   #5 SUCCESS (webhook, 92c121d)
Server memory right after: 158 MB buff/cache, 91 MB available, swap 452 MB used of 2047 MB (Jenkins and the
↪  build fit; Gitea healthy).
```

Listing 36: Build #5, the repair (commit `92c121d`): cause, passing tests, Gitea statuses, and all builds so far (real output).

```
Real output, Nov 2, 2026 (evening US Eastern; UTC timestamps below are already Nov 3). Run by Claude Code on
↪  Brahmi's go-ahead: read-only look inside the Jenkins container (docker exec jenkins ... grep of the build
↪  log, annotation blobs removed) and anonymous reads of Gitea's API.
The failing commit was created on purpose (manual V1.38, Chapter 4, step 4: replace the multiplication by
↪  addition) and pushed by Brahmi to the branch of pull request #10. It was NOT pushed to main and no service
↪  was broken.
```

```
=== The change (commit eb97d7a, "Controlled failure: use addition in total_price to show the merge gate blocks
↪  (issue #9)") ===
pricing.py:   return unit_price * quantity   ->   return unit_price + quantity
Local pre-check on Brahmi's computer before pushing: test_total fails with AssertionError: 13 != 30.

=== Jenkins build #2 of job twintrack/PR-10 (started MANUALLY by Brahmi in Jenkins after the push; the webhook
↪  did NOT start it, see the correction at the end) ===
start: 2026-11-03 00:16:47 UTC (epoch 1793665007)   duration: 5367 ms (about 5 s)   result: FAILURE
Checking out Revision eb97d7aeacc3f6d6933ecbb2739338dcc610e5cf (issue-9-pipeline)
Commit message: "Controlled failure: use addition in total_price to show the merge gate blocks (issue #9)"
[Gitea] Notifying pull request build status: PENDING Build started...
FAIL: test_total (test_pricing.PricingTests.test_total)
AssertionError: 13 != 30
Ran 2 tests in 0.003s
FAILED (failures=1)
ERROR: script returned exit code 1
[Gitea] Notifying pull request build status: FAILURE There was a failure building this commit
Finished: FAILURE
(Builds before it: #1 SUCCESS for commit da16c9b, about 12 s.)

=== Commit statuses Gitea holds for eb97d7a (anonymous API:
↪  /api/v1/repos/group1-startup/twintrack/commits/eb97d7a/statuses), by jenkins-bot ===
2026-11-03T00:16:52 | context 'twintrack/pipeline/pr-main' | failure | There was a failure building this
↪  commit
2026-11-03T00:16:51 | context 'twintrack/pipeline/pr-main' | pending | Build started...
2026-11-03T00:16:47 | context 'twintrack/pipeline/pr-main' | pending | Build queued...
Combined status of eb97d7a: failure (1 status)

=== Branch rule that applies (Gitea API for main) ===
protected: True | required_approvals: 1 | enable_status_check: True | contexts: ['twintrack/pipeline/pr-main']

=== CORRECTION (written after Brahmi told us she had started build #2 herself; revised after the push times
↪  were checked) ===
An earlier version of this file called build #2 "webhook-triggered". That was wrong. Jenkins' own record shows
↪  build #1 began with "Branch indexing" (the repository scan), and build #2's log has no indexing or webhook
↪  line; Brahmi confirmed she ran it manually ("Build Now"). Build #2 started at 00:16:47 UTC (7:16 PM US
↪  Eastern).
Push times, taken from Jenkins' Multibranch Pipeline Events (evidence 93): the failing commit eb97d7a was
↪  pushed at 00:14:35 UTC (7:14 PM US Eastern), and the repair commit 7d7848f at 00:17:24 UTC (7:17 PM),
↪  after build #2. (An earlier version of this file gave 19:02 and 19:09; those were the commits' author
↪  times, not the push times.)
Jenkins did not start a build for the failing push; the repair commit 7d7848f was never built (the next build
↪  number stayed at 3). Cause, found afterwards: the webhook lacked the "Pull Request Synchronized" event
↪  (evidence 92 and 93). After that event was ticked, build #3 was the first one started by the webhook
↪  itself (evidence 94).

=== Note ===
The pull request page that Brahmi captured right after this push (screenshot 90) was taken while the check was
↪  still "pending" (required, not yet reported) and shows the merge box with "Some required checks are
↪  missing" and "0 of 1 official approvals". The red failed state of the required check on the pull request
↪  page was captured later, after the second controlled failure (evidence 96 and 97), and the passing state
↪  after the repair (evidence 99, 100 and 101).
```

Listing 37: Build #2, the first controlled failure (commit `eb97d7a`), started manually, with the correction of an earlier wrong "webhook-triggered" label (real output).

Peer review, merge, and closure. Pull request #10 was assigned for review to SD, the second member, on October 29, 2026, and Gitea required her approval (Figure 5.17, "0 of 1 official approvals granted" until then). On November 2, 2026 (19:51 US Eastern; 00:51 UTC on November 3) SD approved it with a review comment: she checked the three files against the course manual, ran the tests and `py_compile` locally (two tests `ok`), and

suggested a later test for a negative quantity. She merged it 29 seconds later as the merge commit `a0094a8`, and **issue #9 closed automatically** at the same moment (Listing 38). The approval came from SD, the second member, not from the author BD, and it covered the head commit `92c121d` that had passed the required check 18 minutes earlier; no commit was pushed between the passing check and the merge. The merge itself started a Jenkins build of `main` (build #1 of the `main` job, success in about 9 seconds, started by a branch event); its status context is `twintrack/pipeline/head`, because a build of a branch reports a different context than a build of a pull request (`pr-main`), which is the one the branch rule requires.

```
Real output, Nov 2, 2026 evening US Eastern (UTC times below are Nov 3). Run by Claude Code on Brahmi's
↪  go-ahead: anonymous reads of Gitea's public API, and a read-only look inside the Jenkins container on the
↪  server (docker exec ... build.xml and console log; annotation blobs removed).
Purpose: the peer review, the merge, and the closing of the issue for pull request #10 (row "Failure, repair,
↪  and review"), and the Jenkins build of main that the merge started. The review and the merge were done by
↪  SD, the second member, in Gitea; the author of the pull request is BD.

=== Pull request #10 ===
title: Add pricing module, unit tests and Jenkinsfile for CI
state: closed   merged: True   merged_at: 2026-11-03T00:51:29Z   merged_by: sakshi
merge_commit_sha: a0094a823080adff5c314c5d3df0237811be4c9d
base: main   head: issue-9-pipeline 92c121dec9da4c51e5080b198e82be7c405df799
body: Closes #9

=== Review ===
reviewer: sakshi | state: APPROVED | submitted_at: 2026-11-03T00:51:00Z | commit reviewed: 92c121d
comment: "Checked against manual Ch. 4 step 4: pricing.py, tests and Jenkinsfile match. Ran python3 -m
↪  unittest discover -s tests -v locally: 2 tests OK, and py_compile passes. One suggestion for later: a test
↪  for a negative quantity (e.g. -1) would also cover the < 1 boundary."

=== Issue #9 ===
title: Add pricing module, unit tests and Jenkinsfile for CI
state: closed   closed_at: 2026-11-03T00:51:29Z   (closed automatically by the merge, 0 seconds after it)

=== Statuses Gitea holds for the merge commit a0094a8 (posted by jenkins-bot) ===
2026-11-03T00:51:32 | twintrack/pipeline/head | pending | Build queued...
2026-11-03T00:51:46 | twintrack/pipeline/head | pending | Build started...
2026-11-03T00:51:49 | twintrack/pipeline/head | success | This commit looks good
Note: the context for a build of the main BRANCH is twintrack/pipeline/head; builds of pull requests report
↪  twintrack/pipeline/pr-main, which is the context required by the branch rule.

=== Jenkins job folders ===
PR-10, main

=== Jenkins build #1 of the branch job main ===
start: epoch 1793667099941 (2026-11-03 00:51:39 UTC)   duration: 9150 ms (about 9 s)   result: SUCCESS
Cause recorded in build.xml: jenkins.branch.BranchEventCause (started by an SCM event, the push of the merge)
Checking out Revision a0094a823080adff5c314c5d3df0237811be4c9d (main)
Commit message: "Merge pull request 'Add pricing module, unit tests and Jenkinsfile for CI' (#10) from
↪  issue-9-pipeline into main"
[Gitea] Notifying branch build status: PENDING Build started...
+ python3 -m py_compile pricing.py
+ python3 -m unittest discover -s tests -v
Ran 2 tests in 0.000s
OK
[Gitea] Notifying branch build status: SUCCESS This commit looks good
Finished: SUCCESS

=== Sequence ===
00:51:00 UTC review approved; 00:51:29 UTC merged (merge commit a0094a8) and issue #9 closed;
↪  00:51:32-00:51:49 UTC Jenkins statuses for the merge commit; Jenkins build main #1 SUCCESS.
```

Listing 38: Pull request #10: SD's review and approval, the merge (commit `a0094a8`), issue #9 closed automatically, and the Jenkins build of `main`, November 2, 2026 (real output).

5.2.7 Credentials and build boundary (10 points)

Owner: BD. **Status:** described from the observed configuration **PASS**.

Table 5.8: Secrets and where they live (hints only, no values)

Secret	Stored in	Scope and handling
Jenkins one-time unlock password	the Jenkins volume on the server only	Read once on the server to finish setup; never copied anywhere
Jenkins account passwords (`brahmi`, `sakshi`)	password managers of the owners	Sakshi received a temporary password privately and changes it at first sign-in; sign-up is closed
`jenkins-bot` Gitea token	Jenkins credential store (`gitea-jenkins-bot`)	Narrow scopes; Write only on `twintrack`; to be deleted after grading
Webhook secret	none	The webhook only travels over the private Docker network and Jenkins is not reachable from outside; this is a stated limitation
Team address list	the server's untracked `.env`, mode 600	Not in the repository, the compose file, or this PDF

Who can change the Jenkinsfile. Anyone with write access to `twintrack`: BD and SD (and the integration user, which only reads and reports). Because `main` is protected, a change to the `Jenkinsfile` reaches `main` only through a pull request with one approval and a passing required check. Registration on Gitea is closed, so there are no outside accounts that could open a pull request.

Where builds run and the specific risk. Builds run on the **Jenkins controller executor**, inside the Jenkins container, not on a separate agent (allowed by the course manual for a small deployment [23, 1]). The risk: pipeline code is run on the same process that holds Jenkins' credentials and configuration, so a malicious pipeline could read them [24, 25]. Compensating restrictions: only two trusted people can change pipeline code; no outsiders can create accounts or pull requests; the example pipeline uses no credentials step and no repository secret; the container has no Docker socket and is not privileged; its memory is limited; and Jenkins is not reachable from the internet beyond the login page. Untrusted pull requests therefore cannot obtain secrets or privileged Docker access because there are no untrusted contributors in this deployment.

Plugin-update check and operator response. On November 2, 2026 the 60 installed plugins and the core were compared with Jenkins' stable update center: the core (2.580.2) is current; one plugin has a newer version (`workflow-cps`); and no advisory applies to any installed version (Listing 39). No update was applied days before the deadline, to avoid breaking a working, evidenced pipeline; it would be applied after submission by rebuilding the image. A first count of "78 warnings" was wrong (historical advisories) and is corrected in the same listing.

```
Real output, Nov 2, 2026 evening US Eastern. Run by Claude Code on Brahmi's go-ahead from Brahmi's Windows
↪  machine (a Python script that reads Jenkins' public update center,
↪  https://updates.jenkins.io/stable/update-center.actual.json). Nothing was changed on the server.
Purpose (course manual, Chapter 4, row 7): the "plugin-update check".
Method: the 60 plugin versions installed in the image (evidence 72) and the Jenkins core version 2.580.2 were
↪  compared with the stable update center. Security advisories were tested against the INSTALLED versions: an
↪  advisory counts only if its affected-version pattern matches the installed version.

=== Result ===
plugins installed: 60 | with a newer version on the stable update center: 1
  workflow-cps: installed 4395.v738456a_5da_6b_  ->  newer 4403.v471713a_cea_19
update-center security advisories whose affected-version pattern matches an INSTALLED version (plugins + core
↪  2.580.2): 0
core: installed 2.580.2, latest stable offered 2.580.2

Plugins checked individually, installed = latest: gitea 286.v0740a_413c1eb_, git 5.11.0, git-client 6.6.1,
↪  workflow-aggregator 608.v67378e9d3db_1, matrix-auth 3.3, credentials 1514.vb_01eb_014e7d4, script-security
↪  1436.v1082791135e4, branch-api 2.1303.v9f3b_95dc329d, workflow-multibranch 842.v3a_b_59b_57b_e6e.

=== Correction of a first, naive reading ===
A first version of the script printed "security warnings in the update center that name one of these plugins:
↪  78". That number is NOT a finding: the update center lists every historical advisory for a plugin together
↪  with the old version ranges they affected, so counting them says nothing about the installed versions. The
↪  second script above applies the version patterns and finds 0 applicable advisories.

=== Decision ===
No update was applied: one plugin (workflow-cps) has a newer version, no applicable advisory exists, and
↪  changing plugins days before the deadline would risk breaking a working, evidenced pipeline. The update
↪  would be applied after submission by rebuilding the image (plugins are installed at build time, evidence
↪  72) and replacing the container.
```

Listing 39: Plugin and core update check against the stable update center, November 2, 2026 (real output).

Troubleshooting a failed webhook or build.

1. *No build started after a push:* check the webhook's Recent Deliveries in Gitea, then Jenkins' Multibranch Pipeline Events; confirm the events ticked include Pull Request Synchronized; use Scan Repository Now as a fallback.

2. *Build red:* open the build's console; the failing stage and the assertion are printed; reproduce with `python -m unittest discover -s tests -v`.

3. *Status not on the pull request:* check that the Jenkins credential still works and that `jenkins-bot` still has Write; the status context must read `twintrack/pipeline/pr-main`.

4. *Jenkins slow or killed:* look at memory and swap on the server (`free -m`); Jenkins is limited to 640 MiB; restart only the Jenkins container with `docker compose restart jenkins`.

5. *Locked out of the dashboard:* the team address in `.env` may have changed; update it and recreate Caddy.

5.2.8 Manual quality and controls (10 points)

Owner: SD. **Status:** drafted by BD and Claude Code; checked and completed by SD on November 3, 2026 (activity diagram, citation check, troubleshooting pointer, evidence 107–108) **PASS**.

Version, history, and tables. The manual's version is increased from 1.1 to **1.2**. A dated history entry for this update links this chapter and names the Hosts and Passwords changes (1). **Hosts and Passwords tables: changed** (Chapter 2): a Jenkins row in Hosts, the `ci` DNS record, and Passwords hints for the Jenkins accounts, the `jenkins-bot` user and token, the webhook (no secret), and the team address list; no secret is recorded. Figure 5.7 is the adapted UML deployment diagram. New glossary and index terms: Jenkins, Jenkinsfile, webhook, status check, controller executor, multibranch pipeline. Citations are listed in the bibliography. On November 3, 2026, SD opened each cited Jenkins and Gitea page (Jenkins Docker installation, Gitea plugin, Gitea Checks plugin, security, credentials, controller isolation; Gitea webhooks and protected branches): all were available, and the protected-branches page was checked for the two claims this chapter makes with it, required status checks and approvals, and the administrator-override option [21].

Operator restart note. Restart Jenkins with `docker compose restart jenkins` in `~/jenkins` (ready in about 13 seconds, data kept); restart Caddy only with `docker compose up -d caddy` in `~/gitea` after validating the Caddyfile; Gitea needs about 40 seconds to be healthy. Both services restart on their own after a reboot (`unless-stopped`).

Operator troubleshooting note. If a pull request shows no status or the wrong one, follow the procedure in Section 5.2.7. The three checks that found this assignment's real problems are, in order: (1) the pull request's commit list in Gitea, where every head commit should get a `twintrack/pipeline/pr-main` status within seconds of its push (Listing 34); a commit with none was never built, as with `7d7848f`; (2) the repository webhook's recent deliveries and its event selection, which must include *Pull Request Synchronized* (Figure 5.13); (3) the failing Jenkins run's console, where the first assertion or error names the cause (`13 != 30` in builds #2 and #4).

Activity diagram. Figure 5.21 is the course's pull-request-to-merge activity (manual Figure 4.1) annotated with what actually happened on pull request #10. The webhook starts a run, but the merge is gated by the required check reported on the latest head commit, and a passing check still needed SD's approval before the merge.

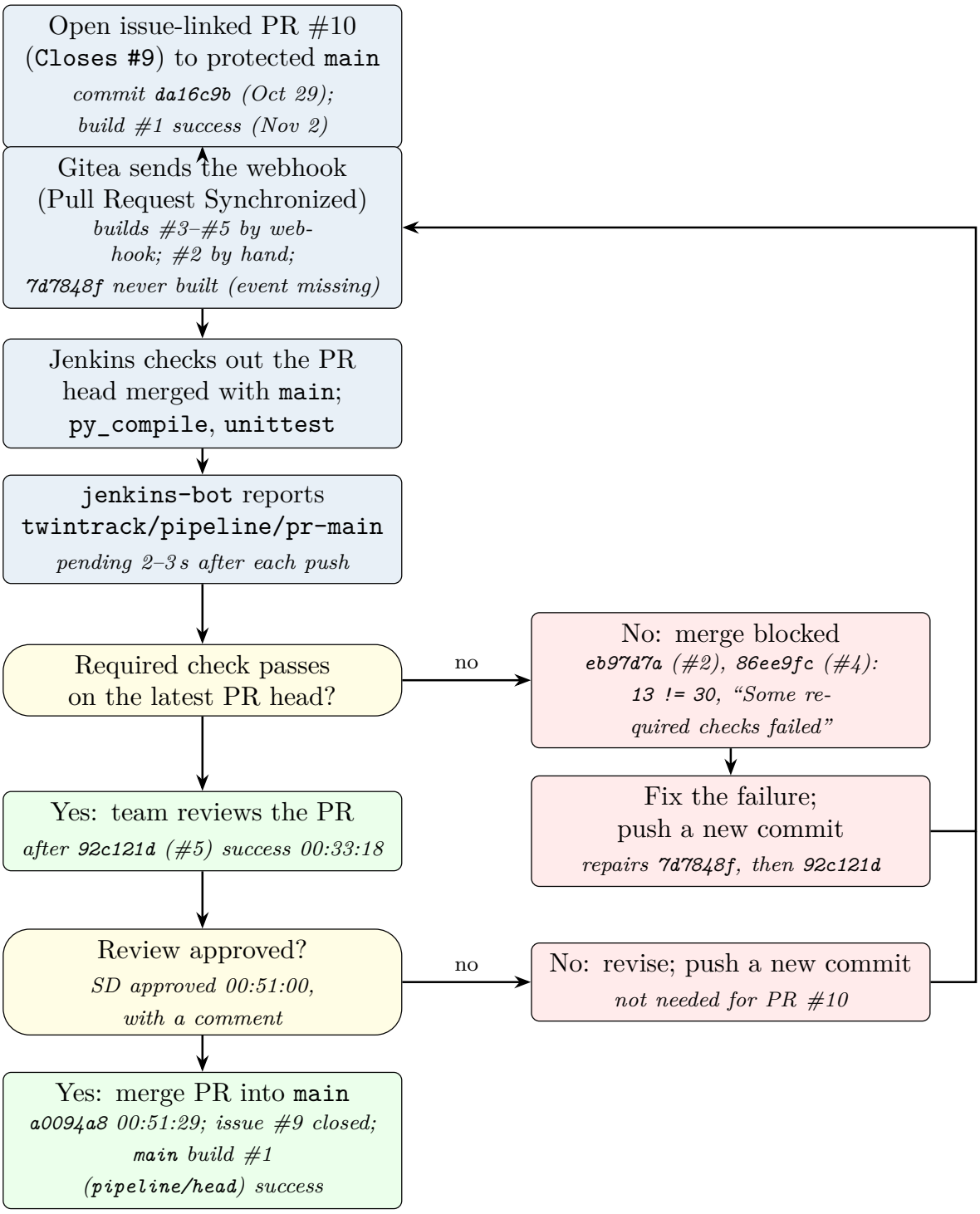


Figure 5.21: Pull-request-to-merge activity for pull request #10, adapted from the course manual's Figure 4.1 [1] with the real commits, Jenkins builds, and times (UTC, November 3, 2026, unless noted; Listing 34). A new commit repeats the build; the check on the latest head commit, not the webhook delivery, gates the merge.

Table 5.9: Failed tests and mistakes of this assignment, and the corrections

#	What went wrong	Correction and evidence
1	The webhook was first created in BD's personal account settings, so it never fired for the organization's repository	Recreated on the repository (Figure 5.11)
2	The webhook lacked the Pull Request Synchronized event: pushes reached Jenkins but no build was scheduled; the repair commit `7d7848f` was never built	Diagnosed from Listing 27 and Figure 5.13; fixed; proven by build #3 (Listing 26)
3	Build #2 was started by hand while the webhook did not work; an earlier evidence file wrongly called it webhook-triggered	Corrected in Listing 37
4	The first reading of the restart time (9 seconds) matched the previous start's log line, and `/login` answered 503 at that moment	Re-measured: about 12.6 seconds (Listing 23)
5	A first plugin check counted 78 historical advisories as findings	Re-tested against the installed versions: 0 apply (Listing 39)
6	A 403 at `/` seen from the team address was first taken for Caddy's block	Headers showed Jenkins' own response (Listing 24)
7	The first push of a test commit to Gitea failed ("Authentication failed") because a deleted token was still saved in the Git credential store	Saved sign-in cleared and replaced; no secret was exposed
8	The repository owner could override the merge rule ("As an administrator, you may still merge")	Accepted and disclosed (Figure 5.19); not enabled to block administrators

Limitations stated plainly. The first real renewal of the Jenkins certificate has not happened. The restart test shows persistence, not backup. The security group probes of 8080, 50000, and 3000 ran from the administrator's computer. The merge button area is outside the cropped screenshot of the blocked state. The webhook has no secret. The peer review of pull request #10 was a single review by SD (one required approval), and her suggestion for a negative-quantity test was not implemented. Jenkins shares a 1 GB host with Gitea and has little memory headroom.

Chapter 6

Monitoring and reliability (SRE)

6.1 Deployment and capacity (15 points)

Owner: SD. **Status:** deployed November 10, 2026; restart test November 11, 2026 **PASS**.

Baseline and capacity. Before anything was added, the server was read without changes (Listing 41): an EC2 `t3.micro` with 913 MiB of RAM, of which only 200 MiB was available and 425 MiB was already in swap, while Gitea, Jenkins, Caddy and the Actions runner were running. Four more containers would not have fit. BD decided to resize the same instance to `t3.small` (same disk and Elastic IP); afterwards the host had 1,909 MiB with 1,064 MiB available and no swap in use (Listing 42). The root disk had 8.4 GB free, enough for the images (about 1.4 GB) and Prometheus' capped storage.

What runs. Four separate containers, all from images published by the projects themselves; nothing is built locally and no Dockerfile is written. Tags are pinned to exact releases (never `latest`) and the pulled image digests are recorded (Listing 43 for the choice, Listing 44 for the digests on the server).

Table 6.1: Monitoring containers, versions, and limits

Container	Image	Role, limit, storage
`prometheus`	`prom/prometheus:v3.16.0`	scrapes every 30 s and stores metrics; 256 MiB; volume `monitoring_prometheus_data`, kept 7 days or 1 GB, whichever comes first
`grafana`	`grafana/grafana:13.3.1`	dashboard and alerts; 384 MiB; volume `monitoring_grafana_data` (users, data source, dashboard, alert rules)
`node-exporter`	`quay.io/prometheus/node-exporter:v1.12.1`	whole-host CPU, memory, root filesystem; 64 MiB; host `/` mounted read-only

Container	Image	Role, limit, storage
`blackbox-exporter`	`quay.io/prometheus/blackbox-exporter:v0.29.0`	HTTPS probes of the Gitea and Jenkins login pages; 64 MiB; no storage

The memory limits make sure that, if memory runs out, a monitoring container is stopped rather than Gitea or Jenkins. Grafana's limit started at 256 MiB, but Grafana used 216 MiB while idle, so it was raised to 384 MiB on the first day (recorded in Listing 44). Retention of 7 days follows the course's lab suggestion; the 1 GB cap protects the root disk that Jenkins builds also use.

Configuration in Gitea. The non-secret files are in the `infra` repository, folder `monitoring`, added through issue #1 and pull request #2. BD checked the Caddy block against the live Caddyfile and the memory limits against the running containers, and merged it on November 11, 2026, 19:42 EST, as merge commit `c03fb1b` (her review was given in the team chat; Gitea records the merge, not a formal approval). The same files run in `~/monitoring` on the server. No Compose service publishes a host port, none is privileged, and none mounts the Docker socket.

```
# Monitoring for the Group 1 startup server (course manual Chapter 5). Non-secret; start from ~/monitoring:
#   docker compose config --quiet && docker compose pull && docker compose up -d
# Four separate containers from project-published images. Design for a ~1 GB host shared with Gitea and
↪  Jenkins:
# - NO published host ports: Prometheus 9090, Node Exporter 9100 and Blackbox 9115 live only on the private
#   `monitoring` network; Grafana 3000 is reached by Caddy by service name on the existing `gitea_web` network.
# - No container gets the Docker socket or privileged mode.
# - Memory caps so that a monitoring container, not Gitea or Jenkins, is killed if memory runs out.
# - Named volumes keep Prometheus metrics history and Grafana state across container replacement.
#   Use `docker compose stop/start`; never `docker compose down -v` (deletes the volumes).
services:
  prometheus:
    image: prom/prometheus:${PROMETHEUS_TAG:?choose a release}
    container_name: prometheus
    restart: unless-stopped
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.path=/prometheus
      - --storage.tsdb.retention.time=7d        # course's suggested lab retention
      - --storage.tsdb.retention.size=1GB       # hard cap so the root disk keeps room for builds
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    mem_limit: 256m
    networks: [monitoring]

  grafana:
    image: grafana/grafana:${GRAFANA_TAG:?choose a release}
    container_name: grafana
    restart: unless-stopped
    environment:
      GF_SERVER_ROOT_URL: "https://metrics.ssw590-g1.tech/"
      GF_AUTH_ANONYMOUS_ENABLED: "false"
      GF_USERS_ALLOW_SIGN_UP: "false"
      GF_ANALYTICS_REPORTING_ENABLED: "false"
      GF_ANALYTICS_CHECK_FOR_UPDATES: "false"
      # No admin/admin window on the public login page: the first admin password is a random value generated
      # on the server (secrets/grafana_admin_password in a mode-700 folder, never printed, never in Git). The
      ↪  owner then
```

```
      # sets the real one with `docker exec -it grafana grafana cli admin reset-admin-password`.
      GF_SECURITY_ADMIN_PASSWORD__FILE: /run/secrets/grafana_admin_password
    secrets: [grafana_admin_password]
    volumes:
      - grafana_data:/var/lib/grafana
    mem_limit: 384m                          # 256m left too little room: Grafana 13.3.1 used ~216 MiB idle
    networks: [monitoring, web]

  node-exporter:
    image: quay.io/prometheus/node-exporter:${NODE_EXPORTER_TAG:?choose a release}
    container_name: node-exporter
    restart: unless-stopped
    pid: host                                # host process namespace, for host-wide CPU/memory figures
    command:
      - --path.rootfs=/host
      - --path.procfs=/host/proc
      - --path.sysfs=/host/sys
    volumes:
      - /:/host:ro,rslave                    # read-only host root, for root-filesystem size/free space
    mem_limit: 64m
    networks: [monitoring]

  blackbox-exporter:
    image: quay.io/prometheus/blackbox-exporter:${BLACKBOX_TAG:?choose a release}
    container_name: blackbox-exporter
    restart: unless-stopped
    command: ["--config.file=/config/blackbox.yml"]
    volumes:
      - ./blackbox/blackbox.yml:/config/blackbox.yml:ro
    mem_limit: 64m
    networks: [monitoring]                   # normal bridge egress: probes go out to the public HTTPS
    ↪ hostnames

networks:
  monitoring: {}                             # private: Prometheus <-> exporters <-> Grafana
  web:
    external: true
    name: gitea_web                          # existing network shared by Caddy, Gitea and Jenkins (reused,
    ↪ not recreated)

secrets:
  grafana_admin_password:
    file: ./secrets/grafana_admin_password       # created on the server only; gitignored

volumes:
  prometheus_data:
  grafana_data:
```

Listing 40: `compose.yaml` of the monitoring stack (`infra/monitoring`): four containers, no published ports, memory limits, named volumes, Grafana also on the existing `gitea_web` network.

Start, update, recover. From `~/monitoring` on the server:

```
cp .env.example .env                         # image tags only
docker compose config --quiet && docker compose pull && docker compose up -d
docker compose ps                            # four containers Up
```

To update, change a tag in `.env`, run `docker compose pull && docker compose up -d`, and record the new digests. To stop and start, use `docker compose stop` and `docker`

`compose start`. **Never** `docker compose down -v`: it deletes both volumes, that is, the metric history and all of Grafana's state. The first Grafana administrator password was a random value generated on the server into `secrets/grafana_admin_password` (folder mode 700, never printed, not in Git), so the default `admin`/`admin` was never valid on the public login page; BD then set her own password with `grafana cli admin reset-admin-password` (Section 6.3).

Deployment diagram. Figure 6.1 adapts the course's Figure 5.1 to this server.

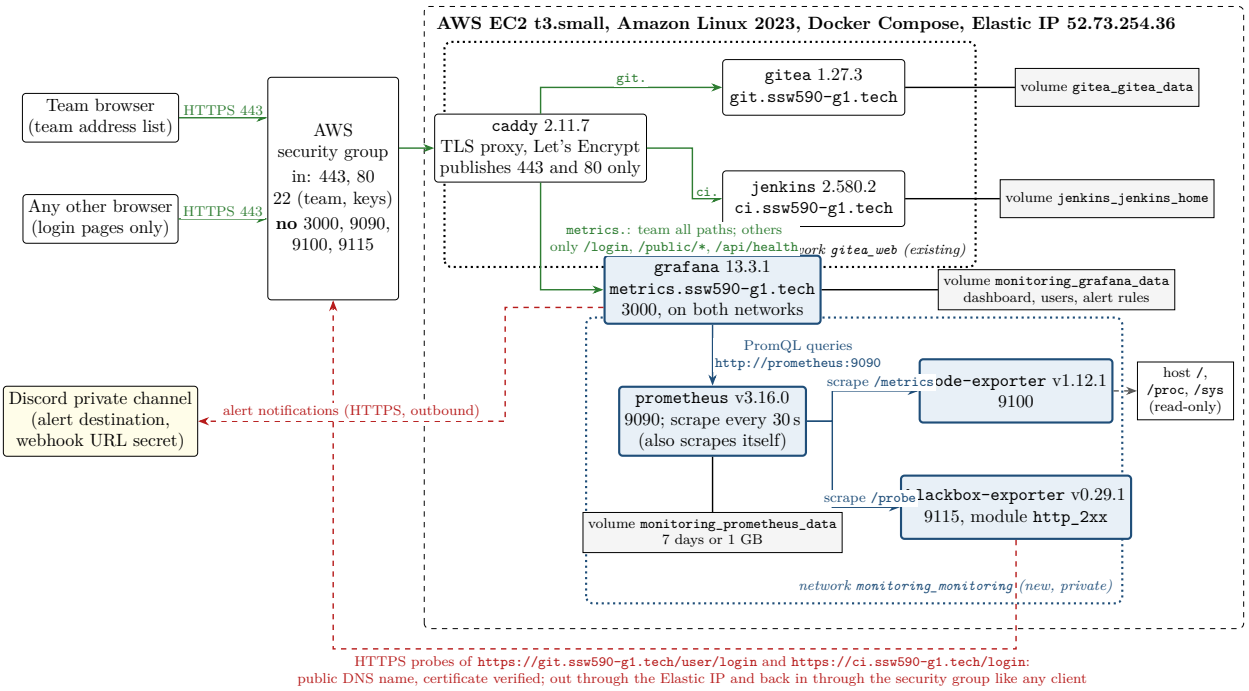


Figure 6.1: Deployment of the Group 1 server with monitoring, adapted from the course manual's Figure 5.1. Blue: the four new monitoring containers. Arrows start at the side that opens the connection. Gitea and Jenkins are probed only through their public HTTPS names; Prometheus, the exporters, and Grafana's port 3000 are reachable only on private Docker networks. Not drawn: the Gitea Actions runner and the Gitea–Jenkins webhook (Chapter 5).

Restart persistence. Right after BD's controlled Jenkins outage and after the SLI window had been read (Section 6.5), Prometheus and then Grafana were restarted with `docker restart`, on November 11, 2026, 8:38–8:40 PM New York time (Listing 45). Both kept the same container IDs and the same named volumes. Prometheus' oldest stored sample (03:15:08 UTC on November 11, the first scrape) and its 720 probe samples per URL from the last six hours were still there; Prometheus was ready again after 3 seconds, and one 30-second Gitea sample was missed (719 instead of 720). Grafana was healthy after 14 seconds, and its data source, both alert rules, the contact point `team-alerts`, both user accounts, and the dashboard (created at 8:25 PM, before the restart) were all still present. A first

reading reported zero dashboards: the script had queried Grafana's legacy dashboard table, while Grafana 13 keeps dashboards in its newer `resource` table; the corrected check is in the same listing. This shows that the volumes persist; it is not a backup and restore. The same test is BD's verification in Section 6.7.

```
=== Server baseline before monitoring (read-only), 2026-11-10 23:26:08 UTC ===
instance type: t3.micro  zone: us-east-1c
--- OS / kernel / arch
PRETTY_NAME="Amazon Linux 2023.12.20261014"
6.18.54-121.166.amzn2023.x86_64
x86_64
--- CPU
2
model name        : Intel(R) Xeon(R) Platinum 8259CL CPU @ 2.50GHz
--- memory (MiB)
               total        used        free      shared  buff/cache   available
Mem:             913         575          60           0         277         200
Swap:           2047         425        1622
--- root filesystem
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1   20G   12G  8.4G  58% /
/dev/nvme0n1p1 21395124224 12391948288 9003175936  58% /
--- load
 23:26:08 up 26 days,  6:09,  0 users,  load average: 0.03, 0.01, 0.00
--- docker
Docker Engine 25.0.16
Docker Compose version v5.6.0
--- containers
gitea        gitea/gitea:1.27.3        Up 8 days (healthy)        22/tcp, 3000/tcp
caddy        caddy:2.11.7        Up 8 days        0.0.0.0:80->80/tcp, :::80->80/tcp, 0.0.0.0:443->443/tcp,
↪  :::443->443/tcp, 443/udp, 2019/tcp
jenkins        group1/jenkins-python:2.580.2        Up 7 days        8080/tcp, 50000/tcp
gitea-runner        gitea/act_runner:0.6.1        Up 3 weeks
--- container memory (one sample)
gitea        1.57%        110.3MiB / 913.1MiB
caddy        0.00%        20.78MiB / 913.1MiB
jenkins        0.14%        271MiB / 640MiB
gitea-runner        0.11%        14.14MiB / 913.1MiB
--- Docker disk usage
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          5         4         6.669GB   5.818GB (87%)
Containers      4         4         217.8MB   0B (0%)
Local Volumes   6         5         275.3MB   0B (0%)
Build Cache     30        0         1.163kB   1.163kB
--- listening TCP sockets on the host (addresses only)
0.0.0.0:22
0.0.0.0:443
0.0.0.0:80
127.0.0.1:37001
[::]:22
[::]:443
[::]:80
--- service versions
gitea version 1.27.3 built with go1.26.7-X:jsonv2 : bindata, timetzdata,
Jenkins login HTTP 200
x-jenkins: 2.580.2
Gitea login HTTP 200
```

Listing 41: Baseline before monitoring: instance type, memory and swap, disk, containers, listening ports, November 10, 2026 (real output).

```
Real output, Nov 10, 2026 (evening US Eastern; UTC timestamps below are already Nov 11). Gate G1 (capacity)
↪  for Assignment 7.
```

```
Decision by Brahmi (BD) after SD's baseline (evidence 110: 913 MiB RAM, 200 MiB available, 425 MiB in swap):
↪  resize the single server from t3.micro to t3.small.
The AWS commands were run by Claude Code on Brahmi's explicit go-ahead for this resize only (stop, change
↪  type, start). Same instance, same disk, same Elastic IP.

=== AWS steps (us-east-1, instance i-0c68ec1bc1e16a414) ===
before:  t3.micro   running  52.73.254.36
stop-instances            -> stopping -> stopped
modify-instance-attribute --instance-type t3.small    -> ok
after:   t3.small    stopped
start-instances           -> pending -> running
after:   t3.small    running  52.73.254.36   (address unchanged)
Outage: Gitea, Jenkins, runner unreachable for the few minutes between stop and the containers' restart.

=== Public pages after the restart (from Brahmi's computer) ===
https://git.ssw590-g1.tech/        HTTP 200
https://ci.ssw590-g1.tech/login    HTTP 200

=== On the server, read-only, 2026-11-11 02:45:47 UTC (up 4 min; load 0.02 0.15 0.08) ===
               total        used        free      shared  buff/cache   available
Mem:            1909         705         524           0         680        1064
Swap:           2047           0        2047
swapon: /swapfile file 2G, 0B used (comes back by itself)
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1   20G   12G  8.4G  58% /

NAME          IMAGE                           STATUS
gitea         gitea/gitea:1.27.3              Up 4 minutes (healthy)
caddy         caddy:2.11.7                    Up 4 minutes
jenkins       group1/jenkins-python:2.580.2   Up 4 minutes
gitea-runner  gitea/act_runner:0.6.1          Up 4 minutes

NAME          MEMORY USED / LIMIT   CPU
gitea         96.54MiB / 1.865GiB   0.04%
caddy         10.93MiB / 1.865GiB   0.00%
jenkins       366.1MiB / 640MiB     0.12%
gitea-runner  6.879MiB / 1.865GiB   0.10%
(Jenkins is higher right after start-up than the 271 MiB in evidence 110; the other values are near the
↪  earlier ones.)

=== Comparison with evidence 110 ===
available RAM  200 MiB  -> 1064 MiB
total RAM      913 MiB  -> 1909 MiB
swap in use    425 MiB  -> 0 MiB
All four containers restarted on their own (restart policy); the disk is unchanged.

=== Cost note (to be confirmed on AWS's price page) ===
t3.small costs roughly twice t3.micro per hour (about 0.0104 USD/hr more). Accepted by Brahmi; revisit after
↪  grading.
```

Listing 42: Capacity decision by BD: resize from `t3.micro` to `t3.small` and the host afterwards, November 10, 2026 (real output, BD's evidence).

```
Image tags and digests chosen for the monitoring stack, read from the registries 2026-11-10 23:27:22 UTC by
↪  Claude Code for SD.
Server architecture: x86_64 (linux/amd64). Index digest = multi-arch list; amd64 digest = what the server will
↪  run.
Also: local validation of the drafted configuration (docker compose config, promtool, blackbox
↪  --config.check).

prom/prometheus:v3.16.0
    index sha256:e04987641b57704f408e31f6373ba4138ecf27cb4a165d24aabb6c38fddd0f06
    amd64 sha256:89c1a644817e9ad70fa2af76c84898e327c1de7063735f962e8313b1e28672c3
grafana/grafana:13.3.1
```

```
    index sha256:0615ec4fbd4072004b7d86b46ded63013a74b7d6eb1119273dd89e8fa25c13cf
    amd64 sha256:1c88285f1dd391a62232c010e6c224846e6a73bdc583b3365be2eba614b7919e
quay.io/prometheus/node-exporter:v1.12.1
    index sha256:1b4e4438faca4dd7e001dd445d161a4a2091b0fededa84093b3a8dfeae1f1be0
    amd64 sha256:da83fae85603c4e47e6c68369a7d746e2dda683dc35ea2e234b4f171e0d92798
quay.io/prometheus/blackbox-exporter:v0.29.1
    index sha256:ea1c7aa3ef3f307cd03e85f34abb604478ccb90b567fc690da0cd9776da76b13
    amd64 sha256:66eae352a79e6f677aeddd8c25a9bdb017586dc5fbd230f22329213d891f94bd

$ docker compose config --quiet
compose config OK
$ promtool check config prometheus.yml
Checking /etc/prometheus/prometheus.yml
 SUCCESS: /etc/prometheus/prometheus.yml is valid prometheus config file syntax
$ blackbox_exporter --config.check
time=2026-11-10T23:27:14.900Z level=INFO source=main.go:100 msg="(go=go1.27.3, platform=linux/arm64,
↪  user=root@ab03ed60a291, date=20261021-18:03:52, tags=unknown)"
time=2026-11-10T23:27:14.901Z level=INFO source=config.go:144 msg="Configuration file change detected,
↪  reloading the configuration."
time=2026-11-10T23:27:14.901Z level=INFO source=main.go:108 msg="Config file is ok exiting..."
```

Listing 43: Chosen tags with index and linux/amd64 digests, and local validation of the configuration, November 10, 2026 (real output).

```
Real output, Nov 10, 2026 evening US Eastern (server clock UTC, already Nov 11). Assignment 7, rows 1 and 2.
↪  Run by Sakshi (Claude Code over SSH as ec2-user).
Server: t3.small after BD's resize (evidence 140). Files copied from assignment-7/monitoring/ to ~/monitoring;
↪  .env copied from .env.example (tags only).

=== Change made before the first start (security) ===
Brahmi's Caddy block keeps /login public, and the Caddy log already showed scanners on the new name (evidence
↪  141).
So Grafana never started with admin/admin: compose passes GF_SECURITY_ADMIN_PASSWORD__FILE from a Compose
↪  secret,
~/monitoring/secrets/grafana_admin_password = `openssl rand -base64 33` (44 bytes), created on the server,
↪  never printed,
folder mode 700, gitignored. Brahmi replaces it with `docker exec -it grafana grafana cli admin
↪  reset-admin-password`.

=== 2026-11-11 03:14-03:15 UTC: docker compose config --quiet && docker compose pull && docker compose up -d
↪  ===
config OK
 Image prom/prometheus:v3.16.0 Pulled
 Image grafana/grafana:13.3.1 Pulled            (node-exporter and blackbox-exporter pulled likewise)
 Volume monitoring_prometheus_data Created
 Volume monitoring_grafana_data Created
 Network monitoring_monitoring Created
 Container node-exporter Started / prometheus Started / grafana Started / blackbox-exporter Started

$ docker compose ps
NAME                IMAGE                                          STATUS          PORTS
blackbox-exporter   quay.io/prometheus/blackbox-exporter:v0.29.1   Up 20 seconds   9115/tcp
grafana             grafana/grafana:13.3.1                         Up 20 seconds   3000/tcp
node-exporter       quay.io/prometheus/node-exporter:v1.12.1       Up 21 seconds   9100/tcp
prometheus          prom/prometheus:v3.16.0                        Up 20 seconds   9090/tcp
(PORTS shows container ports only; no host bindings.)

=== docker inspect (03:16:20 UTC) ===
/prometheus        restart=unless-stopped mem=268435456 privileged=false ports={} mounts=prometheus.yml(ro),
↪  monitoring_prometheus_data:/prometheus
/grafana           restart=unless-stopped mem=268435456 privileged=false ports={}
↪  mounts=monitoring_grafana_data:/var/lib/grafana, secret file
/node-exporter     restart=unless-stopped mem=67108864  privileged=false ports={} mounts=/:/host (ro,rslave)
/blackbox-exporter restart=unless-stopped mem=67108864  privileged=false ports={} mounts=blackbox.yml(ro)
```

```
No container mounts /var/run/docker.sock.

=== RepoDigests of the pulled images (match the index digests in evidence 111) ===
prom/prometheus:v3.16.0
↪  prom/prometheus@sha256:e04987641b57704f408e31f6373ba4138ecf27cb4a165d24aabb6c38fddd0f06 amd64
grafana/grafana:13.3.1
↪  grafana/grafana@sha256:0615ec4fbd4072004b7d86b46ded63013a74b7d6eb1119273dd89e8fa25c13cf amd64
quay.io/prometheus/node-exporter:v1.12.1
↪  quay.io/prometheus/node-exporter@sha256:1b4e4438faca4dd7e001dd445d161a4a2091b0fededa84093b3a8dfeae1f1be0
↪  amd64
quay.io/prometheus/blackbox-exporter:v0.29.1  quay.io/prometheus/blackbox-exporter@sha256:ea1c7aa3ef3f307cd03
↪  e85f34abb604478ccb90b567fc690da0cd9776da76b13 amd64
`docker buildx imagetools inspect` (SD's Mac) confirms each index's linux/amd64 entry is the amd64 digest in
↪  evidence 111.

=== Prometheus /api/v1/targets (job instance health lastScrape lastError) ===
blackbox   blackbox-exporter:9115                   up 2026-11-11T03:16:08 ''
https      https://git.ssw590-g1.tech/user/login    up 2026-11-11T03:16:16 ''
https      https://ci.ssw590-g1.tech/login          up 2026-11-11T03:16:19 ''
node       node-exporter:9100                       up 2026-11-11T03:16:16 ''
prometheus prometheus:9090                          up 2026-11-11T03:16:12 ''

=== Instant queries (03:16 UTC) ===
probe_success{instance="https://git.ssw590-g1.tech/user/login"}            1
probe_success{instance="https://ci.ssw590-g1.tech/login"}                  1
probe_http_status_code  git 200   ci 200
probe_duration_seconds  git 0.014027061   ci 0.025555427
probe_ssl_earliest_cert_expiry  git 1799858155   ci 1801368269
node_memory_MemTotal_bytes      2002350080
node_memory_MemAvailable_bytes  823209984
node_filesystem_size_bytes{device="/dev/nvme0n1p1",fstype="xfs",mountpoint="/"}   21395124224
node_filesystem_avail_bytes{device="/dev/nvme0n1p1",fstype="xfs",mountpoint="/"}  6994649088
count(node_cpu_seconds_total{mode="idle"})  2

=== Same moment on the host ===
$ free -b
               total        used        free      shared  buff/cache   available
Mem:      2002350080  1031618560    74612736     1048576   896118784   804671488
Swap:     2147479552   170827776  1976651776
$ df -B1 /
Filesystem       1B-blocks        Used  Available Use% Mounted on
/dev/nvme0n1p1 21395124224 14400479232 6994644992  68% /
Comparison: MemTotal equal to free's total; filesystem size equal to df 1B-blocks; avail differs by 4096 bytes
↪  (one block,
taken a few seconds apart); MemAvailable 823 MB vs free 805 MB, sampled ~10 s apart. Root fs is 32.7% free
↪  (68% used,
up from 58% because of the ~1.4 GB of new images).

=== Host listening sockets (sudo ss -ltnp) - unchanged: only 22, 80, 443 public ===
0.0.0.0:80 docker-proxy    0.0.0.0:443 docker-proxy    0.0.0.0:22 sshd    127.0.0.1:41143 containerd   (+ IPv6
↪  [::]:22/80/443)

=== Memory per container (docker stats --no-stream) ===
grafana 210.1MiB / 256MiB   prometheus 45.57MiB / 256MiB   node-exporter 11.23MiB / 64MiB   blackbox-exporter
↪  14.51MiB / 64MiB
gitea 102MiB   caddy 20.49MiB   jenkins 330.7MiB / 640MiB   gitea-runner 10.91MiB

=== Adjustment at 03:18 UTC: Grafana memory cap 256m -> 384m ===
Reason: 216 MiB used at idle, too close to the cap; an OOM kill would also stop alerting. Host available ~780
↪  MiB.
$ docker compose up -d   ->  Container grafana Recreate / Recreated / Started   (volume
↪  monitoring_grafana_data kept)
After restart: grafana 316.8MiB / 384MiB, OOMKilled=false, restarts=0; host available 747 MiB, swap used 165
↪  MiB of 2047.

=== Grafana from inside the network (from the caddy container) ===
GET http://grafana:3000/api/health ->
↪  {"database":"ok","version":"13.3.1","commit":"da3aa3ff574e7dbbf808c66b6daa744defed0441"}
```

```
POST http://grafana:3000/login {"user":"admin","password":"admin"} -> HTTP/1.1 401 Unauthorized
grafana log: msg="Failed to authenticate request" ... error="[password-auth.invalid] invalid password"
```

Listing 44: Deployment: pull and start, container settings, pulled digests, Prometheus targets, first probe results, host values, Grafana limit change, November 10, 2026 (real output).

```
Real output, Nov 11, 2026, 8:38-8:40 PM New York time (2026-11-12 01:38:39-01:40:08 UTC). Assignment 7, row 1
↪  (restart persistence);
also BD's row 7 verification. Run by Sakshi (Claude Code over SSH) from ~/monitoring, right after the SLI
↪  window was read (evidence 116),
so the restart gap falls outside that window. Script: assignment-7/sd-drafts/restart-test.sh. Containers
↪  restarted one at a time with
`docker restart`; no volume was removed. Grafana values are read from a COPY of grafana.db (titles and counts
↪  only, no secrets).

$ bash restart-test.sh | tee restart-test-20261112.log
=== Restart persistence test, started 2026-11-12 01:38:39 UTC on ip-172-31-29-240.ec2.internal ===
--- BEFORE (2026-11-12 01:38:39 UTC) ---
    /prometheus id=99345c8a4e02 started=2026-11-11T03:14:59.665762703Z restarts=0
    ↪  volumes=monitoring_prometheus_data
    /grafana id=7eb45a891fa9 started=2026-11-11T03:17:23.06653391Z restarts=0 volumes=monitoring_grafana_data
  oldest stored sample (prometheus_tsdb_lowest_timestamp_seconds):
    {'instance': 'prometheus:9090', 'job': 'prometheus'} 1794366908
  probe samples stored in the last 6 h:
    {'instance': 'https://ci.ssw590-g1.tech/login', 'job': 'https'} 720
    {'instance': 'https://git.ssw590-g1.tech/user/login', 'job': 'https'} 720
  Grafana state (from a copy of grafana.db):
    dashboards: 0: []
    [note added by SD: wrong query, see "Correction" below; the dashboard 'Group 1 Startup Server (SRE)'
    ↪  exists in Grafana 13's resource table]
    data sources: 1: ['prometheus (prometheus)']
    alert rules: 2: ['https-service-down', 'root-disk-below-15-percent']
    contact points (names only): 2: ['empty', 'team-alerts']
    users: 2: ['admin (server admin)', 'sakshi']
--- docker restart prometheus (2026-11-12 01:38:40 UTC) ---
prometheus
    prometheus ready at 2026-11-12 01:38:43 UTC
--- docker restart grafana (2026-11-12 01:38:43 UTC) ---
grafana
    grafana healthy at 2026-11-12 01:38:57 UTC
--- waiting 70 s for two new scrapes ---
--- AFTER (2026-11-12 01:40:07 UTC) ---
    /prometheus id=99345c8a4e02 started=2026-11-12T01:38:40.728113306Z restarts=0
    ↪  volumes=monitoring_prometheus_data
    /grafana id=7eb45a891fa9 started=2026-11-12T01:38:45.163823402Z restarts=0 volumes=monitoring_grafana_data
  oldest stored sample (prometheus_tsdb_lowest_timestamp_seconds):
    {'instance': 'prometheus:9090', 'job': 'prometheus'} 1794366908
  probe samples stored in the last 6 h:
    {'instance': 'https://ci.ssw590-g1.tech/login', 'job': 'https'} 720
    {'instance': 'https://git.ssw590-g1.tech/user/login', 'job': 'https'} 719
  Grafana state (from a copy of grafana.db):
    dashboards: 0: []
    [note added by SD: wrong query, see "Correction" below; the dashboard 'Group 1 Startup Server (SRE)'
    ↪  exists in Grafana 13's resource table]
    data sources: 1: ['prometheus (prometheus)']
    alert rules: 2: ['https-service-down', 'root-disk-below-15-percent']
    contact points (names only): 2: ['empty', 'team-alerts']
    users: 2: ['admin (server admin)', 'sakshi']
  collection gap: probe samples in the last 5 min (expected about 10 per URL; fewer = the gap):
    {'instance': 'https://git.ssw590-g1.tech/user/login', 'job': 'https'} 9
    {'instance': 'https://ci.ssw590-g1.tech/login', 'job': 'https'} 10
  targets now:
    {'instance': 'blackbox-exporter:9115', 'job': 'blackbox'} 1
    {'instance': 'prometheus:9090', 'job': 'prometheus'} 1
```

```
    {'instance': 'node-exporter:9100', 'job': 'node'} 1
    {'instance': 'https://git.ssw590-g1.tech/user/login', 'job': 'https'} 1
    {'instance': 'https://ci.ssw590-g1.tech/login', 'job': 'https'} 1
=== finished 2026-11-12 01:40:08 UTC ===

=== Correction to the "dashboards: 0" lines above (checked 01:40:31 UTC, after the test) ===
The script's dashboard query read Grafana's legacy `dashboard` table. Grafana 13 stores dashboards in unified
↪  storage
(table `resource`, group dashboard.grafana.app), so 0 was a wrong query, not a missing dashboard. Read-only
↪  check of the same database copy:
grafana started=2026-11-12T01:38:45.163823402Z
dashboard uid=group1-sre title='Group 1 Startup Server (SRE)' created=2026-11-12T00:25:18Z generation=1
↪  panels=7
  panel titles: ['1. Whole-host CPU utilization (%)', '2. Available host memory (bytes)', '3. Root filesystem
  ↪  free (%)', '6. HTTPS probe duration (seconds)', '4. Gitea HTTPS availability', '5. Jenkins HTTPS
  ↪  availability', 'Scrape health (up: 1 = scrape worked)']
history versions: 3
The dashboard was created at 00:25:18 UTC, before the Grafana restart at 01:38:45 UTC, and is unchanged after
↪  it.
The script was fixed afterwards to read the resource table.

=== Reading ===
- Same container IDs and the same named volumes before and after; only StartedAt changed (restart, not
↪  re-creation).
- Prometheus: oldest stored sample 1794366908 (= 2026-11-11 03:15:08 UTC, the first scrape) before and after:
↪  history kept.
  720 probe samples per URL in the last 6 h before and after (719 for Gitea after: one sample lost to the
  ↪  restart gap).
- Grafana: data source, both alert rules, contact point team-alerts, both users, and the dashboard are still
↪  present.
  ('empty' is Grafana's built-in placeholder contact point.)
- Collection gap: 9 Gitea samples instead of about 10 in the last 5 minutes = one missed 30 s scrape while
↪  Prometheus restarted
  (ready 3 s after the restart command); Grafana was healthy 14 s after its restart. All five targets up
  ↪  afterwards.
- This shows that the volumes persist across container restarts. It is not a backup and restore.
```

Listing 45: Restart test of Prometheus and Grafana: state before and after, readiness times, collection gap, and the corrected dashboard check, November 11, 2026 (real output).

6.2 Metrics and HTTPS probes (15 points)

Owner: SD. **Status:** all targets up and host values verified November 10, 2026 **PASS**.

What is scraped. Prometheus scrapes four jobs every 30 seconds, all by private service name on the `monitoring` network (Listing 46): itself (`prometheus:9090`), the Node Exporter (`node-exporter:9100`), the Blackbox Exporter's own metrics (`blackbox-exporter:9115`), and the `https` probe job. Grafana reads only from Prometheus; it does not contact Gitea, Jenkins, or the exporters. Right after the start, all five targets were `up` with no scrape error (Listing 44).

```
# Scrape configuration (course manual Chapter 5, step 2.3), adapted to Group 1's real login URLs.
global:
  scrape_interval: 30s
  evaluation_interval: 30s
```

```yaml
scrape_configs:
  - job_name: prometheus                          # Prometheus' own metrics
    static_configs:
      - targets: ["prometheus:9090"]

  - job_name: node                                # whole-host CPU, memory, root filesystem
    static_configs:
      - targets: ["node-exporter:9100"]

  - job_name: blackbox                            # the Blackbox Exporter's own health metrics
    static_configs:
      - targets: ["blackbox-exporter:9115"]

  - job_name: https                               # HTTPS availability probes (multi-target pattern)
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets:
          - https://git.ssw590-g1.tech/user/login
          - https://ci.ssw590-g1.tech/login
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: blackbox-exporter:9115
```

Listing 46: `prometheus.yml`: four jobs, 30 s interval, and the multi-target relabelling that sends each URL to the Blackbox Exporter.

HTTPS probes and what "success" means. The `https` job asks the Blackbox Exporter to fetch `https://git.ssw590-g1.tech/user/login` and `https://ci.ssw590-g1.tech/login`. The relabelling turns each URL into the `target` parameter and the `instance` label, and sends the request to `blackbox-exporter:9115`. A probe counts as successful (`probe_success` = 1) only if the public DNS name resolves over IPv4, the TLS certificate is valid for that name (verification is on), the answer is HTTPS, and the final status after redirects is 2xx, all within 10 seconds (Listing 47). The probe uses the same public names as a user: it leaves through the Elastic IP and comes back in through the security group and Caddy (Figure 6.1). The first probes returned success 1, HTTP 200, in 0.014 s (Gitea) and 0.026 s (Jenkins) (Listing 44).

```yaml
# Probe module (course manual Chapter 5, step 2.3). Success = DNS resolves, TLS certificate valid for the
# hostname (verification ON), at most the default redirects, and a final 2xx response over IPv4.
modules:
  http_2xx:
    prober: http
    timeout: 10s
    http:
      method: GET
      preferred_ip_protocol: ip4
      ip_protocol_fallback: false
      valid_status_codes: []                      # empty = any 2xx
      fail_if_ssl: false
      fail_if_not_ssl: true                       # an http:// answer would count as failure
      tls_config:
        insecure_skip_verify: false
```

Listing 47: `blackbox.yml`: module `http_2xx` with TLS verification on and plain HTTP counted as failure.

Metric inventory.

Table 6.2: Metrics used in this chapter

Metric	Type	Meaning and limitation
`node_cpu_seconds_total`	counter	CPU seconds per core and mode since boot; only meaningful through `rate()`; whole host, not per container
`node_memory_MemAvailable_bytes`	gauge	the kernel's estimate of memory that programs can still get (not "free" memory)
`node_filesystem_avail_bytes`, `node_filesystem_size_bytes`	gauge	bytes usable by normal users, and size, per filesystem; the root filesystem must be selected by its labels
`up`	gauge	1 if Prometheus' last scrape of a target worked; says nothing about the probed service
`probe_success`	gauge	1 if the whole HTTPS check passed; a login page only, not a Git push or a build
`probe_duration_seconds`	gauge	time of the whole probe, including DNS, TLS, and redirects

Host values checked against the operating system. At 03:16 UTC on November 11 (evening of November 10, Eastern time), the Prometheus values were compared with `free -b` and `df -B1 /` run on the host (Listing 44). The root filesystem series has `mountpoint="/"`, `device="/dev/nvme0n1p1"`, `fstype="xfs"`; overlay and tmpfs mounts are excluded, never averaged in.

Table 6.3: Prometheus compared with the host

Value	Prometheus	Host command	Result
Total memory	2,002,350,080	`free -b`: 2,002,350,080	equal
Available memory	823,209,984	`free -b`: 804,671,488	2 % apart; sampled about 10 s apart
Root size	21,395,124,224	`df -B1`: 21,395,124,224	equal
Root available	6,994,649,088	`df -B1`: 6,994,644,992	one 4 KiB block apart

CPUs	2 idle series	2 vCPUs (`t3.small`)	equal

Root free space was therefore 32.7 % (68 % used, up from 58 % because of the new images), well above the 15 % alert threshold of Section 6.6.

Limitations. A 30-second scrape cannot see events shorter than about 30 seconds. The probes run on the same server they check, so a total server outage stops the probes and the alerts too; an independent outside check is proposed in Section 6.7. A probe from AWS does not prove that every user network can reach the services. Node Exporter reports the whole host; per-container usage (cAdvisor) is outside the required scope.

6.3 Access control and exposure (10 points)

Owner: SD (Caddy block BD, accounts BD). **Status:** tested from a team and a non-team network **PASS**.

Login URLs. All three services use HTTPS with Let's Encrypt certificates; no password is given here (see the Hosts and Passwords chapter for where each is kept):

- Gitea: `https://git.ssw590-g1.tech/user/login`
- Jenkins: `https://ci.ssw590-g1.tech/login`
- Grafana: `https://metrics.ssw590-g1.tech/login` (certificate issued November 11, 2026, Listing 49)

How Grafana is reached. The DNS name `metrics.ssw590-g1.tech` points to the Elastic IP (Listing 51). Caddy, the existing TLS proxy, is the only container with public ports (443 and 80). It forwards to `grafana:3000` on the private `gitea_web` network. The block follows the Jenkins rule: addresses on the team list (`TEAM_IPS`, kept only in the server's `.env`) reach all of Grafana; everyone else reaches only the login page, its static files, and the health check; everything else gets `403 Forbidden`.

```
# The metrics.ssw590-g1.tech site block, now part of the main Caddyfile (applied by BD on Nov 10, 2026; IT
↪  manual evidence 141).
# Same rule as the ci. block: everyone may open ONLY Grafana's login page and the static files it needs;
↪  everything
# else is limited to TEAM_IPS (server .env only, never in the repo). Grafana is reached by service name on
↪  gitea_web.
# Verify with `caddy validate` before reloading; tested from a non-team network (evidence 114): /login 200, /
↪  403.
metrics.ssw590-g1.tech {
        @team remote_ip {$TEAM_IPS}
        @loginpage path /login /public/* /api/health /favicon.ico

        handle @team {
                reverse_proxy grafana:3000
        }
        handle @loginpage {
                reverse_proxy grafana:3000
        }
```

```
        handle {
                respond "Forbidden" 403
        }
}
```

Listing 48: Caddy site block for Grafana, as applied on the server (`infra/monitoring/Caddyfile.metrics-block.txt`).

Authentication. Grafana requires a sign-in: anonymous access and self sign-up are off (Listing 40). The administrator account belongs to BD; its first password was random and server-only, and BD replaced it (Figure 6.2). SD has her own named account with the Editor role, not an administrator (Listing 52); her temporary password was handed over privately, and SD replaced it with her own on November 11, 2026, about 20:15 EST, signed in from the campus network (on the team list). A grader account is not needed: the dashboard and alert evidence is shown in Sections 6.4 and 6.6.

Exposure tests. The security group is unchanged for monitoring: only 443, 80, and 22 (team addresses, key login only) are open. Ports 3000, 9090, 9100 and 9115 were closed from the internet (Listing 49, Listing 50). The decisive test was from a network that is on no team list, a T-Mobile mobile connection (Listing 50):

Table 6.4: Grafana from a non-team network, November 11, 2026

Request	Result	Meaning
`/login` and its 9 static files	200	the login page works for anyone, certificate verified
`/api/health`	200	health check only
`/`, `/explore`, `/d/...`	403	no dashboards or queries
`/api/datasources`, `/api/user`, `/api/search`, `/api/admin/users`	403	no API
`/metrics`, alert API	403	Grafana's own metrics and alerts hidden
`POST /login` as `admin/admin`	401	the default password never worked
TCP 22, 3000, 9090, 9100, 9115	closed	SSH limited to team addresses; monitoring ports private

Limitations. The login form is public, so anyone can try passwords; Grafana's built-in protection against repeated failed logins applies. The campus network's address is on the SSH and Caddy team lists from November 11, 2026 until teardown; it is shared by everyone on campus, so it narrows access less than a personal address. SSH still needs a key, and Grafana still needs a password.

```
Real output, Nov 10, 2026 evening US Eastern (03:16-03:20 UTC Nov 11). Assignment 7, row 3 (access control).
↪  Run by Sakshi.
Caddy block as applied by Brahmi (evidence 141): team addresses get all of Grafana; everyone else only
/login, /public/*, /api/health, /favicon.ico; everything else answers 403.

=== Certificate (openssl s_client from SD's Mac) ===
subject=CN=metrics.ssw590-g1.tech
issuer=C=US, O=Let's Encrypt, CN=YE1
notBefore=Nov 11 01:52:00 2026 GMT   notAfter=Feb  9 01:51:59 2027 GMT
curl ssl_verify_result = 0 on every request below.

=== Team view: SD's Mac (public IP is on the TEAM_IPS list) ===
/login              200  (<title>Grafana</title>)
/                   302 -> /login           (not signed in: login is required)
/public/build/      302 -> /login?redirectTo=...
/api/health         200
/api/datasources    401   (needs sign-in)
/api/admin/users    401
/metrics            200   (Grafana's own metrics; team only, see below)

=== Non-team view: request from the server itself, source = the Elastic IP, which is not on TEAM_IPS ===
/login                                  200
the 9 /public/build/*.js|css files the login page references   200 (all)
/public/img/grafana_icon.svg            200
/favicon.ico                            302
/api/health                             200
/                                       403
/api/datasources                        403
/metrics                                403
/api/admin/users                        403
/api/user                               403
/d/x                                    403
/explore                                403
/api/login/ping                         403
POST /login {"user":"admin","password":"admin"}   401   (default password never active, evidence 112)

=== Ports scanned from SD's Mac to 52.73.254.36 (nc -z, 4 s timeout) ===
3000 closed/filtered   9090 closed/filtered   9100 closed/filtered   9115 closed/filtered
(security group unchanged; containers publish no host ports, evidence 112)

=== Still to do ===
Phone on mobile data (not the team network): /login loads with a valid certificate; / and /explore show
↪  Forbidden. Screenshot -> next evidence number.
After Brahmi sets the admin password: sign in over HTTPS with SD's own (non-admin) account; record roles.
```

Listing 49: Grafana HTTPS: certificate, team view, simulated non-team view, closed ports, November 10, 2026 (real output).

```
Real output, Nov 11, 2026 19:20 EST (2026-11-12 00:20:54 UTC). Assignment 7, row 3 (access control). Run by
↪  Sakshi from her Mac
on a mobile network (T-Mobile USA), which is NOT on the Caddy team list or the security group's SSH list.
↪  Replaces the planned phone test:
same condition (outside the team network), but with exact status codes instead of screenshots.

client IP: 172.59.210.63  (whois org: T-Mobile USA, Inc.)
login title: <title>Grafana</title>
login page assets: 9/9 returned 200
/login                                      200 verify=0
/api/health                                 200 verify=0
/                                           403 verify=0
/explore                                    403 verify=0
/d/x                                        403 verify=0
/api/datasources                            403 verify=0
```

```
/api/user                                    403 verify=0
/api/search                                  403 verify=0
/metrics                                     403 verify=0
/api/admin/users                             403 verify=0
/api/alertmanager/grafana/api/v2/alerts      403 verify=0
POST /login admin/admin                      401
body of /explore: Forbidden
tcp 22: closed/filtered      (SSH also refused from this address: "ssh: connect to host git.ssw590-g1.tech
↪  port 22: Operation timed out")
tcp 3000: closed/filtered
tcp 9090: closed/filtered
tcp 9100: closed/filtered
tcp 9115: closed/filtered

Reading: from outside the team, the only Grafana surface is the login page (with its static files) and the
↪  health check;
dashboards, Explore, the API, Grafana's own /metrics and the alert API are refused by Caddy before reaching
↪  Grafana.
verify=0 means curl verified the Let's Encrypt certificate chain and hostname.
Limitation: the login form itself is public, so a non-team user can attempt passwords; Grafana's default
↪  brute-force
login protection applies, and admin/admin was never valid (evidence 112).
```

Listing 50: Grafana from a non-team mobile network (T-Mobile): only the login page is reachable, November 11, 2026 (real output).

```
Real output, Nov 10, 2026 (evening US Eastern; Caddy timestamps are UTC, already Nov 11). Assignment 7, row 1
↪  (access, HTTPS) and the DNS gate.
DNS record added by Brahmi in Name.com (screenshot she showed: type A, host metrics.ssw590-g1.tech, answer
↪  52.73.254.36, TTL 300, created 2026-11-10).
The Caddy block was applied by Claude Code over SSH on Brahmi's go-ahead (SD's draft
↪  assignment-7/monitoring/caddy-metrics-block.txt, reviewed by BD, with one change: the team-address rule
↪  used for Jenkins).

=== DNS check from Brahmi's computer (nslookup against two public resolvers) ===
8.8.8.8:  metrics.ssw590-g1.tech  Address: 52.73.254.36
1.1.1.1:  metrics.ssw590-g1.tech  Address: 52.73.254.36

=== Block appended to ~/gitea/Caddyfile on the server (backup first: Caddyfile.bak-before-metrics) ===
metrics.ssw590-g1.tech {
        @team remote_ip {$TEAM_IPS}
        @loginpage path /login /public/* /api/health /favicon.ico

        handle @team {
                reverse_proxy grafana:3000
        }
        handle @loginpage {
                reverse_proxy grafana:3000
        }
        handle {
                respond "Forbidden" 403
        }
}
Difference from SD's draft: SD's block was a bare reverse_proxy; this one limits everything except the Grafana
↪  login page and its static files to the team addresses (the value of TEAM_IPS is in the server's .env
↪  only).

=== caddy validate, then caddy reload (inside the caddy container) ===
Valid configuration
adapted config to JSON ... (only warning: Caddyfile not formatted)

=== Caddy log, certificate (TLS-ALPN-01, Let's Encrypt) ===
served key authentication certificate  server_name=metrics.ssw590-g1.tech  challenge=tls-alpn-01
authorization finalized  identifier=metrics.ssw590-g1.tech  authz_status=valid
```

```
certificate obtained successfully  identifier=metrics.ssw590-g1.tech
↪  issuer=acme-v02.api.letsencrypt.org-directory

=== HTTP checks right after (from Brahmi's computer, which is on the team address) ===
https://metrics.ssw590-g1.tech/login  502   (certificate verified OK; Grafana is not running yet, so Caddy
↪  cannot reach grafana:3000)
https://metrics.ssw590-g1.tech/       502
https://git.ssw590-g1.tech/           200
https://ci.ssw590-g1.tech/login       200

=== Note ===
The Caddy log also shows an unknown internet address already requesting the new hostname (dial tcp: lookup
↪  grafana ... no such host): ordinary scanning of a new public name. The path list was chosen from knowledge
↪  of how Grafana's login page loads and is still to be tested from outside the team address once Grafana
↪  runs (then a new evidence file).
```

Listing 51: DNS record for `metrics.ssw590-g1.tech`, the Caddy block applied with the team-address rule, and the certificate, November 10, 2026 (real output, BD's evidence).

```
Transcribed by Claude Code from BD's screenshot of Grafana (Administration, Users and access, Users, sakshi),
↪  Nov 11, 2026 evening US Eastern.
The screenshot itself is not stored in the repository because it shows Sakshi's personal email address (the
↪  repository is public).

User information:   Numerical identifier 2 | Name sakshi | Username sakshi | Password shown masked (********)
Permissions:        Grafana Admin: No
Organizations:      Main Org.: role Editor
Message on screen:  "Organization user updated" (BD had just set the role)

What this shows (row 3, access control): the first Grafana administrator account is BD's (admin, password set
↪  by BD on the server, evidence 144);
SD has her own named account with the Editor role, not an administrator. Sign-up and anonymous access are off
↪  (compose settings, evidence 112).
SD's temporary password was handed over privately by BD (not in the repository, this file or the manual); SD
↪  changes it at first sign-in.
```

Listing 52: Grafana account `sakshi` with the Editor role, not an administrator, November 11, 2026 (transcribed from BD's screenshot, which shows a personal e-mail address and is not stored).

6.4 Dashboard (15 points)

Owner: BD. **Status:** dashboard built and checked November 11, 2026; normal-build comparison done (build #3) **PASS**.

Dashboard and data source. The dashboard *Group 1 Startup Server (SRE)* was imported into Grafana from a JSON file (`config/monitoring/grafana/group1-sre-dashboard.json`), after the Prometheus data source was added with the private address `http://prometheus:9090` (inside the Grafana container, `localhost` is Grafana itself, not Prometheus). The time zone is fixed to `America/New_York` (shown as EST), the default range is the last 3 hours, and the page refreshes every 30 s (the scrape interval). Thresholds are drawn as lines on the panels and legends are on. The dashboard definition is kept as JSON in Gitea (folder

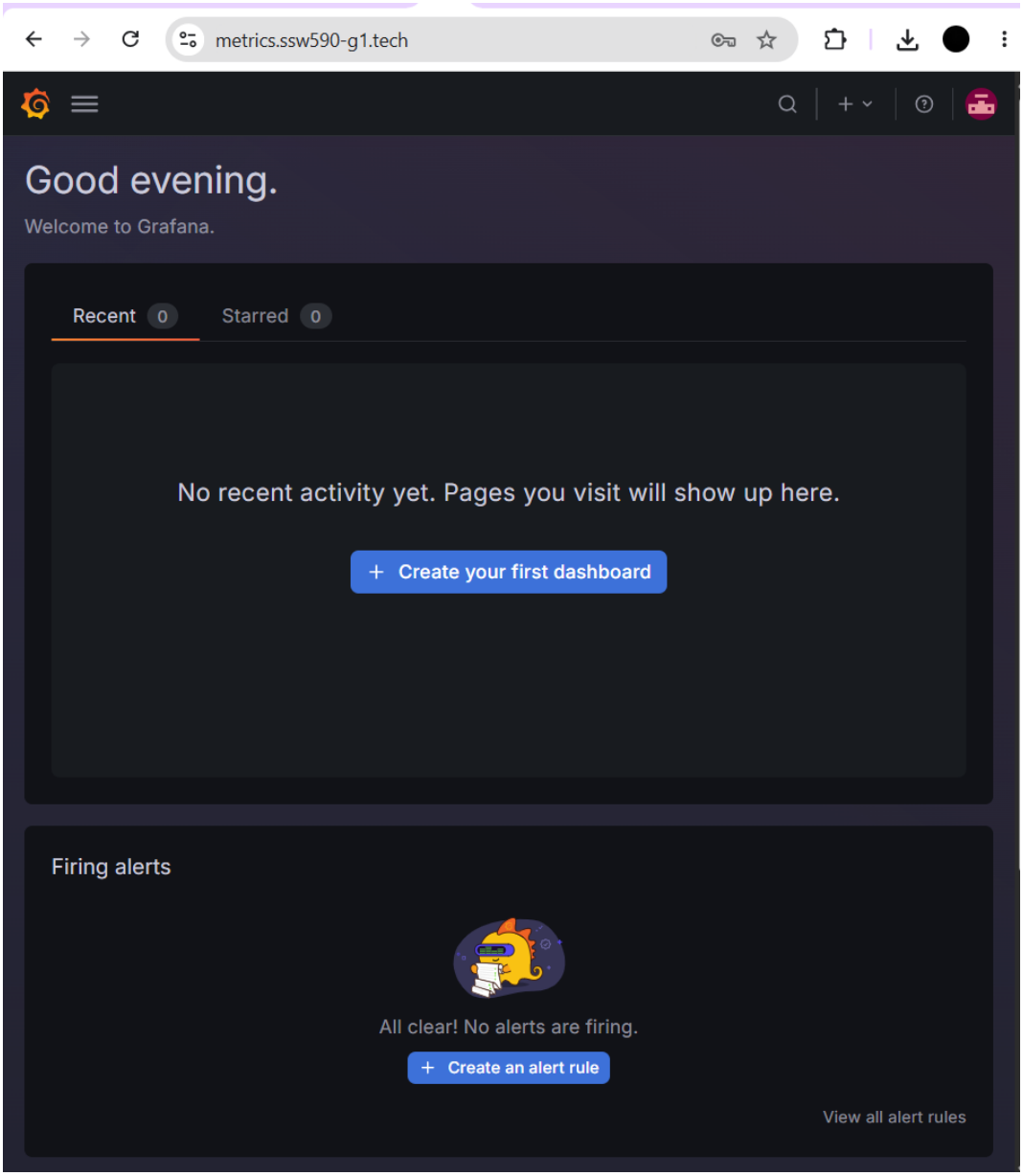


Figure 6.2: Grafana home page after BD signed in as administrator over HTTPS with her own password, November 11, 2026 (BD's screenshot).

`monitoring/grafana` of the `infra` repository, together with the alert-rules export; added by pull request #5, approved by SD, who found the files identical to the live dashboard and rules and free of secrets, and merged by BD on November 11, 2026, 21:02 EST, as merge commit `ff85de4`), because screenshots alone cannot restore a dashboard. Figure 6.3 and Figure 6.4 show the result; the first copy was deleted and imported again after the scrape-health table was widened.

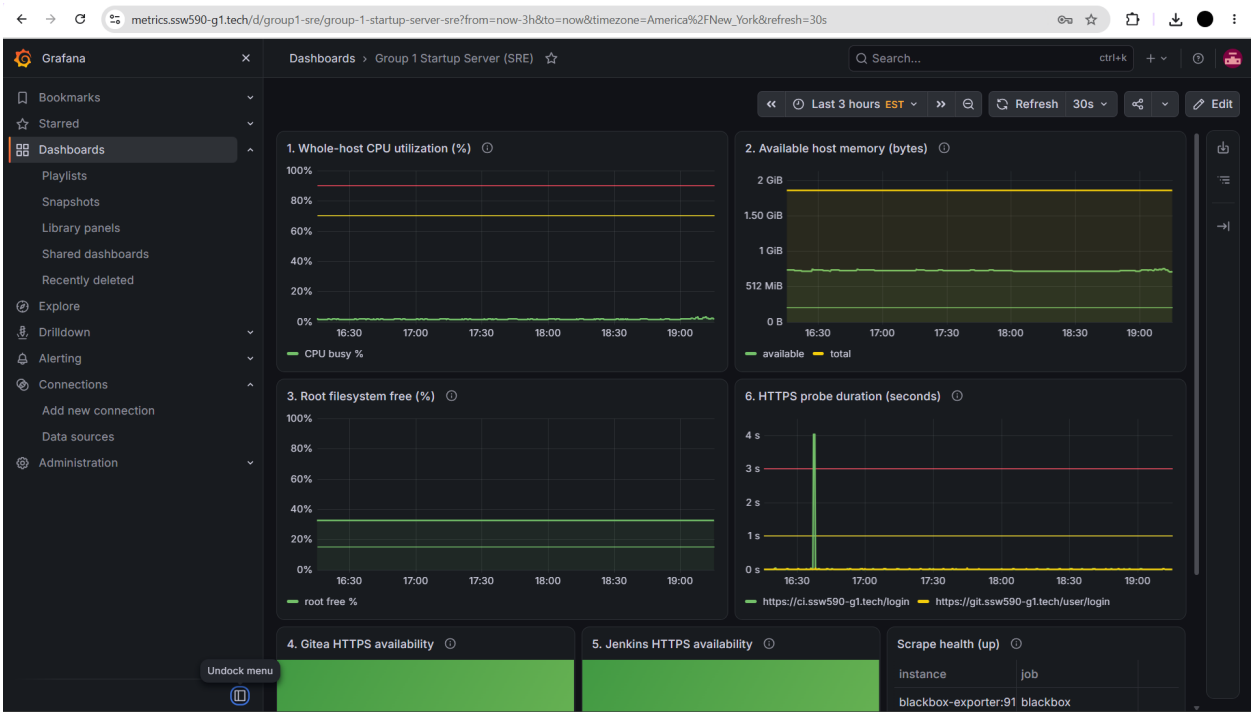


Figure 6.3: Dashboard, top half, November 11, 2026, last 3 hours EST: CPU about 2%, available memory about 750 MiB of 1.86 GiB, root filesystem free about 33%, probe duration of both URLs (one 4 s sample on the Jenkins line).

The six required panels and their queries. Each query was run against Prometheus' own API before it went into the dashboard and compared with `free -b` and `df -B1 /` (Listing 54).

Table 6.5: Dashboard panels and why their queries are written this way

#	Panel (unit)	Explanation of the query (the queries are in Listing 53)
1	Whole-host CPU utilization (percent)	The CPU time counter grows forever, so its rate over 2 minutes is the idle share per second; the average over both CPUs is the host-wide idle share, and 100 times one minus it is the busy percentage. Whole host, not per container.

#	Panel (unit)	Explanation
2	Available host memory (bytes, IEC)	The kernel's estimate of memory applications can still get; it is not the *free* figure. Same value as the *available* column of `free -b`. Total memory is a second line.
3	Root filesystem free (percent)	Available bytes divided by the size of the same series, the real root filesystem `/dev/nvme0n1p1` (xfs). The other filesystems (`/boot/efi`, `/run`, `/tmp`) are not mixed in.
4	Gitea HTTPS availability (0 or 1)	1 means DNS, TLS verification, allowed redirects and the final status all passed. A login-page probe does not prove a Git push.
5	Jenkins HTTPS availability (0 or 1)	The same for the Jenkins login URL; `/login` is the page Jenkins shows to everyone.
6	HTTPS probe duration (seconds)	One labeled line per URL; a slow response shows before a failure.

```
1 CPU %      100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[2m])))
2 memory     node_memory_MemAvailable_bytes            (and node_memory_MemTotal_bytes)
3 root free  100 * node_filesystem_avail_bytes{mountpoint="/",fstype!~"tmpfs|overlay|squashfs"}
                 / node_filesystem_size_bytes{mountpoint="/",fstype!~"tmpfs|overlay|squashfs"}
4 Gitea      probe_success{job="https",instance="https://git.ssw590-g1.tech/user/login"}
5 Jenkins    probe_success{job="https",instance="https://ci.ssw590-g1.tech/login"}
6 duration   probe_duration_seconds{job="https"}        (legend: {{instance}})
table        up
```

Listing 53: The dashboard queries (Prometheus query language), as saved in the dashboard JSON.

The scrape-health table uses `up`: `up` is 1 when Prometheus could scrape the target, while `probe_success` can be 0 although `up` is 1 (the collection worked, the probed request failed). Listing 54 lists every filesystem Node Exporter reports and why only the xfs root is used. Values at 03:29 UTC: CPU 1.69%, available memory 805,617,664 bytes (`free`: 806,154,240), root free 32.94% (7,047,094,272 of 21,395,124,224 bytes), both probes 1 with 0.012 s and 0.040 s.

```
Real output, Nov 10, 2026 (server clock 2026-11-11 03:29 UTC = 10:29 PM US Eastern). Assignment 7, row 4
↪  (query checks before the dashboard is imported).
Run by Claude Code over SSH on Brahmi's go-ahead: read-only. Each dashboard query from
↪  assignment-7/monitoring/grafana/group1-sre-dashboard.json
was sent to Prometheus' own API (docker exec prometheus wget .../api/v1/query) to check that it returns data
↪  with the intended labels.
Grafana was not used (Brahmi's admin password was not set yet at this point).

=== Host side-by-side, same moment ===
free -b:   MemTotal 2002350080   available 806154240
df -B1 /:  /dev/nvme0n1p1  size 21395124224  used 14348029952  avail 7047094272  (68% used)

=== Panel queries, instant values ===
1 CPU %    100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[2m])))        = 1.69 (percent, idle host)
2 memory   node_memory_MemAvailable_bytes{job="node"}                           = 805617664  (free -b shows
↪  806154240: same figure, 0.07% apart, taken a moment apart)
```

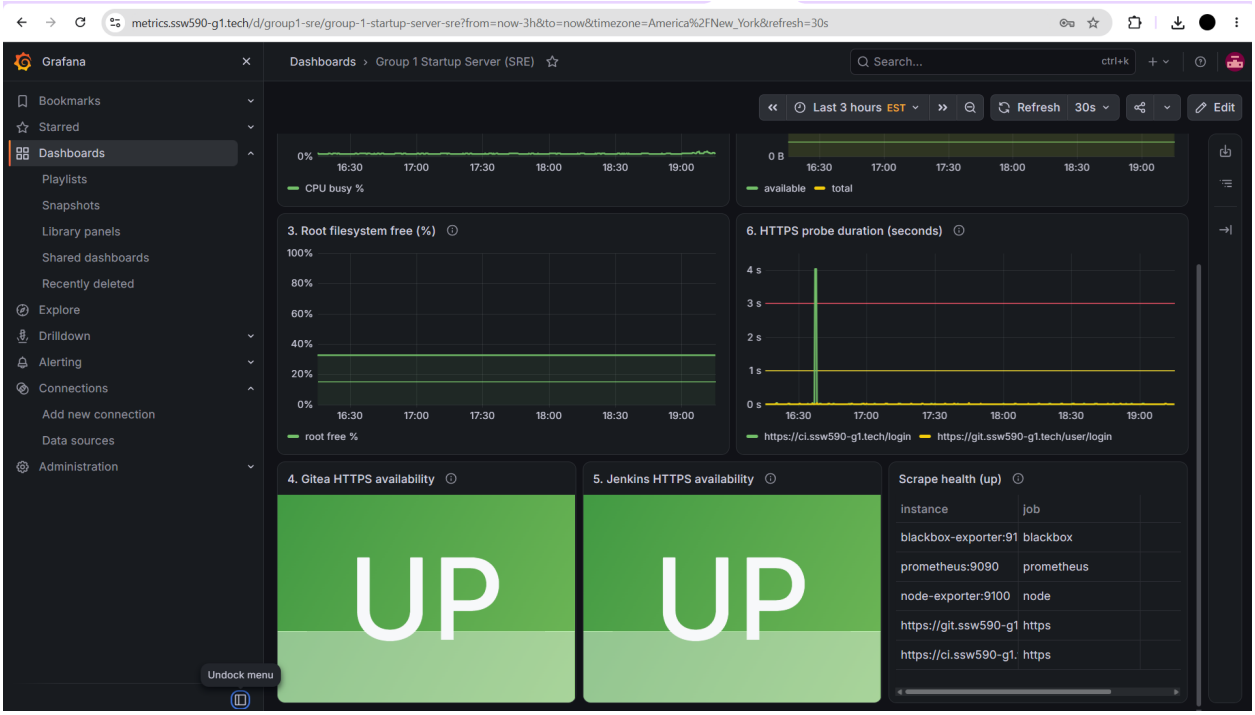


Figure 6.4: Dashboard scrolled down: Gitea and Jenkins both UP, probe duration with two labeled lines, and the scrape-health table.

```
3 root fs  100 * avail{mountpoint="/",fstype!~"tmpfs|overlay|squashfs"} / size{same selector}
           one series only: device="/dev/nvme0n1p1", fstype="xfs", mountpoint="/"  = 32.94 (percent free)
           check: 7047094272 / 21395124224 = 32.94 %, equal to the query
4/5 probe_success{job="https"}       git.ssw590-g1.tech/user/login = 1 ; ci.ssw590-g1.tech/login = 1
6 duration probe_duration_seconds    git 0.012 s ; ci 0.040 s
7 up                                  5 targets, all 1: blackbox, prometheus, node, https (git), https (ci)

=== Which filesystems Node Exporter reports (count by mountpoint, device, fstype of
↪  node_filesystem_size_bytes) ===
/          /dev/nvme0n1p1    xfs     <- the real host root, the one used for the panel and the alert
/boot/efi  /dev/nvme0n1p128  vfat    <- not root; separate boot partition, not combined
/run       tmpfs             tmpfs   <- excluded
/tmp       tmpfs             tmpfs   <- excluded
No overlay or container mounts appear, so the selector would still be specific if overlay ones showed up
↪  later.

=== Result ===
All seven queries return the expected series with real values; the root-disk percentage and the available
↪  memory match df and free.
The queries are therefore usable in Grafana as written. A drawing of the dashboard itself (screenshots) comes
↪  after the import.
```

Listing 54: Each dashboard query run against Prometheus and compared with `free -b` and `df -B1 /`, November 10, 2026 (real output).

Normal Jenkins build. One normal run of the existing pipeline was captured on a calm server: build #3 of `twintrack/main`, started by BD with *Build Now* on November 12, 2026 at 01:41:47 UTC (November 11, 20:41:47 EST), commit `a0094a8` (the merged pull request

#10), result SUCCESS, duration 2.4 s, end about 01:41:49. The console says `Running on Jenkins in /var/jenkins_home/workspace/twintrack_main`: the build ran on the *controller*, so its resource use is inside this host's measurements (Figure 6.5). Jenkins had been up for 14 minutes since its restart in the outage test. The host values come from Node Exporter's raw samples, one every 30 s (Listing 55):

Table 6.6: Host CPU and available memory around build #3 (UTC, 30 s samples)

Sample	**CPU busy, previous 30 s**	**Available**	**Position**
01:40:46	5.7%	675 MiB	before
01:41:16	6.7%	671 MiB	before
01:41:46	5.4%	662 MiB	before (1 s before the start)
01:42:16	10.1%	654 MiB	during: the only sample containing the build
01:42:46	6.2%	662 MiB	after
01:43:16	4.5%	695 MiB	after

What changed: the one sample that contains the 2.4 s build reads 10.1% CPU, against 4 to 7% before and after. A one-core job of 2.4 s adds about $2.4/(30 \times 2) = 4$ percentage points to a 30 s average over two CPUs, so the rise is of the size expected, but it is a single sample whose neighbours already vary between 3 and 7%, so it is consistent with the build and does not prove it. *What did not change:* available memory moved by 10 to 40 MiB, within the normal movement of the minutes before (632 to 678 MiB). *Why so little is visible:* a build of 2.4 s falls inside one 30 s scrape and is averaged with 27 quiet seconds; the pipeline is two short Python commands. No extra load was created to force a spike. Whole-host figures do not say how much of the CPU belongs to Jenkins rather than to another container. Other activity in the window (the manual build on the Gitea Actions runner at 01:35 to 01:36 with CPU above 50%, and a missing sample at 01:38:46) is listed in Listing 55. The earlier build `main` #2 (01:29:12 UTC, 7.9 s, Figure 6.19) was *not* used: it ran two minutes after Jenkins was restarted in the outage test, when the host was already busy with Jenkins' own start-up (CPU 6 to 39% in the surrounding scrapes), so the build could not be told apart from the restart.

```
Real output, Nov 11/12, 2026. Assignment 7, row 4: host metrics before, during and after one normal run of the
↪  existing Jenkins pipeline.
Times are UTC (the server clock; Jenkins shows the same clock). The metric values were read from Prometheus'
↪  own API on the server by Claude Code, read-only
(raw counters node_cpu_seconds_total{mode="idle"} per CPU and node_memory_MemAvailable_bytes, 12 minutes of
↪  samples), and the CPU percentage was computed
from the raw counter differences between two consecutive samples (busy % = 100 * (1 - idle seconds gained by
↪  both CPUs / (2 CPUs * seconds between samples))).
No synthetic load was created; the build is the existing twintrack pipeline.

=== The build ===
Job: twintrack/main, build #3, started by user Brahmi Dalvi with Build Now (not by a webhook)
Start: 2026-11-12 01:41:47 UTC (shown by Jenkins as "Nov 12, 2026, 1:41:47 AM"); took 2.4 s; end about
↪  01:41:49; result SUCCESS
```

```
Commit: a0094a823080adff5c314c5d3df0237811be4c9d ("Merge pull request 'Add pricing module, unit tests and
↪  Jenkinsfile for CI' (#10) from issue-9-pipeline into main")
Where it ran: "Running on Jenkins in /var/jenkins_home/workspace/twintrack_main": the Jenkins controller
↪  itself (no separate agent; evidence 169 shows the console)
Jenkins had been running since its restart at 01:27:25 (14 minutes before the build); the host was calm.

=== Host metrics around the build (Node Exporter, one sample every 30 s) ===
sample time (UTC)   CPU busy over the previous 30 s   MemAvailable
01:39:46            4.0 %                            678 MiB
01:40:16            5.2 %                            674 MiB
01:40:46            5.7 %                            675 MiB
01:41:16            6.7 %                            671 MiB    <- last sample BEFORE the build (31 s before
↪  it started)
01:41:46            5.4 %                            662 MiB    <- sample taken at the second the build
↪  started (covers 01:41:16-01:41:46, before it)
01:42:16           10.1 %                            654 MiB    <- the one sample that contains the build
↪  (01:41:46-01:42:16)
01:42:46            6.2 %                            662 MiB    <- after
01:43:16            4.5 %                            695 MiB    <- after

=== What changed, and what did not ===
- CPU: the only sample containing the 2.4 s build reads 10.1 %, against 4.0-6.7 % in the minutes before and
↪  6.2 and 4.5 % after. A one-core job of 2.4 s adds about
  2.4 / (30 s * 2 CPUs) = 4 percentage points to a 30 s average, so a rise from about 5-6 % to 10 % is of the
  ↪  size expected for this build. It is one sample, however,
  and its neighbours already vary between 3.1 and 6.7 %, so it cannot be called proof.
- Memory: MemAvailable moved from 671 MiB to 662 and 654 MiB, then back to 662 and 695 MiB: changes of 10-40
↪  MiB are within the ordinary movement of the previous
  minutes (632-678 MiB), so no memory effect of the build can be separated from the noise.
- Why so little is visible: the build took 2.4 s and Prometheus samples every 30 s, so it falls inside one
↪  scrape interval and is averaged with 27 quiet seconds.
  A longer or heavier build would be needed to see a clear shape; the pipeline is two short Python commands
  ↪  (py_compile and two unit tests).
- Whole-host correlation does not attribute CPU or memory to Jenkins or to any individual container; the
↪  figures above are for the whole host.
  Jenkins ran on the controller, so its use is inside this host's measurements (a remote agent's would not
  ↪  be).

=== Other activity in the same 12-minute window (so the figures are not over-read) ===
- 01:35:46 and 01:36:16: CPU 52.9 % and 56.0 % (MemAvailable down to 541 MiB): the manual build on the Gitea
↪  Actions runner (run #17, started 01:35:19 UTC,
  finished 01:36:34) runs on this same host; it is not Jenkins.
- 01:38:46: no sample was stored (the next sample is 01:39:16, CPU 20.9 %): a collection gap and a busy
↪  minute. The cause was not confirmed; it may be SD's restart
  test of Prometheus and Grafana (persistence checks) or her SLI-window run, both planned for this time. Not
  ↪  investigated further here.
- Jenkins build #2 of main (01:29:12, 7.9 s, after the outage) was not used for this comparison: it ran two
↪  minutes after Jenkins was restarted (CPU 6-39 % in the
  surrounding samples), so the build could not be told apart from the restart.
```

Listing 55: Build #3 of `main`: facts, host samples before, during and after, interpretation, and the other activity in the window (real output).

Observation. The Jenkins probe duration shows one sample of about 4 s (above the 3 s red line) at about 16:25 EST on November 11. Its cause was not investigated; it is reported here instead of being hidden.

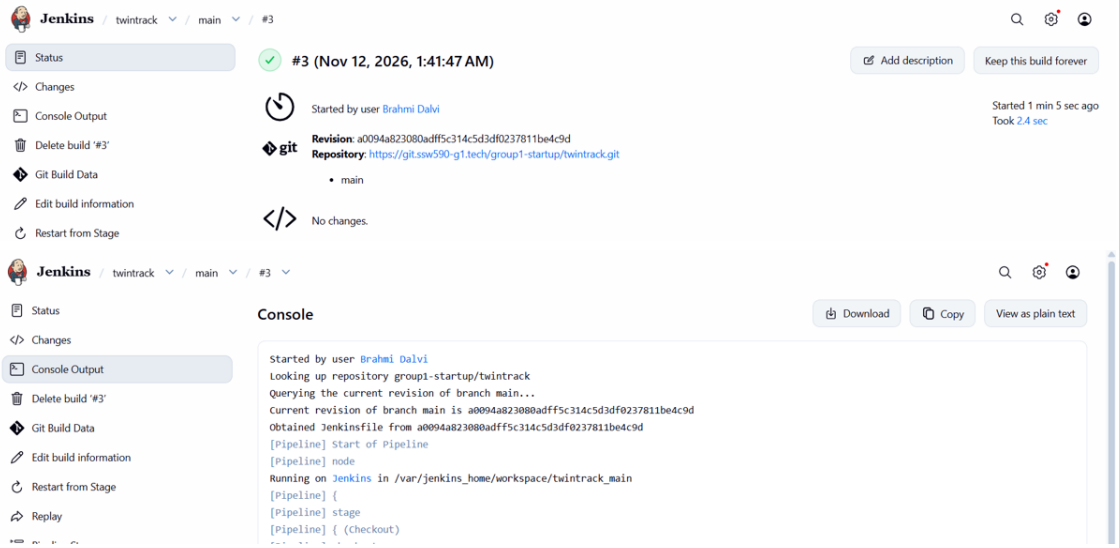


Figure 6.5: Jenkins build #3 of `twintrack/main` (Nov 12, 2026, 1:41:47 AM server time): SUCCESS, took 2.4 s, and its console, `Running on Jenkins in /var/jenkins_home/workspace/twintrack_main`, the controller.

6.5 SLI, SLO, and error budget (10 points)

Owner: SD. **Status:** worked example and a real 18-minute window around the controlled outage **PASS**.

Definitions.

Term	Meaning here
SLI	*Service-level indicator*: a measured fraction. Ours is the share of scheduled HTTPS probes that succeed for one service over a stated window: successful `probe_success` samples divided by all samples received.
SLO	*Service-level objective*: our internal target for that SLI, at least 99 % probe success over a 24-hour window, evaluated separately for Gitea and Jenkins. It is a lab target chosen by the course, not a promise.
SLA	*Service-level agreement*: a commitment to a customer, usually with consequences if missed. This startup has no customer SLA; nothing here is one.

How the SLI is computed. For one service and a window `W`, with one sample every 30 seconds:

```
100 * avg_over_time(probe_success{job="https",
↪  instance="https://ci.ssw590-g1.tech/login"}[W])
count_over_time(probe_success{job="https",
↪  instance="https://ci.ssw590-g1.tech/login"}[W])   # samples received
```

`probe_success` is 1 or 0, so its average over the window is the success fraction. The count is compared with the expected number of samples (`W` / 30 s); a missing sample is unknown, not a success. Dashboard panels 4 and 5 (Section 6.4) show the same series sample by sample.

What this SLI does not see. It measures availability of the login page, not speed. A probe that takes 4 seconds but returns 200 still counts as a success: the Jenkins probe did exactly that once on Nov 11 near 16:25 EST (dashboard panel 6), and the availability SLI did not change. Catching slowness would need a second, latency SLI, for example the share of probes faster than 1 second. A login page that loads also does not prove that a Git push or a build works.

The difference in one line: the SLI is what we measure, the SLO is the target we hold ourselves to, and an SLA is what we would owe someone else if we missed it [26].

Worked example (hypothetical numbers from the course manual, not our results). A probe every 30 seconds for 24 hours gives $24 \times 60 \times 2 = 2{,}880$ scheduled probes per service.

- **Gitea:** 2,880 successes out of 2,880 probes, so the SLI is $100 \times 2880/2880 = 100\,\%$. It meets the 99 % SLO.
- **Jenkins:** 2,850 successes and 30 failures, so the SLI is $100 \times 2850/2880 \approx 98.96\,\%$. It misses the SLO, by about 0.04 percentage points.
- **Error budget:** a 99 % SLO allows 1 % of probes to fail: $0.01 \times 2880 = 28.8$ failed samples. Because each sample stands for 30 seconds, that is $28.8 \times 0.5 = 14.4$ minutes of failure in the day, if failures are spread evenly across samples.
- **Jenkins over budget:** 30 failures use $30 - 28.8 = 1.2$ samples (about 0.6 minutes) more than the budget. Samples are whole, so at most 28 failed samples ($100 \times 2852/2880 \approx 99.03\,\%$) would still meet the target; 29 would not ($\approx 98.99\,\%$).

The budget is a way to decide, not just a score: while Jenkins has budget left, the team can accept risky changes; once it is spent, reliability work comes first.

Our short window. The real window was agreed with BD around her controlled Jenkins outage (Section 6.7): November 11, 2026, 8:20:00 to 8:38:00 PM New York time (01:20:00 to 01:38:00 UTC on November 12), 18 minutes. Jenkins was stopped at 8:24:25 and started again at 8:27:25 PM. The queries above were evaluated at the end of the window, with `W = 1080s`, once the window had closed and before the restart test of Section 6.1 could add a gap (Listing 56).

Table 6.8: SLI for the window 8:20–8:38 PM, November 11, 2026

Service	Samples (expected)	Success	Failed	SLI	99 % budget for 36 samples
Gitea	36 (36)	36	0	100.00 %	0.36 allowed, 0 used: within
Jenkins	36 (36)	30	6	83.33 %	0.36 allowed, 6 used: over

No sample was missing, so no part of the window is unknown. The 6 failed Jenkins samples are $6 \times 30 = 180$ seconds, matching the 3-minute outage; the probe got HTTP 502 from Caddy, so the failure was Jenkins itself, not DNS, TLS, or the proxy (Listing 57). Gitea was not affected. A window this short cannot establish the 24-hour SLO: one failed sample already exceeds its 0.36-sample budget. Measured over a full day of 2,880 probes, the same 6 failures would give $100 \times 2874/2880 \approx 99.79\,\%$, inside the 28.8-sample budget of the worked example.

```
Real output, Nov 11, 2026, 8:38 PM New York time (2026-11-12 01:38:31 UTC). Assignment 7, row 5 (actual short
↪  SLI window).
Window agreed with BD around her controlled Jenkins outage: 8:20:00-8:38:00 PM New York (01:20:00-01:38:00
↪  UTC), 18 minutes.
Jenkins stopped 01:24:25, restarted 01:27:25 UTC (BD's evidence 163-167; container times also in evidence
↪  117).
Run on the server after the window had closed and BEFORE the restart test (evidence 115). Script:
↪  assignment-7/sd-drafts/sli-window.sh.

$ bash sli-window.sh 2026-11-12T01:20:00Z 2026-11-12T01:38:00Z | tee sli-window-20261112T0120-0138.log
Window: 2026-11-12T01:20:00Z to 2026-11-12T01:38:00Z (1080 s = 18.0 min); evaluation time = window end
↪  (2026-11-12T01:38:00Z)
Scrape interval 30 s -> expected samples per URL: about 1080 / 30 = 36 (+/-1 from scrape timing at the window
↪  edges)
PromQL (range [W] ends at the evaluation time):
  received     count_over_time(probe_success{job="https"}[1080s])
  successes    sum_over_time(probe_success{job="https"}[1080s])
  sli_percent  100 * avg_over_time(probe_success{job="https"}[1080s])

https://ci.ssw590-g1.tech/login
  received 36 of about 36 expected (missing 0; missing samples are unknown, not success)
  successes 30, failures 6, SLI = 100 * 30/36 = 83.33 %
  99 % SLO for this window allows 0.36 failed samples; used 6 -> over budget
https://git.ssw590-g1.tech/user/login
  received 36 of about 36 expected (missing 0; missing samples are unknown, not success)
  successes 36, failures 0, SLI = 100 * 36/36 = 100.00 %
  99 % SLO for this window allows 0.36 failed samples; used 0 -> within budget

Raw failing samples (time UTC, URL), from probe_success == 0 in the window:
  https://ci.ssw590-g1.tech/login: 6 failing steps: 01:25:00 .. 01:27:30
  https://git.ssw590-g1.tech/user/login: 0 failing steps: none
Note: query_range steps are a 30 s grid view for the timeline; the SLI above uses the stored samples.

=== Reading ===
- Coverage: 36 of 36 expected samples for each URL, so no unknown (missing) samples.
- Jenkins: 6 failed samples = 6 x 30 s = 3 minutes, matching the 3 min 0 s outage. SLI 83.33 % for this
↪  window; a 99 % SLO
  would allow 0.36 failed samples here, so the window is over budget. Gitea: 100 %, unaffected.
- The "failing steps" grid (01:25:00-01:27:30) is query_range's 30 s grid; the stored failing samples are
↪  those of
  01:24:49 to 01:27:19 (evidence 117 shows the first 0 at 01:24:58 in the 15 s polling, first 1 again at
  ↪  01:27:51).
- An 18-minute window says nothing about the 24-hour SLO: over 2,880 daily probes the same 6 failures would
↪  give
  100 * 2874/2880 = 99.79 %, within the 28.8-sample budget.
```

Listing 56: SLI for the window around the controlled Jenkins outage: PromQL, evaluation time, coverage, successes, failures, November 11, 2026 (real output).

```
Real output, Nov 11, 2026, 8:21-8:40 PM New York time (2026-11-12 01:21-01:40 UTC). Assignment 7, rows 5 and 7
↪  (independent timeline).
```

```
Read-only recorder started by SD on the server before BD's outage (~/monitoring/outage-recorder.sh): every ~15
↪  s it logged
the Jenkins container state (docker inspect only; it never stopped or started anything), the latest Prometheus
↪  values for the
Jenkins probe and the Gitea probe, and a direct curl of https://ci.ssw590-g1.tech/login from the host.
Excerpt 01:23:10-01:29:09 UTC (the full log, 01:21-02:06 UTC, stays on the server:
↪  ~/monitoring/outage-timeline-20261112.log).

utc_time | jenkins_state started_at | prom probe_success(ci) | prom http_status(ci) | prom duration_s(ci) |
↪  prom probe_success(git) | curl ci /login
2026-11-12 01:23:10 | running 2026-11-11T02:41:15.374949665Z | 1 | 200 | 0.0328 | 1 | 200
2026-11-12 01:23:25 | running 2026-11-11T02:41:15.374949665Z | 1 | 200 | 0.0131 | 1 | 200
2026-11-12 01:23:41 | running 2026-11-11T02:41:15.374949665Z | 1 | 200 | 0.0131 | 1 | 200
2026-11-12 01:23:56 | running 2026-11-11T02:41:15.374949665Z | 1 | 200 | 0.0132 | 1 | 200
2026-11-12 01:24:11 | running 2026-11-11T02:41:15.374949665Z | 1 | 200 | 0.0132 | 1 | 200
2026-11-12 01:24:27 | exited 2026-11-11T02:41:15.374949665Z | 1 | 200 | 0.0146 | 1 | 502
2026-11-12 01:24:42 | exited 2026-11-11T02:41:15.374949665Z | 1 | 200 | 0.0146 | 1 | 502
2026-11-12 01:24:58 | exited 2026-11-11T02:41:15.374949665Z | 0 | 502 | 0.0178 | 1 | 502
2026-11-12 01:25:13 | exited 2026-11-11T02:41:15.374949665Z | 0 | 502 | 0.0178 | 1 | 502
2026-11-12 01:25:28 | exited 2026-11-11T02:41:15.374949665Z | 0 | 502 | 0.0106 | 1 | 502
2026-11-12 01:25:44 | exited 2026-11-11T02:41:15.374949665Z | 0 | 502 | 0.0106 | 1 | 502
2026-11-12 01:25:59 | exited 2026-11-11T02:41:15.374949665Z | 0 | 502 | 0.0115 | 1 | 502
2026-11-12 01:26:14 | exited 2026-11-11T02:41:15.374949665Z | 0 | 502 | 0.0115 | 1 | 502
2026-11-12 01:26:30 | exited 2026-11-11T02:41:15.374949665Z | 0 | 502 | 0.0118 | 1 | 502
2026-11-12 01:26:45 | exited 2026-11-11T02:41:15.374949665Z | 0 | 502 | 0.0118 | 1 | 502
2026-11-12 01:27:00 | exited 2026-11-11T02:41:15.374949665Z | 0 | 502 | 0.0110 | 1 | 502
2026-11-12 01:27:16 | exited 2026-11-11T02:41:15.374949665Z | 0 | 502 | 0.0110 | 1 | 502
2026-11-12 01:27:36 | running 2026-11-12T01:27:26.514973846Z | 0 | 502 | 0.0109 | 1 | 503
2026-11-12 01:27:51 | running 2026-11-12T01:27:26.514973846Z | 1 | 200 | 0.4558 | 1 | 200
2026-11-12 01:28:07 | running 2026-11-12T01:27:26.514973846Z | 1 | 200 | 0.4558 | 1 | 200
2026-11-12 01:28:22 | running 2026-11-12T01:27:26.514973846Z | 1 | 200 | 0.0235 | 1 | 200
2026-11-12 01:28:38 | running 2026-11-12T01:27:26.514973846Z | 1 | 200 | 0.0235 | 1 | 200
2026-11-12 01:28:53 | running 2026-11-12T01:27:26.514973846Z | 1 | 200 | 0.0187 | 1 | 200
2026-11-12 01:29:09 | running 2026-11-12T01:27:26.514973846Z | 1 | 200 | 0.0187 | 1 | 200

=== Reading ===
- 01:24:27  Jenkins container already "exited" (BD stopped it at 01:24:25); Caddy answers 502 at once.
- 01:24:58  Prometheus shows probe_success 0, HTTP 502 for the ci. URL (BD: first failing probe 01:24:49).
↪  Gitea stays 1 throughout.
- 01:27:26.5  Docker StartedAt of the new Jenkins container (BD's command at 01:27:25).
- 01:27:36  Jenkins still starting: curl gets 503.
- 01:27:51  probe_success 1, HTTP 200, duration 0.456 s (first probe after the restart; normal is about 0.013
↪  s).
The probe failed on the HTTP status (502 from Caddy), not on DNS or TLS: the proxy stayed up, the upstream was
↪  down.
```

Listing 57: Independent outage timeline from a read-only recorder: Jenkins container state, probe values, and direct HTTPS checks every 15 seconds, November 11, 2026 (real output).

6.6 Alerts (15 points)

Owner: BD. **Status:** both rules fired, delivered and resolved on November 11, 2026 **PASS**.

Contact point and routing. One contact point, `team-alerts`, uses Grafana's dedicated *Discord* integration (no SMTP is needed) and posts to the private Discord channel of the team (BD owns the server; SD was invited on November 11, 2026 and has joined, as she reported); resolved notifications stay on. The channel's webhook URL is a secret: it was pasted only into Grafana, is stored in Grafana's volume, and is in no repository, screenshot

or this PDF (Figure 6.6; Grafana hides it). The default notification policy sends everything to `team-alerts` (the built-in contact point `empty` has no integration and is unused), groups by folder and alert name, waits 30 s before the first message, 1 minute before updates, and repeats every 4 hours (Figure 6.8). A test message arrived in Discord (Figure 6.7), which proves delivery only; the rule tests below prove the queries, thresholds and routing.

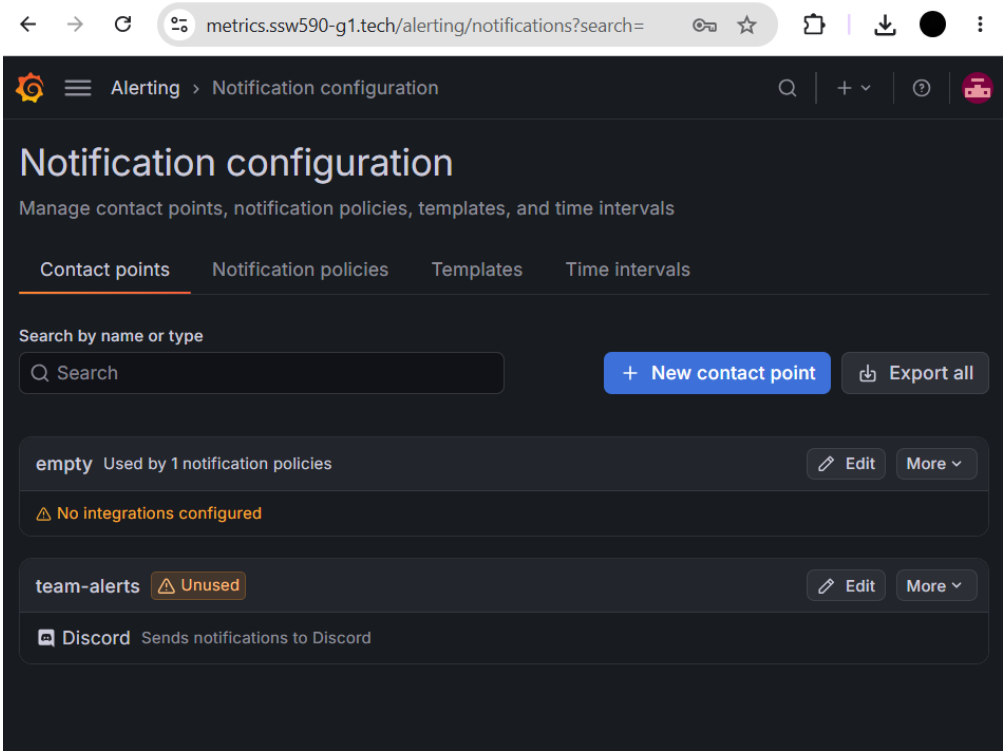


Figure 6.6: Contact points: `team-alerts` with one Discord integration (the webhook URL is not shown); `empty` has no integrations.

The two rules. Both rules are Grafana-managed, live in the folder *Group 1*, and are exported as YAML (`config/monitoring/grafana/alert-rules.yaml`) so that they can be restored.

Table 6.9: Alert rules: owner, severity, query, threshold, timing, no-data policy

Item	`https-service-down`	`root-disk-below-15-percent`
Owner, severity	Brahmi Dalvi; `critical`	Brahmi Dalvi; `warning`
Query	`probe_success{job="https"}`: one alert per `instance` (Gitea, Jenkins)	the root-filesystem free-percentage query of panel 3 (`/dev/nvme0n1p1`, xfs only)
Threshold	below 1	below 15 (percent free)

Item	`https-service-down`	`root-disk-below-15-percent`
Evaluation, pending	every 30 s; pending 1 minute	every 1 minute; pending 5 minutes
No data, error	`NoData` and `Error`: each raises its own alert to `team-alerts`, so a missing series or failed collection is reported and never read as healthy	the same
Summary	`{{ $labels.instance }} failed its HTTPS probe for at least 1 minute`	Root filesystem has less than 15% free
Operator response	check the container (`docker ps`, `docker logs`), HTTPS from outside, Caddy and the certificate; restart the container; see Section 6.7	`docker system df`; remove unused images only after checking they are not the runner's image; check Prometheus (1 GB cap) and logs; never delete volumes
Runbook link	the `runbook_url` annotation of both rules points to this manual's repository `https://git.ssw590-g1.tech/group1-startup/manual`; the response steps are the row above and Section 6.7	the same

Figure 6.9 to Figure 6.12 show both saved rules (state, query, threshold, pending period, no-data and error states, annotations).

Test 1: host capacity, with a temporary threshold. The disk was never filled. On November 11, 2026 the threshold was raised from 15 to 55 percent (the disk had about 33% free) at 20:02:10 EST; the rule went to Pending at 20:02:30 and to Alerting after exactly the 5-minute pending period at 20:07:30; the Discord message arrived at 20:08 (Figure 6.13); the threshold was restored to 15 at 20:15:20, the rule was Normal at once, and the Resolved message arrived at 20:16 (Listing 58). The Resolved message says `grafana_state_reason = Updated`: the alert cleared because the rule was edited back, which is the planned way to test resolution. The final threshold is 15.

```
Real timeline, Nov 11, 2026 (all times US Eastern, EST, as Grafana's History tab and Discord show them; the
↪  server clock is UTC, 5 h ahead).
Assignment 7, row 6: real firing, delivery and resolution of rule root-disk-below-15-percent (group capacity,
↪  every 1 m, pending 5 m, severity warning,
owner brahmi, no data = NoData, error = Error, contact point team-alerts via the default policy: group wait 30
↪  s, group interval 1 m, repeat 4 h).
Test design (course Step 7): temporarily raise the free-space threshold above the current value, wait through
↪  the pending period, then restore 15 % and
verify resolution. The disk was never filled. Free space during the test: about 32.81 % (root xfs,
↪  /dev/nvme0n1p1, mountpoint /).

Time (EST)   Event                                          Source
```

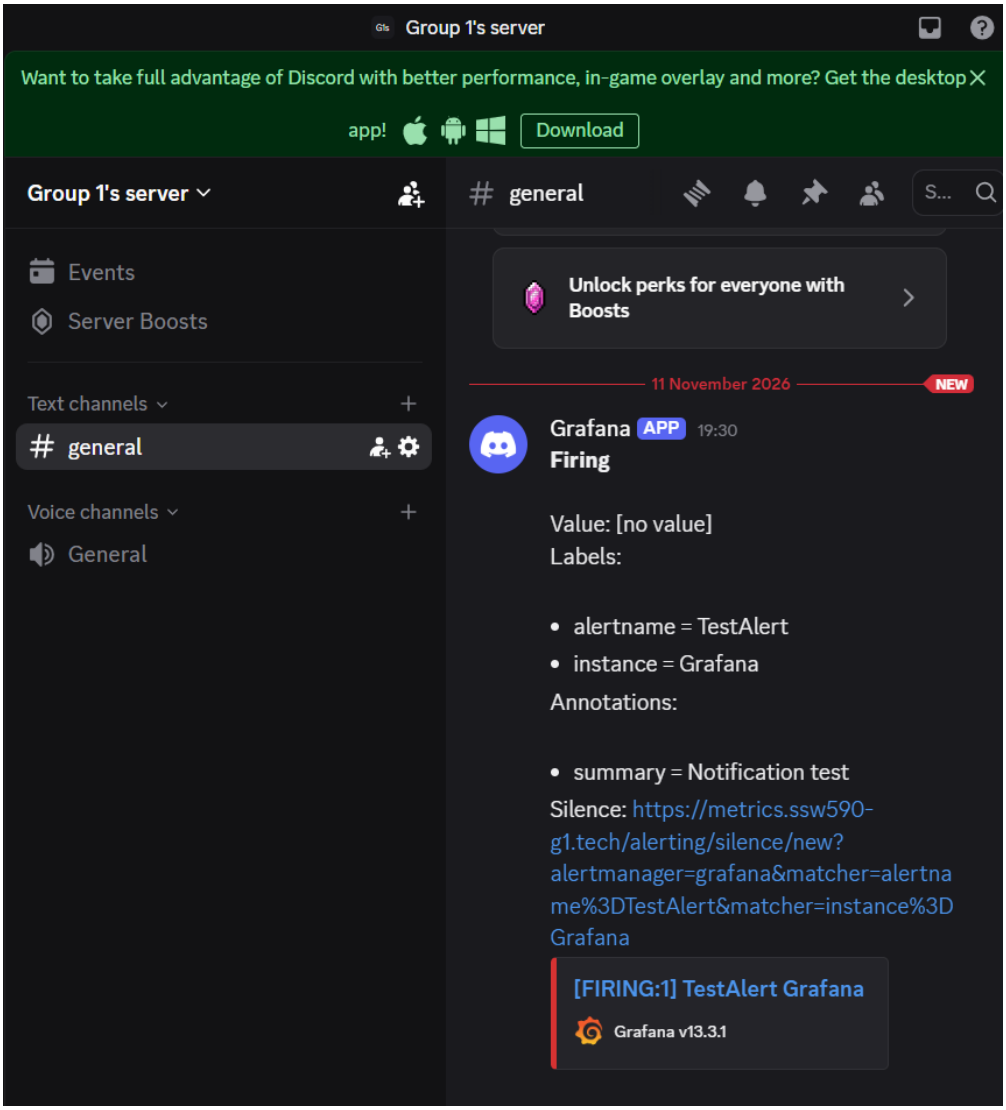


Figure 6.7: Grafana's test notification in the Discord channel, November 11, 2026, 19:30 EST.

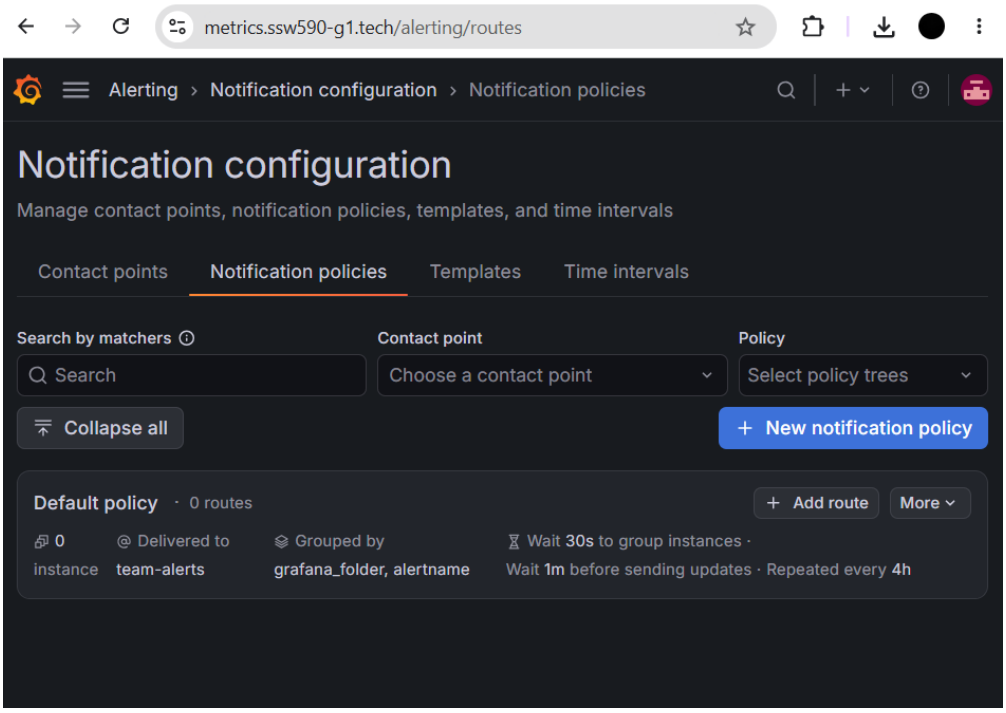


Figure 6.8: Default notification policy: delivered to `team-alerts`, grouped by `grafana_folder` and `alertname`, group wait 30 s, group interval 1 m, repeat 4 h.

```
20:02:10     Threshold edited from 15 to 55 and saved          Grafana History: "Normal (Updated)" (evidence
↪  161)
20:02:30     Pending (A = 32.809085, C = 1)                    Grafana History (evidence 156 screenshot,
↪  161)
20:07:30     Alerting / Firing (A = 32.808320, C = 1)          Grafana History (evidence 157, 161)
20:08        Discord message "Firing", Value A=32.8083...      Discord (evidence 158)
20:14-20:15  Threshold edited back from 55 to 15 and saved     Grafana History: "Normal (Updated)" 20:15:20
↪  (evidence 159, 161)
20:15:20     State back to Normal                              Grafana History (evidence 159, 161)
20:16        Discord message "Resolved", Value [no value]      Discord (evidence 160)

Calculated delays:
- Edit to Pending: 20 s (the next 1-minute evaluation tick).
- Pending to Alerting: exactly 5 min 0 s = the configured pending period.
- Alerting to Discord message: under 30 s (group wait 30 s; Discord shows the minute 20:08).
- Restore edit to Normal: 0 s (the update itself resets the state, "Normal (Updated)").
- Restore to Resolved message: about 40 s (group interval 1 m / group wait).
Two notes for honesty: (1) The Resolved message says "grafana_state_reason = Updated": the alert did not clear
↪  because free space changed but because the rule was edited back to 15; that is the planned way to test
↪  resolution. (2) The Discord times are shown to the minute only.
Final settings: threshold 15 (the rule page in evidence 159 reads "Is below 15"); no silences, no other
↪  change. The rule export (assignment-7/monitoring/grafana/alert-rules.yaml) already has 15.
```

Listing 58: Real timeline of the disk-capacity test: raised to 55, Pending, Firing, Discord, restored to 15, Normal, Resolved.

Test 2: service availability. The second rule was tested by the controlled Jenkins outage (Section 6.7): it went Pending at 20:25:10 EST, Firing at 20:26:10, a Discord message arrived

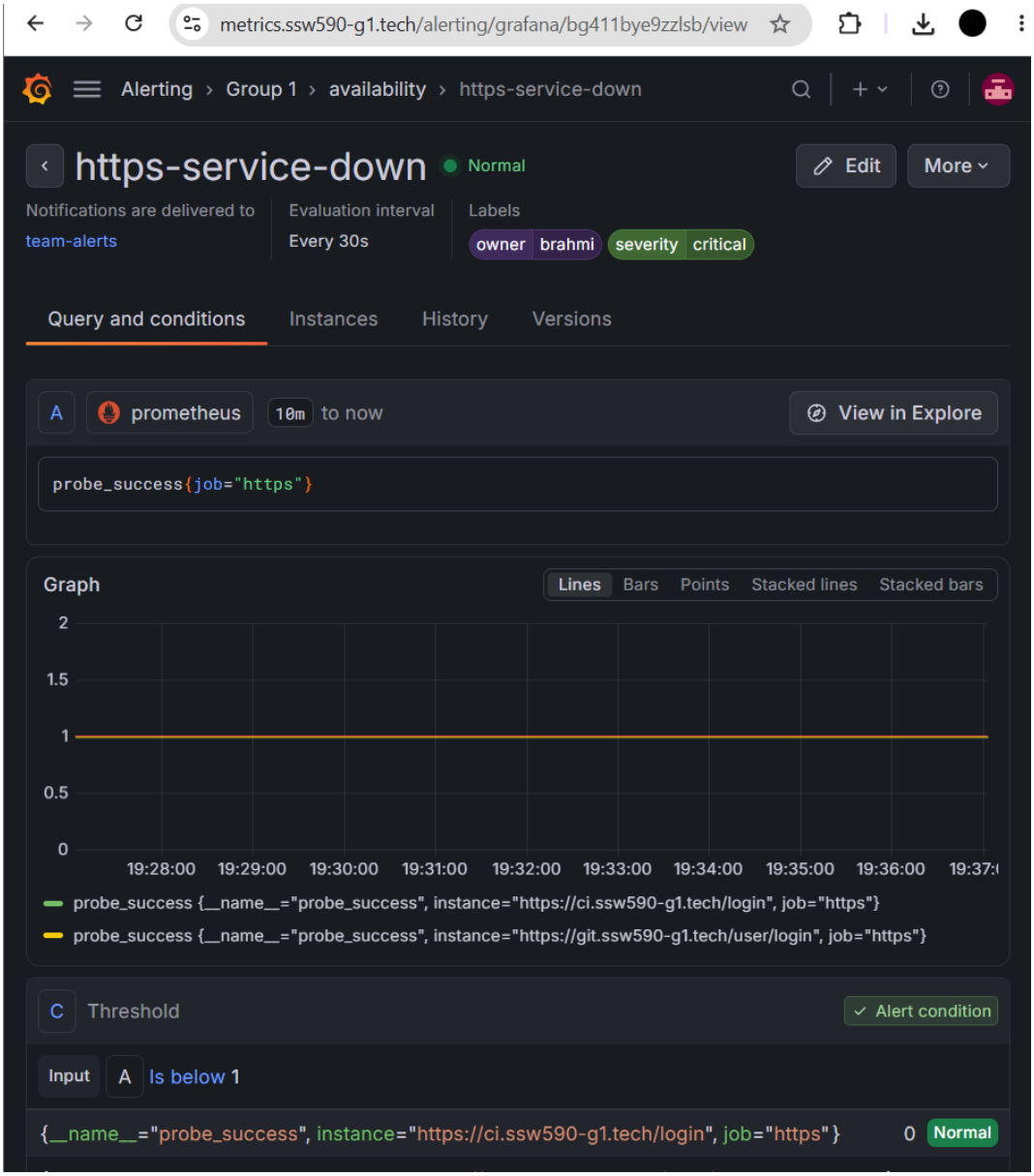


Figure 6.9: Rule `https-service-down`: query, threshold "below 1", labels `owner` and `severity`, notifications to `team-alerts`.

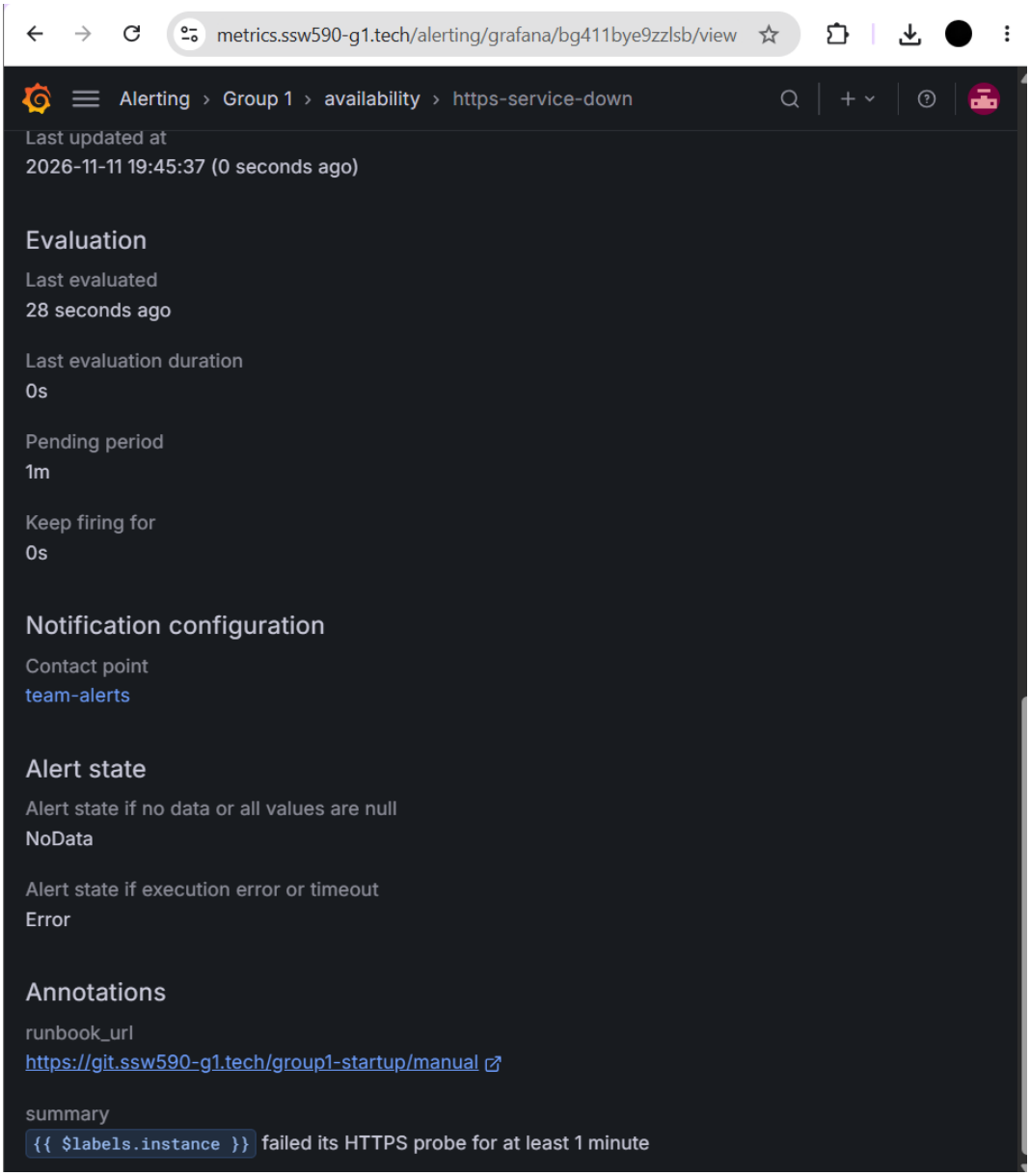


Figure 6.10: The same rule, lower part: pending 1 m, no data `NoData`, error `Error`, `runbook_url` and `summary`.

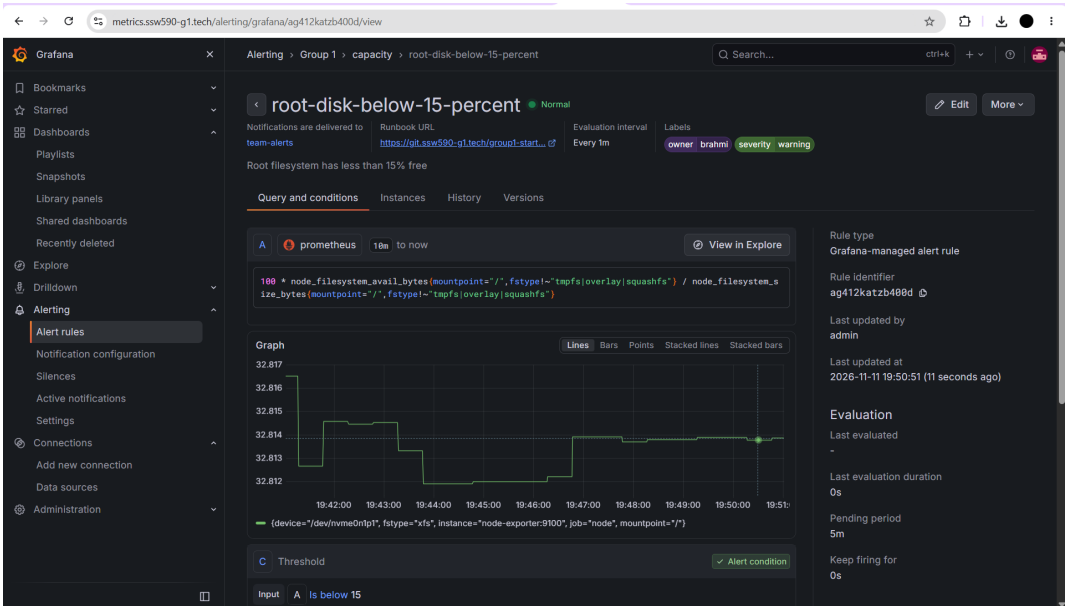


Figure 6.11: Rule `root-disk-below-15-percent`: query for the xfs root, graph at about 32.8%, threshold "below 15".

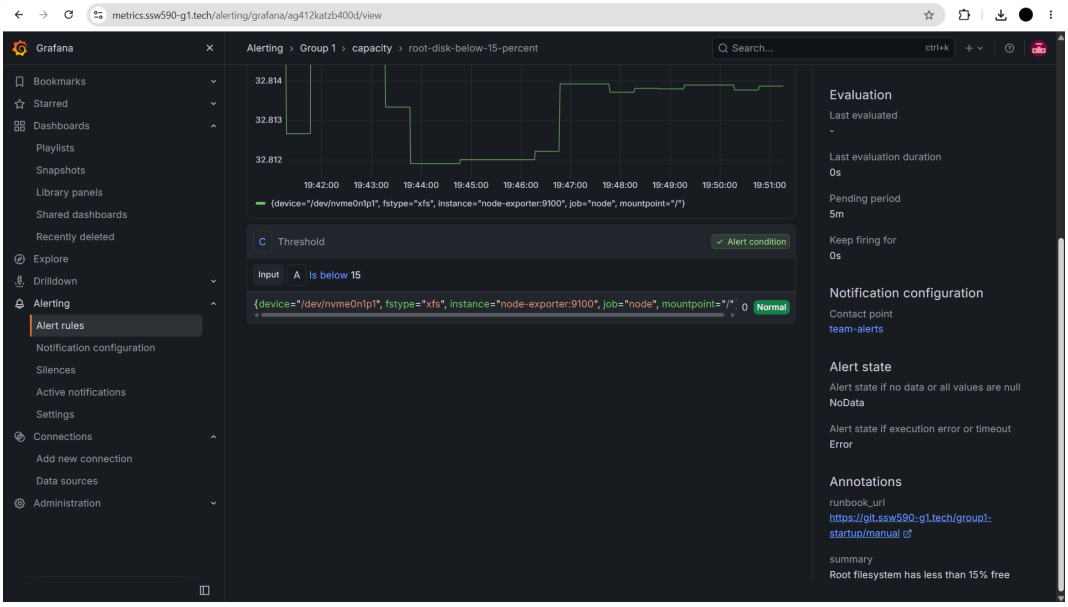


Figure 6.12: The same rule, lower part: pending 5 m, `NoData`, `Error`, `runbook_url`, `summary`.

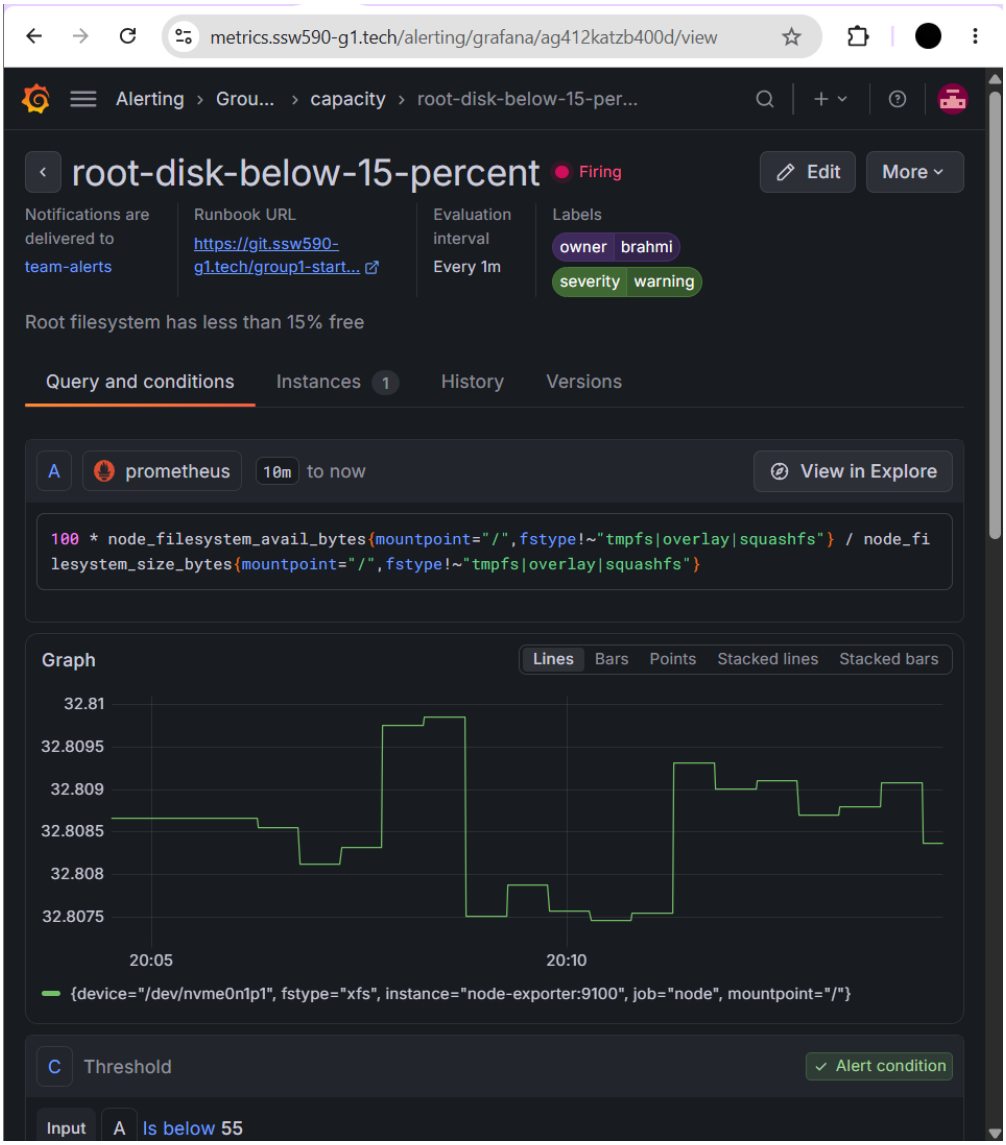


Figure 6.13: Disk rule Firing with the temporary threshold 55, 20:08 EST.

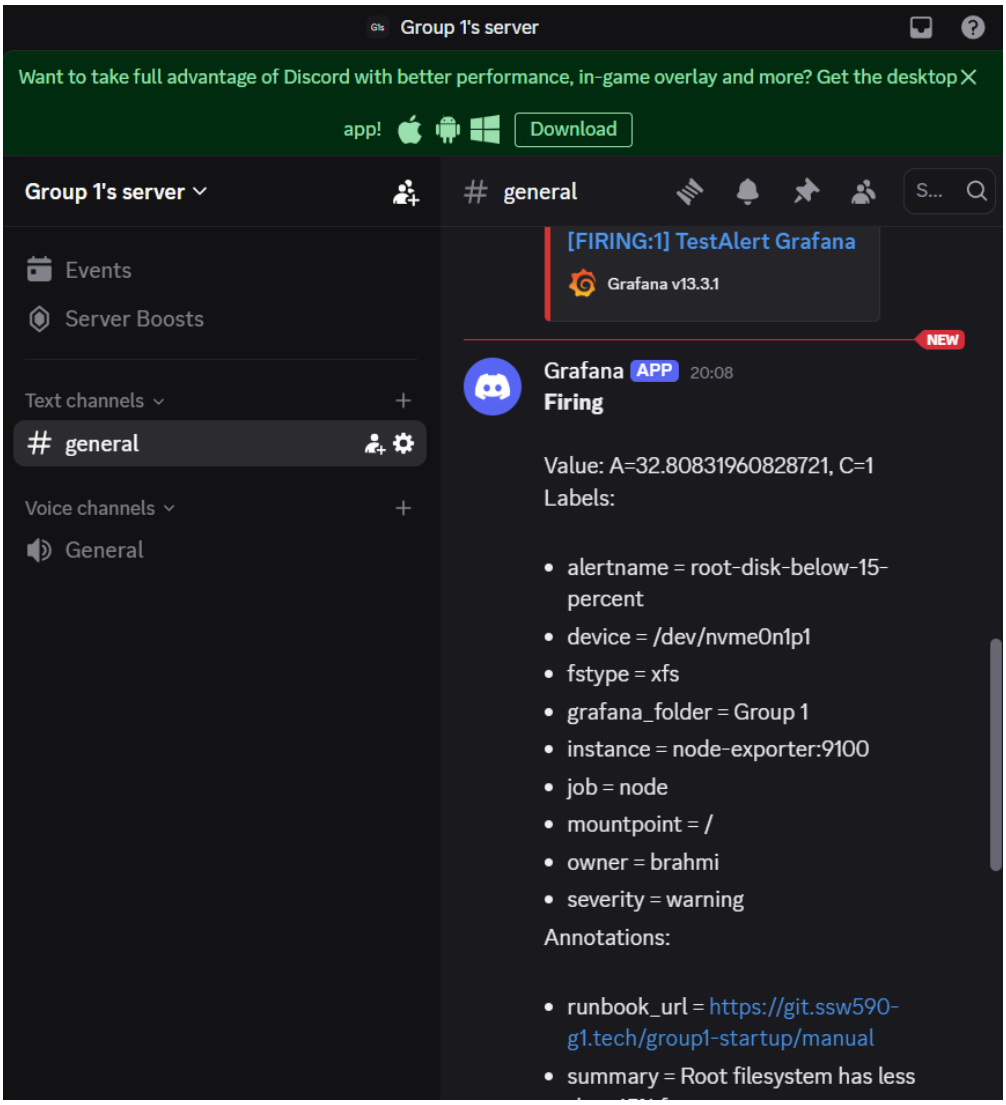


Figure 6.14: The Discord Firing message for the disk rule, 20:08 EST.

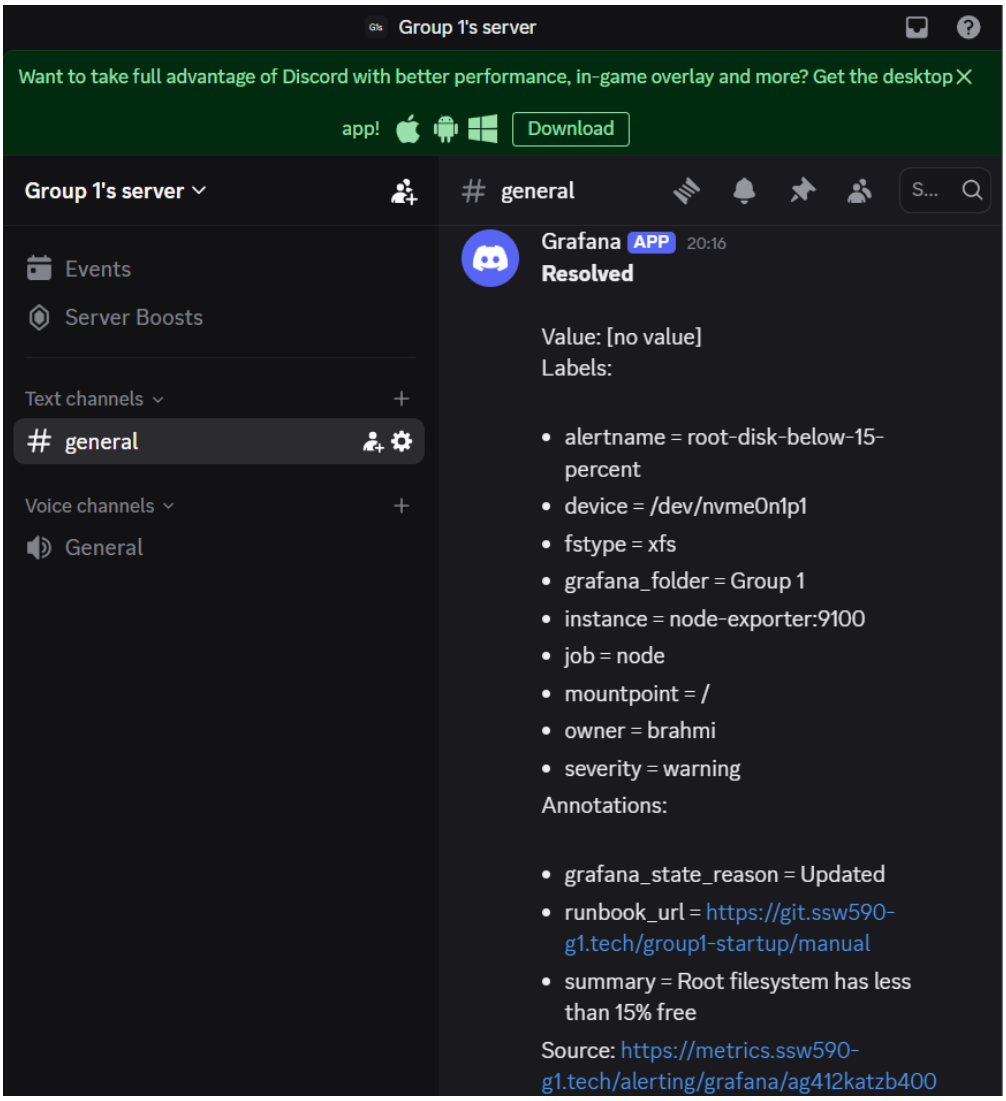


Figure 6.15: The Discord Resolved message, 20:16 EST, after the threshold was restored to 15.

at 20:26 (Figure 6.17), and after the restart the rule was Normal at 20:28:10 with a Resolved message at 20:28 (Figure 6.18, Figure 6.16).

6.7 Controlled incident and review (10 points)

Owner: BD, with SD recording. **Status:** outage run and verified November 11, 2026; blameless review written and two issues opened **PASS**.

Exercise. BD and SD agreed, both being on campus, to stop only the Jenkins container for about three minutes. No build was running. Gitea and monitoring stayed up; no volume was touched and the instance was not stopped. BD ran `docker stop jenkins` and, after the Discord message, `docker start jenkins` in an SSH session, each with the time printed by `date -u`.

Table 6.10: Incident timeline, November 11 to 12, 2026

UTC	EST	Event
01:24:25	20:24:25	Injection: `docker stop jenkins`
01:24:49	20:24:49	First failing probe (24 s later)
01:25:10	20:25:10	Rule Pending
01:26:10	20:26:10	Rule Firing (after the 1-minute pending period)
about 01:26	20:26	Discord Firing message received; read, then acknowledged by restarting (exact time not recorded)
01:27:25	20:27:25	Restart: `docker start jenkins` (3 min 0 s after the stop)
01:27:49	20:27:49	First successful probe
01:28:10	20:28:10	Rule Normal
about 01:28	20:28	Discord Resolved message
01:29:12	20:29:12	Jenkins `main` build #2 started by BD: SUCCESS in 7.9 s
01:29:14–01:29:20		Gitea receives the `twintrack/pipeline/head` status for `a0094a8`: pending, pending, success

Detection and restoration. From the stop to the first failing probe: 24 s (at most one 30 s scrape). From the stop to the rule's Firing state: 1 min 45 s. From the stop to the Discord message: about 2 min 15 s (Discord shows only the minute; the 30 s group wait follows Firing). From the stop to the first successful probe: 3 min 24 s; from the restart to it: 24 s (Jenkins' own start-up is about 13 s). From the first successful probe to Normal: 21 s. The settings (scrape 30 s, evaluation 30 s, pending 60 s, group wait 30 s) allow at most 2 min 30 s before a message, so the observed delay is as configured. Gitea stayed at 1 in all 18 samples around the outage while 6 Jenkins samples failed (Listing 59).

```
Real timeline, Nov 11/12, 2026. Assignment 7, rows 6 and 7. Server clock and Prometheus are UTC; Grafana and
↪  Discord show US Eastern (EST = UTC - 5 h).
```

```
Agreed exercise (BD and SD both on campus, SD recording): stop only the Jenkins container for about 3 minutes
↪  with `docker stop jenkins`, restart it with
`docker start jenkins`. No volume touched, the instance not stopped, Gitea and monitoring kept running. No
↪  build was running (Jenkins idle; no build started in the previous 10 minutes).
Commands run by BD in an SSH session on the server (campus address); the times were printed by `date -u` in
↪  the same command line.

Event (UTC)  EST       What happened / source
01:24:19             last probe = 1 (Jenkins)                                    Prometheus samples of
↪  probe_success (below)
01:24:25     20:24:25 INJECTION: `docker stop jenkins` printed 01:24:25 UTC      server terminal
01:24:49     20:24:49 first failing probe (probe_success = 0), 24 s after stop   Prometheus
01:25:10     20:25:10 rule https-service-down: Pending (A = 0, C = 1)            Grafana History (evidence
↪  163)
01:26:10     20:26:10 rule: Alerting / Firing (after the 1 m pending period)     Grafana History (evidence
↪  163)
about 01:26  20:26    Discord "Firing" message, instance https://ci.ssw590-g1.tech/login   Discord (evidence
↪  164; Discord shows the minute only)
(acknowledged)       BD read the message and then restarted Jenkins; no separate acknowledgement time was
↪  recorded (at the latest 01:27:25)
01:27:25     20:27:25 RESTART: `docker start jenkins` printed 01:27:25 UTC       server terminal (3 min 0 s
↪  after the stop)
01:27:49     20:27:49 first successful probe (probe_success = 1), 24 s after the restart   Prometheus
01:28:10     20:28:10 rule back to Normal (A = 1, C = 0)                         Grafana History (evidence
↪  163)
about 01:28  20:28    Discord "Resolved" message                                  Discord (evidence 165)
01:29:12     20:29:12 Jenkins build twintrack/main #2, Build Now by BD: SUCCESS in 7.9 s, revision a0094a8
↪  (the merged main commit)   Jenkins (evidence 166)
01:29:14-01:29:20     Gitea commit statuses of a0094a8, context twintrack/pipeline/head: pending (queued)
↪  01:29:14, pending (started) 01:29:19, success "This commit looks good" 01:29:20   Gitea API, read without
↪  a token

Raw samples of probe_success for the Jenkins login URL (one per 30 s, UTC; Prometheus range query read at
↪  about 01:30):
  01:21:19=1 01:21:49=1 01:22:19=1 01:22:49=1 01:23:19=1 01:23:49=1 01:24:19=1 | 01:24:49=0 01:25:19=0
  ↪  01:25:49=0 01:26:19=0 01:26:49=0 01:27:19=0 | 01:27:49=1 01:28:19=1 01:28:49=1 01:29:19=1 01:29:49=1
Failing samples: 6 (01:24:49 to 01:27:19). The Gitea login URL stayed 1 in every sample of the same range
↪  (01:21:16 to 01:29:46): only Jenkins was affected.

Calculated times (start and end events named):
- Time to first failing probe: stop 01:24:25 to first failed sample 01:24:49 = 24 s (at most one scrape
↪  interval, 30 s).
- Time to detection by the alert rule: stop to Firing state 01:26:10 = 1 min 45 s (24 s to the failed probe,
↪  the next 30 s evaluation at 01:25:10 starts Pending, then the 60 s pending period).
- Time to notification: stop to the Discord message, shown at 20:26 = between 1 min 35 s and 2 min 34 s
↪  (Discord shows the minute only; Firing 01:26:10 plus the 30 s group wait gives about 01:26:40, so about 2
↪  min 15 s).
- Outage as set: stop to restart = 3 min 0 s. Restoration time, stop to first successful probe = 01:24:25 to
↪  01:27:49 = 3 min 24 s; restart to first successful probe = 24 s (Jenkins' own start-up about 13 s, see
↪  Chapter 4). By the samples, the last good sample 01:24:19 to the first good sample 01:27:49 spans 3 min 30
↪  s.
- Time to resolution: first successful probe 01:27:49 to rule Normal 01:28:10 = 21 s; the Resolved message
↪  shows 20:28.
- Compared with the settings (scrape 30 s, evaluation 30 s, pending 60 s, group wait 30 s: at most 2 min 30 s
↪  expected before a message): observed about 2 min 15 s, as expected.

Verification after recovery:
- Jenkins HTTPS login page reachable (a first check by Claude Code at 01:27:36, 11 s after the restart, still
↪  showed the probe at 0, as expected during start-up).
- Passing pipeline: build main #2 SUCCESS (evidence 166).
- Gitea reachable; commit statuses from Jenkins reached Gitea (twintrack/pipeline/head success for a0094a8 at
↪  01:29:20), so the status integration through Jenkins still works.
- The Prometheus and Grafana restart test (SD's evidence, shared with row 1) follows after SD's SLI window has
↪  ended.
Difference from Chapter 4: that failure was a bad commit stopped by the merge gate (evidence 96-101, Listing
↪  36 and 37); this one is an outage of the CI service itself, found by monitoring.
```

Listing 59: Real timeline of the controlled Jenkins outage with the raw probe samples, the calculated times and the recovery checks.

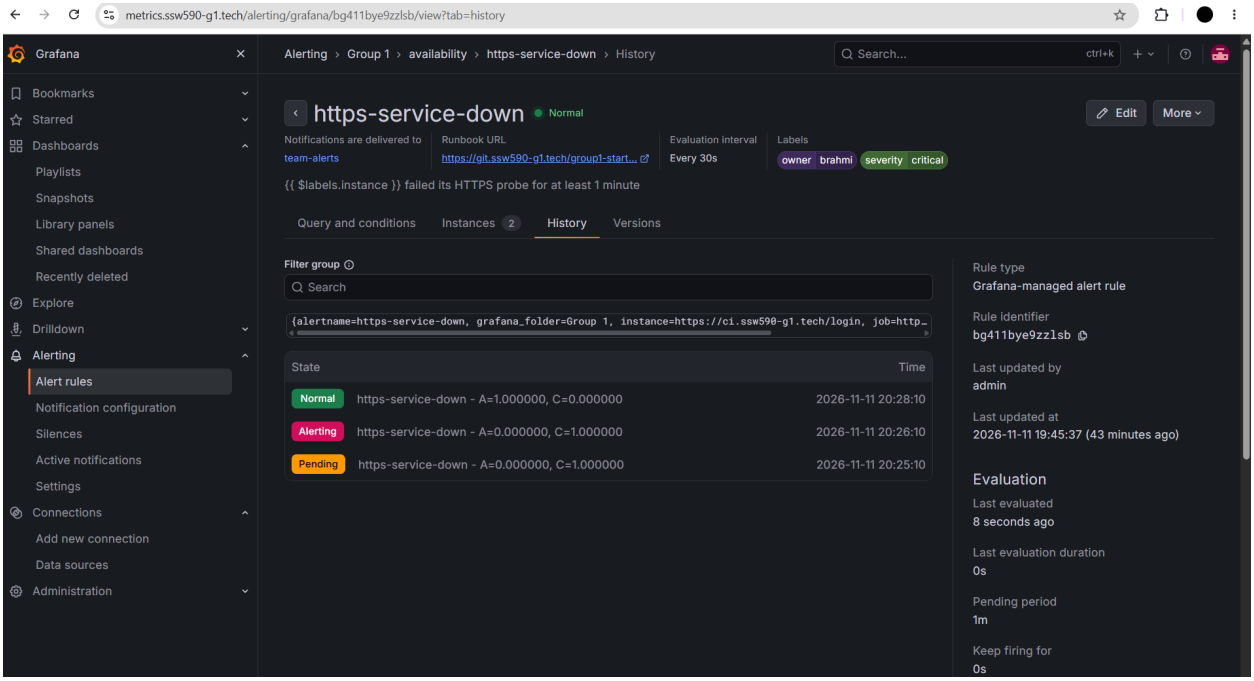


Figure 6.16: History of the rule for the Jenkins instance: Pending 20:25:10, Alerting 20:26:10, Normal 20:28:10 (EST).

Verification. After the restart, Jenkins' login page answered, a pipeline run of `main` passed (Figure 6.19), and the commit status of that run reached Gitea, so the Jenkins-to-Gitea status path works again. The Prometheus and Grafana restart test followed and is described with the deployment (Section 6.1); the record is Listing 45, which also shows the dashboard still present after Grafana's restart (read from Grafana's database, because Grafana 13 keeps dashboards in a new storage table).

Compared with Chapter 4. In Chapter 5 a *bad commit* failed its tests and Gitea's merge gate refused the merge, while Jenkins itself was healthy. Here the commit was fine and the *CI service* was down: nothing failed in Jenkins' tests, no commit status changed, and only monitoring noticed. The two checks cover different failures and neither replaces the other.

Blameless review. *Impact:* Jenkins was unavailable for 3 min 24 s as seen by the probes (stop to first successful probe); only the team uses it, no build was running, and Gitea stayed up, so nobody lost work. *Cause:* a deliberate injection, `docker stop jenkins`, agreed beforehand by BD and SD. *Response:* the rule went Firing after 1 min 45 s, the Discord message arrived about half a minute later, BD restarted Jenkins after reading it, and Jenkins answered 24 s later; a build and the Gitea status path were checked straight away. *What worked:* the alert fired for the right service only, its summary named the URL,

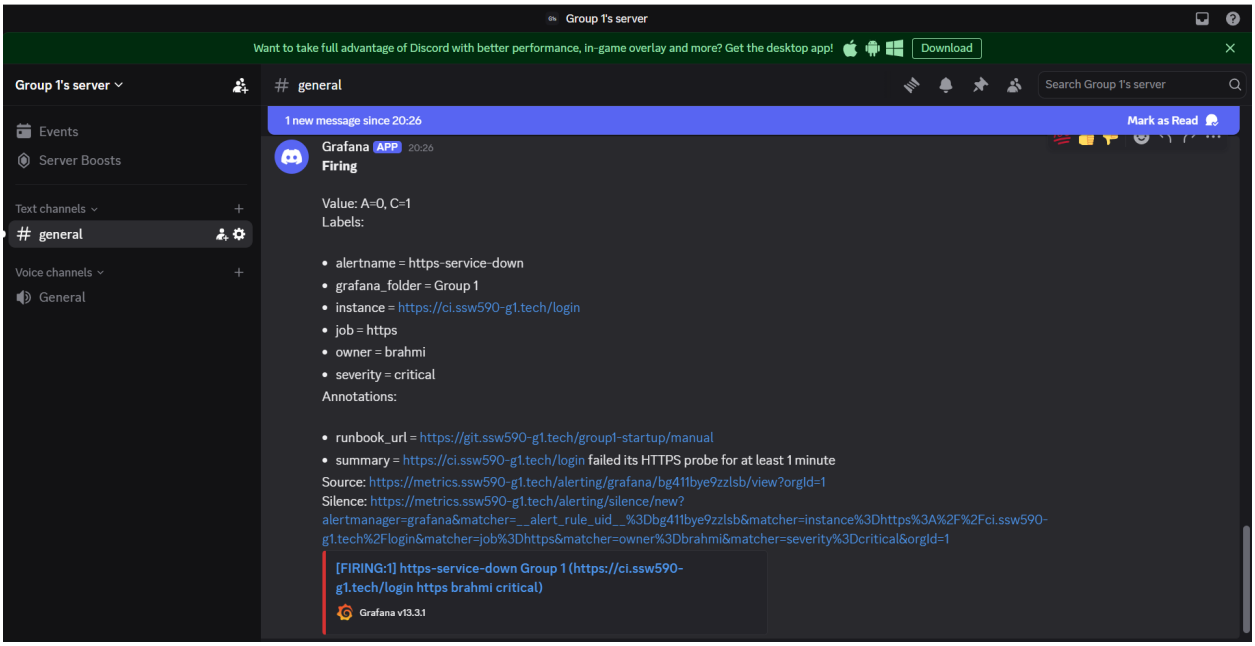


Figure 6.17: Discord Firing message for `https://ci.ssw590-g1.tech/login`, 20:26 EST.

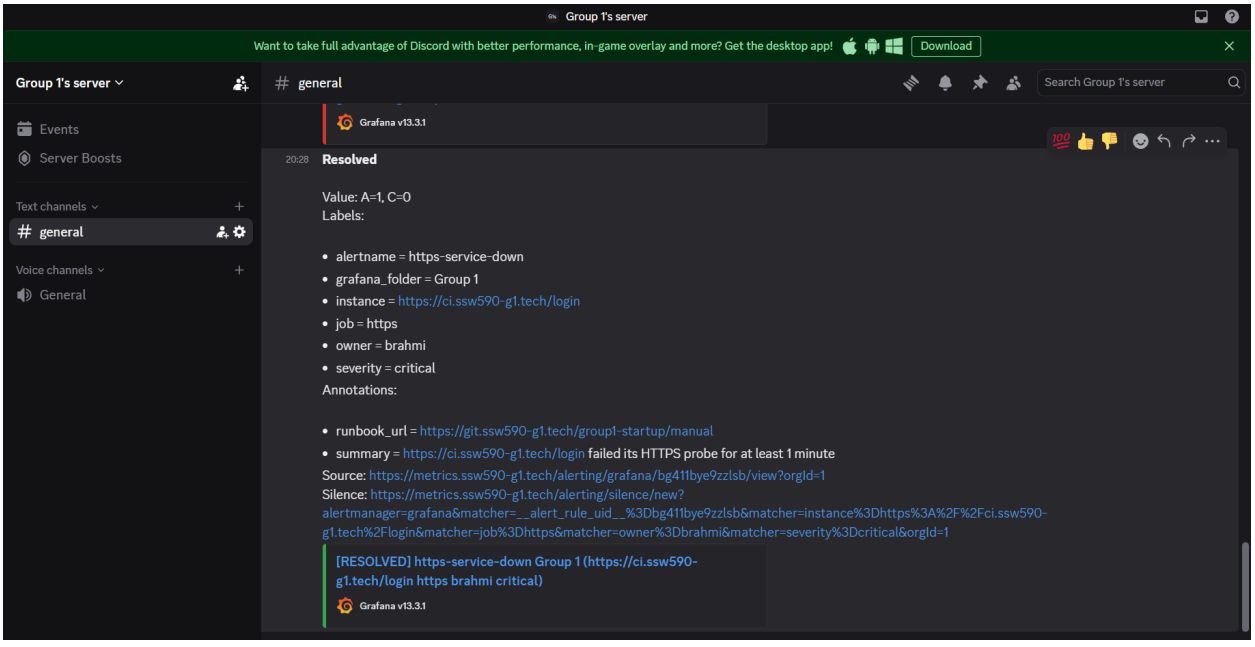


Figure 6.18: Discord Resolved message, 20:28 EST.

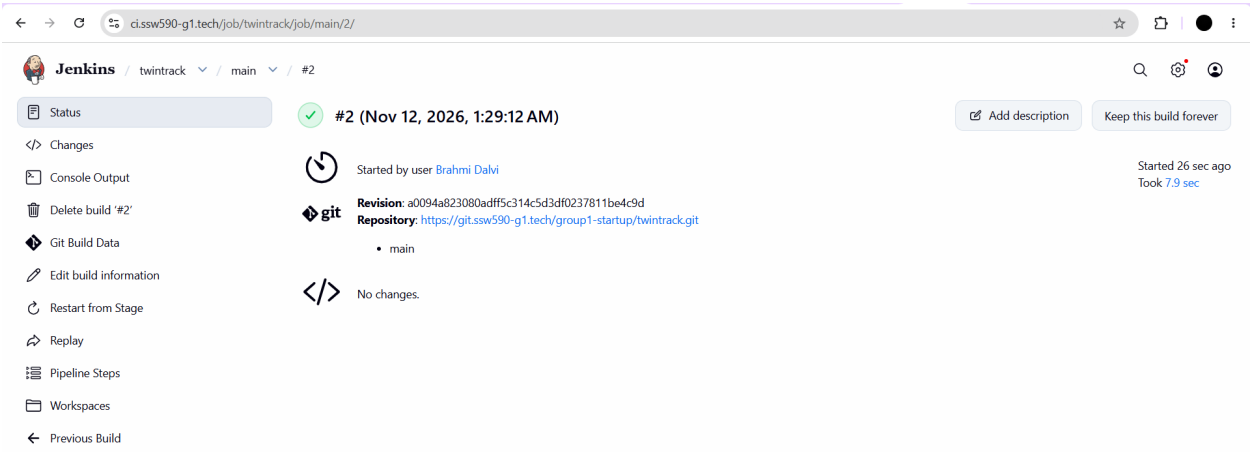


Figure 6.19: Jenkins `twintrack/main` build #2 after the outage, Nov 12, 2026, 1:29:12 AM server time (UTC): SUCCESS in 7.9 s.

the Resolved message followed 21 s after recovery, and the numbers matched the configured delays. *What was weak:* the exact acknowledgement time was not written down, the runbook link in the rules is still the manual repository and not a specific page, the one team contact point is a single Discord channel (if nobody reads it, nothing happens), and the probe that detected the outage runs on the same host as the services. Measurement gaps: one probe location only, and it runs on the same server, so an outage of the whole host would also stop monitoring; probes every 30 s cannot see shorter outages; the first failing probe time (01:24:49) differs from SD's 15 s polling (Listing 57) by a few seconds, which is the sampling interval; and the 4 s probe sample in Section 6.4 was not explained. No person was at fault: the failure was injected on purpose.

Improvement issues. Two prioritized issues are recorded in the `infra` repository in Gitea: **(1) Monitoring shares the monitored server** (owner BD, high priority): add a probe that runs outside this host, for example a free external uptime check or a scheduled job on another machine, because a failure of the whole instance also stops Prometheus, Grafana, and the alerts. **(2) Back up the monitoring state and test notifications on a schedule** (owner SD, medium priority): copy the Grafana and Prometheus volumes off the host and send a monthly test notification, because the restart test shows persistence only, not a backup that can be restored. They are issue #3 (assigned to BD) and issue #4 (assigned to SD), both open on November 12, 2026.

6.8 Manual changes in this version (10 points)

Owner: BD. **Status:** written November 11, 2026; scanned November 12, 2026 **PASS**.

Version and history. The manual's version rose from 1.2 to **1.3** (title page and 1, the document history): the entry dated November 11, 2026 names the authors (BD, SD, and the AI assistant), lists what changed in this chapter, says that the Hosts and Passwords tables *changed*, and links to Chapter 6 and Chapter 2.

Hosts and Passwords changes. **Hosts:** the instance type is `t3.small` (resized November 10); SSH is allowed from two single addresses, the home address and the campus address (kept for the later assignments; the campus address is shared by others on that network); the new host `metrics.ssw590-g1.tech` with Grafana, Prometheus, Node Exporter, and Blackbox Exporter, their versions, memory limits, and private ports (only Grafana reaches Caddy); the DNS record `metrics`; the monitoring notes of the Gitea and Jenkins rows. **Passwords (hints only):** the Grafana administrator (a random first password, then BD's own, set at a prompt on the server), the Grafana user `sakshi` (temporary password handed over privately, changed at first sign-in), the Discord webhook URL of the contact point (stored only inside Grafana), the Discord server, and the extended team address list. No secret is written in either table.

Glossary, references, and attribution. The glossary has new entries for SRE, SLI, SLO, SLA, error budget, Prometheus, Grafana, exporter, and probe. The bibliography has the Chapter 5 sources of the course manual [27, 28, 29, 30, 31, 32, 33, 34] and the SRE Workbook chapter on SLOs [26]; the entries listed as *not separately opened* were not opened by the authors. Dashboard queries and the Prometheus and Blackbox configuration follow the course manual's examples [1], adapted to the real URLs; nothing was copied from an imported dashboard. The AI Prompt Log has the record for this assignment: what the AI assistant did and where it made mistakes, and that no password, token, or webhook URL was ever given to it.

Build and review. The chapter was reviewed by both authors in the manual repository as commit `f54a119`, built successfully by Gitea Actions run #22 (1 min 20 s). SD found no unfinished-text or pending marker left in her rows 1, 2, 3, and 5 in the build of the commit before it. The PDF handed in is built by the next run, from the commit that follows `f54a119` and differs from it only in this paragraph and in three status labels (rows 4, 7, and 8); the build identifier on the title page and in the page footer names that commit. Before it was handed in, the text of the built PDF was scanned for unfinished-text markers, undefined or duplicate references, and sensitive strings (the webhook URL, tokens, the two SSH addresses, and personal email addresses); none was found.

Bibliography

[1] D. Muresan, *DevOpsStartup: A Startup DevSecOps Assignment Manual, Version 1.18, Student Edition*, Stevens Institute of Technology, 2026, sSW 590 course manual (PDF, dosManual-V1.18.pdf); Chapters 1 and 2 govern this chapter.

[2] Docker, "Packet filtering and firewalls," https://docs.docker.com/engine/network/packet-filtering-firewalls/, cited from the course manual bibliography; re-verify before relying on it.

[3] Gitea, "Installation with docker," https://docs.gitea.com/installation/install-with-docker/, cited from the course manual bibliography; re-verify before relying on it.

[4] Caddy, "Automatic https," https://caddyserver.com/docs/automatic-https, cited from the course manual bibliography; re-verify before relying on it.

[5] Jenkins Project, "Gitea plugin," https://plugins.jenkins.io/gitea/, listed as reference 23 of the course manual (Chapter 4); opened by SD on 2026-11-03 and found available.

[6] GitHub Education, "Github student developer pack: Domain offers," https://education.github.com/pack?sort=az&tag=Domains, offer eligibility and terms must be checked at the time of use.

[7] Amazon Web Services, "New: AWS public IPv4 address charge and public IP insights," https://aws.amazon.com/blogs/aws/new-aws-public-ipv4-address-charge-public-ip-insights/, states a charge of 0.005 USD per public IPv4 address per hour from 2024-02-01. Taken from a search-result summary; confirm on the current pricing page. Accessed 2026-10-15.

[8] ——, "Migrate your Amazon EBS volumes from gp2 to gp3 and save up to 20% on costs," https://aws.amazon.com/blogs/storage/migrate-your-amazon-ebs-volumes-from-gp2-to-gp3-and-save-up-to-20-on-costs, source for the gp3 rate of 0.08 USD per GB-month in us-east-1, with third-party guides; the official pricing page was not read. Accessed 2026-10-15.

[9] Holori, "AWS t3.micro price, us-east-1," https://calculator.holori.com/aws/ec2/t3.micro/us-east-1, third-party calculator: 0.0104 USD per hour on demand. Not AWS's own page. Accessed 2026-10-15.

[10] Gitea, "Database preparation," https://docs.gitea.com/installation/database-prep/, cited from the course manual bibliography; accessed 2026-10-12.

[11] ——, "Configuration cheat sheet," https://docs.gitea.com/administration/config-cheat-sheet/, cited from the course manual bibliography; re-verify before relying on it.

[12] Name.com, "Domain safe (previously known as advanced security)," https://www.name.com/support/articles/360001351388-domain-safe-previously-known-as-advanced-security, support article, last updated 2026-06-09. Its body text could not be retrieved in full; the free-certificate statement comes from search-result summaries of it. Accessed 2026-10-15.

[13] Let's Encrypt, "Challenge types," https://letsencrypt.org/docs/challenge-types/, cited from the course manual bibliography; re-verify before relying on it.

[14] Name.com, "Renewing domain safe ssl certificates," https://www.name.com/support/articles/360020995114-renewing-domain-safe-ssl-certificates, support article. The 90-day renewal window is from a search-result summary of it. Accessed 2026-10-15.

[15] GitHub Community, "Namecheap domain, discussion 148485," https://github.com/orgs/community/discussions/148485, a student reports being asked to pay for the Namecheap SSL certificate after claiming the free domain; one report, not confirmed. Accessed 2026-10-15.

[16] Gitea, "Install gitea runner with docker," https://docs.gitea.com/runner/2/installation/docker/, listed as reference 14 of the course manual (Chapter 3); not separately opened by the authors.

[17] ——, "Gitea runner labels," https://docs.gitea.com/runner/labels/, listed as reference 13 of the course manual (Chapter 3); not separately opened by the authors.

[18] ——, "Gitea actions variables," https://docs.gitea.com/usage/actions/actions-variables/, accessed 2026-10-19.

[19] Jenkins Project, "Installing jenkins with docker," https://www.jenkins.io/doc/book/installing/docker/, listed as reference 24 of the course manual (Chapter 4); opened by SD on 2026-11-03 and found available.

[20] Gitea, "Webhooks," https://docs.gitea.com/usage/repository/webhooks/, listed as reference 18 of the course manual (Chapter 4); opened by SD on 2026-11-03 and found available.

[21] ——, "Protected branches," https://docs.gitea.com/usage/access-control/protected-branches/, listed as reference 16 of the course manual (Chapter 4); opened by SD on 2026-11-03 and found available.

[22] Jenkins Project, "Gitea checks plugin," https://plugins.jenkins.io/gitea-checks/, listed as reference 22 of the course manual (Chapter 4); not installed in this deployment; opened by SD on 2026-11-03 and found available.

[23] ——, "Controller isolation," https://www.jenkins.io/doc/book/security/controller-isolation/, listed as reference 20 of the course manual (Chapter 4); opened by SD on 2026-11-03 and found available.

[24] ——, "Credentials," https://www.jenkins.io/doc/book/security/credentials/, listed as reference 21 of the course manual (Chapter 4); opened by SD on 2026-11-03 and found available.

[25] ——, "Securing jenkins," https://www.jenkins.io/doc/book/security/, listed as reference 26 of the course manual (Chapter 4); opened by SD on 2026-11-03 and found available.

[26] S. Thurgood, D. Ferguson, A. Hidalgo, and B. Beyer, "Implementing SLOs," The Site Reliability Workbook, Google, https://sre.google/workbook/implementing-slos/, 2018.

[27] Grafana Labs, "Configure the prometheus data source," https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/, listed as reference 24 of the course manual (Chapter 5); not separately opened by the authors.

[28] ——, "Grafana alerting," https://grafana.com/docs/grafana/latest/alerting/, listed as reference 25 of the course manual (Chapter 5); not separately opened by the authors.

[29] ——, "Run grafana docker image," https://grafana.com/docs/grafana/latest/setup-grafana/installation/docker/, listed as reference 26 of the course manual (Chapter 5); not separately opened by the authors.

[30] Prometheus Authors, "Blackbox exporter," https://github.com/prometheus/blackbox_exporter, listed as reference 39 of the course manual (Chapter 5); not separately opened by the authors.

[31] ——, "Installation and docker persistence," https://prometheus.io/docs/prometheus/latest/installation/, listed as reference 40 of the course manual (Chapter 5); not separately opened by the authors.

[32] ——, "Node exporter: host and container deployment," https://github.com/prometheus/node_exporter, listed as reference 41 of the course manual (Chapter 5); not separately opened by the authors.

[33] ——, "Prometheus configuration," https://prometheus.io/docs/prometheus/latest/configuration/configuration/, listed as reference 42 of the course manual (Chapter 5); not separately opened by the authors.

[34] ——, "Prometheus query functions," https://prometheus.io/docs/prometheus/latest/querying/functions/, listed as reference 43 of the course manual (Chapter 5); not separately opened by the authors.

[35] DigitalOcean, "How to configure firewall rules," https://docs.digitalocean.com/products/networking/firewalls/how-to/configure-rules/, cited from the course manual bibliography; re-verify before relying on it.

[36] Gitea, "Configuration cheat sheet: Actions," https://docs.gitea.com/administration/config-cheat-sheet/, aRTIFACT_RETENTION_DAYS defaults to 90; accessed 2026-10-19.

Appendix A

Document Authoring Guide

A.1 How to Use This Appendix

Use this appendix to maintain the template's file conventions, build process, and review rules. Update it when the document structure or toolchain changes.

A.2 Template Provenance

This template was developed by **Darian Muresan, Ph.D.**, in conjunction with **OpenAI Codex** and **Anthropic Claude Code**. Keep this attribution distinct from the assignment-specific instructor metadata.

A.3 File Naming and Responsibilities

Every chapter file shall use the `devops` prefix, a PascalCase chapter name, and the `.tex` suffix. Each such file shall contain one complete chapter, beginning with `\chapter{...}`, and all of its sections and subsections. Do not split one chapter into section-level `.tex` files. For example, all of Chapter 2 is contained in `devopsExampleAssignment.tex`. Do not add requirements or architecture-design chapters; this template records a DevOps learning activity and its operational evidence.

A.4 Synonyms and Controlled Terms

Use one preferred term for each important concept. If an assignment uses equivalent terms, record them here before release. For example, state whether "continuous delivery" refers to continuous delivery or continuous deployment (CD) generally or specifically excludes automatic production deployment. Add important terms to `devopsGlossary.tex` and index them where used.

A.5 Building the PDF

The template requires a LaTeX distribution with `pdflatex`, `bibtex`, `makeglossaries`, and `makeindex`. Install the tools using the platform instructions below, then run one of the following platform builds.

A.5.1 macOS and Linux Build

```
make -f makefileMacLinux pdf
```

A.5.2 Windows PowerShell Build

```
.\makefilePowerShell.ps1 -Action pdf
```

The completed build regenerates references, bibliography, glossary, and index. Review the log and visually inspect affected PDF pages. Do not edit generated auxiliary files or the PDF directly.

A.6 Install LaTeX Tools on macOS

A.6.1 Recommended Installation

Install the current full MacTeX distribution. It includes TeX Live, `pdflatex`, `bibtex`, `latexmk`, `makeglossaries`, `makeindex`, and useful applications such as TeXShop and TeX Live Utility. The full distribution is the least troublesome choice for this template because it includes the packages used by the glossary, table, listing, color, and hyperlink features.

Either download the signed installer from the MacTeX web site and complete the macOS installation dialog, or, if Homebrew is already installed, run:

```
brew install --cask mactex
```

Close and reopen Terminal after installation. If the TeX commands are not found, add the MacTeX binary directory to the current shell session:

```
export PATH="/Library/TeX/texbin:$PATH"
```

Do not place this command in a shell profile until it has been verified on the local computer.

A.6.2 Verification and Compilation

From the template directory, verify the tools:

```
command -v pdflatex bibtex latexmk makeglossaries makeindex
```

Each command should print a path. Then create the PDF:

```
make -f makefileMacLinux pdf
```

To remove generated intermediate files while keeping the PDF, run:

```
make -f makefileMacLinux clean
```

A.7 Install LaTeX Tools on Windows

A.7.1 Recommended Installation

Install the current Basic MiKTeX distribution using the official MiKTeX Windows installation guide. During installation, choose a per-user installation when appropriate and set missing-package installation to *Ask me first* or *Yes*. The latter is convenient for a student machine with Internet access; the former makes each package installation explicit.

Open MiKTeX Console after installation, apply available updates, and install the `latexmk` package if the verification command below cannot find it. On an organization-managed computer, obtain approval before installing software or changing execution policy.

A.7.2 Verification and Compilation

Open a new PowerShell window in the template directory and verify:

```
Get-Command pdflatex,bibtex,latexmk,makeglossaries,makeindex
```

Each command should display a command record. Build the PDF with:

```
.\makefilePowerShell.ps1 -Action pdf
```

If PowerShell blocks the script under a managed execution policy, ask the system administrator for the approved procedure. Do not bypass institutional security policy. To remove intermediate build files, run:

```
.\makefilePowerShell.ps1 -Action clean
```

A.8 Common Build Problems

Symptom	Resolution
Command not found	Close and reopen the terminal. On macOS, verify the TeX binary path; on Windows, verify MiKTeX completed installation and is on the user PATH.
Missing LaTeX package	Use MiKTeX Console or its approved on-the-fly package installation prompt on Windows. On macOS, install the full MacTeX distribution or use TeX Live Utility to add the missing package.
References or glossary incomplete	Run the supplied platform build instead of a single `pdflatex` command; it runs BibTeX, the glossary tool, and the index tool in the required order.
Script is blocked	Follow the institution's approved PowerShell policy or run the documented build commands with an approved administrator.

A.9 Authoring and Review Rules

Replace bracketed placeholders with assignment facts. Keep operational examples safe, bounded, and free of credentials. Link activities to evidence, retain history, and state unknowns or instructor decisions explicitly. Rebuild after structural, cross-reference, table, glossary, or index changes.

Appendix B

AI Agent Usage Guide

B.1 How to Use This Appendix

An AI agent that creates, reviews, or maintains an assignment shall read this file first. It shall identify the task in the routing table, read the minimum authoritative sources, and expand context only for a discovered dependency or inconsistency. This guide is maintained in LaTeX so it is visible, versioned, and reviewable with the assignment.

B.2 Governing Principles

The authored LaTeX files, versioned automation, and approved assignment materials are authoritative. The PDF is a review artifact and shall not be edited directly. AI may draft, explain, analyze, check consistency, and assist with builds. It shall not invent completion, evidence, access, costs, grades, approvals, command output, security posture, or operational results.

B.3 Chapter File Convention

Every assignment chapter shall be stored in exactly one file named with the `devops` prefix, a PascalCase chapter name, and the `.tex` suffix. For example, Chapter 2 is fully contained in `devopsExampleAssignment.tex`. A chapter file shall begin with its `\chapter{...}` command and contain all of that chapter's sections and subsections. Do not split one chapter across files, create section-only `.tex` files, or use `\input` or `\include` to assemble a chapter from smaller fragments.

Supporting non-chapter files may use a different name only when they are not chapter content: `prologue.tex`, `abstract.tex`, `devopsGlossary.tex`, and `devopsReferences.bib` are examples. Appendix chapters follow the same chapter-file convention.

B.4 AI Quick Start

1. Read this guide and identify the requested outcome.
2. Select the closest task below and load only its listed sources.
3. Inspect supplied facts and report consequential gaps or assumptions.
4. Make narrowly scoped changes; preserve stable IDs, glossary terms, and history.
5. Never request, print, store, or commit credentials, tokens, personal data, or production data.
6. Build the complete PDF, inspect the log, visually inspect affected pages, and record material AI use in Chapter C.

B.5 Task Routing

Table B.1: AI Task-Routing Table

Task	**Minimum authoritative sources**
Initialize an assignment	`devopsAssignment.tex`, `prologue.tex`, `devopsIntroduction.tex`, `devopsExampleAssignment.tex`, and the instructor's assignment facts.
Develop a workflow	`devopsExampleAssignment.tex`, source repository materials, and authorized environment constraints.
Define evidence	`devopsExampleAssignment.tex` and controlled execution outputs.
Review a retrospective	`devopsExampleAssignment.tex` and the submitted workflow or runbook.
Edit AI guidance	`devopsAIAgentGuide.tex`, `devopsAIPromptLog.tex`, `devopsDocumentGuide.tex`, and `devopsAssignment.tex`.
Compile or troubleshoot	`devopsAssignment.tex`, `devopsDocumentGuide.tex`, the affected chapter, and the build log.

B.6 DevOps-Specific Rules

1. Prefer versioned, repeatable automation over undocumented manual steps.
2. Treat deployment output, monitoring data, and screenshots as evidence only when they reflect an actual execution.
3. Design failure exercises to be isolated, reversible, and authorized.
4. Apply least privilege and never place secrets in LaTeX, source control, prompts, screenshots, or logs.

5. State expected cloud cost, cleanup action, and resource ownership for each remote-environment activity.

6. Identify whether a recommendation is a proposal, an instructor directive, or demonstrated evidence.

B.7 Example AI Requests

B.7.1 Create an Assignment Brief

```
Read devopsAIAgentGuide.tex and use the Initialize an Assignment context.
From the supplied SSW 590 objective and constraints, fill only supported
placeholders in the introduction and activity brief. Propose clear, observable
completion criteria, a safe cleanup action, and a rubric. Do not invent cloud
access, due dates, policy, costs, or approved tools. Update history, compile,
and report instructor decisions still needed.
```

B.7.2 Review a Student Workflow

```
Read devopsAIAgentGuide.tex and the workflow, evidence, and retrospective
chapters. Review the supplied repository and sanitized evidence for
reproducibility, least privilege, observability, rollback, and cleanup.
Separate verified facts from gaps and recommendations. Do not claim that an
unexecuted command passed, and do not request credentials or operate resources.
```

B.8 Human Decisions and Information Boundaries

The instructor retains authority over assignment scope, grading, due dates, approved tooling, access, budgets, security posture, exceptions, and policy. An AI agent shall surface alternatives and evidence, not represent a decision as approved. Record material prompts, assumptions, corrections, and unresolved issues in Chapter C.

B.9 Completion Checklist

- Scope, deliverables, environment, constraints, and cleanup are clear.
- Activity identifiers and evidence references are valid and unique.
- Examples contain no secrets, personal data, or unsafe production actions.
- New controlled terms are added to the glossary and indexed.
- The newest approved change is recorded in document history.
- The complete build succeeds with no unresolved references or missing files.

- A human reviews all decisions requiring instructional or operational authority.

Appendix C

AI Prompt Log

C.1 How to Use This Appendix

Record AI assistance that materially affects wording, automation, evidence interpretation, troubleshooting, or feedback: enough context for a reviewer to see the input, output, assumptions, and human corrections. Never record credentials or personal data.

C.2 Record: IT Manual Skeleton and Gitea Chapter Outline

Date: October 12, 2026; Tool and model: Claude Code (Claude Sonnet 5.5); Task: set up this separate IT manual and prepare the Gitea chapter before any infrastructure exists. Inputs: the course manual [1] (Version 1.18, Chapters 1 and 2 read in full; it is logged here as the governing source), and the conventions of the earlier `report/` document. Output used: the manual skeleton, the Gitea chapter outline with eight rubric-aligned subsections and an extra-credit subsection, the Hosts and Passwords tables, a draft cloud-agnostic Compose configuration under `gitea/`, and a work split between the two members. The assistant created no cloud resources, executed no deployment, and recorded no test results; every evidence item is **PENDING**. The first draft of the skeleton was derived from `report/` because the course template was not yet available; it was rebuilt from the Canvas template (version 1.3) after Brahmi Dalvi supplied it, which also required merging the per-section files into the single `devopsGitea.tex` chapter file the template's AI guide requires. Human decisions recorded: SQLite is the final database choice; Brahmi Dalvi owns the cloud deployment, DNS, HTTPS, and firewall work; the 20-point backup/restore extra credit is optional and unassigned (it was later not attempted). Human review and corrections: Brahmi Dalvi reviewed the skeleton, supplied the Canvas template to replace it, and set the decisions listed above. Each author's own review of the later drafts is recorded in that author's record below.

C.3 Record: SD Regions of the Gitea Chapter (Draft)

Date: October 12, 2026; Tool and model: Claude Code (Claude Opus 5.5); Task: carry out Phase A of SD's plan (`assignment-4/SAKSHI.md`) without any cloud resources: local rehearsal of the Gitea and Caddy stack, a persistence rehearsal, and drafts of the `giteaDatabase`, `giteaWorkflow`, `giteaArchitecture`, and `giteaManualQuality` regions. Inputs: the course manual [1] Chapters 1–2 and Appendix A, the team's `README.md` and `SAKSHI.md`, and the draft files in `gitea/`. Output used: the four region drafts, the architecture figure, the script `assignment-4/sd/persistence-test.sh`, a backlog draft, and glossary, bibliography, and history lines. The assistant ran the stack on SD's workstation at `https://localhost` with Caddy's local certificate authority, created a throwaway local administrator whose generated password was kept in a private file outside the repository and never printed, and ran the persistence script twice (the first run failed on a quoting bug, recorded in Table 3.19). Only the rehearsal's container and object IDs are quoted, inside a box marked as not graded; every VM result is **PENDING**. The assistant did not edit BD's regions, `gitea/`, or the root file, and created no cloud resources. Human review and corrections: SD read each draft in the built PDF and approved every push. She ran the server tests herself: the persistence test, with her password typed at a hidden prompt, and the row 6 setup script, with her own token. She took the Gitea screenshots and, with BD, chose a lighter evidence set for row 6 (one public API listing plus three screenshots instead of one screenshot per step). Later work in the same tool and model, October 15, 2026: the row 3 results and figures, the row 7 rewrite for the live AWS deployment (read-only commands run on the server with SD's key), the row 6 subsection, and the row 8 quality pass, including the secrets and address scan. Once during the session SD pasted a Gitea access token into the chat; the assistant did not use it and asked her to delete it in Gitea. The token is not recorded anywhere in this manual or the repository.

C.4 Record: BD Deployment, Evidence, and Drafted Regions

Date: October 12–15, 2026; Tool and model: Claude Code (Claude Sonnet 5.5); Task: carry out Brahmi Dalvi's part of the Gitea assignment: claim the Student Pack domain, create the AWS resources, deploy Gitea and Caddy, verify them, capture evidence, and draft the domain, deployment, HTTPS, and firewall subsections and the Hosts and Passwords tables. Inputs: the course manual [1], BD's screenshots, BD's answers and go-ahead before each step, and real command output from BD's computer and the server. Output used: the security group, key pair, `t3.micro` instance, Elastic IP, first-boot script `gitea/aws/ec2-user-data.sh`, the deployed Compose stack, the evidence files under `evidence/`, and the drafted text.

What the assistant did and did not do. It ran the AWS command line and SSH commands from BD's computer, each after BD's explicit approval, and saved the real output (Chapter 3). It never saw a password or private key: BD created the first Gitea administrator in her own terminal, the SSH private keys were written to files outside the repository

and never printed, and the temporary password for Sakshi's account was shared by BD outside the chat. The assistant verified Sakshi's SSH login from the server's log and read her persistence-test output against the pass conditions in `assignment-4/SAKSHI.md`; it did not verify her Gitea sign-in.

Human decisions the assistant did not make: use AWS (instructor approved), a new instance rather than reusing the earlier one, SQLite, declining the Namecheap authorization because it asked for read and write access to all repositories, declining the paid domain protection, the card handling, and which tests to run. Problems the assistant met and reported rather than hid: a Windows character encoding crash in the AWS tool that made a first check look like a wait, an access-denied on a public image parameter, PowerShell quoting that broke a remote script, a numbering clash in the evidence files, and layout errors in the first draft of the firewall figure. Unverified items are marked pending in the text, notably the certificate renewal test, the second-network firewall test, AWS prices, and the second certificate option. Human review and corrections: Brahmi Dalvi approved each paid or external step before the assistant ran it, supplied every screenshot, and performed the account-creation and token steps herself, so the assistant never handled a password or token. She made the decisions listed above, corrected her mailbox's time zone (which had made an order date look a day off), and chose to turn automatic renewal off and delete the stored card. Assistant errors found and fixed during the group's review were an evidence-numbering clash, a mangled formatting command in the firewall section, and two assumptions about where passwords are stored that nobody had stated.

C.5 Record: SD Rows of the Automated Manual Build Chapter

Date: October 19–20, 2026; Tool and model: Claude Code (Claude Opus 5.5); Task: SD's rows 3, 4, 6, and 8 of Chapter 4. Inputs: the course manual [1] Chapter 3, the team's split in the course repository (`assignment-5/`), Gitea's documentation for Actions variables and configuration [18, 36], and BD's runner. Output used: the standalone manual repository, the job image `ci/Dockerfile`, the workflow, `ci/local-build.sh`, the build identifier mechanism, the change, failure, and repair series (runs #3–#6), evidence 60–66, and rows 3, 4, 6, and 8. The assistant checked the Gitea documentation before relying on context names and artifact retention, and tested every command locally before pushing; it assumed `upload-artifact@v4` would work, which run #1 disproved, and switched to v3. Screenshots of public pages were taken by a headless browser without signing in; evidence was read from Gitea's public API without a token. SD made the first push herself, with her own credentials typed in her own terminal; later pushes used the credential stored on her computer, and no credential was given to the assistant. Human review and corrections: SD read the drafts and approved each push; SD and BD agreed the ownership split.

C.6 Record: Jenkins Integration and Chapter (Assignment 6)

Date: October 27 to November 2, 2026; Tool and model: Claude Code (Claude Sonnet 5.5); Task: plan the assignment from the course manual [1], prepare the Jenkins deployment files, run the server-side steps with BD's go-ahead, collect evidence, and draft Chapter 5. Input: the course manual Chapter 4 (V1.38), the earlier chapters of this manual, the live server (read-only checks and steps approved one at a time), Gitea's and Jenkins' public interfaces, and screenshots and pasted logs from BD. Output used: the plan and task split, the Jenkins `Dockerfile` and compose file, the Caddy block, the build and restart checks, the failure and repair commits, evidence 70–105, and the draft text of all eight subsections (rows 4, 5, 6, and 8 are SD's rows and were drafted while SD was unavailable). Assumptions and mistakes, corrected: the assistant first reported that the Gitea webhook had worked on a real push, but BD had started that build by hand, and the Jenkins records showed it (Listing 37); a restart-time reading of 9 seconds matched the previous start's log (Listing 23); a count of 78 plugin advisories was not a finding (Listing 39); and a 403 at the dashboard was first attributed to Caddy but came from Jenkins (Listing 24). No credential was given to the assistant: BD typed or pasted every token and password herself, and Git pushes to Gitea used the sign-in already stored on her computer. Human review and corrections: BD ran the browser steps, captured the screenshots, and approved each server change; SD reviewed and merged pull request #10 herself (Listing 38), and her review of the drafted rows (4, 5, 6, and 8) was still pending when this entry was written.

C.7 Record: SD Review of Rows 4, 5, 6, and 8 of the Jenkins Chapter

Date: November 2–3, 2026; Tool and model: Claude Code (Claude Opus 5.5); Task: SD's review and completion of the rows of Chapter 5 that BD drafted for her (4, 5, 6, 8), plus SD's review and merge of pull request #10. Inputs: the course manual [1] Chapter 4, BD's drafts and evidence 70–106, and Gitea's public API. What the assistant did: compared `pricing.py`, the test file, and the `Jenkinsfile` on the pull request with the manual and ran the tests locally before SD's review; checked every commit ID, build number, status, time, and the branch rule in the drafts against Gitea's records (all matched); added the push-time and status round trip (Listing 34), a clean-clone test run of the merged `main` (Listing 33), the activity diagram (Figure 5.21), the operator troubleshooting note, and a check of every cited Jenkins and Gitea page; and stated that the second failure and repair were committed one second apart but pushed separately. No Jenkins or Gitea credential was given to the assistant; SD approved and merged the pull request herself in Gitea. Human review: SD read the changes and approved the push.

C.8 Record: BD's Rows 4, 6, 7, and 8 of the Monitoring Chapter

Date: November 10–12, 2026; Tool and model: Claude Code (Claude Sonnet 5.5); Task: BD's part of Chapter 6 (rows 4, 6, 7, 8) and the server steps behind it. Inputs: the course manual [1] Chapter 5, SD's evidence 110–117, and BD's screenshots. What the assistant did: resized the instance from `t3.micro` to `t3.small` with the AWS command line on BD's explicit go-ahead (stop, change type, start; Listing 42); applied the Caddy block for the Grafana name after reviewing SD's draft and adding the team-address rule (Listing 51); ran read-only queries against Prometheus and compared every dashboard query with `free` and `df` (Listing 54); wrote the dashboard JSON and the drafts of the alert rules, the runbook, and the incident timeline; computed the delays from Prometheus' samples and Grafana's history (Listing 58, Listing 59); cropped screenshots to remove browser tabs, other servers, and a personal email address; and drafted the text of rows 4, 6, 7, and 8. Mistakes and corrections: the first plan said no-data and error would be set to *Alerting*, while the saved rules use `NoData` and `Error`, which also notify, so the plan was corrected; an early note said the disk had 42% free when it had 33% after the new images; the first panel table overflowed its column and was split; the first build used for the dashboard comparison ran two minutes after a restart and was replaced by a calmer one (Listing 55); and the assistant tried to add the campus address to the firewall and to the team-address list itself, but the permission system blocked both changes as weakening security, so BD made them herself. No password, token, or webhook URL was given to the assistant: BD typed her Grafana password at a prompt on the server and pasted the Discord webhook only into Grafana, and Git pushes to Gitea used the sign-in she entered. Human review: BD ran the Jenkins stop and start of the outage herself, clicked through Grafana, created the contact point, approved each server change, and reviewed and merged SD's infra pull request in Gitea; SD recorded the outage and ran the SLI window and the restart test.

Glossary

artifact A file produced by a workflow run and stored with that run for download; kept for a limited retention period, so not a backup. 180

automation A repeatable, version-controlled procedure executed by software rather than undocumented manual steps. 180

container An isolated, runnable instance of a container image that packages an application with its dependencies. 180

container image An immutable, layered template built from a Dockerfile from which containers are started. 180

controller executor A build slot on the Jenkins controller itself, as opposed to a separate build agent; pipeline code then runs on the same process that holds Jenkins' configuration. 180

deployment The controlled delivery of a versioned change to an environment. 180

Docker Compose A tool that runs a multi-container application from a versioned YAML file. 180

error budget The amount of failure an SLO allows: 1% of 2,880 probes in a day is 28.8 failed probes (14.4 minutes). 180

evidence A retained record of an actual execution, observation, or review. 180

exporter A small program that exposes metrics of a host or service in the format Prometheus scrapes; here Node Exporter (the host) and Blackbox Exporter (probes). 180

Gitea A self-hosted Git service with issues, pull requests, and organizations. 180

Gitea Actions Gitea's built-in continuous integration, compatible with GitHub Actions workflow files stored under `.gitea/workflows/`. 180

Grafana An open-source tool that draws dashboards from stored metrics and evaluates alert rules, sending notifications to a contact point. 180

Jenkins An open-source automation server that runs build and test pipelines; here a controller in a container that tests pull requests of the product repository. 180

Jenkinsfile A text file in the product repository that defines a Jenkins pipeline as code (stages and commands). 180

job container A temporary container that a runner starts from a fixed image for one workflow job and removes afterwards, so each build starts clean. 180

Kanban board A visual workflow board with columns such as To Do, In Progress, and Complete, used to make work and work-in-progress visible. 180

multibranch pipeline A Jenkins job type that scans a repository and creates a pipeline job for each branch or pull request that contains a Jenkinsfile. 180

named volume Storage managed by Docker and mounted into a container; it outlives the container and is removed only explicitly. 180

observability The ability to infer system behavior from signals such as logs, metrics, traces, and health checks. 180

persistence Data surviving replacement of the container that uses it, because it lives on a mounted volume. 180

pipeline An automated sequence that builds, tests, packages, and/or deploys a change. 180

probe A scheduled test request made from outside a service, such as an HTTPS request to a login page, whose result is recorded as 1 (success) or 0 (failure). 180

Prometheus An open-source monitoring system that scrapes numeric metrics from targets every 30 seconds, stores them as time series, and answers queries. 180

pull request A request to merge a branch into another, reviewed and approved before merging; in Gitea a description containing "Closes #N" closes issue N on merge. 180

rollback A controlled return to a known acceptable version or state after a failed or harmful change. 180

runbook A documented operational procedure for a repeatable task or incident response. 180

runner A program (here `act_runner`) that registers with Gitea, picks up workflow jobs, and runs them, typically in temporary job containers. 180

security group A stateful virtual firewall attached to an AWS instance that lists the inbound and outbound traffic it allows. 180

SLA Service level agreement: a commitment to a customer, possibly with consequences when missed; this exercise creates no SLA. 180

SLI Service level indicator: a measured value that describes how a service behaves; here the share of scheduled HTTPS probes that succeed in a window. 180

SQLite An embedded SQL database stored in a single file and accessed as a library, with no server process, network port, or credential. 180

SRE Site reliability engineering: running a service with measured reliability targets, monitoring, alerts, and a written review of every incident. 180

status check A named result (pending, success, failure) that a service such as Jenkins reports on a commit; Gitea can require it before a pull request is merged. 180

synonyms Equivalent terms used in this assignment; record preferred terms to avoid ambiguity in instructions and evidence. 180

TeX Live A cross-platform distribution of TeX, LaTeX, and their packages; Overleaf compiles projects with it. 180

user data A script supplied when an EC2 instance is launched that runs once as root at first boot. 180

webhook An HTTP request that Gitea sends to another service when an event such as a push or a pull-request update happens; here it tells Jenkins to build. 180

Acronyms

AI artificial intelligence. 180

AMI Amazon Machine Image. 180

AWS Amazon Web Services. 180

CD continuous delivery or continuous deployment. 167, 180

CI continuous integration. 180

DNS Domain Name System. 180

DORA DevOps Research and Assessment. 180

EC2 Amazon Elastic Compute Cloud. 180

IaC infrastructure as code. 180

SLO service-level objective. 180

SWOT strengths, weaknesses, opportunities, and threats. 180

TLS Transport Layer Security. 180

UML Unified Modeling Language. 180

Index

AI agent, 171
    chapter file convention, 171
    DevOps workflow, 171
AI prompt log, 175
Appendix
    AI Agent Usage Guide, 171
    Document Authoring Guide, 167
artifact, 65
automated manual build, 55
    quality controls, 72

build identifier, 65

continuous integration, 75
controlled failure, 68, 107

document conventions, 167
document history, v

error budget, 143

Gitea, 12
    architecture and operation, 47
    database and persistence, 26
    firewall and external exposure, 37
    https and certificate renewal, 30
    manual quality and controls, 51
    optional extra credit: verified backup and restore, 54
    pinned gitea deployment, 18
    startup team workflow, 42
    student pack domain and dns, 13
Gitea Actions, 55

hosts table, 3

introduction, 1
IT manual, 1

Jenkins, 75
    quality controls, 116
Jenkinsfile, 99
job container, 61

LaTeX, 167
latexmk, 61

macOS, 168
MacTeX, 168
merge gate, 75, 102
MiKTeX, 169
monitoring, 120
Muresan, Darian, iii

passwords table, 3
pipeline, 99
pull request, 42

runbook, 47
runner, 61

SLA, 143
SLI, 143
SLO, 143
SQLite, 26
SRE, 120
startup DevSecOps process, 12
status check, 102
synonyms, 167

template attribution, iii
TeX Live, 61
troubleshooting
    failed build, 73

Windows, 169